

- 集成电路的互连线
- 互连线延时模型
- 互连线的信号完整性问题

第六章

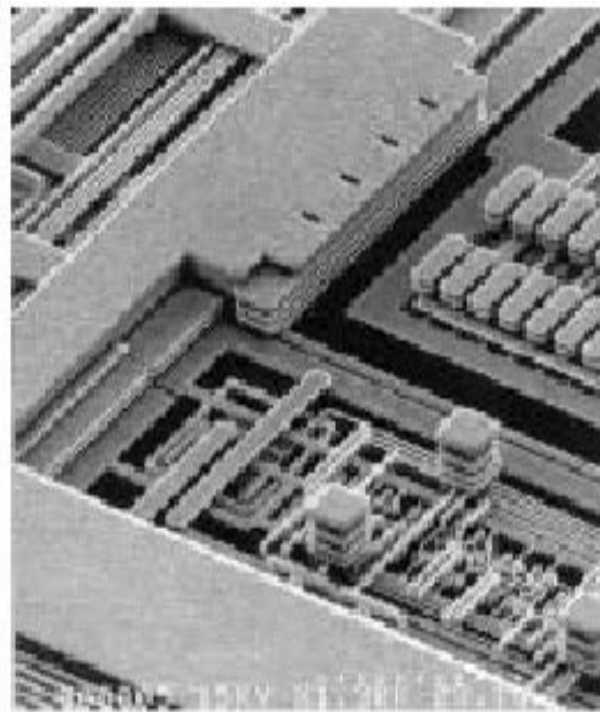
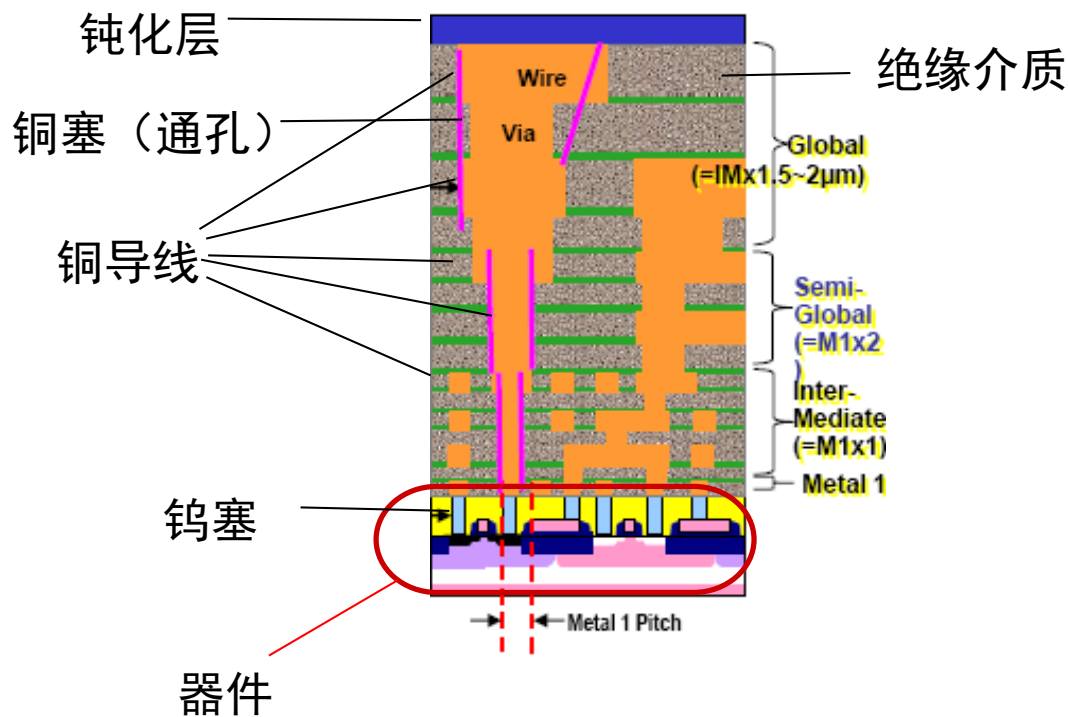
互连线与互连技术

6.1 集成电路的互连线

集成电路中的导线

- ❖ 集成电路中的导线采用铝或者铜（现代先进工艺采用铜）。
- ❖ 导线和导线之间通过二氧化硅为主的绝缘介质进行隔离
- ❖ 采用多层布线（目前先进工艺布线层达到10层左右），不同布线层之间通过通孔垂直连接

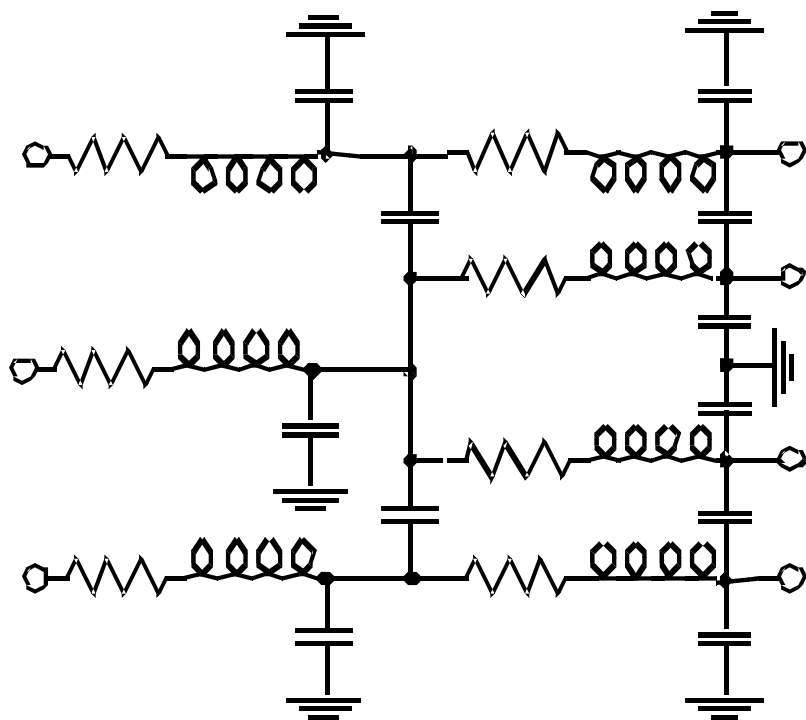
现代集成电路的互连线



导线材料：铝或者铜
导线和导线之间通过绝缘介质隔离
多层布线，不同布线层之间通过通孔连接

导线模型

完整的电路模型



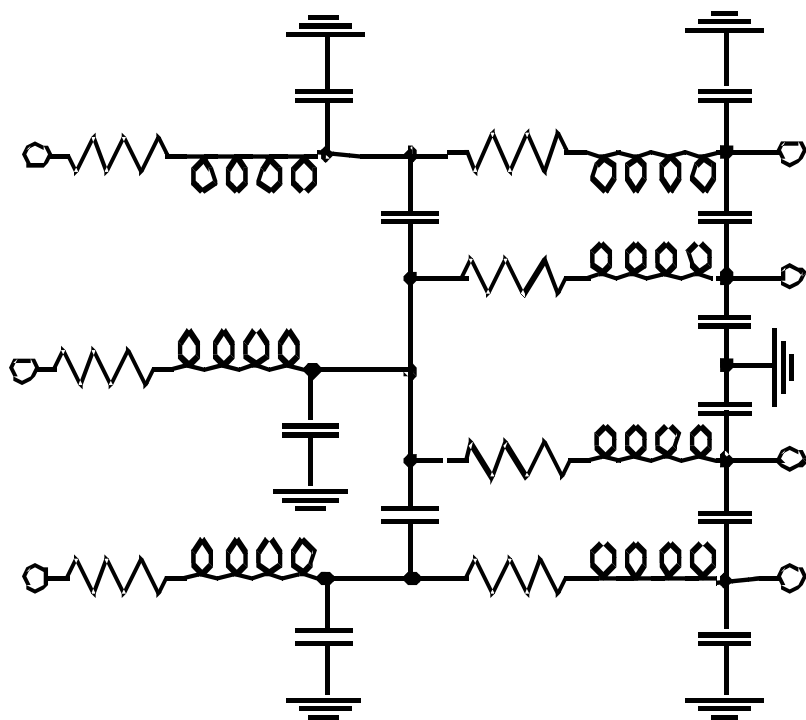
物理结构



每段导线存在一定的寄生电阻、寄生电感，导线和衬底间存在寄生电容，相邻的导线之间存在线间电容。

导线模型

完整的电路模型

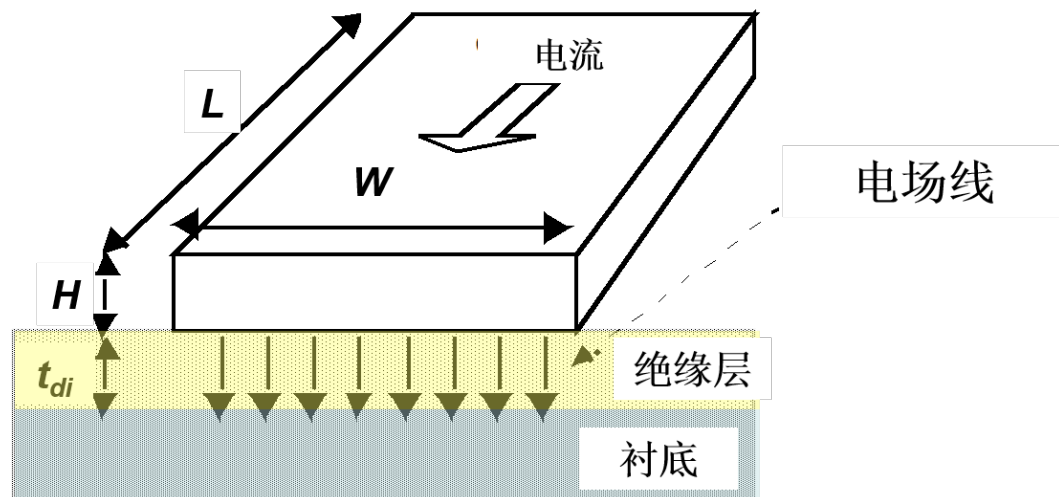


物理结构



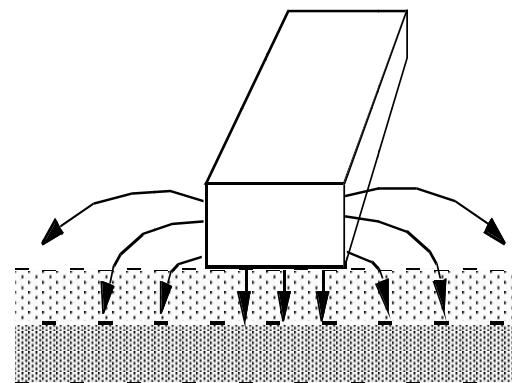
每段导线存在一定的寄生电阻、寄生电感，导线和衬底间存在寄生电容，相邻的导线之间存在线间电容。

互连线的寄生电容——对地电容



$$C_{int} = \frac{\epsilon_{di}}{t_{di}} WL$$

* 芯片的衬底通常要么接电源要么接地，可以看做交流地平面



边缘电容（导线侧壁与衬底间的电容）

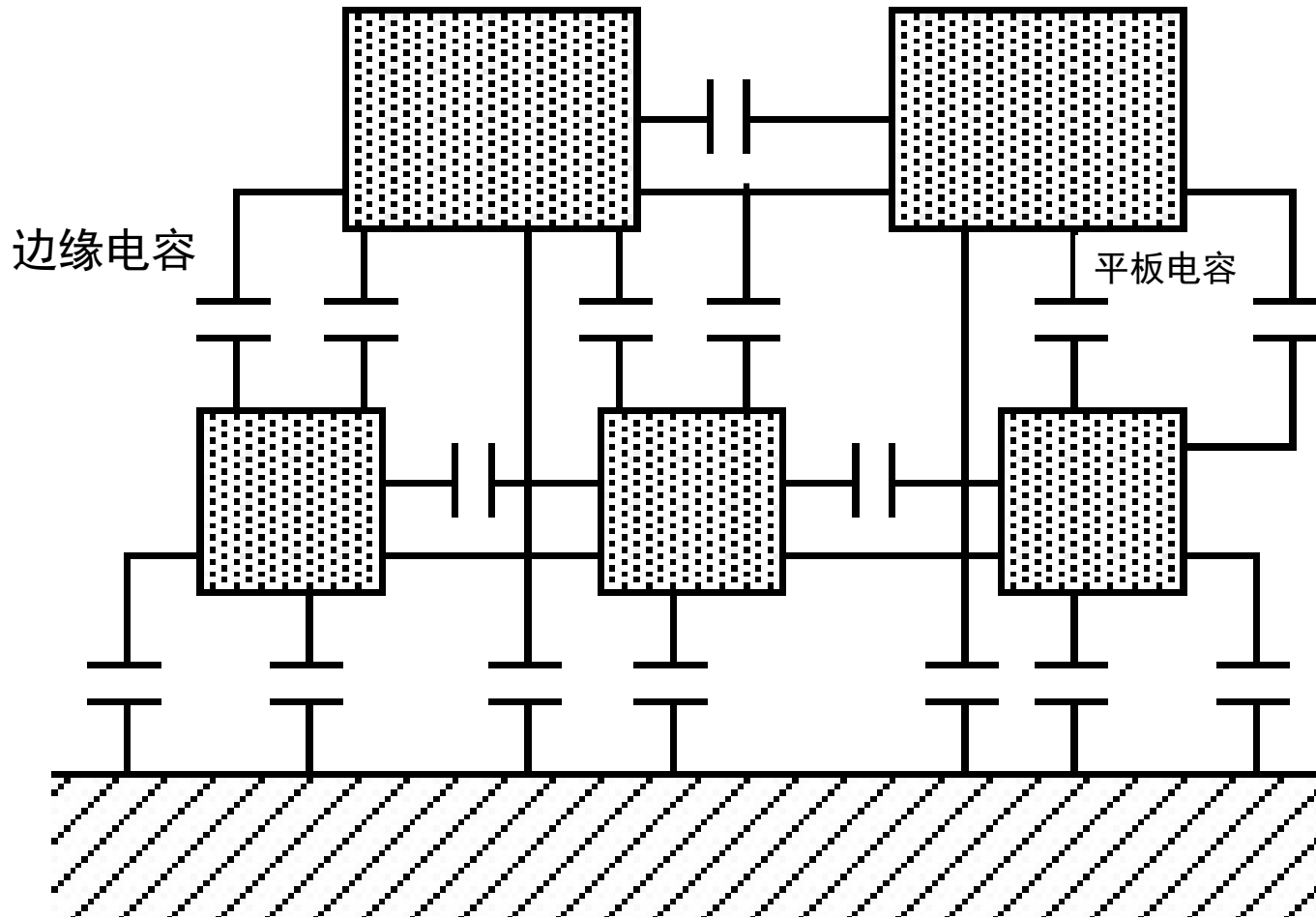
$$C_{wire} = C_{pp} + C_{fringe}$$

平行板
电容

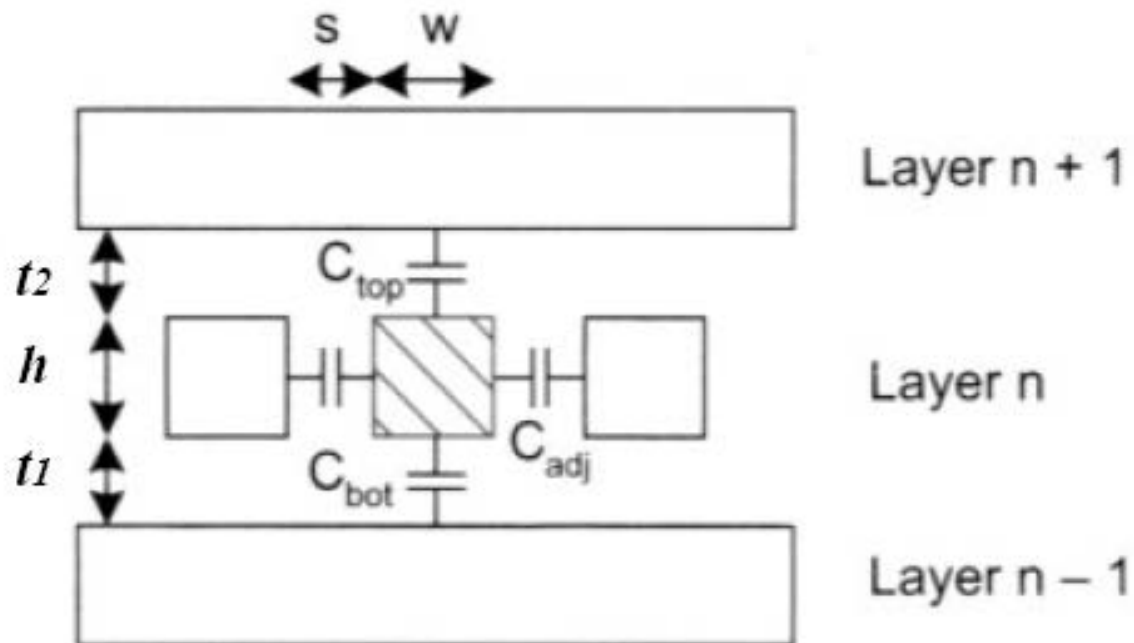
边缘电容

大 W/H ，总电容接近平行板电容
小 W/H ，边缘电容占主要部分

互连线的寄生电容——线间电容

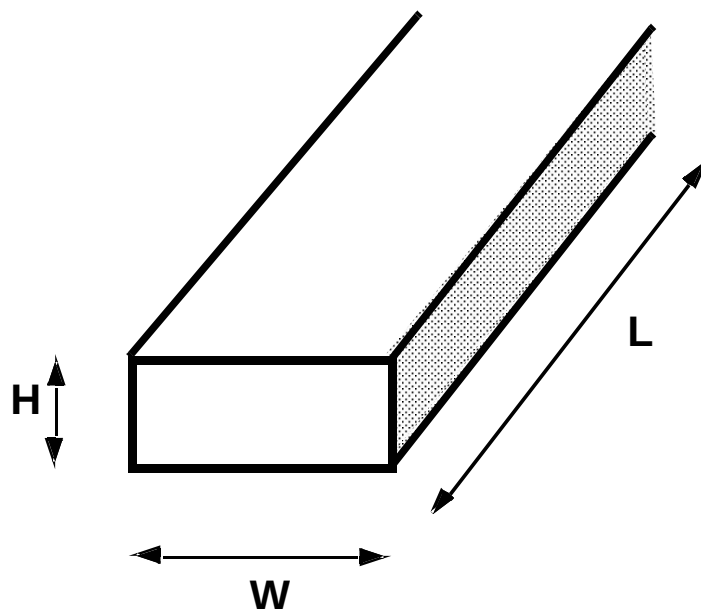


互连线电容



如果周围导线都是连接固定电位，则互连线的总的对地电容等于所有寄生电容之和。

互连线电阻



$$R = \frac{\rho L}{HW}$$

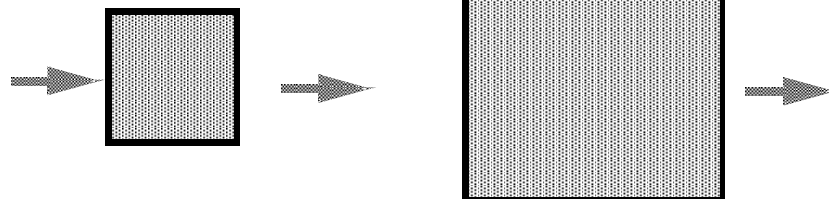
方块电阻 (Sheet Resistance)

$R_{\square}$

R_1

$\equiv$

R_2



• 典型值

- * 金属(铝, 铜)—— $0.05 \Omega/\square$
- * 多晶硅—— $10 \sim 15 \Omega/\square$
- * 扩散区 (N^+) —— $20 \sim 30 \Omega/\square$

导线的总电阻 = 串联的方块数 $\times$ 材料方块电阻值

接触电阻

❖ 层间转换（接触孔和通孔）——接触电阻

❖ 应尽可能减少接触孔和通孔

❖ 接触孔

- 大接触孔具有小的接触电阻
- 电流集聚效应：对于较大接触孔，电流往往集中在接触孔的周边——采用多个小尺寸接触孔减小接触电阻

互连线的寄生电感

一段导线的寄生电感 l 和寄生电容 c 之间的关系：

$$cl = \varepsilon\mu$$

ε : 周围介质介电常数

μ : 周围介质磁导率

电磁波在导线中的传播速度：

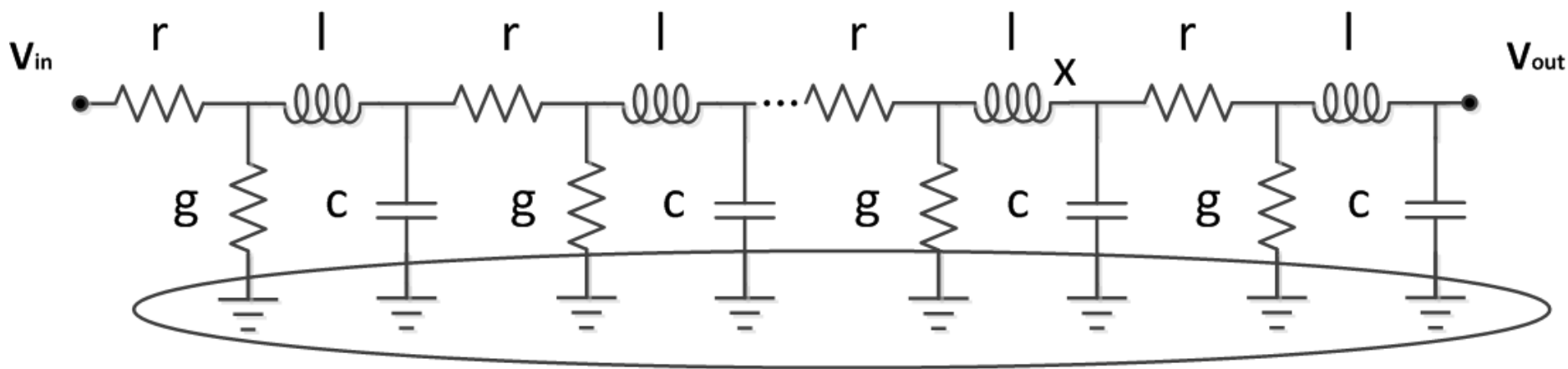
$$v = \frac{1}{\sqrt{lc}} = \frac{1}{\sqrt{\varepsilon\mu}} = \frac{c_0}{\sqrt{\varepsilon_r \mu_r}} \quad c_0: \text{真空中的光速}$$

电感的影响

- 集成电路芯片内部的导线一般寄生电感值很小，可以忽略
- 考虑电感的条件：低电阻、高开关频率
- 高频电路中的寄生电感会引起：
 - 振荡
 - 过冲
 - 阻抗失配引起的信号反射
 - 线间互感

互连线的寄生效应

- 集成电路互连线存在寄生的电阻、电容和电感
- 实际处理时忽略那些影响不大的寄生效应



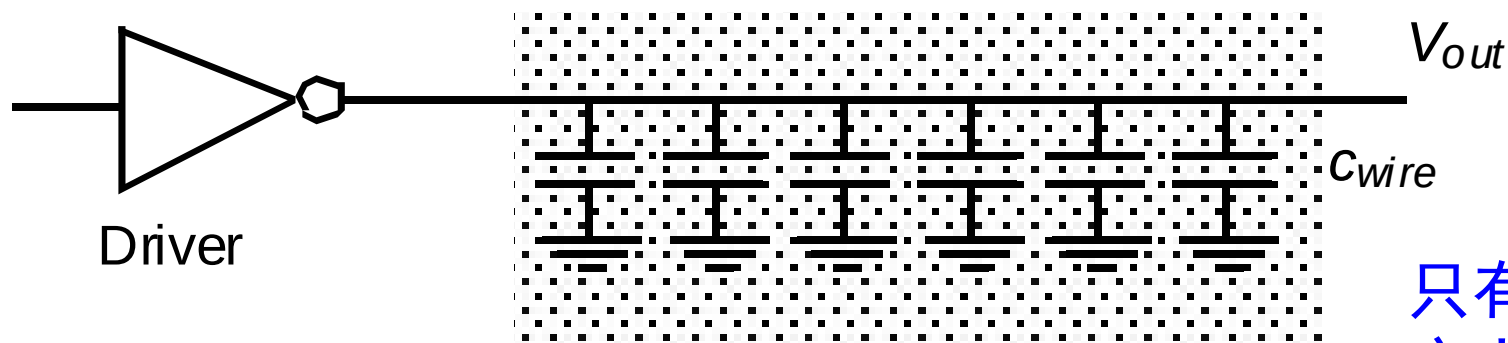
6.2

互连线延时模型

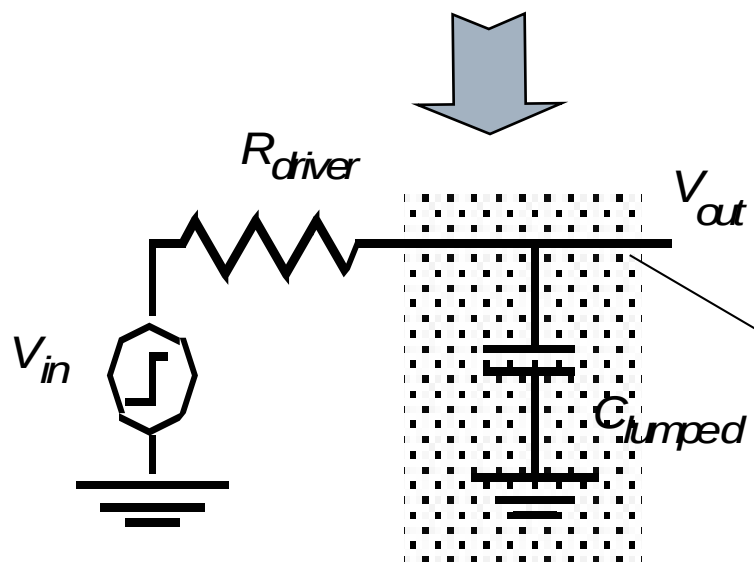
6.2.1

集总电容模型

集总模型 (Lumped Model)



只有寄生电容占支配地位



等电势，无延时，只考虑了负载效应

集总电容模型

- 当导线较短时，寄生电阻和寄生电感可以忽略，只考虑寄生电容。
- 由于忽略了电阻和电感，导线上各处的电势总是相同的，即导线上各处的信号之间没有延时——导线延时等于0。
- 导线对于电路延时的影响只有增加的负载电容。

6.2.2

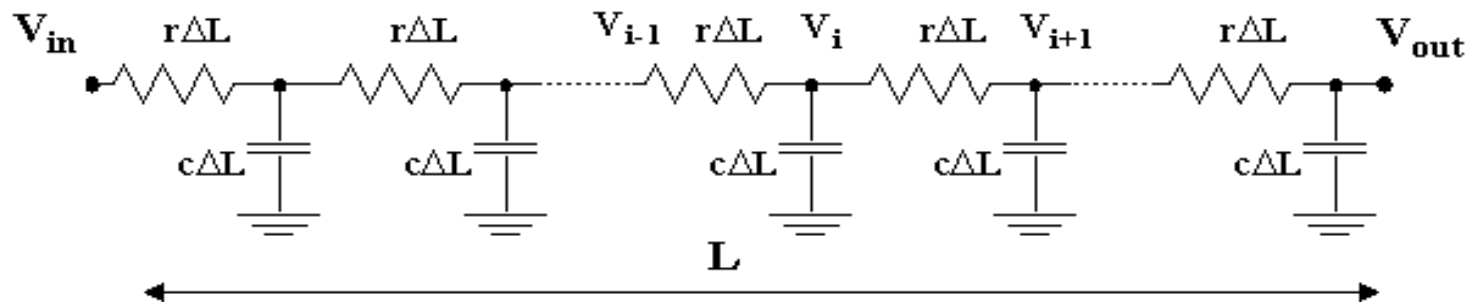
分布rc模型

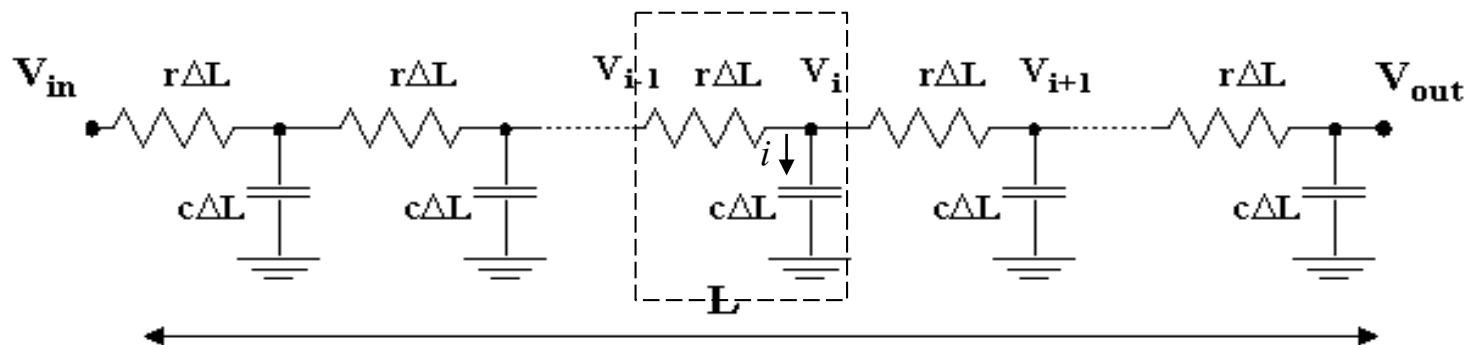
分布rc模型

当导线长度较长（相对于门延时）、信号速度较快时需要考虑寄生电阻(考虑互连线延时)。

条件：

(1) $t_{pRC} > t_{pgate}$; 并且 (2) 导线输入信号的 $t_{edge} < R_w C_w$.



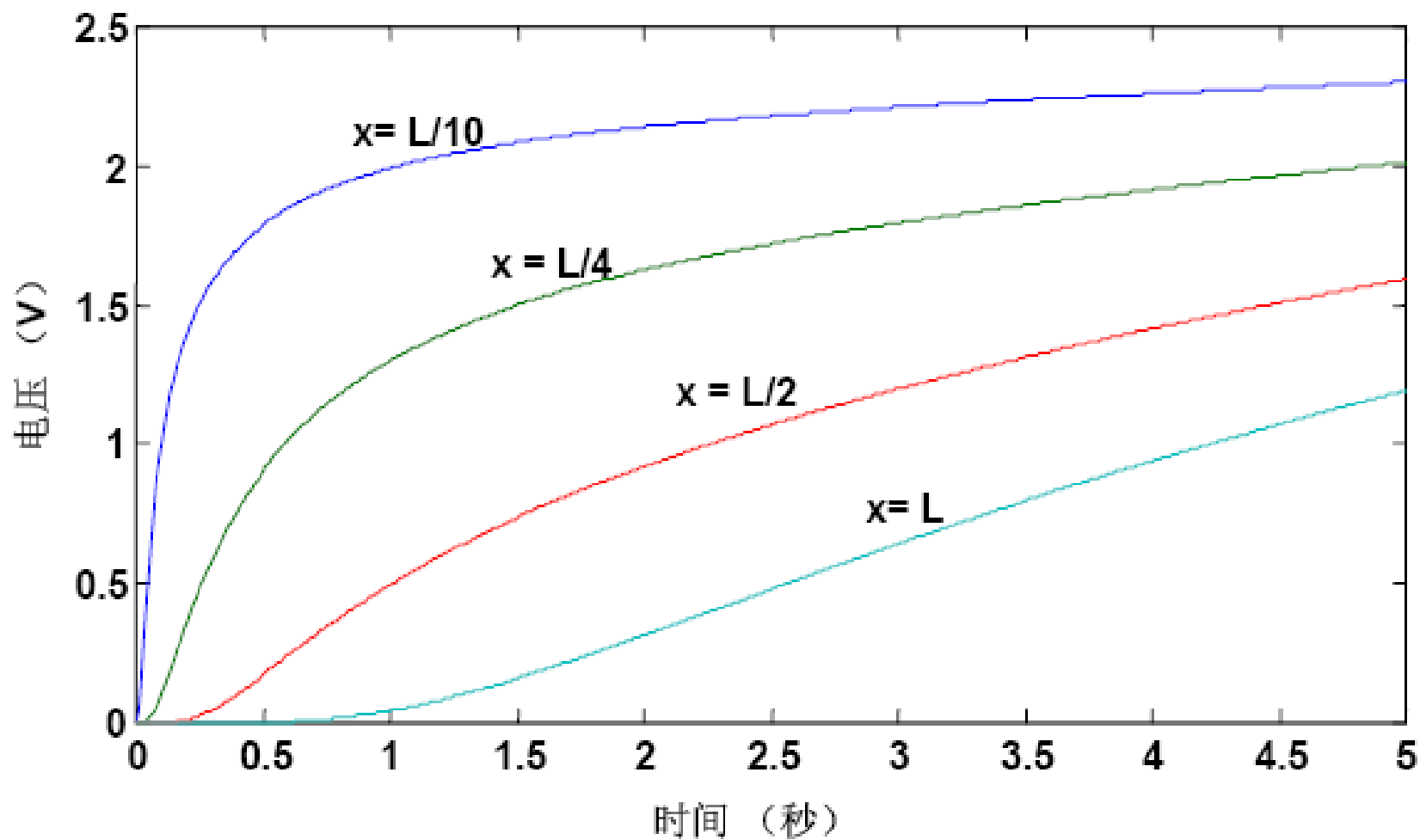


$$c\Delta L \frac{\partial V_i}{\partial t} = \frac{(V_{i+1} - V_i)}{r\Delta L} + \frac{(V_{i-1} - V_i)}{r\Delta L}$$

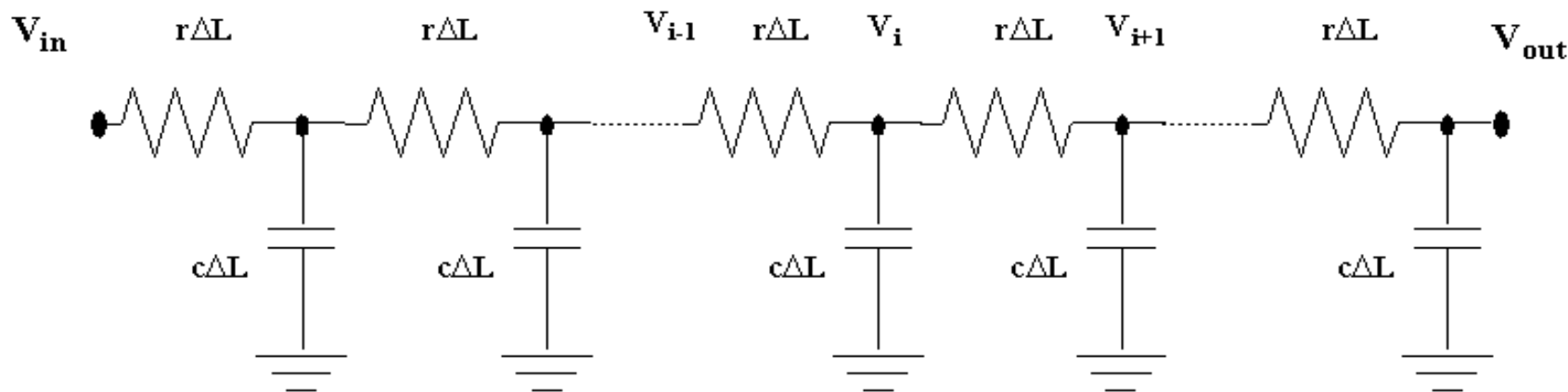
当 $\Delta L \rightarrow 0$, 用长度微分 dx 代替, 对应的电阻压降变为沿 x 方向的电压微分 dv

$$\Rightarrow \quad rc \frac{\partial v}{\partial t} = \frac{\partial^2 v}{\partial x^2} \quad \text{扩散方程}$$

RC导线的阶跃响应与时间及位置的关系



rc链的Elmore延时



总长度: L

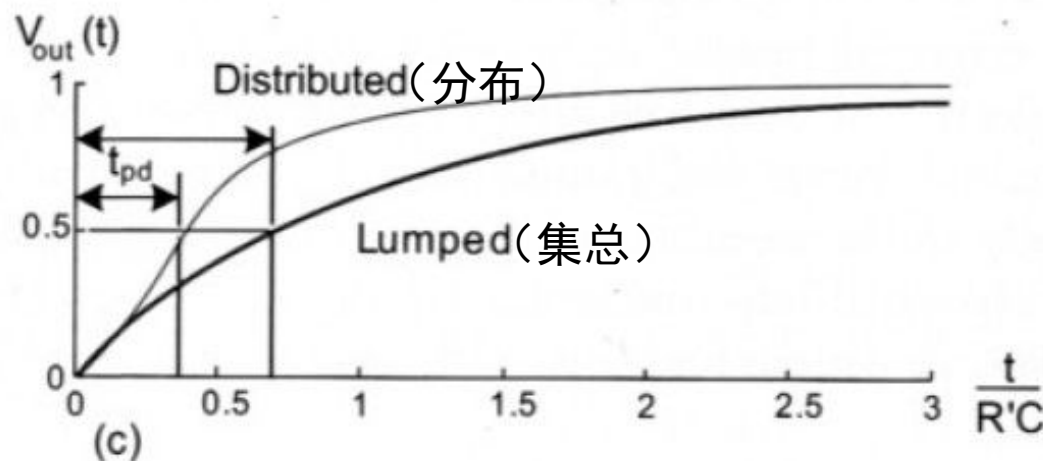
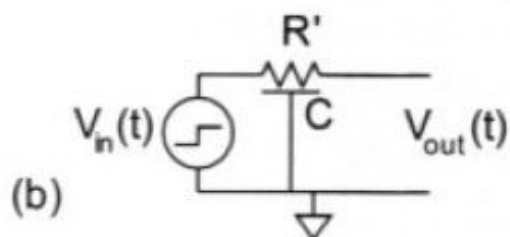
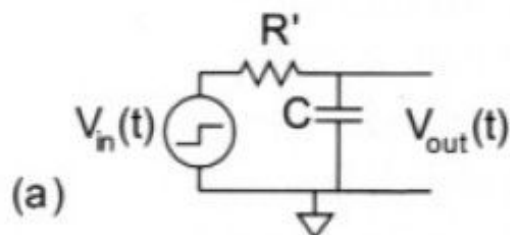
总电阻 R (集总电阻) $= rL$

总电容 C (集总电容) $= cL$

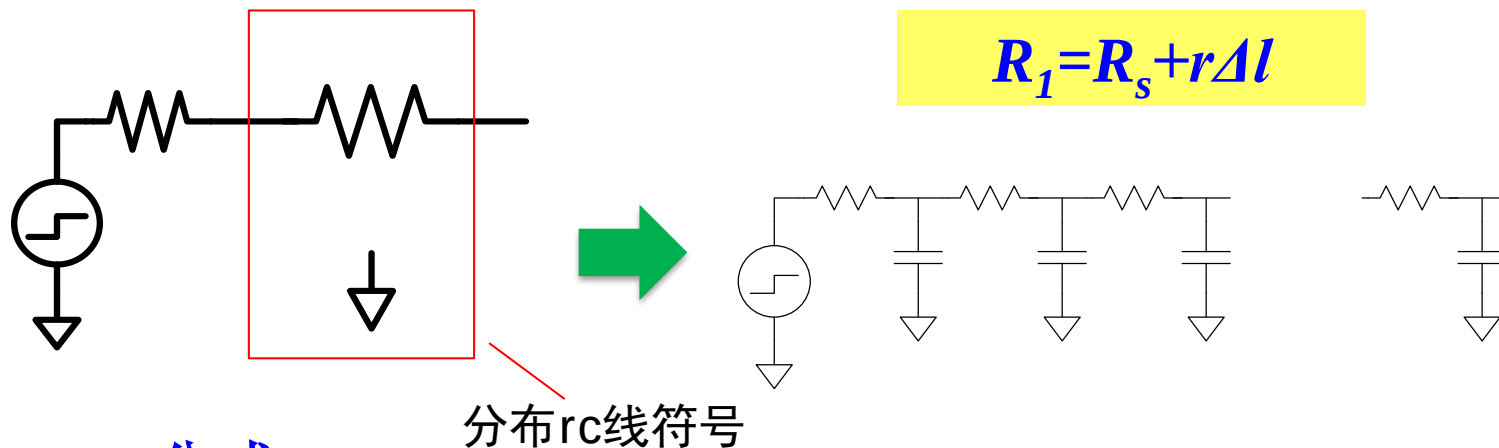
$$\begin{aligned}\tau_{DN} &= \sum_{i=1}^N C_i \left(\sum_{j=1}^i R_j \right) = \sum_i C_i R_{ii} = \sum_i c\Delta l \cdot \sum_j i \cdot r\Delta l \\ &= rcL^2 \frac{N(N+1)}{2N^2} = RC \frac{N+1}{2N} \xrightarrow{N \rightarrow \infty} \frac{RC}{2} = \frac{rcL^2}{2}\end{aligned}$$

集总与分布RC的阶跃响应

电压范围	集总RC网络	分布rc网络
$0 \rightarrow 50\% (t_p)$	$0.69 RC$	$0.38 RC$
$0 \rightarrow 63\% (\tau)$	RC	$0.5 RC$
$10\% \rightarrow 90\% (t_r)$	$2.2 RC$	$0.9 RC$

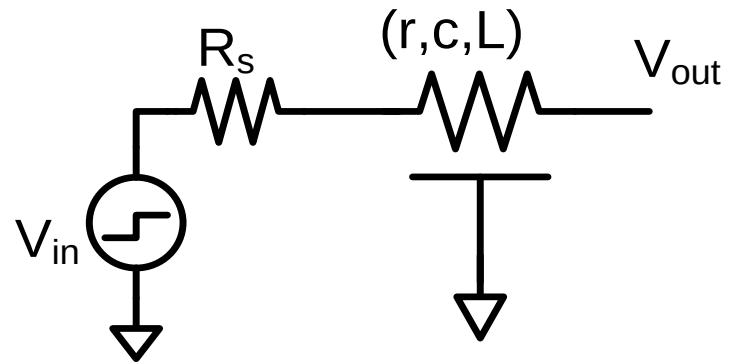


考虑驱动器内阻 = R_S 时 RC 线的延时



Elmore公式

$$\begin{aligned}
 \tau_D &= R_1 C_1 + (R_1 + R_2) C_2 + \cdots + (R_1 + R_2 + \cdots + R_N) C_N \\
 &= (R_S + r\Delta l) c\Delta l + (R_S + r\Delta l + r\Delta l) c\Delta l + \cdots + (R_S + Nr\Delta l) c\Delta l \\
 &= NR_S c\Delta l + r\Delta l c\Delta l + (2r\Delta l) c\Delta l + \cdots + Nr\Delta l c\Delta l \\
 &= R_S cL + \frac{1}{2} r c L^2 \\
 \therefore \tau_D &= R_S C_w + \frac{1}{2} r c L^2 \\
 t_p &= 0.69 R_S C_w + 0.38 r c L^2
 \end{aligned}$$



$$t_p = 0.69R_s \cdot cL + 0.38rcL^2$$

逻辑门延时 互连线延时

小结

互连线模型：

- 集总电容模型：低速短导线
- 分布rc模型：高速长导线， t_r (t_{pgate}) $< RC$

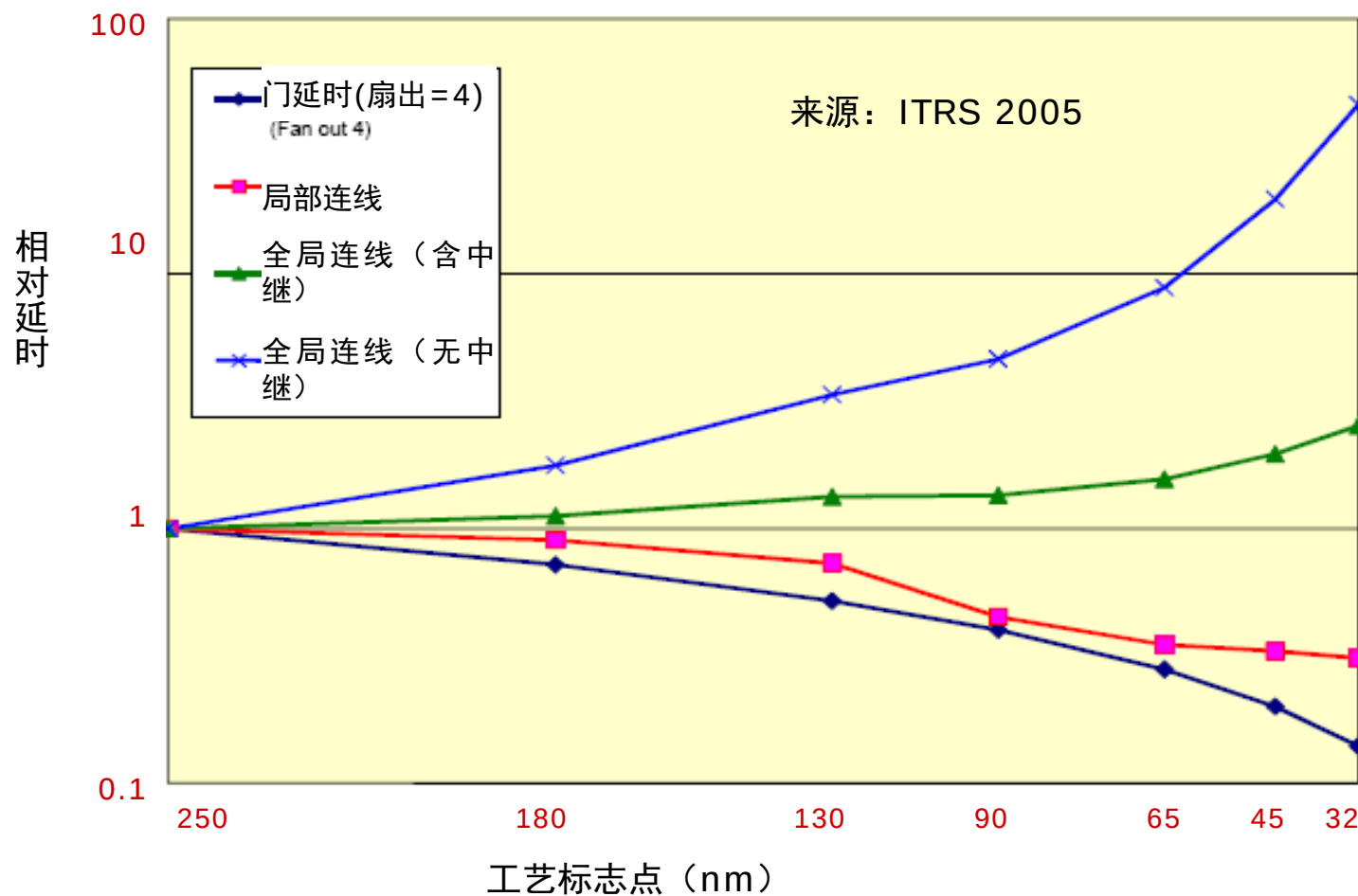
延时与导线长度的平方成正比

6.2.3

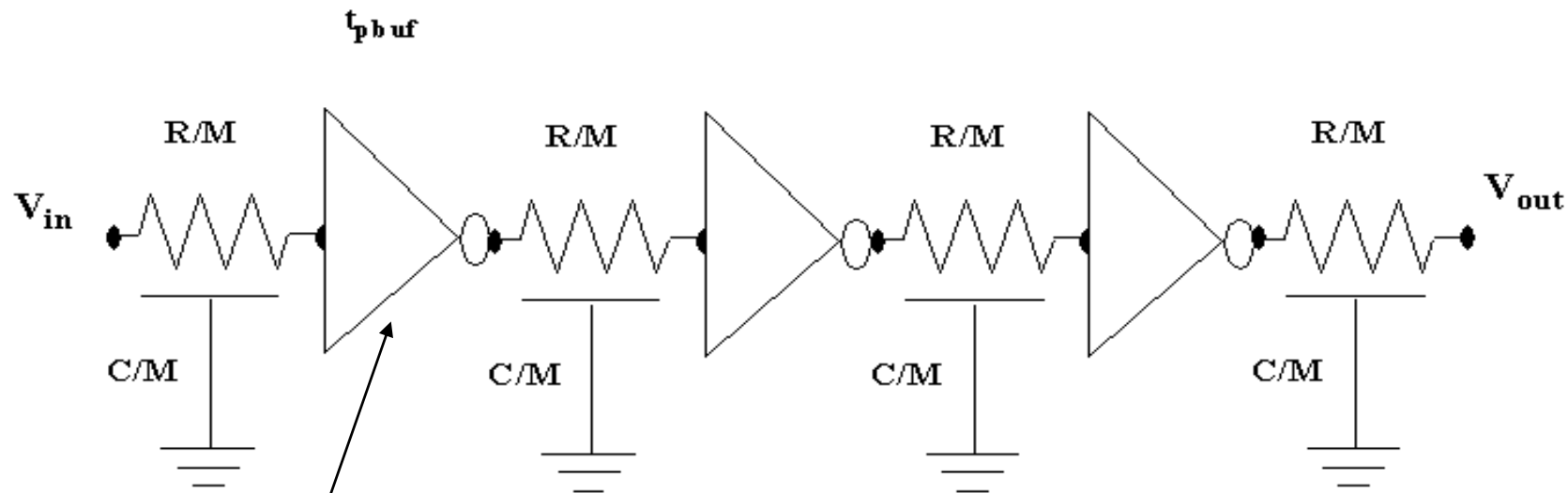
互连线延时的优化

互连问题

互连线延时占主要成份



优化方法1——插入中继器



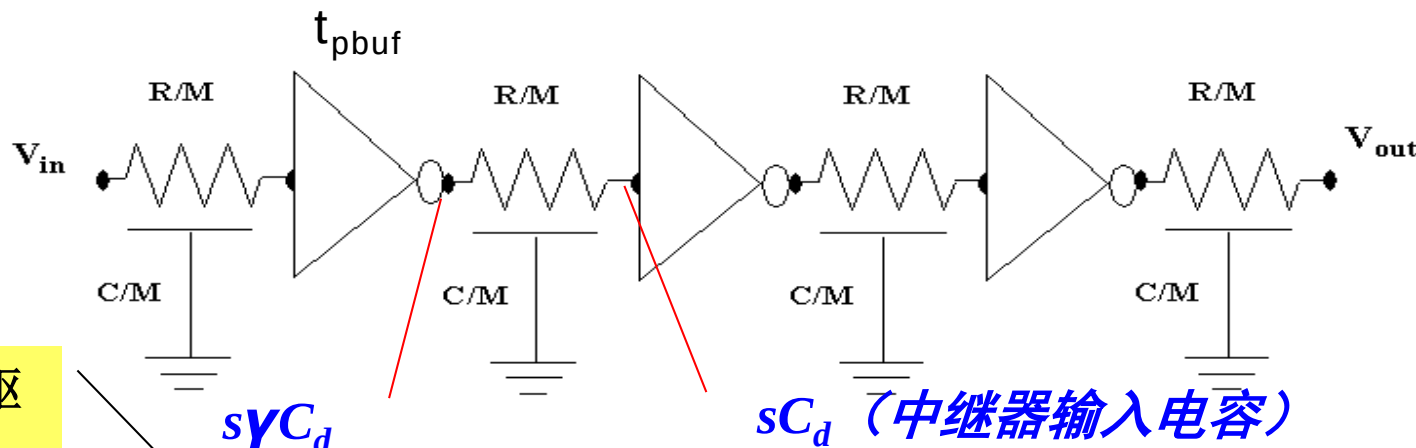
中继器
(Repeater)

$$t_p = 0.38M \left(\frac{L}{M} \right)^2 rc + Mt_{pbuf}$$

$$\Rightarrow M = L \sqrt{\frac{0.38rc}{t_{pbuf}}}$$

$$t_{p,opt} = 2\sqrt{t_{pwire(unbuffered)}t_{pbuf}}$$

插入中继器（考虑负载）



$$t_p = M \left(0.69 \frac{R_d}{s} (s\gamma C_d + \frac{cL}{M} + sC_d) + 0.69 \left(\frac{rL}{M} \right) (sC_d) + 0.38rc \left(\frac{L}{M} \right)^2 \right)$$

$$M_{opt} = L \sqrt{\frac{0.38rc}{0.69R_d C_d (\gamma + 1)}} = \sqrt{\frac{t_{pwire(unbuffered)}}{t_{p1}}}$$

$$s_{opt} = \sqrt{\frac{R_d c}{r C_d}}$$

$$t_{p,min} = (1.38 + 1.02\sqrt{\gamma + 1}) L \sqrt{R_d C_d rc} \quad (\text{线性})$$

导线临界长度

$$L_{crit} = \frac{L}{M_{opt}} = \sqrt{\frac{t_{p1}}{0.38rc}} \quad t_{p1}: \text{单位尺寸}(s=1) \text{ 中继器的传播延时}$$

(与互连线长度无关的常数)

插入中继器后的导线延时

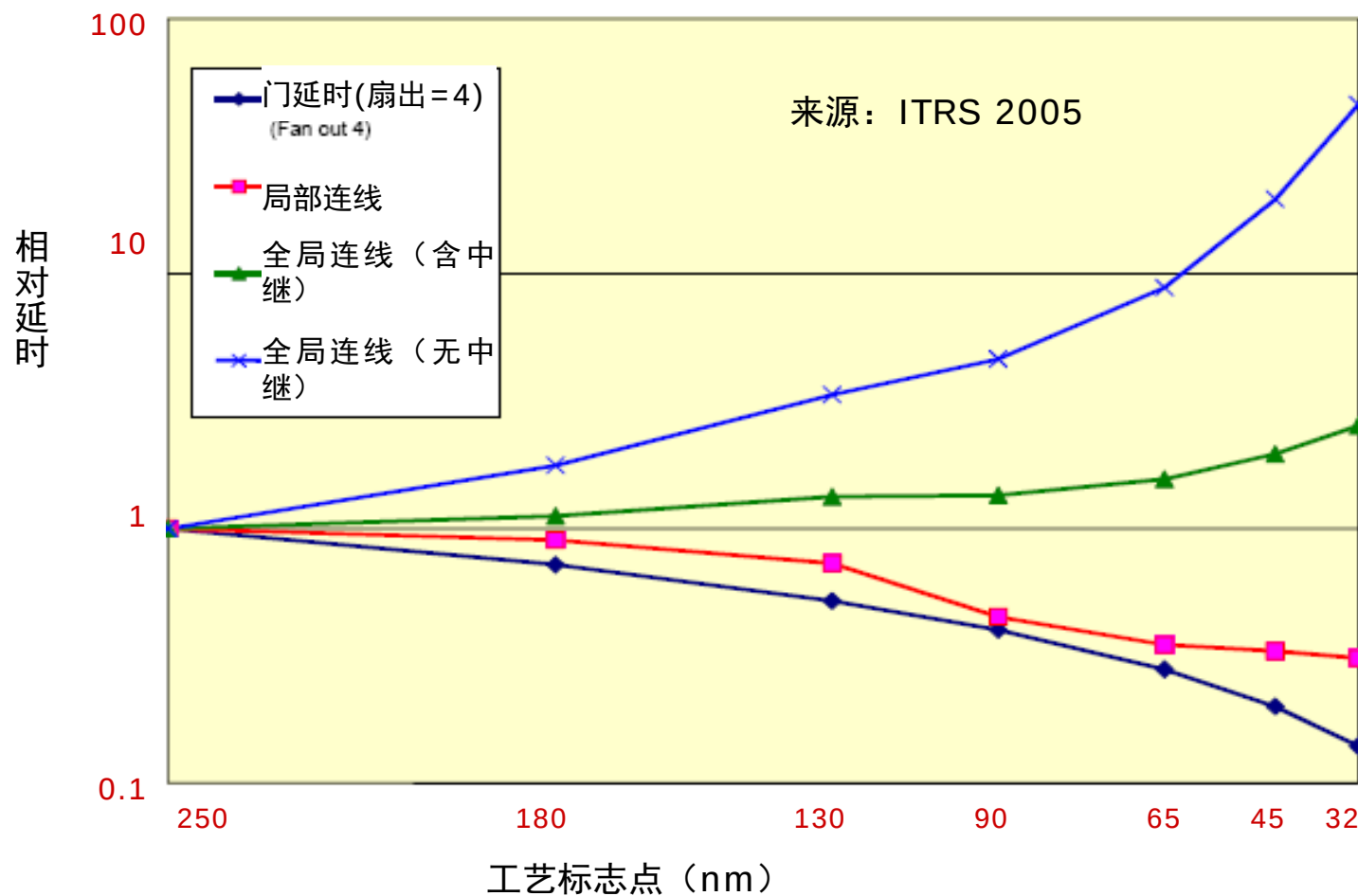
$$t_{p, crit} = \frac{t_{\dot{p}, min}}{m_{opt}} = 2 \left(1 + \sqrt{\frac{0.69}{0.38(1 + \gamma)}} \right) t_{p1}$$

导线中继的规律

- 对于给定的工艺和互连层，存在着中继器间导线段的最优长度，这段导线的延时和布线层无关!
- 插入中继器只有在导线长度超过 L_{crit} 两倍的时候才有意义。

互连问题

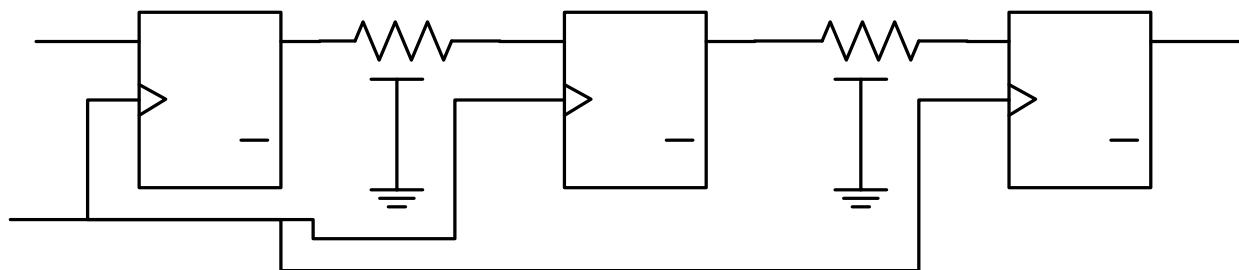
互连线延时占主要成份



方法2：导线流水线

❧ 互连结构优化

❧ 把一条长导线分成k段，每段导线的延时都小于时钟周期。



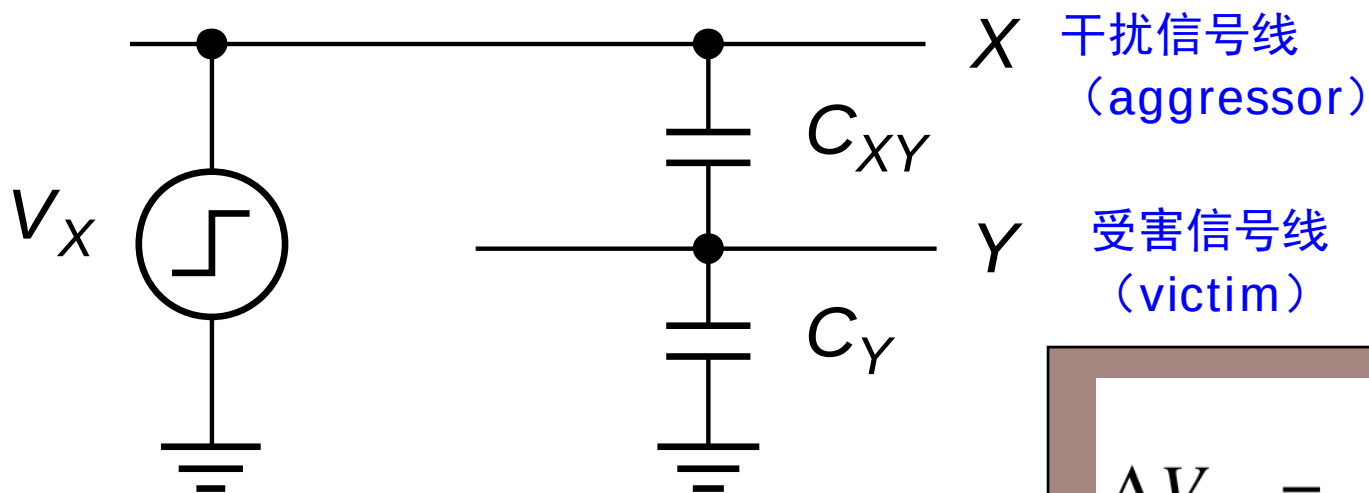
6.3 互连线的信号完整性问题

6.3.1

电容串扰

电容串扰（Cross Talk）

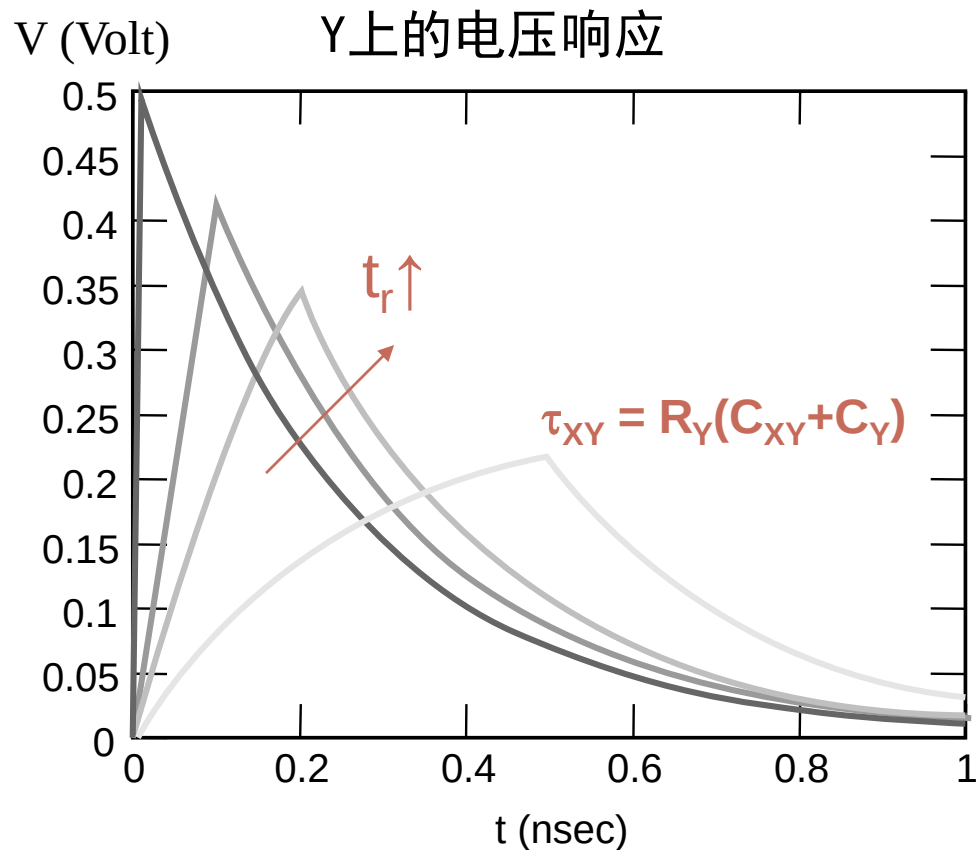
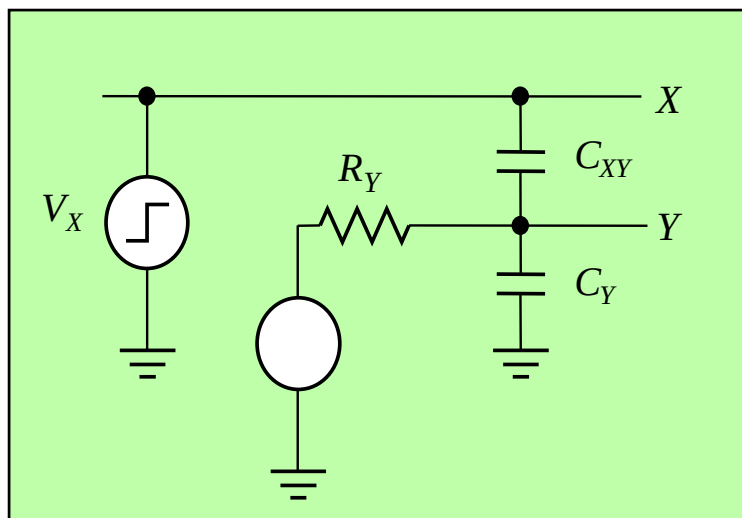
浮空线情况：



$$\Delta V_Y = \frac{C_{XY}}{C_Y + C_{XY}} \Delta V_X$$

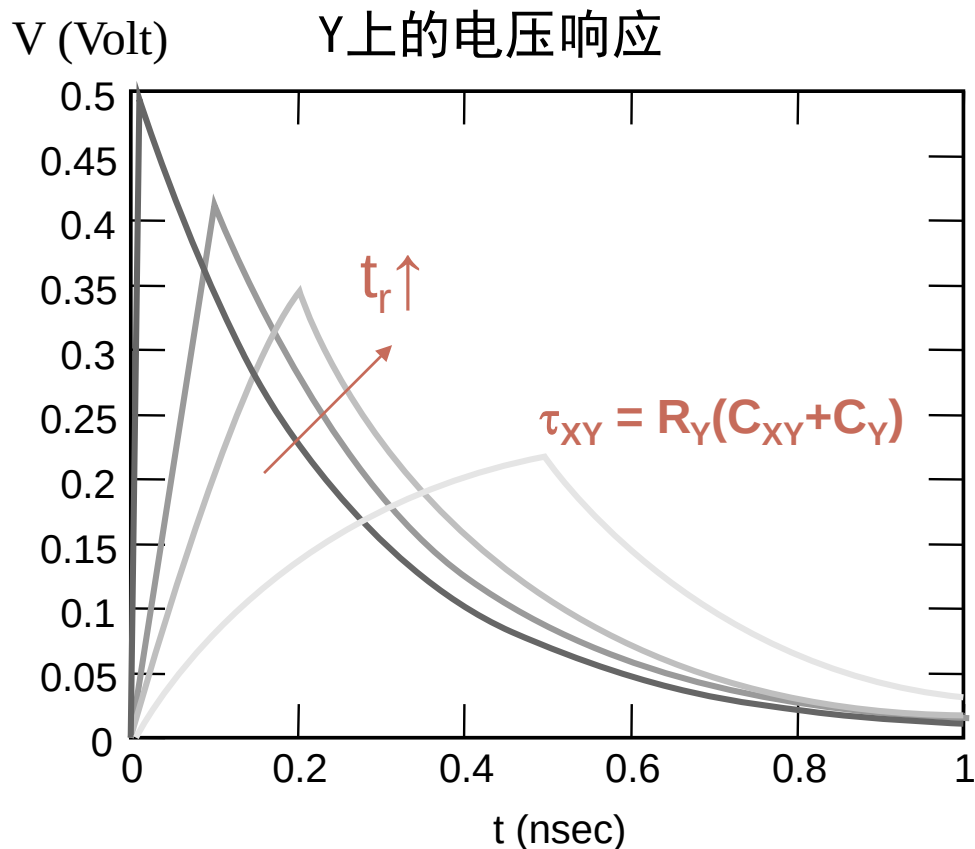
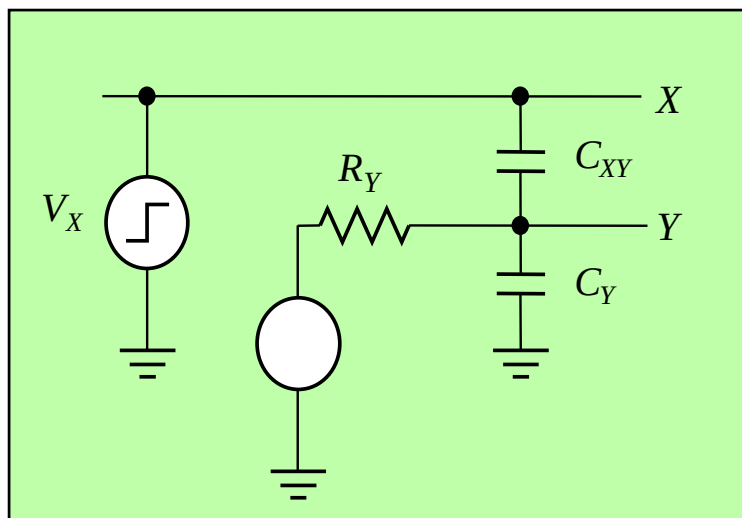
干扰噪声的幅度与耦合电容的大小和干扰信号的幅度有关。

被驱动线的串扰



上升时间越大，受害线产生的干扰峰值越小。

被驱动线的串扰

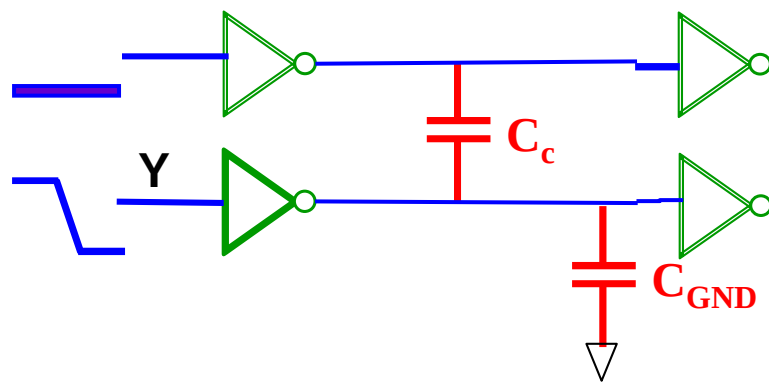


R_Y 的大小影响Y节点的放电时间常数，在同样的干扰信号上升时间下，放电越快，在Y上的噪声脉冲幅度越低。

保持导线的驱动阻抗较低（特别是使得 $\tau_{xy} < t_r$ ）对降低电容串扰的影响很有帮助。

电容串扰对传播延时的影响

相邻导线不翻转

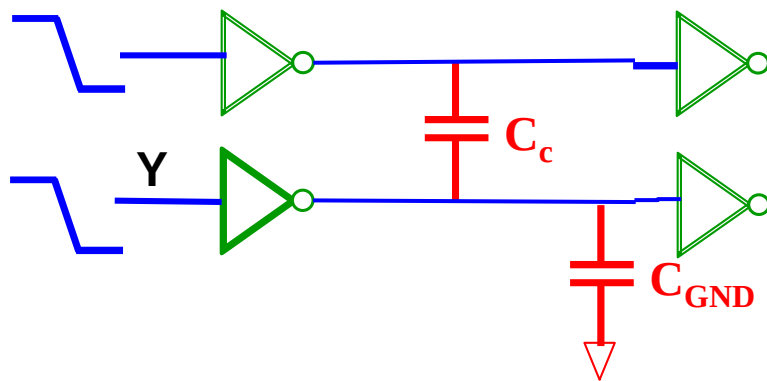


C_c 是对地电容

$$C_L = C_{GND} + C_C$$

电容串扰对传播延时的影响

相邻导线同向翻转



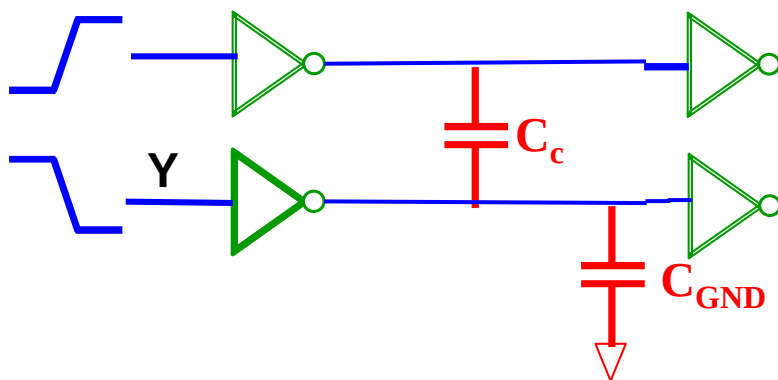
C_c 上的电压不变——
不存在充放电。

$$t_p = \frac{\Delta Q}{I} \triangleq C_L \cdot \frac{\Delta V}{I} \implies C_{eq} \triangleq \Delta Q / \Delta V \quad \text{等效电容等于0}$$

$$C_L = C_{GND}$$

电容串扰对传播延时的影响

相邻导线反向翻转



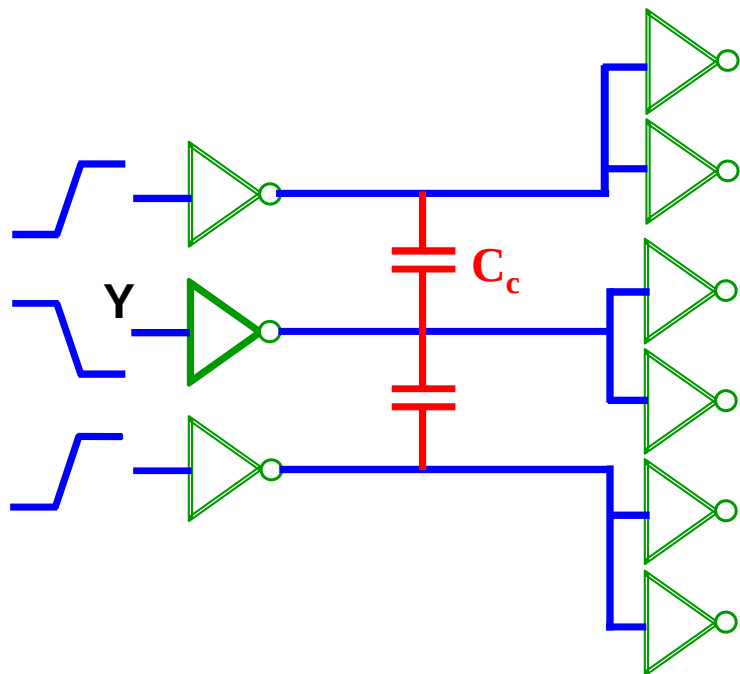
C_c 上的电压变化等于 $2V_{DD}$ 。

密勒效应: $\Delta Q = C_c \cdot 2V_{DD} \implies C_{eq} = C_c \cdot \frac{2V_{DD}}{V_{DD}} = 2C_c$

$$C_L = C_{GND} + 2C_c$$

当与受害线相邻的导线向相反方向变化时延时增加更多!

电容串扰对传播延时的影响



延时与相邻导线的翻转方向有关。

$$C_{GND} \leq C_L \leq C_{GND} + 4C_C$$

总线延时与相邻导线翻转的关系

N位总线，N条平行导线

$$t_{p,k} = g C_w (0.38 R_w + 0.69 R_D)$$

bit $k-1$	bit k	bit $k+1$	Delay factor g
↑	↑	↑	1
↑	↑	—	$1 + r$
↑	↑	↓	$1 + 2r$
—	↑	—	$1 + 2r$
—	↑	↓	$1 + 3r$
↓	↑	↓	$1 + 4r$

r =单位长度的线间电容/单位长度的导线对地电容

导线延时的估计

- ❗ 延时的实际值不仅取决于周围导线的耦合电容值，还取决于信号的翻转时序
- ❗ 噪声对延时的影响相差非常大——不可忽略
- ❗ 实际延时具有不可预见性，带来的验证成本大大增加
 - 逻辑路径的延时会影响时序电路的时序关系，时序关系错误会引起时序错误，最终导致电路的功能错误

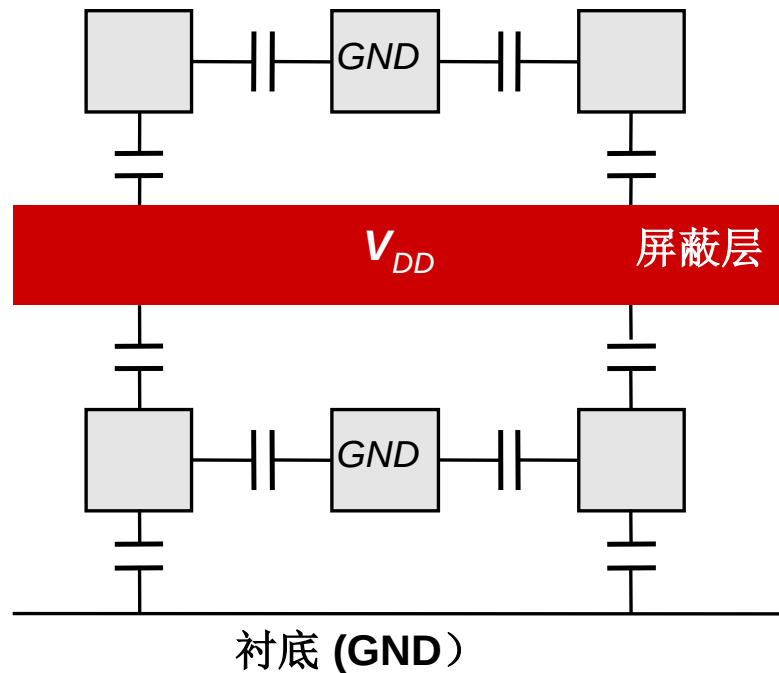
克服电容串扰的方法

1. 增大驱动强度，尽量**避免浮空节点**
2. 把敏感节点（动态节点）远离全摆幅信号（隔离）
3. 使得信号的上升和下降时间尽可能长
4. **不要使两条信号线平行布置很长的距离**

克服电容串扰的方法

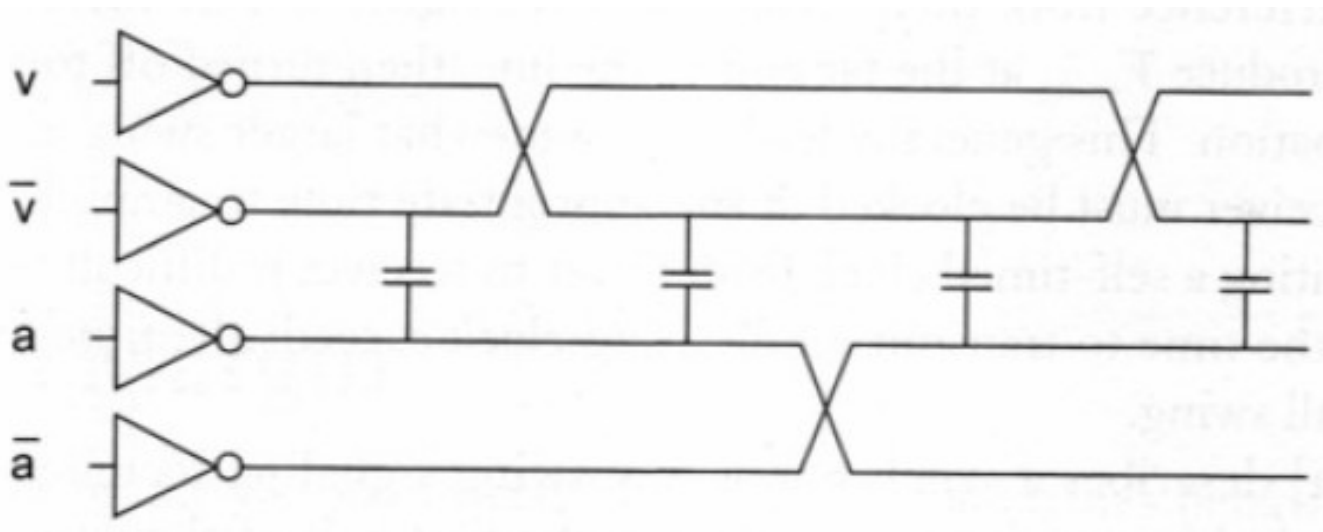
5. 对于敏感的信号线增加屏蔽

- 在两条信号线间插入电源或地线；
- 不同层次的信号之间用一个GND或 V_{DD} 金属平面相隔



克服电容串扰的方法

6. 把在不同时间翻转的总线相间放置，使得两两相邻的导线不会同时翻转
7. 采用差分信号，并将差分信号线“绞合”——使串扰信号变为共模噪声。



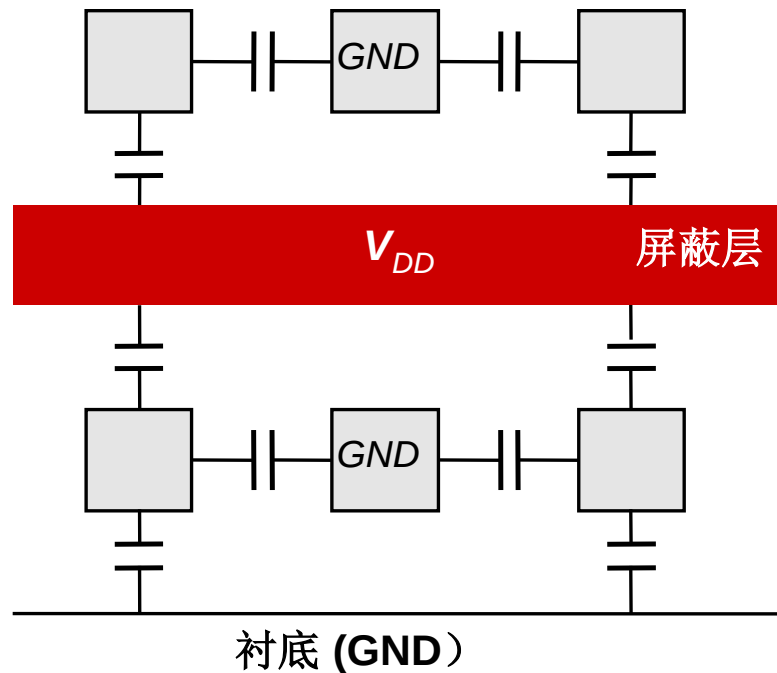
克服电容串扰的方法

1. 增大驱动强度，尽量**避免浮空节点**
2. 把敏感节点（动态节点）远离全摆幅信号（隔离）
3. 使得信号的上升和下降时间尽可能长
4. **不要使两条信号线平行布置很长的距离**

克服电容串扰的方法

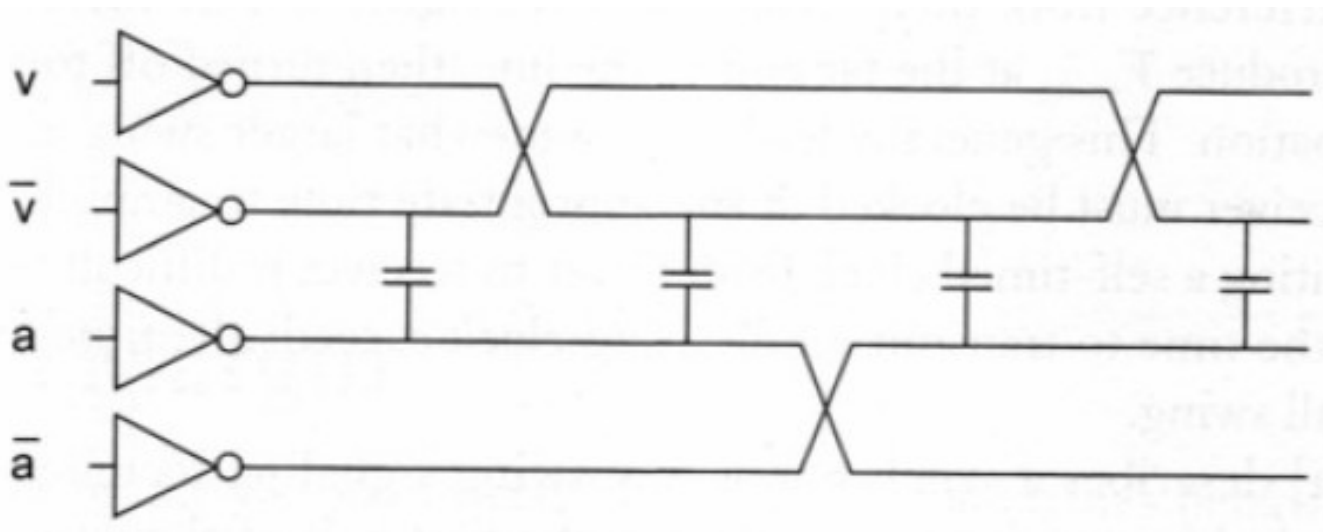
5. 对于敏感的信号线增加屏蔽

- 在两条信号线间插入电源或地线；
- 不同层次的信号之间用一个GND或 V_{DD} 金属平面相隔



克服电容串扰的方法

6. 把在不同时间翻转的总线相间放置，使得两两相邻的导线不会同时翻转
7. 采用差分信号，并将差分信号线“绞合”——使串扰信号变为共模噪声。



6.3.2

IR DROP

电阻引入的噪声（IR drop）

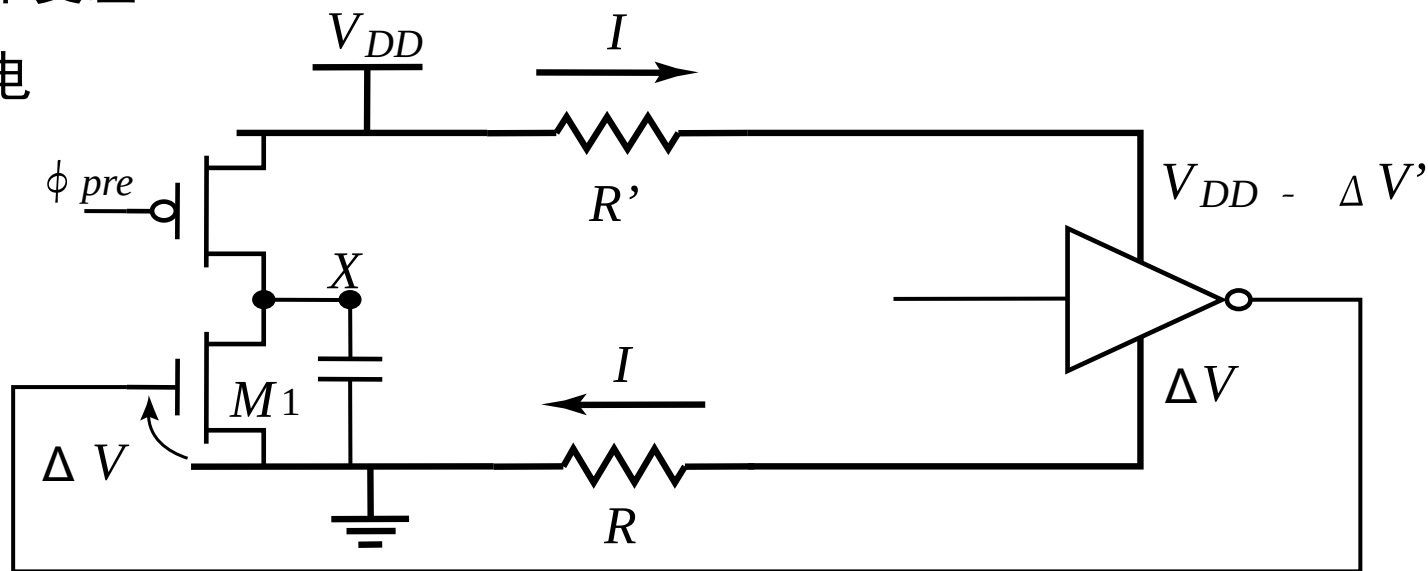
- 电源线寄生电阻的存在会造成远离电源端的电路的供电电压降低——距离越远压降越大

- V_{DD} 下降会减小噪声容限

- 片内电压存在差异

- 逻辑电平变差

- 引起漏电



IR drop的动态性

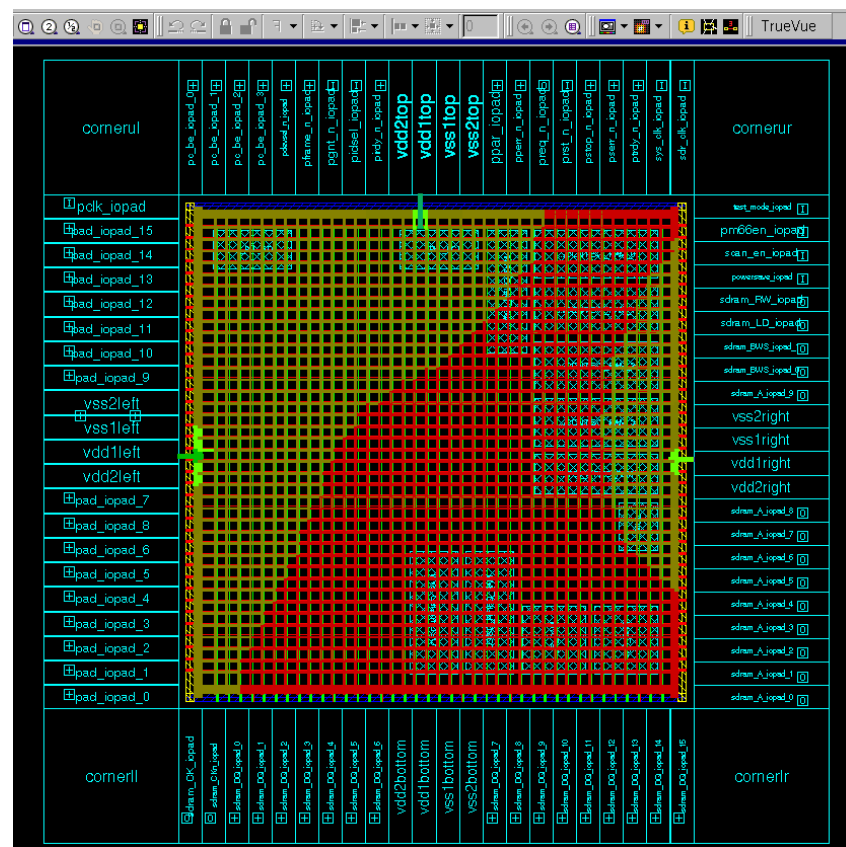
电源电流的波动会引起电源电压的波动

- 最大的IR drop与峰值电流有关，峰值电流与电路的开关活动有关——IR drop是一种动态现象
- 引起延时抖动

降低IR drop的方法

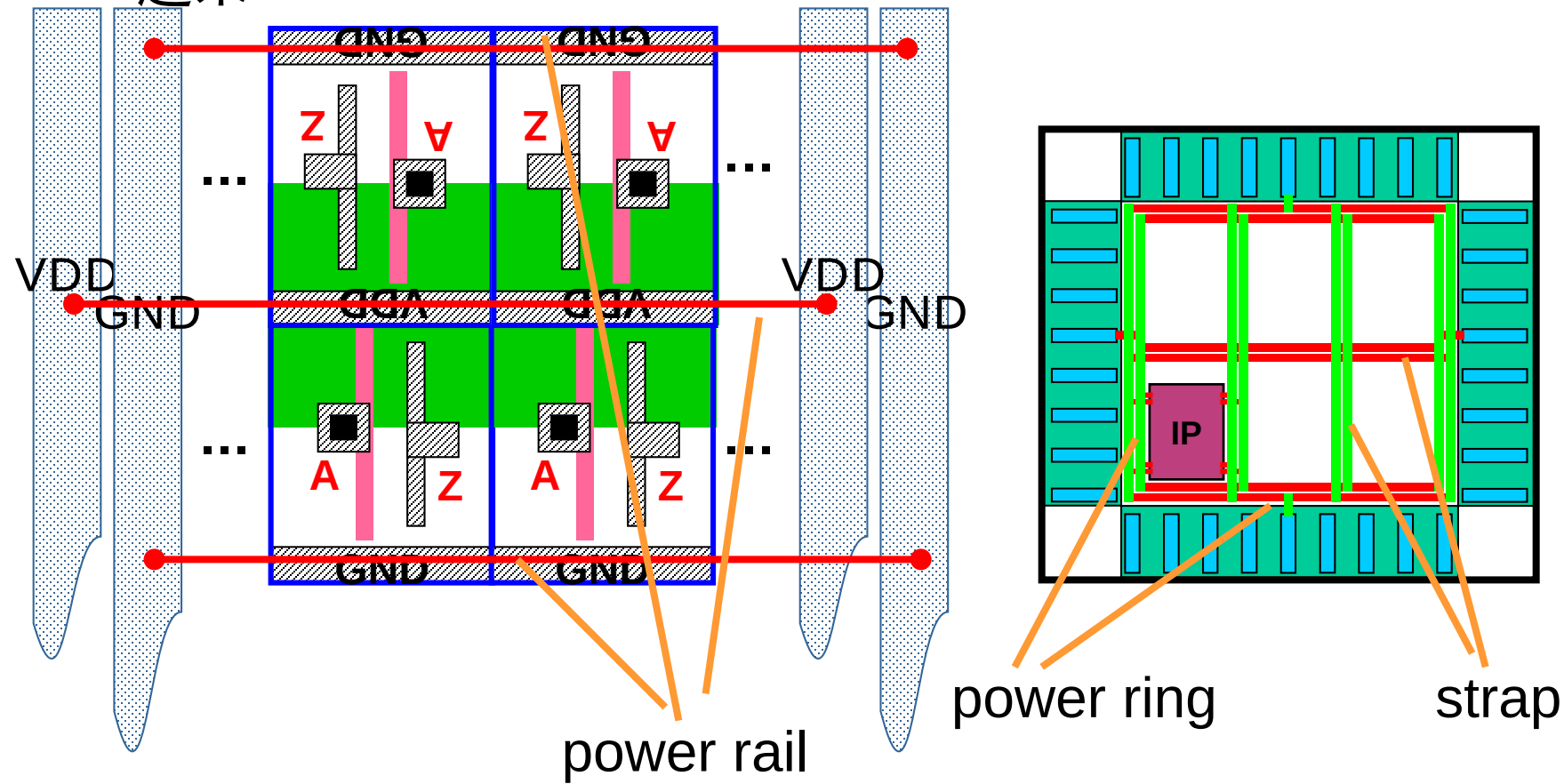
优化电源网络，减小电阻：

- 采用网络结构进行电源分布



电源网格

在Metal 1布置低层电源，在高层金属层把电源线“捆绑”起来



降低IR drop的方法

减小导线电阻：

- 保证电源线宽度——与流过的电流相适应
- 电源捆绑时要用多接触孔

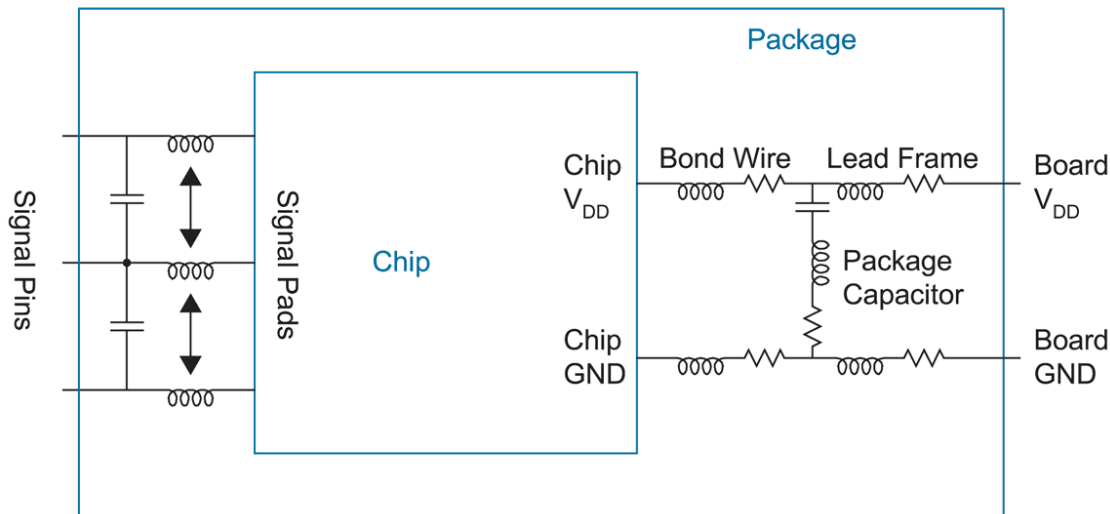
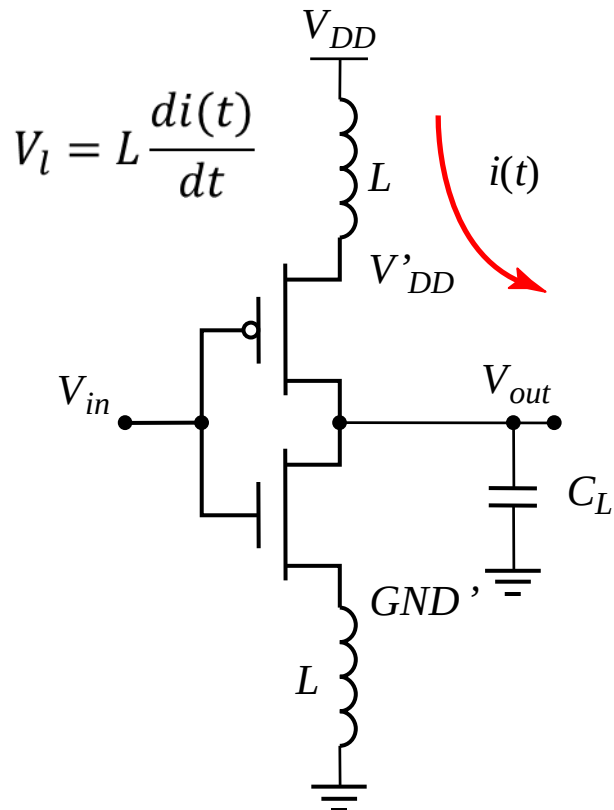
减小峰值电流

- 控制动态功耗
- 采用异步电路——分散电路开关的时间
- 控制输入输出模块的功耗

6.3.3

$L(di / dt)$

L di/dt



寄生电感的一个重要来源是压焊线和芯片封装引起的。可达到nH量级。导线越长寄生电感越大。

寄生电感对电源电压和地线的影响:

- 电流变化导致电压变化

电压变化的大小与电路切换的速度和电流的大小有关。

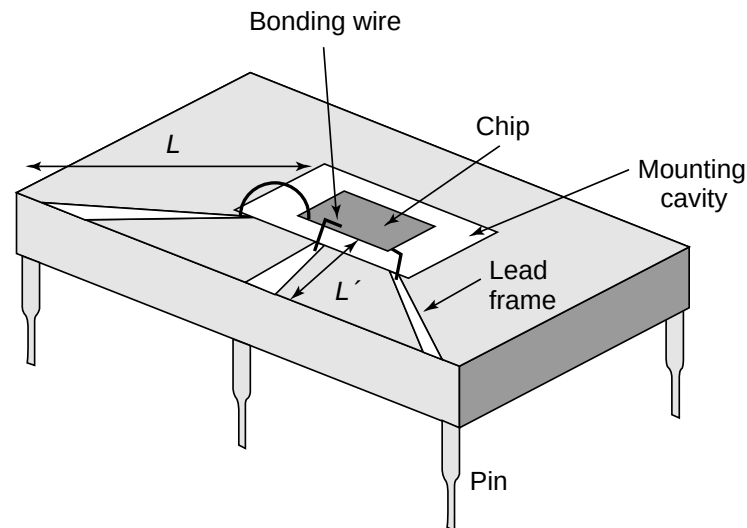
解决 Ldi/dt 问题的方法

- ❗ 把I/O pad和芯片内核电路的电源引脚分开。
- ❗ 增加片外信号的上升和下降时间到所允许的最大值。
 - 最好的驱动器是在输出端具有最大允许的上升和下降时间，并满足所规定的延时
- ❗ 安排好消耗大电流的翻转使它们不会同时发生。
 - 同步电路的电路翻转比较集中，大部分在时钟沿处
 - 采用异步电路可以分散电路翻转的时间，降低最大电流

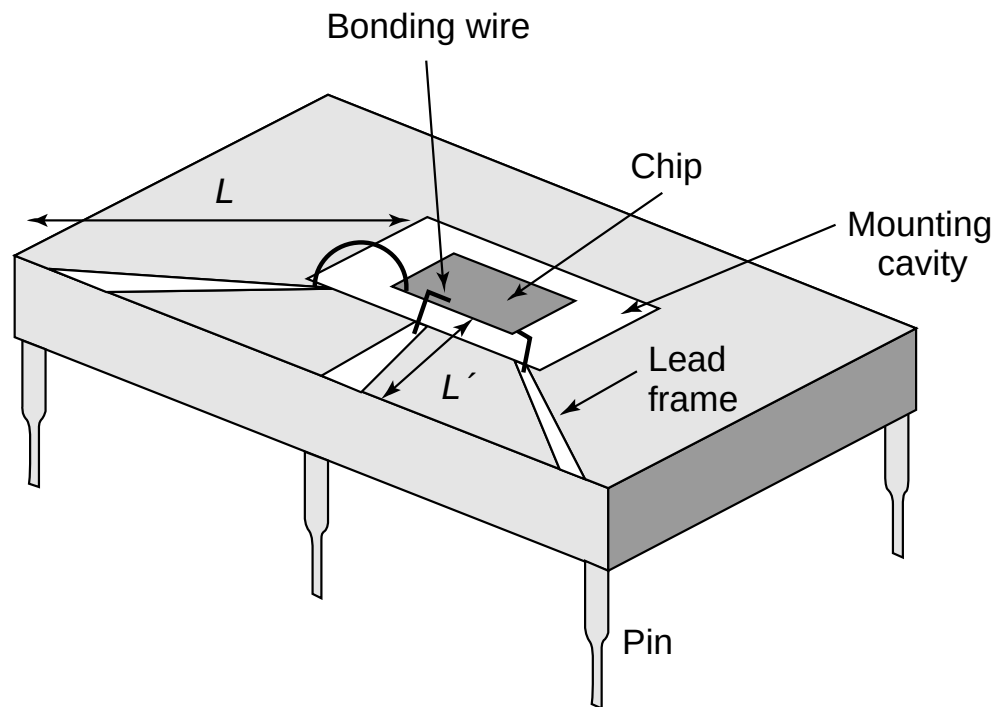
解决 Ldi/dt 问题的方法

减小寄生电感

- 采用多组电源地引脚（多个寄生电感并联）。
- 仔细选择封装中电源地引脚的位置。



解决 Ldi/dt 问题的方法

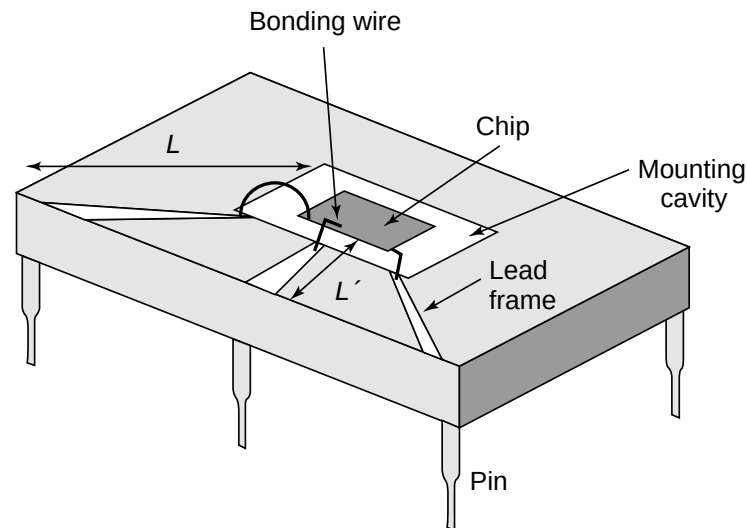


位于封装四角处的导线和压焊线的电感明显较大，不宜用做电源地引脚。

解决Ldi/dt问题的方法

减小寄生电感

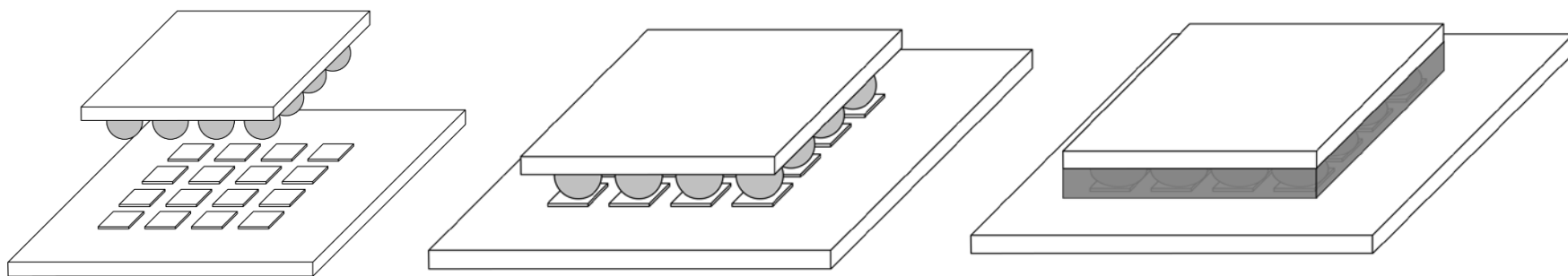
- 采用多组电源地引脚（多个寄生电感并联）。
- 仔细选择封装中电源地引脚的位置。
 - 位于封装四角处的导线和压焊线的电感明显较大，不宜用做电源地引脚。
- 使用先进的封装技术，如倒装焊、焊球



先进封装

❧ 倒装焊 (flip-chip)

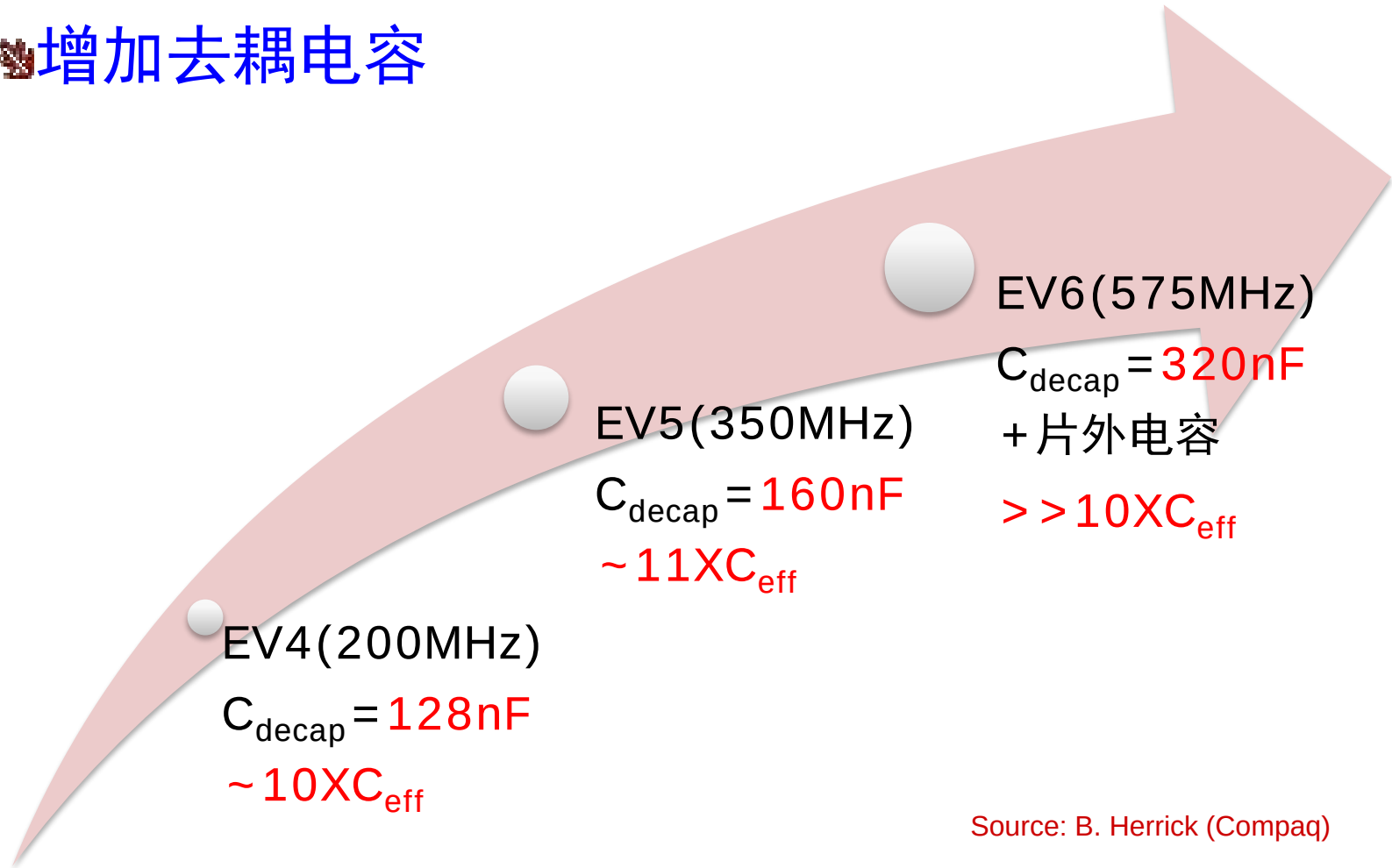
❧ 球阵列 (Ball Grid Array, BGA) 封装



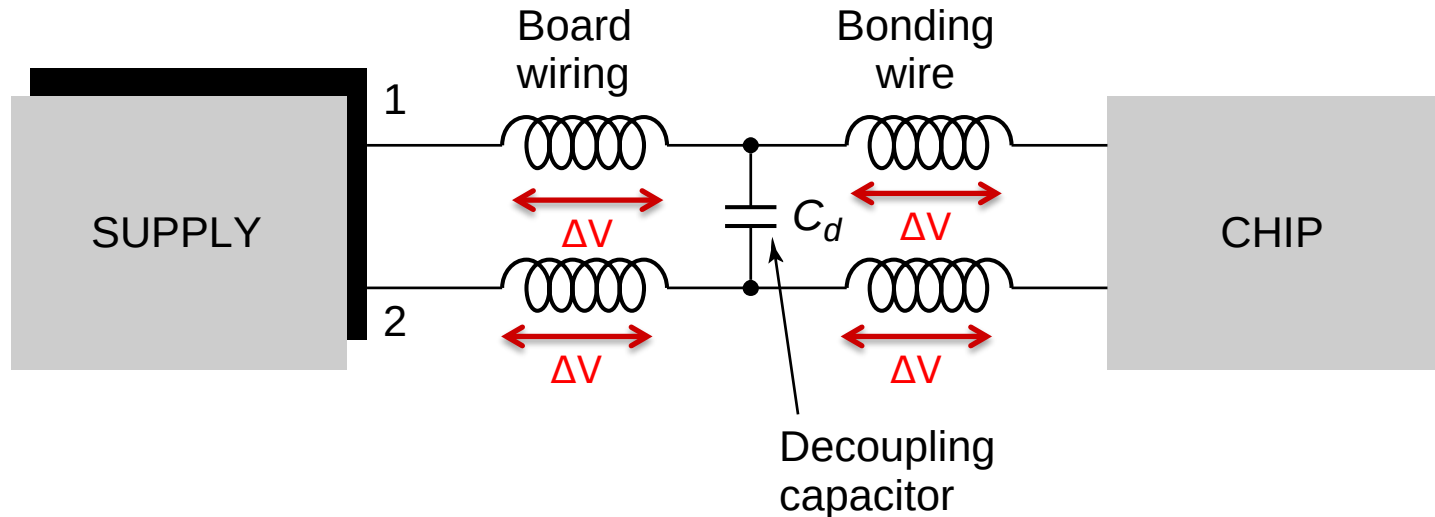
图片来源: wikipedia

解决Ldi/dt问题的方法

增加去耦电容



去耦电容的放置方法



去耦电容的位置: **就近放置!**

- 电路板上: 放在电源引脚处
- 芯片内: 放置在电源线下, 靠近大的驱动器

6.3.4

小结

互连线的信号完整性问题小结

线间电容引起串扰问题

- 影响因素：干扰信号幅度、翻转速度、受害信号线的驱动电阻、线间电容大小
- 影响：信号噪声、延时不确定（与数据相关）
- 解决方法：减小信号幅度、隔离、屏蔽、避免存在高阻节点、差分信号、补偿和抵消技术（如绞合走线）

寄生电阻引起电源压降问题

- 影响因素：寄生电阻、电源电流
- 影响：电源压降（动态）、漏电
- 解决方法：电源地线用网络分布，借助IR drop分析工具，控制峰值电流

互连线的信号完整性问题小结（2）

寄生电感（特别是电源地引线电感）引起电源波动问题

- 影响因素：电流大小，逻辑门的翻转速度
- 影响：电源波动
- 解决办法：多个电源引线（一个电源线5-10个驱动器）、内核与IO驱动器电源分离、缩短封装引线长度、去耦电容、控制驱动器的翻转速度和峰值电流

总结（续）

互连线问题的考虑

- 信号线和时钟线主要考虑串扰问题
- 电源线主要考虑IR drop和 $L \, di/dt$ 问题
- 为了减小时钟线的延时偏差要考虑串扰问题，时钟驱动器要注意电源电压的波动和IR drop问题
- 全局互连（长导线）主要考虑延时问题
- 总线处易发生同时翻转的信号线平行走线的情况，要特别注意串扰问题
- 动态节点和高阻节点、模拟信号线对串扰特别敏感

总结（续）

互连线工程

- 时钟信号线要适当增加与周围导线的间距，高速电路中要采用屏蔽
- 总线克服串扰的方法：
 - 电源/地线屏蔽
 - 把不同时间的翻转的两组总线相间放置
 - 绞合布线等方法抵消串扰
- 长导线要插入中继电器，长距离传输还可以采用互连线流水线

总结（续）

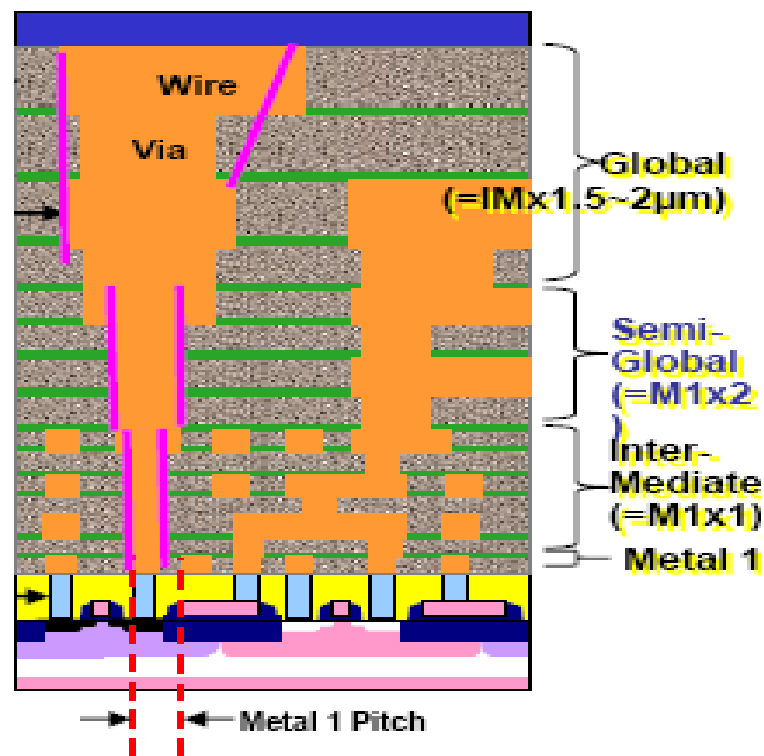
- ❖ 片内、片外和电路板上要放置必要的去耦电容，抑制电源线噪声
- ❖ 避免出现动态节点和浮空导线

6.4

互连线的scaling down

互连线密度的提升

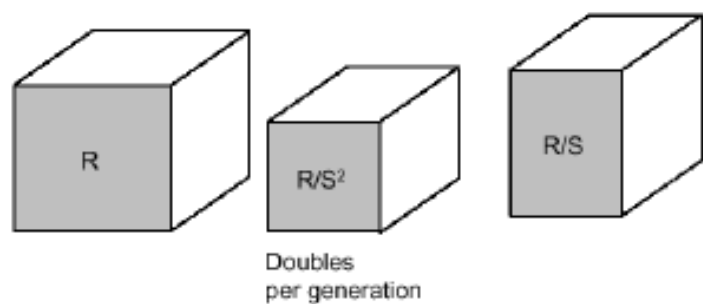
可以加工的最小的互连线节距（pitch）也与工艺的特征尺寸有关。



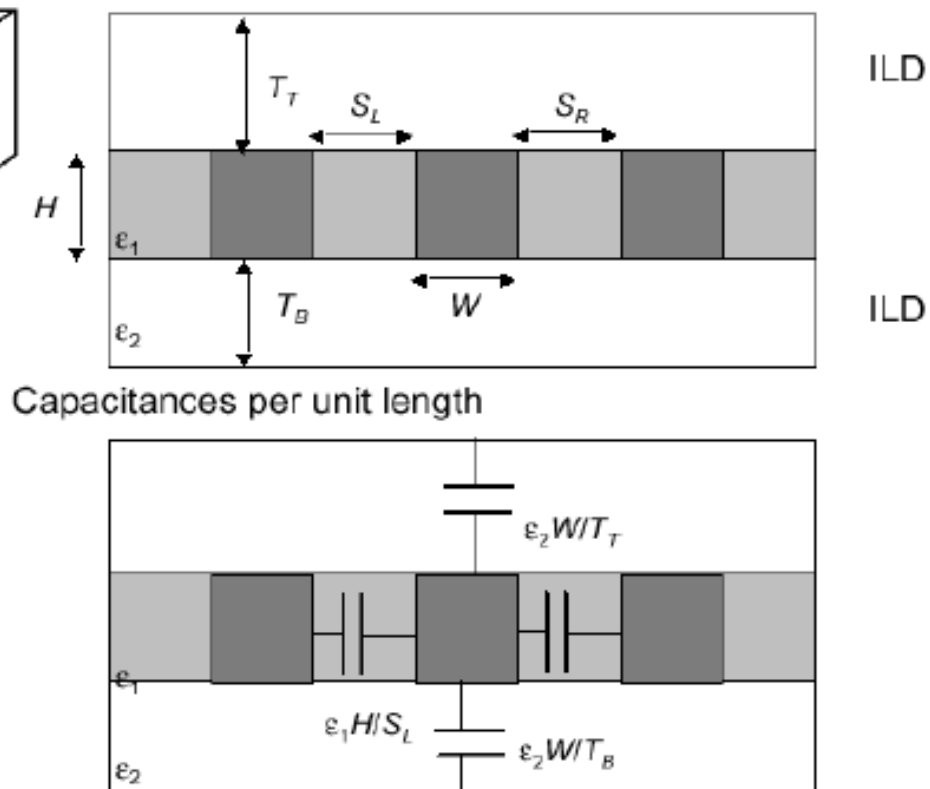
Pitch=宽度+间隔

互连参数与尺度的关系

互连线的电阻



互连线的电容

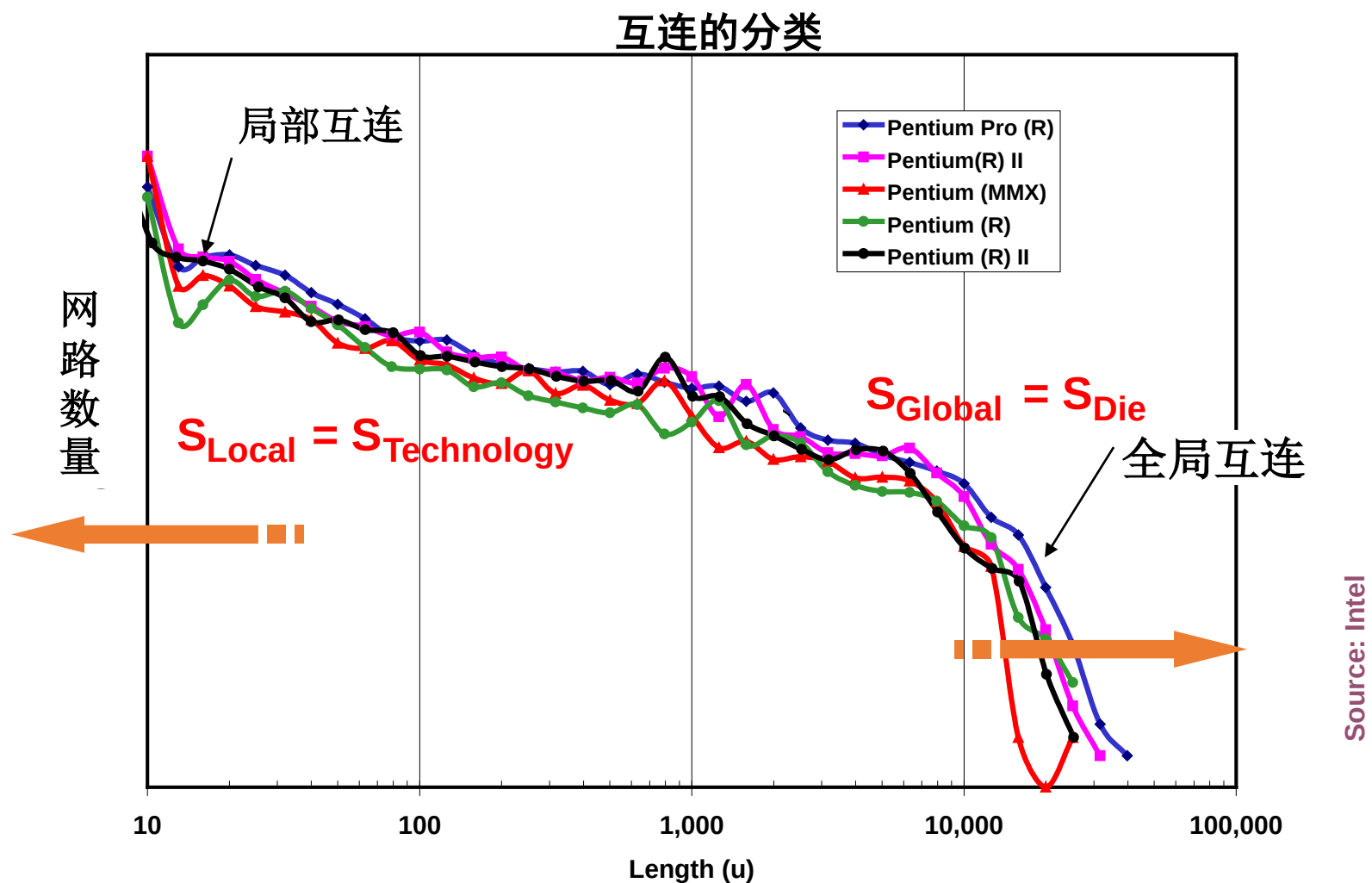


互连线的理想尺寸缩小

- 导线宽度和厚度、介质层厚度按同一比例 ($S > 1$) 缩小

参数	关系式	变化
W, H, t	/	$1/S$
单位长度电阻 R	$1/WH$	S^2 延时增大
单位长度底板电容 C	W/t	1
单位长度侧壁电容 C_c	H/t	1

Scaling down中的互连线长度



互连线的恒电阻尺寸缩小

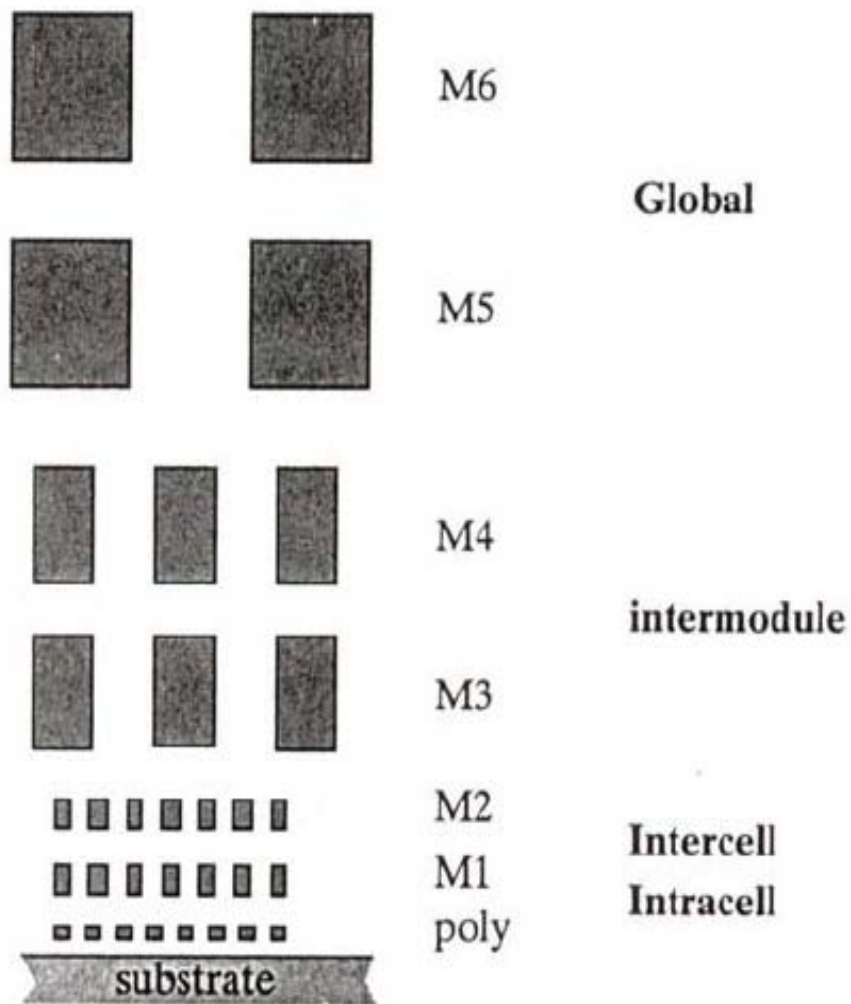
- 恒电阻尺寸缩小：导线宽度（ W ）和节距（ t ）按比例缩小时，导线厚度（ H ）保持不变。
- 影响：使性能得到改善，但使边缘和线间电容（串扰）增加

参数	关系式	变化	
W, t	/	$1/S$ （ H 不变）	
单位长度电阻 R	$1/WH$	S	随着导线长度的缩短，总电阻不变
单位长度侧壁电容 C_c	H/t	S	串扰增加

互连工艺改进策略

- 采用分层互连线：
 - 全局连线：保持对电阻的控制
 - 局部连线：集成密度和低电容是关键
- 短线应随特征尺寸一起缩小
- 长导线采用宽厚导线

集成电路中的布线层规划



- 宽厚和间距大的连线布置在高层
 - **M5**及以上采用较厚连线:
 - 全局连线、电源线
- 密集和较薄的连线布置在低层
 - **M1**、**M2**、**M4**、(**M5**): 功能块连线
 - **M1**: 局部(单元内)连线

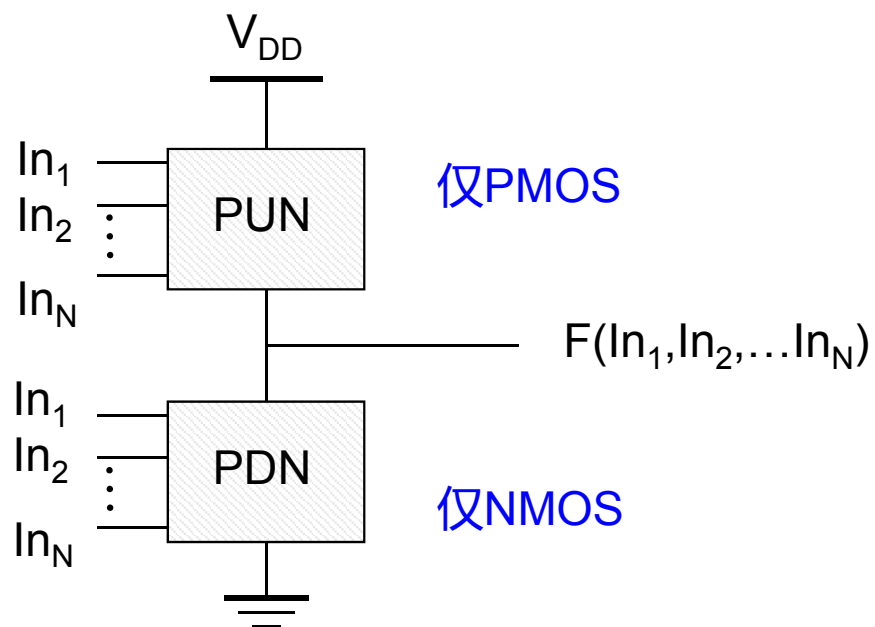
第七章 组合逻辑门的设计 (上)

组合逻辑

- 定义：组合逻辑——在任何时刻电路的输出与其当前输入信号间的关系服从某个布尔表达式（假设通过逻辑门的瞬态响应已经稳定），而不存在任何从输出返回至输入的连接。

7.1 静态互补CMOS逻辑的特点

静态互补CMOS逻辑门



PUN 和PDN 是互补的逻辑网络（导通条件相反）。

静态CMOS逻辑的特点

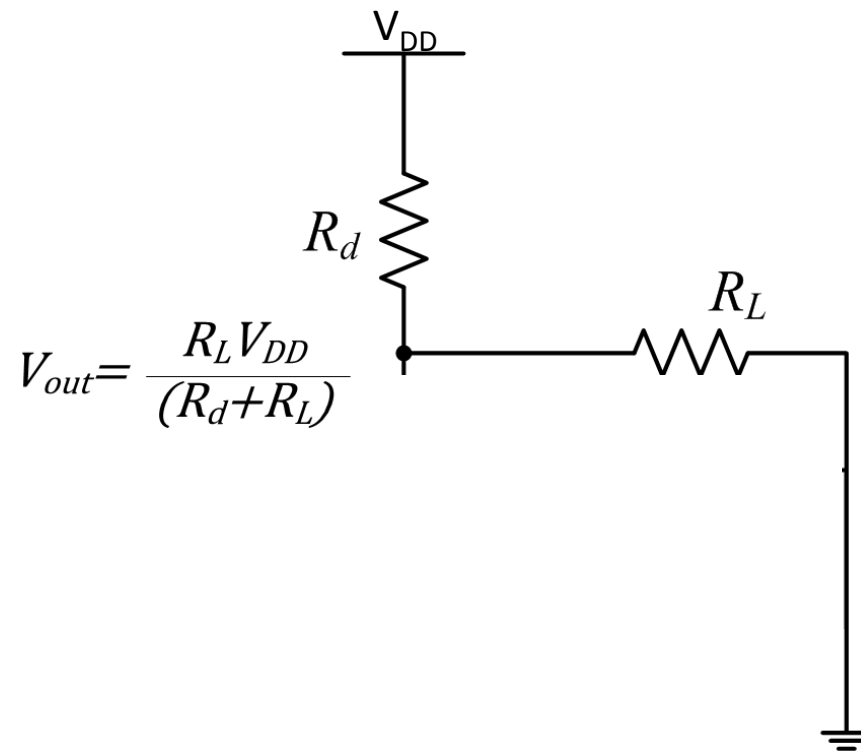
- 全摆幅; ——噪声容限高
- 无比逻辑: 逻辑电平与器件的相对尺寸无关; ——容易设计
- 静态逻辑: 稳态时总是存在输出到 V_{DD} 或者GND的通路, 输出阻抗低 (抗干扰能力强)

静态逻辑（静态电路）：每一时刻每个门的输出通过一个低阻路径连到电源或地上，同时任何时候该门的输出即为该电路实现的布尔函数值（忽略切换期间的瞬态效应）。

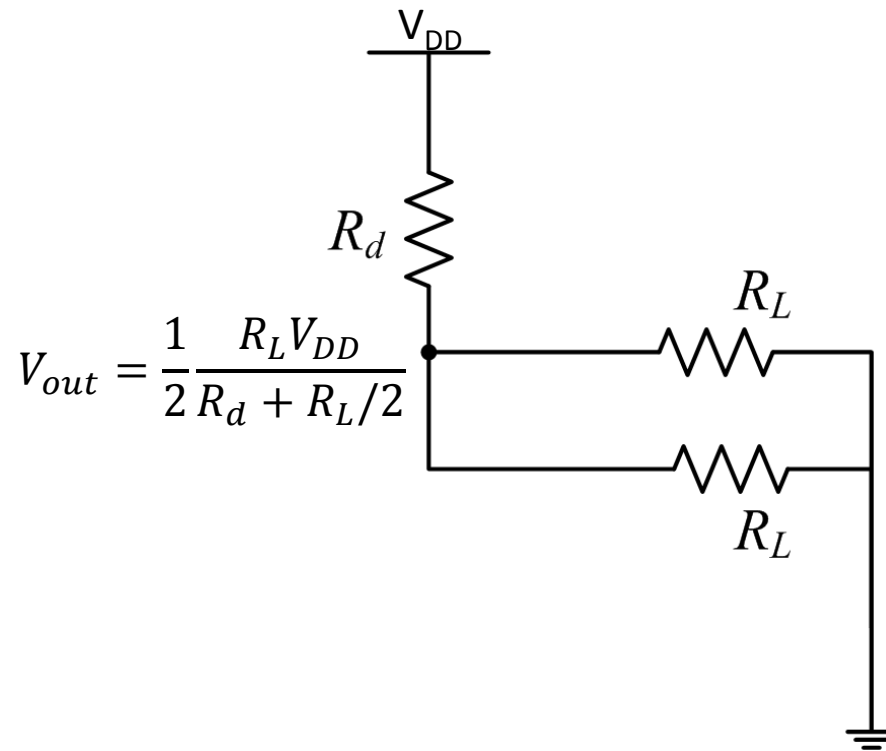
静态CMOS逻辑的特点

- 全摆幅; ——噪声容限高
- 无比逻辑: 逻辑电平与器件的相对尺寸无关; ——容易设计
- 静态逻辑: 稳态时总是存在输出到 V_{DD} 或者GND的通路, 输出阻抗低
- 无静态功耗 (不包括器件漏电流功耗)
- 具有可再生性
- 超高的输入阻抗; ——逻辑功能不受扇出的影响

输入阻抗的影响



输入阻抗的影响



如果输入阻抗较小，扇出会影响输出点电位。

静态CMOS逻辑的特点

- 全摆幅; ——噪声容限高
- 无比逻辑: 逻辑电平与器件的相对尺寸无关; ——容易设计
- 静态逻辑: 稳态时总是存在输出到 V_{DD} 或者GND的通路, 输出阻抗低
- 无静态功耗 (不包括器件漏电流功耗)
- 具有可再生性
- 超高的输入阻抗; ——逻辑功能不受扇出的影响
- VTC对工艺偏差不敏感

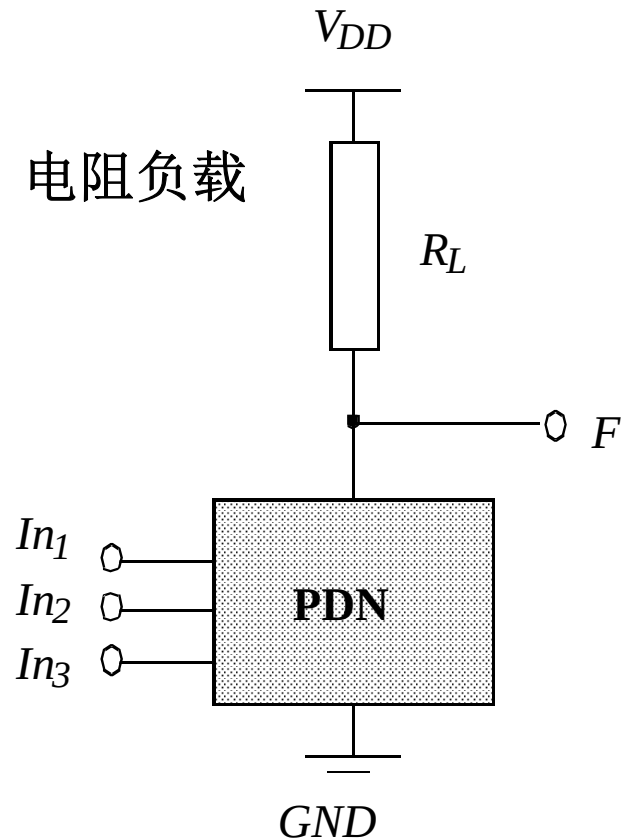
是应用最广的逻辑类型, 主流EDA工具都是面向静态CMOS逻辑设计的。

7.2 有比逻辑

7.2.1 伪NMOS逻辑门的静态特性

有比逻辑

简化一个开关网络，以减少实现所需晶体管数目：

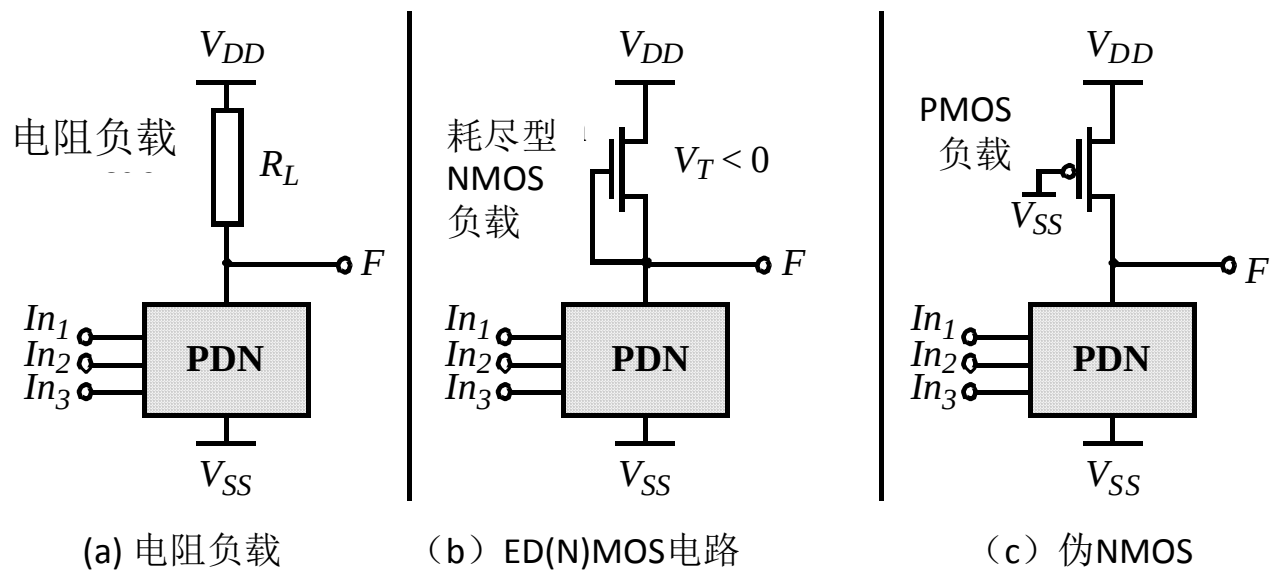


- N 个晶体管 + 负载

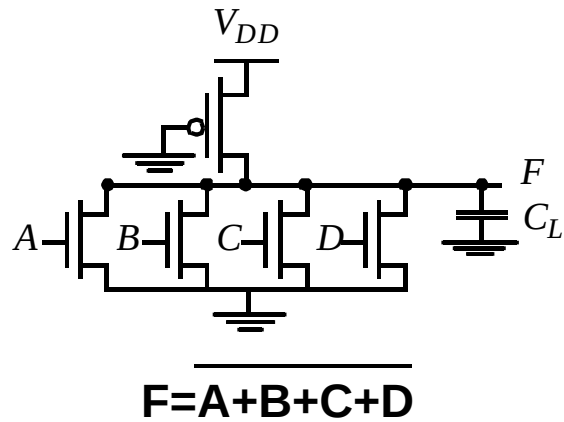
- $V_{OH} = V_{DD}$

- $V_{OL} = \frac{R_{PDN}}{R_{PDN} + R_L} V_{DD}$

有比逻辑门电路

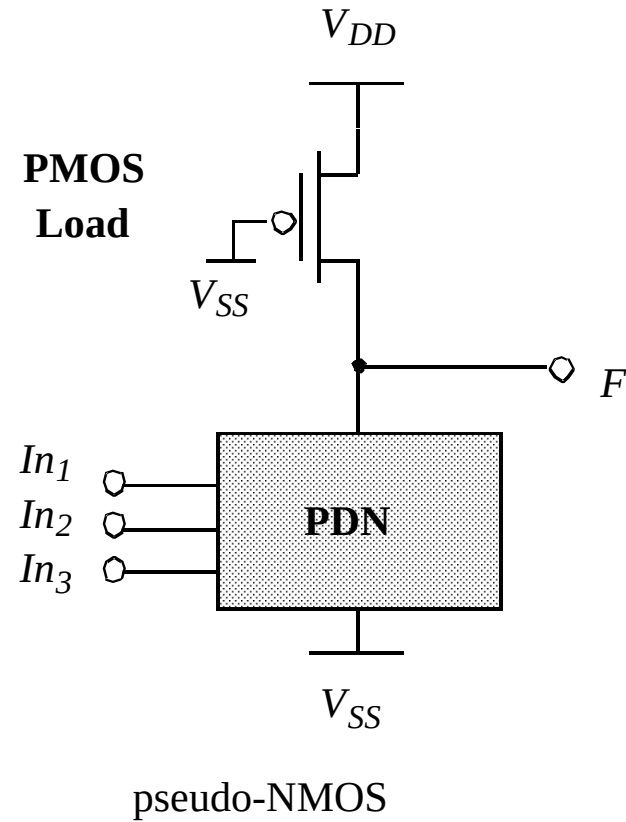


伪NMOS



- 晶体管数 **$N+1$**

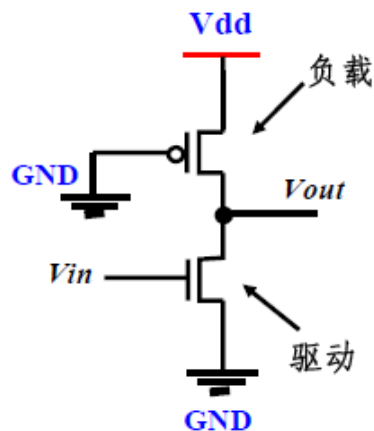
- 有比电路: V_{OL} 和晶体管尺寸有关



伪NMOS反相器的静态特性

对于短沟道器件

$$k_n \left((V_{DD} - V_{Tn}) V_{OL} - \frac{V_{OL}^2}{2} \right) + k_p \left((-V_{DD} - V_{Tp}) \cdot V_{DSATp} - \frac{V_{DSATp}^2}{2} \right) = 0$$



假设 V_{OL} 相对于驱动电压 $(V_{DD} - V_T)$ 很小，
而 V_{Tn} 与 V_{Tp} 在数值上相等，

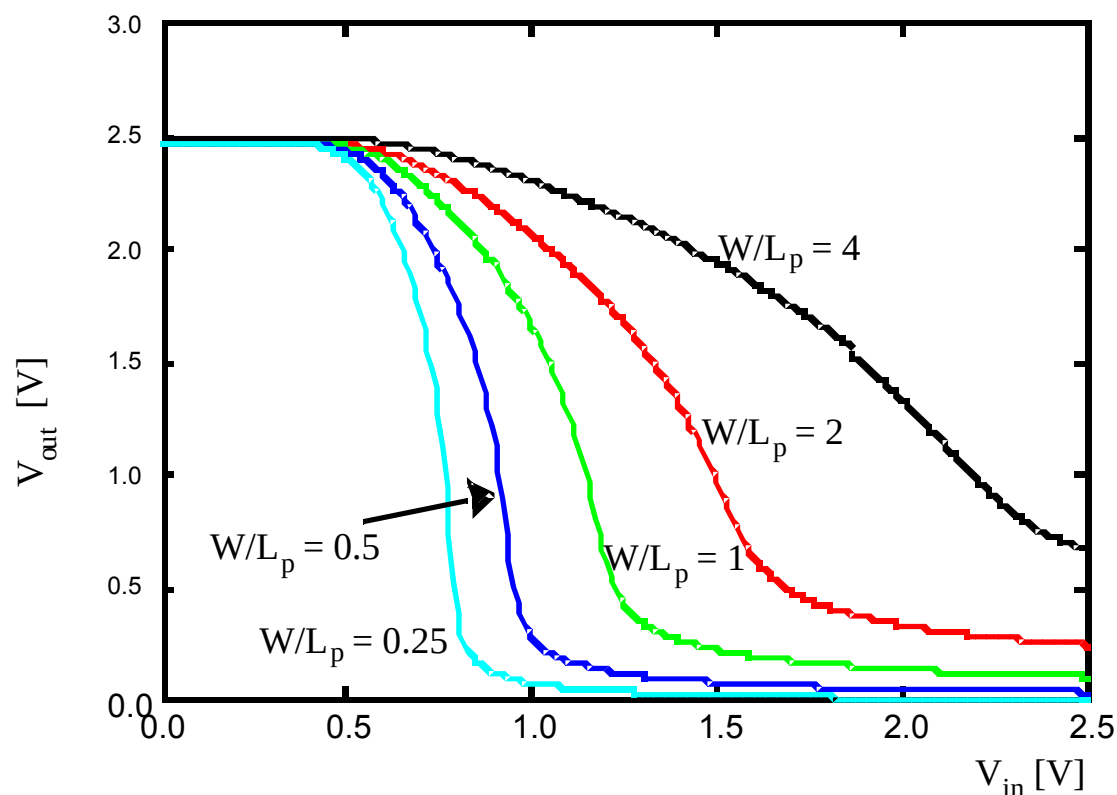
于是 V_{OL} 可以近似为：

$$V_{OL} \approx \frac{k_p (V_{DD} + V_{Tp}) \cdot V_{DSATp}}{k_n (V_{DD} - V_{Tn})} \approx \frac{\mu_p \cdot W_p}{\mu_n \cdot W_n} \cdot V_{DSATp}$$

不同 W_p/W_n 下的VTC

- 固定NMOS的 (W/L) ，随着PMOS的 $(W/L)_p$ 增大， V_{OL} 升高
- 为使在设计空间的各个边界上都能有合适的噪声容限，一般需要比较保守地选择负载管（弱PMOS管）

$$V_{OL} \propto \frac{W_p}{W_n}$$

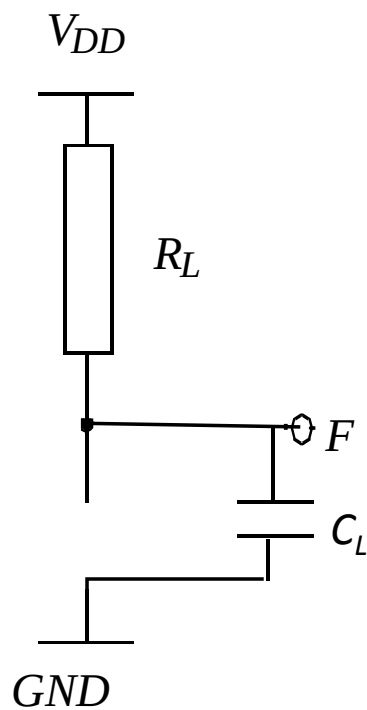


7.2 有比逻辑

7.2.2 伪NMOS逻辑门的传播延时

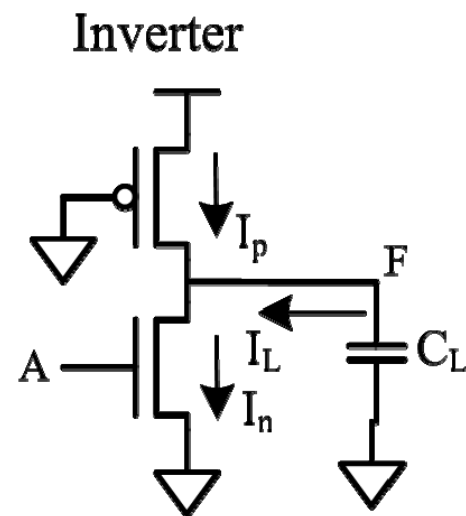
伪NMOS逻辑的传播延时

- 上拉



$$t_{pLH} = 0.69R_L C_L$$

- 下拉



$$I_L = I_n - I_p$$

$$t_{pHL} = \frac{C_L V_{DD}}{2I_L}$$

伪NMOS延时的特点

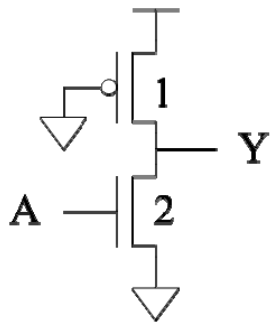
- 不对称响应，上拉和下拉的延时模型不同
- 负载管尺寸 W_p 同时影响上拉驱动能力和下拉驱动能力：
 - 减小 W_p 可以缩短下拉延时，但同时会增大上拉延时
 - ——电路设计时要选择合适的 W_p
- 由于存在负载管的竞争，伪NMOS反相器的下拉性能比互补CMOS反相器差
- 对前一级的负载小——每个输入只连接一个晶体管

Wp/Wn的确定

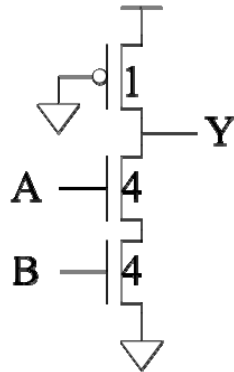
- 较小的Wp可以降低 V_{OL} ，减小面积与输出负载，但同时会带来 t_{pLH} 的增加，需要综合考虑。
- 本课程中如无特别说明，伪NMOS逻辑采用 $Wp=Wn/2$

不同伪NMOS逻辑门的性能

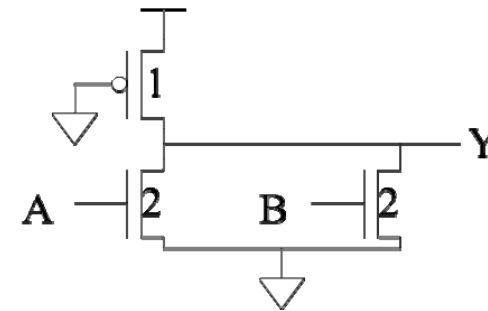
反相器



NAND2



NOR2



- NOR2门的性能比NAND2门好
- 伪NMOS更适合实现NOR门，在大扇入NOR门实现上相比互补CMOS逻辑门有优势，面积更小，逻辑努力更小

7.2 有比逻辑

7.2.3 伪NMOS逻辑门的功耗与特点

伪nMOS的静态功耗

- 输出低电平时存在直流通路

$$P_{low} = V_{DD} I_{low} \approx V_{DD} \left| k_p' \frac{W}{L} \left((-V_{DD} - V_{TP}) V_{DSATp} - \frac{V_{DSATp}^2}{2} \right) \right|$$

PMOS管的导通电阻越大静态功耗越大。

- $V_{OL} > 0$ ，使得后级电路的NMOS亚阈值漏电流增大。

伪nMOS逻辑的特点

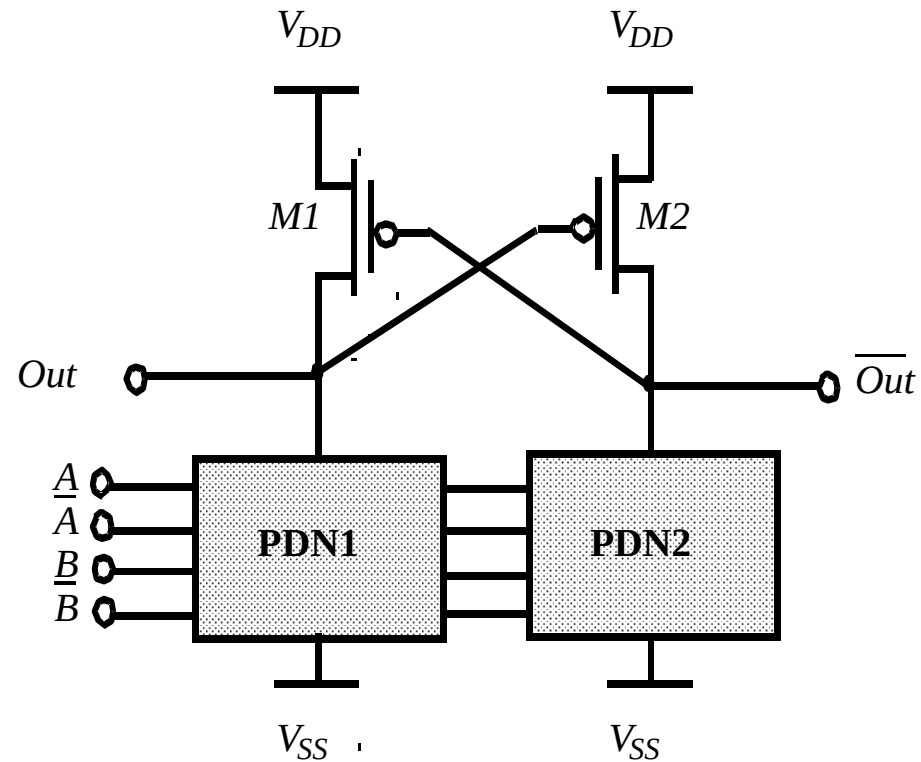
- 晶体管数少 ($N+1$)，只用N管实现逻辑，减少了输入电容,适用于大扇入NOR门
- 有比电路， V_{OL} 和晶体管尺寸有关，必须在性能、功耗和噪声容限间折中，使得设计相对困难。
 - W_p 增大-噪声容限减小、 t_{pLH} 缩短、 t_{pHL} 增大、功耗增大
- $V_{OL} \neq 0$ ，噪声容限小。功能要求 $V_{OL} < V_{IL}$,出于功耗考虑， $V_{OL} < V_T$
- 静态功耗大限制了应用。

7.2 有比逻辑

7.2.4 差分串联电压开关逻辑

DCVSL

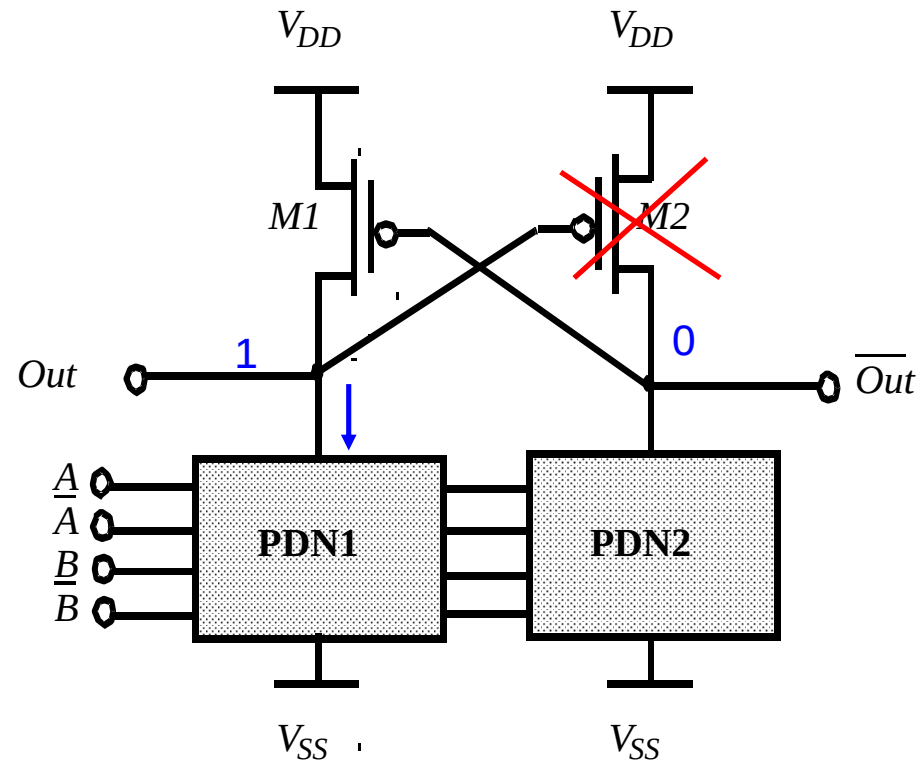
- 交叉耦合的**PMOS**形成正反馈的有源负载
- 差分输入/差分输出



Differential Cascode Voltage Switch Logic (DCVSL)
差分串联电压开关逻辑

DCVSL

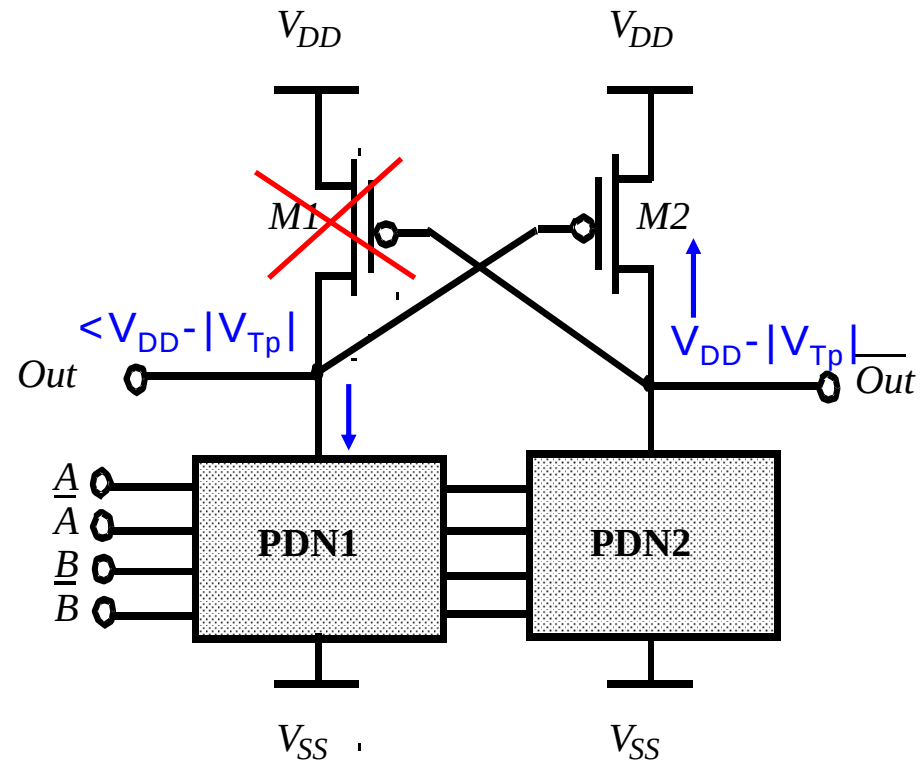
- 交叉耦合的**PMOS**形成正反馈的有源负载
- 差分输入/差分输出



Differential Cascode Voltage Switch Logic (DCVSL)
差分串联电压开关逻辑

DCVSL

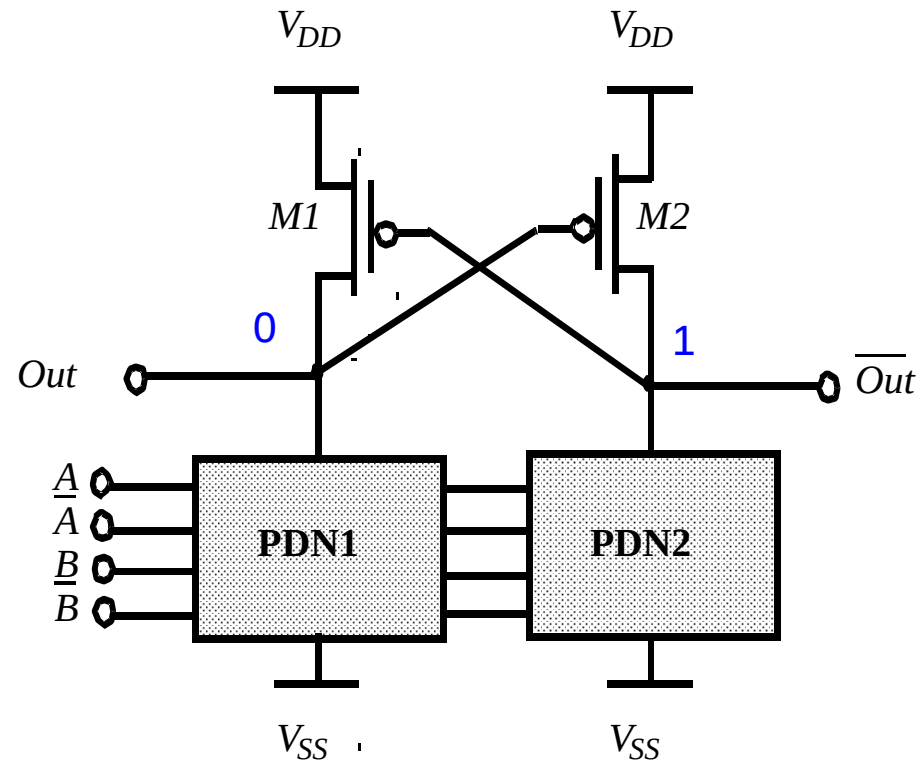
- 交叉耦合的**PMOS**形成正反馈的有源负载
- 差分输入/差分输出



Differential Cascode Voltage Switch Logic (DCVSL)
差分串联电压开关逻辑

DCVSL

- 交叉耦合负载
- 轨线到轨线的摆幅
- 无静态功耗

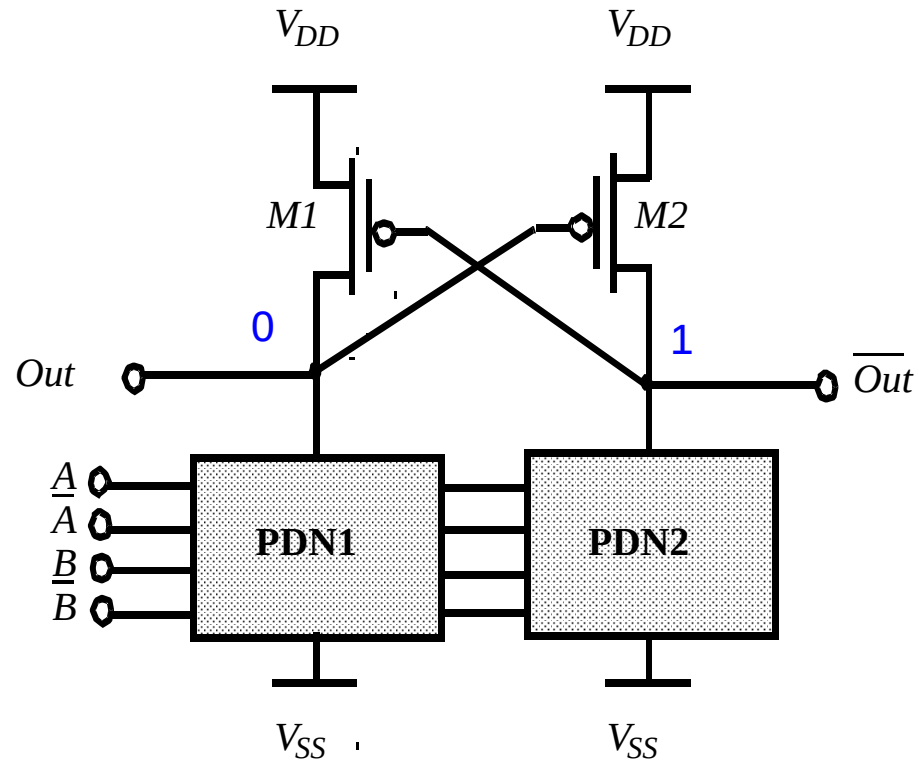


Differential Cascode Voltage Switch Logic (DCVSL)
差分串联电压开关逻辑

DCVSL

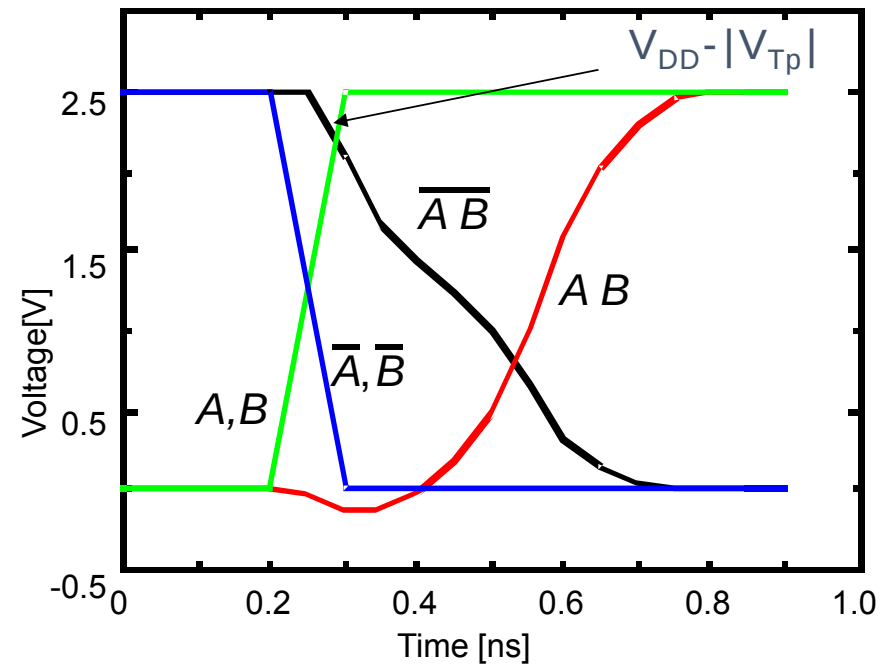
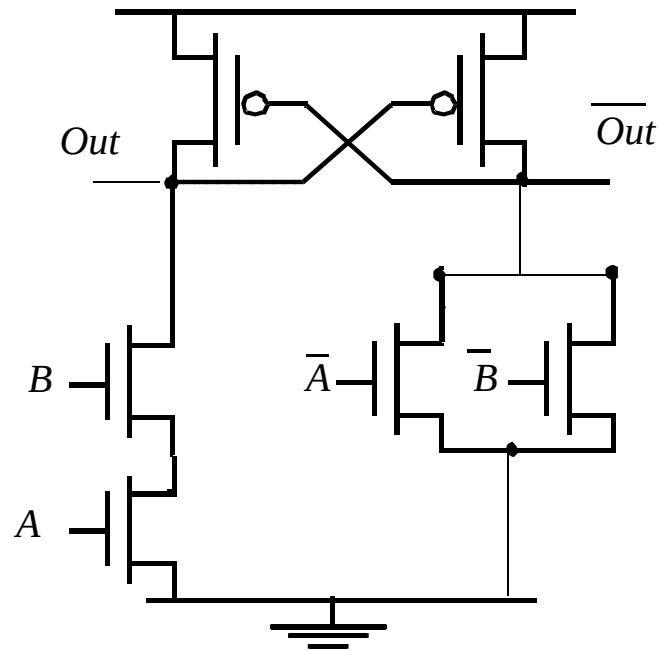
Out由1向0翻转时，
PDN1必须足够强使得Out低于 $V_{DD}-|V_{Tp}|$

仍然是有比逻辑！

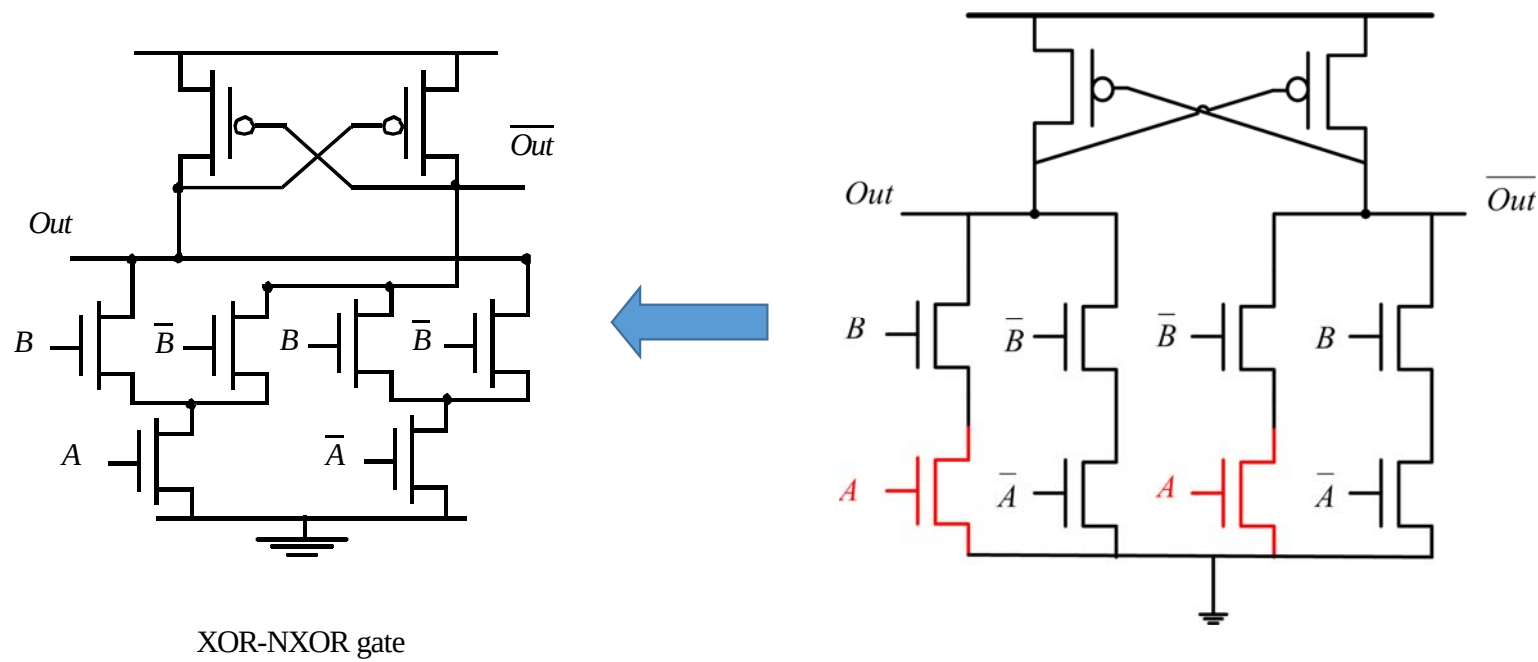


Differential Cascode Voltage Switch Logic (DCVSL)
差分串联电压开关逻辑

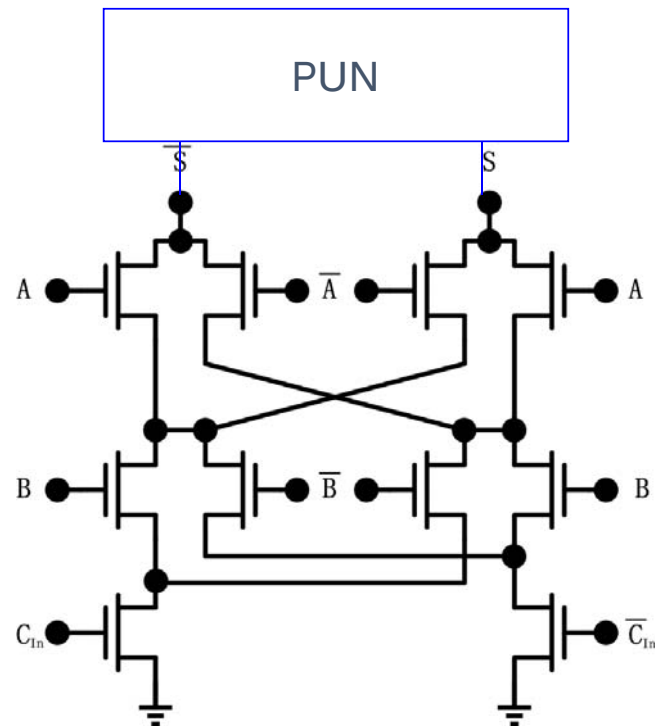
DCVSL 瞬态响应



DCVSL 逻辑共享



DCVSL全加器



$$S = A \oplus B \oplus C_{in}$$

DCVSL特点

- **结构**: 逻辑互补的**NMOS** 下拉管, 交叉连接的**PMOS** 上拉管。
- **逻辑形成**:
 - 具有伪**NMOS**特点: 所有逻辑由**N** 管实现, 减少了输入电容, 在速度上有潜在优势。
 - 不同于伪**NMOS**: 存在的交叉反馈使**CVSL** 的输出最终都能稳定在正确的电平上。
- **设计考虑**: 交叉**P** 管尺寸的选择很重要,
 - **P** 管太大时产生的上拉电流会使下拉速度变慢,
 - 但**P** 管太小时将需较长时间才能把互补输出上拉到高电平。
- **面积**: 同时要求正反输入, 面积大, 但在要求互补输出或两个下拉网络能**共享逻辑**时比较有利, 比较适合于实现多输入的**NOR** 结构。
- **功耗**: 无静态功耗, 但同时有上升和下降过渡 (开关活动性大), 因此**有较大的翻转 (Cross-over) 电流和功耗**

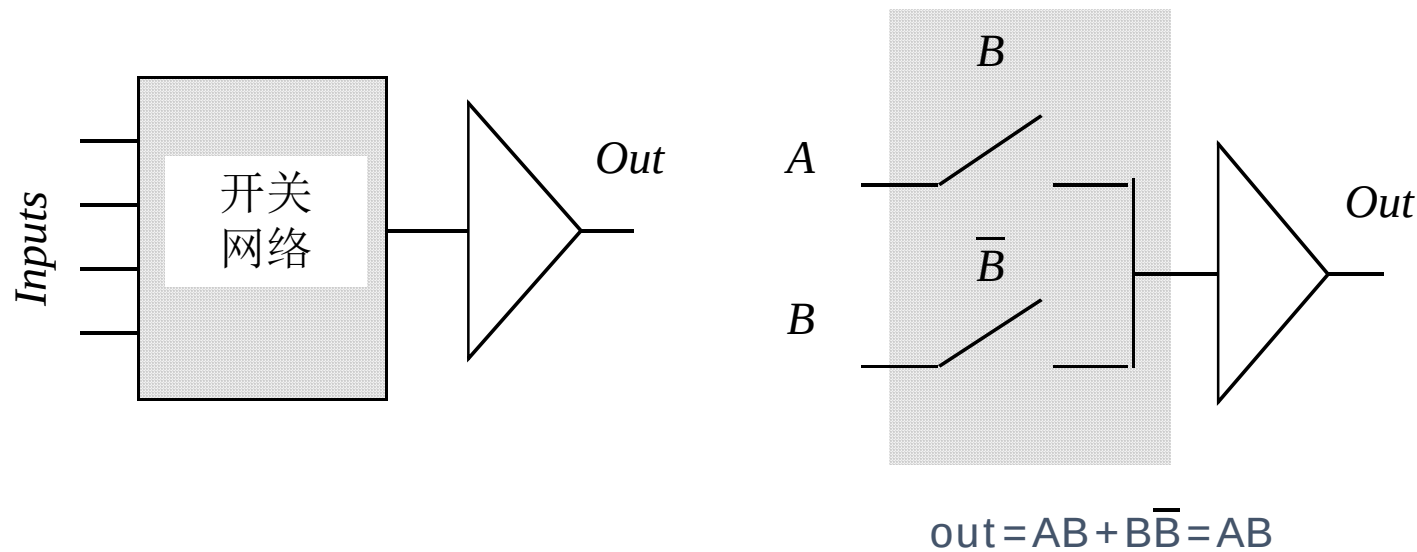
} 折中

7.3 传输管逻辑

7.3.1 传输管逻辑的工作原理

传输管逻辑原理

通过允许输入驱动源漏端来减少晶体管数目。

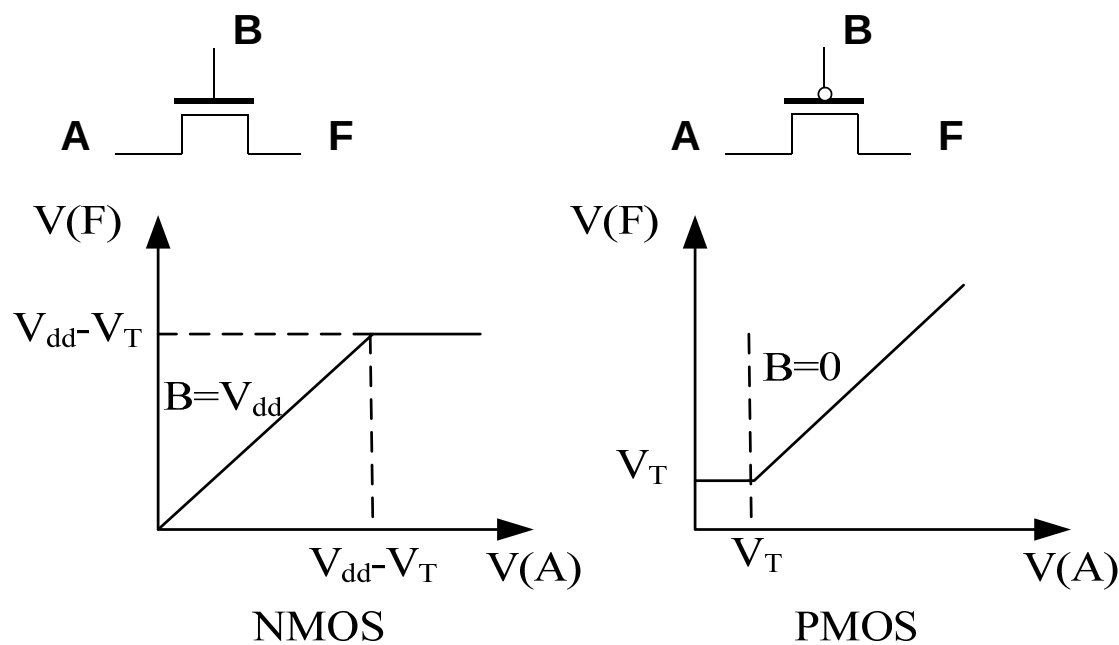


• 要保证在任何时刻都有一个开关路径导通！

• N个晶体管

阈值损失

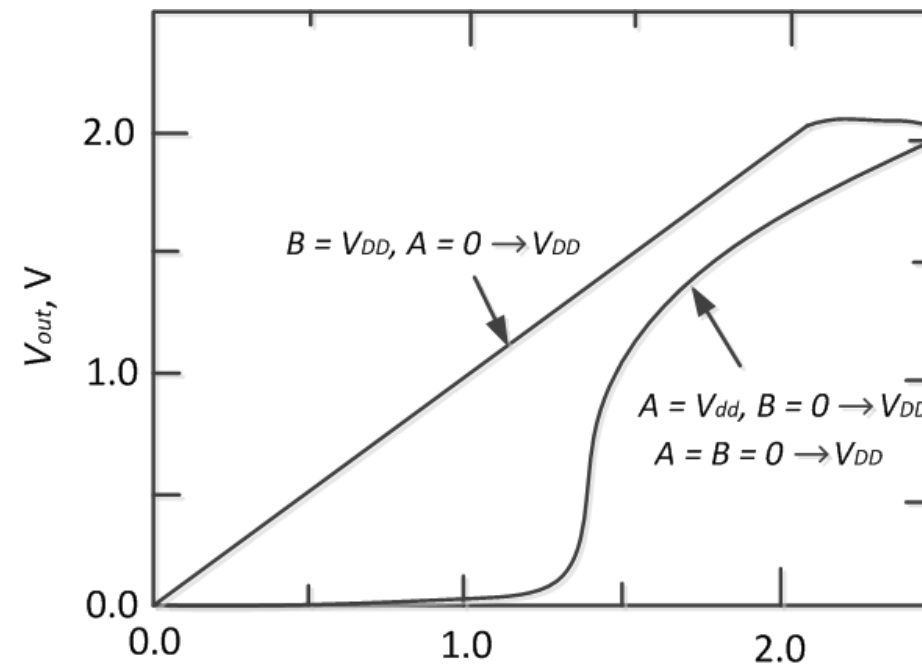
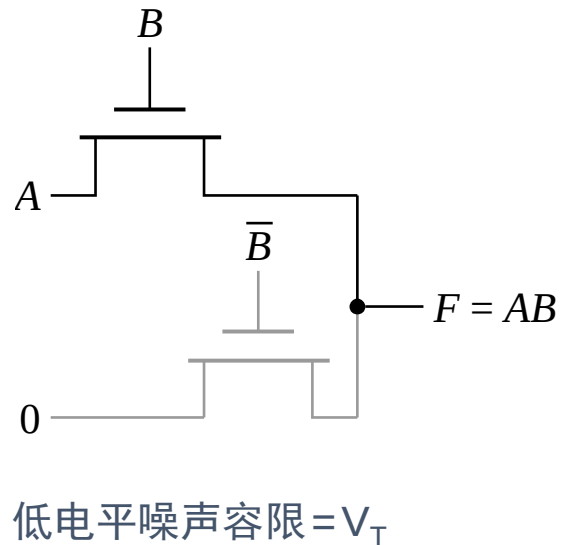
NMOS传输高电平存在阈值损失；
PMOS传输低电平存在阈值损失。



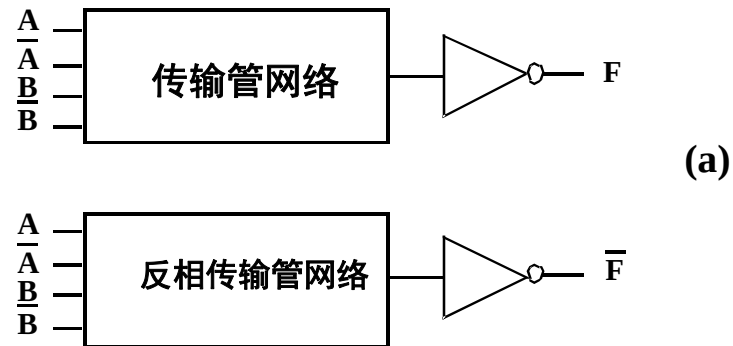
传输管逻辑的电压传输特性

不能再生，需要通过插入CMOS反相器来弥补

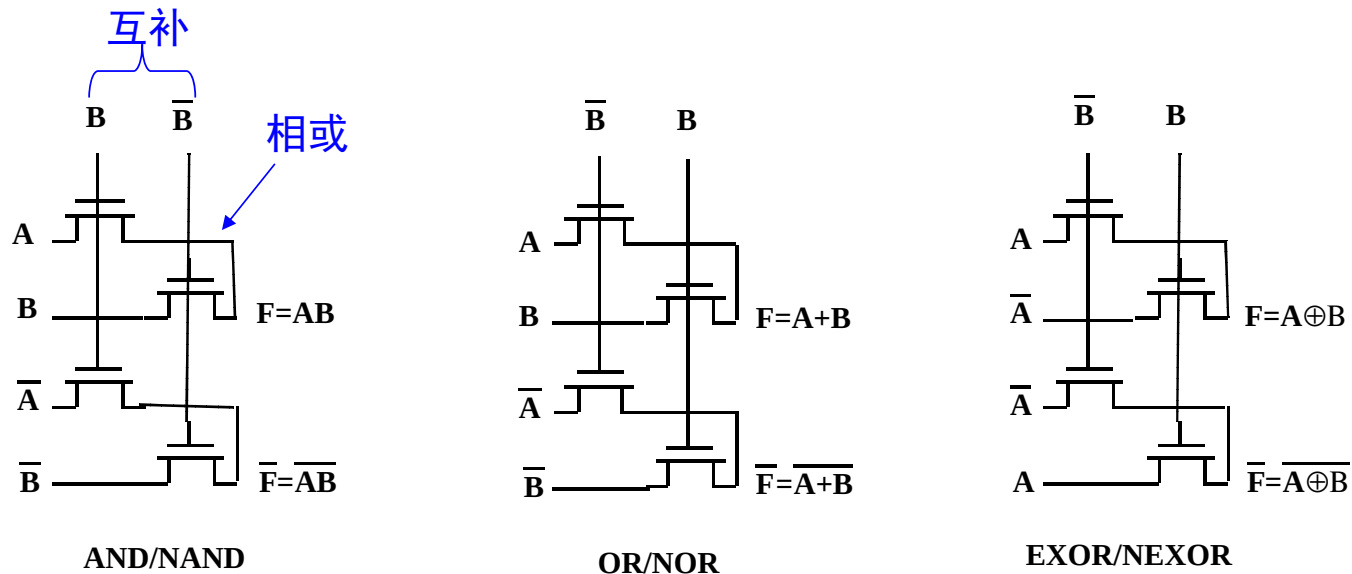
NMOS 传输管AND门



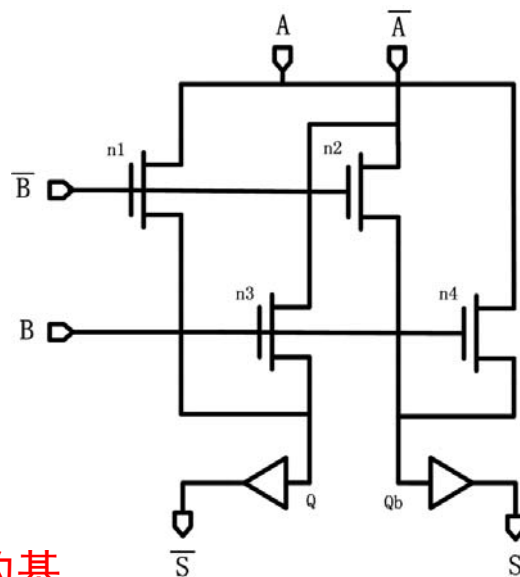
互补传输管逻辑 (CPL)



每个网络包括两个导通条件互补的支路并联；两个网络结构相同，不同网络中相同导通条件的支路输入信号互补。



CPL全加器

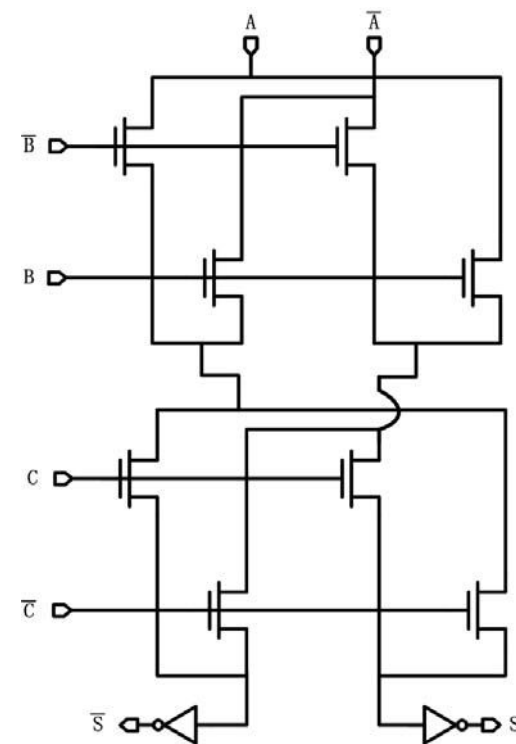


半加器

所有逻辑都由相同的基本拓扑结构单元实现。

$$F = AS + \overline{BS}$$

$$\overline{F} = \overline{AS} + \overline{BS}$$



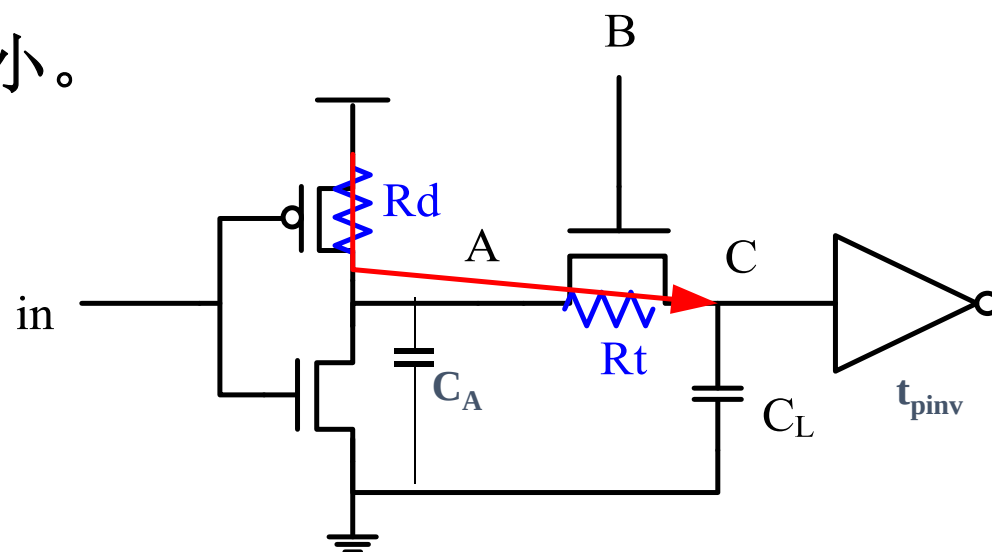
全加器

CPL静态特性的特点

- 简单，比静态CMOS 电路的晶体管数大大减少。
- 静态逻辑
 - 输出节点（经前级驱动逻辑门）总是连至 V_{DD} 或GND。
- 信号再生能力差，一般需要在输出端采用互补CMOS反相器做驱动，对输出信号进行整形
- 噪声容限等于NMOS管的阈值电压，噪声容限小
- 存在阈值损失，信号输出不是轨到轨的
- 具有互补功能（互补的输入—互补的输出），需产生互补信号的额外电路以及额外的布线开销。但在需要正反输出信号时比较有利。
- 设计具有模块化的特点，易于构成逻辑和设计库。

传输管的延时

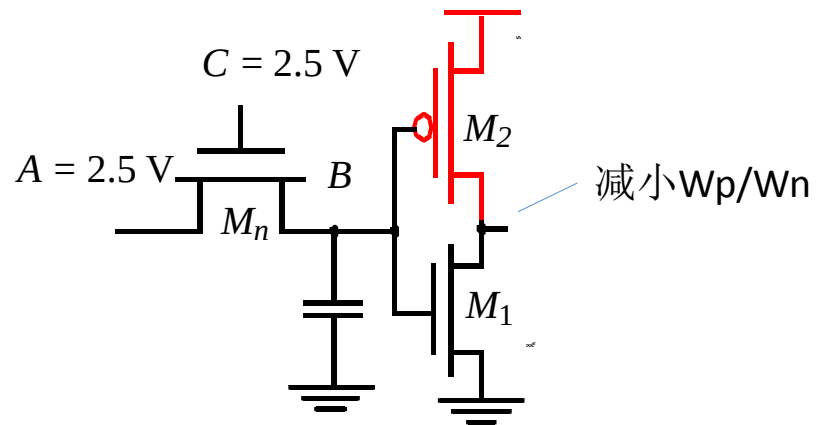
- 对驱动它的电路的负载效应小。
- 延时与驱动门有关。



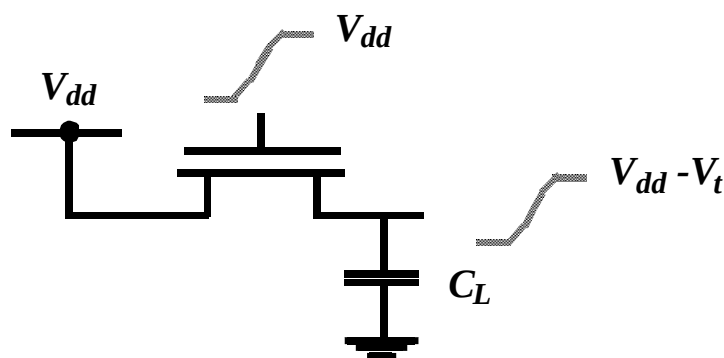
$$t_p = 0.69(R_d + R_t)C_L + t_{p\text{inv}} + 0.69R_dC_A$$

传输管逻辑延时优化

- 输出反相器应设计成Low-skewed 的，以克服传输管输出上升速度缓慢的缺点。



传输管逻辑门的功耗



$$E_{01 \rightarrow} = C_L \cdot V_{dd} \cdot (V_{dd} - V_t)$$

- 具有小的信号摆幅，**可以利用小摆幅来降低充放电功耗。**（例如，存储器中的小摆幅位线）
- 晶体管数量上，寄生电容小，功耗低。（不要忘记前级和后级电路的功耗！）

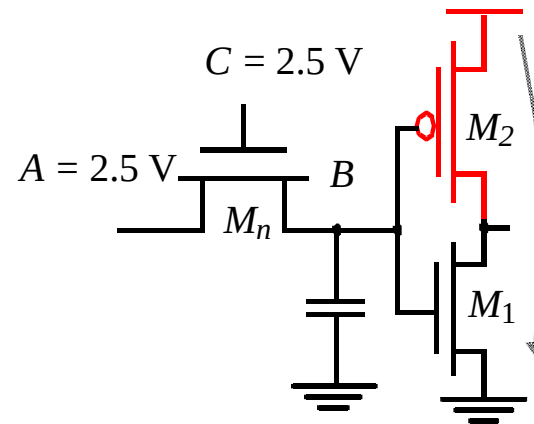
7.3 传输管逻辑

7.3.2 阈值损失补偿技术

7.3.2.1 电平恢复技术

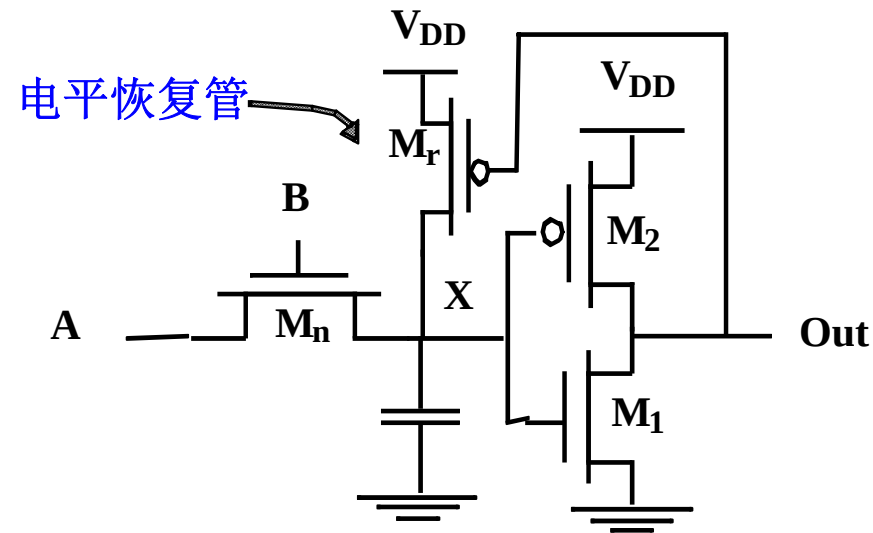
阈值损失带来的问题

- 阈值损失会引起后级电路的静态漏电流，增加电路的静态功耗。



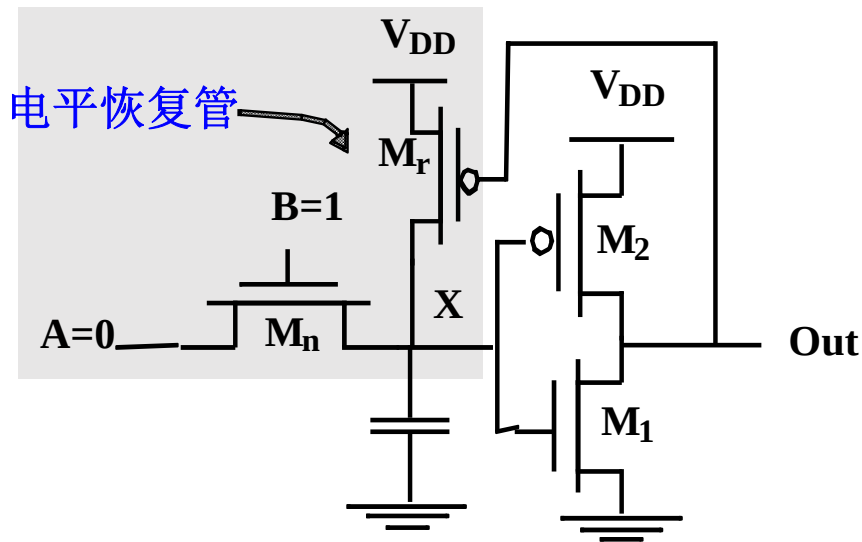
注意：由于体效应的存在，实际电平损失超过 V_{T0}

电平恢复（阈值损失补偿1）



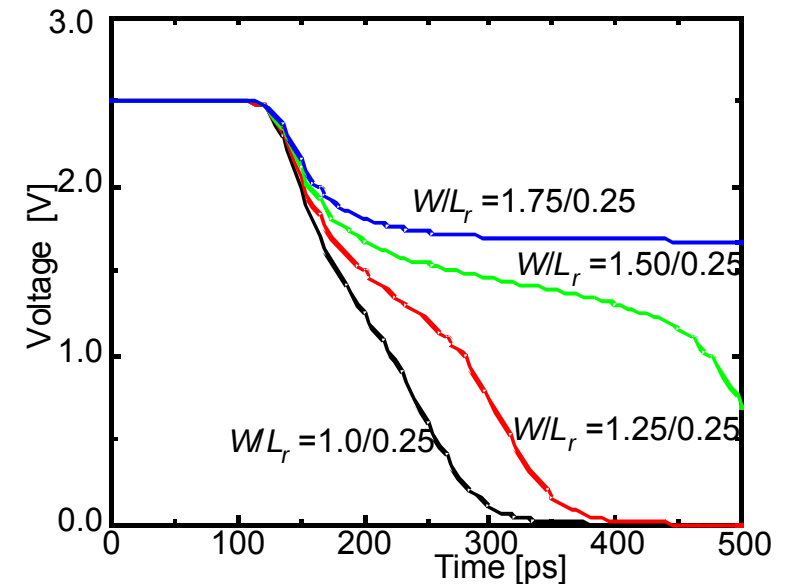
- 优点：全摆幅
- 缺点：
 - 电平恢复管增加了负载电容；
 - 电平恢复管在X由高向低翻转时削弱了放电电流。

电平恢复（阈值损失补偿1）



🔴 **Mn应强于Mr:** $A=0$, $B=1$ 时, 应保证X的电平低于反相器阈值
(是一个有比电路。)

🔴 注意: 传输晶体管下拉电路可能会有几个晶体管串联（堆叠）在一起

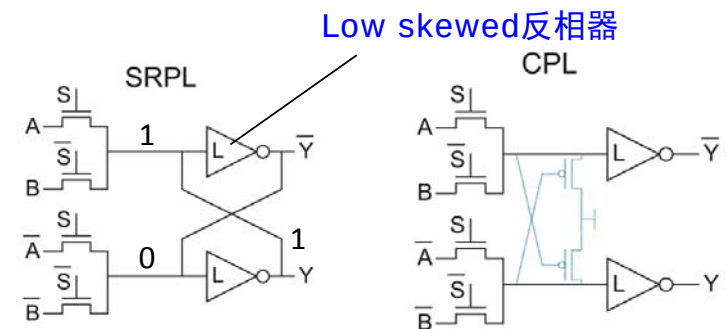
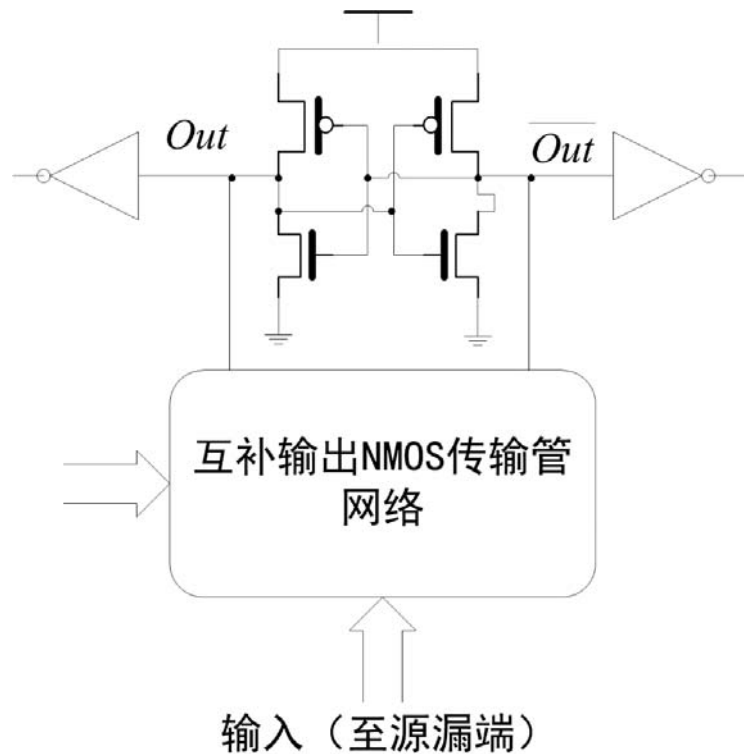


$A=0, B=1$ 时X点电压的波形

低电平随着Mr的强度增大而升高, 传播延时也随之增长。

差分电平恢复电路

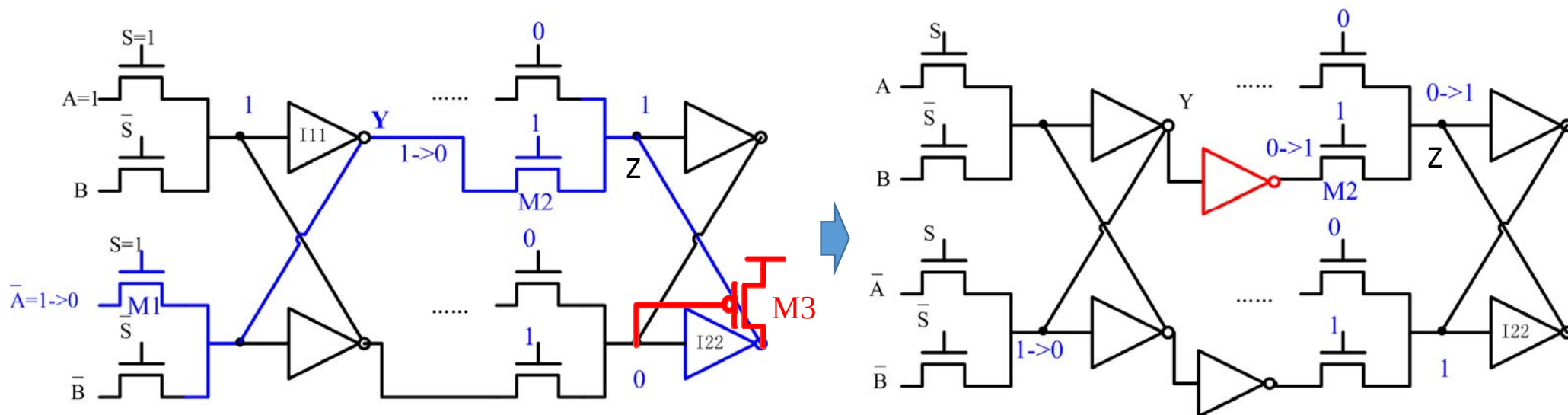
- 又称：摆幅恢复传输管逻辑（SRPL）
- 正反馈，加速输出变化，以较少的晶体管数代价获得较小的延时



删掉摆幅恢复反相器中的NMOS管，即变为了带电平恢复管的CPL电路。

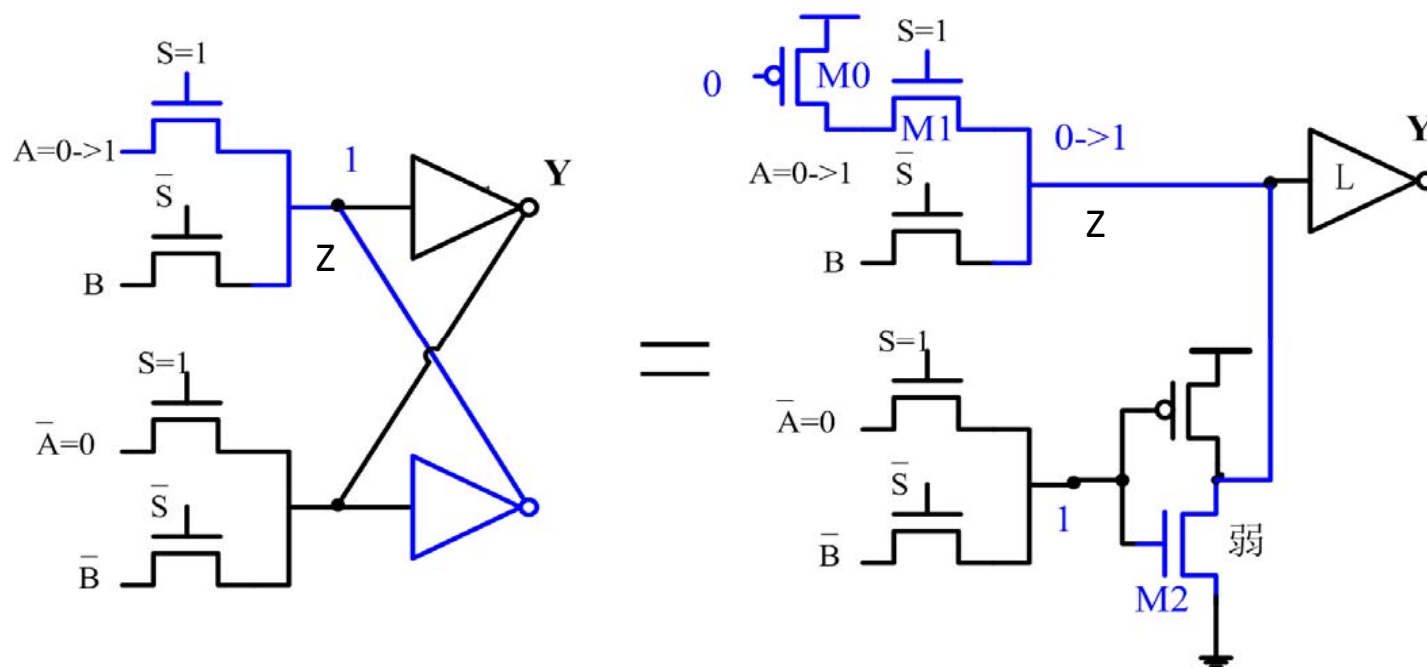
摆幅恢复传输管逻辑设计中的注意事项

- 串联SRPL门之间要用CMOS缓冲器，
 - 否则串联门的电平恢复器件之间的竞争会降低性能。



M1下拉，M2-M3上拉，形成竞争

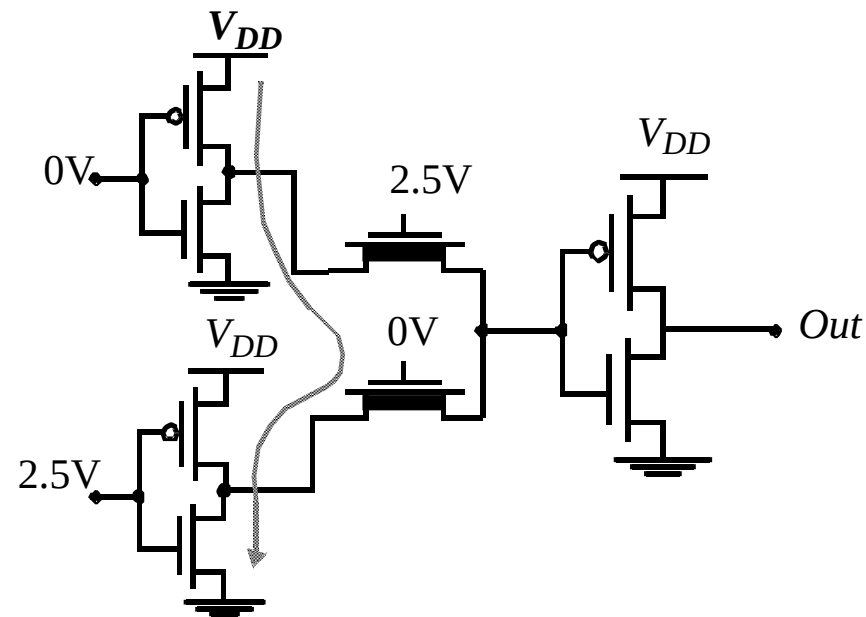
摆幅恢复传输管逻辑设计中的注意事项



由于传输管传输高电平时驱动能力弱，交叉耦合反相器中的NMOS必须足够弱，以保证传输管上拉时的性能。

7.3.2.2 低阈值传输管

使用 $V_T=0$ 的单管传输门（阈值损失补偿2）

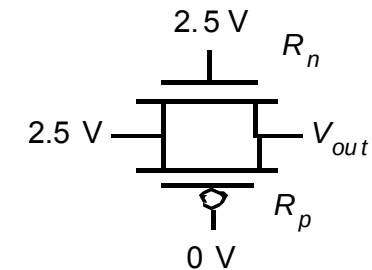
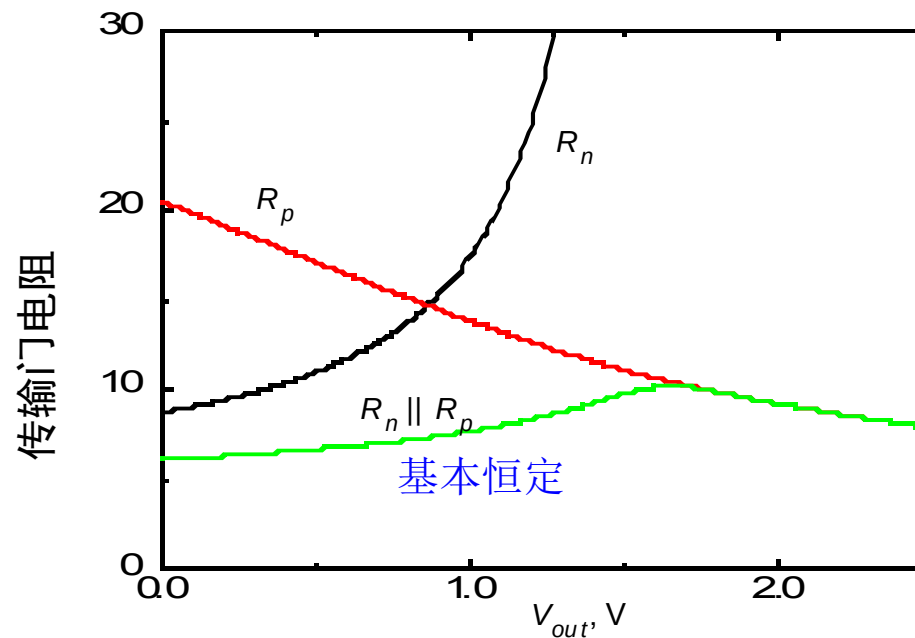
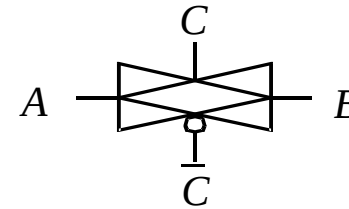
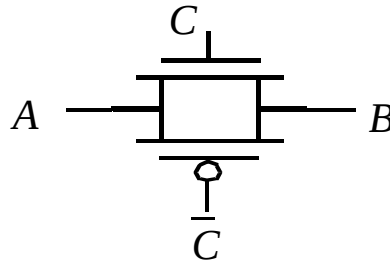


但要注意漏电流

7.3.1.3 CMOS传输门

CMOS传输门（阈值损失补偿3）

传输门
(transmission gate)

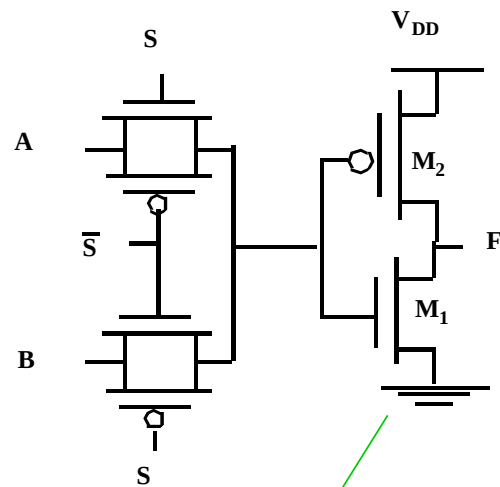


通常 $\frac{W_p}{W_n} = 10$

a10 过大的 W_p 会增加寄生电容，在NMOS导通的范围内电阻主要由NMOS电阻决定。
adm, 2011/11/7

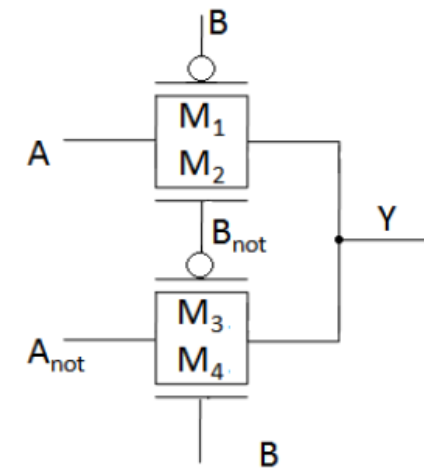
传输门逻辑

多路选择器逻辑：



传输门同样不具有再生性，输出端要加反相器驱动。

异或门：

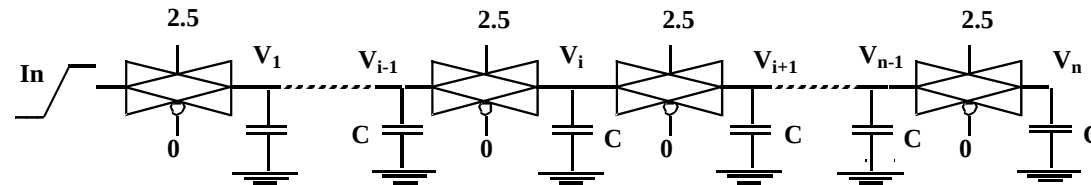


传输门逻辑的性能

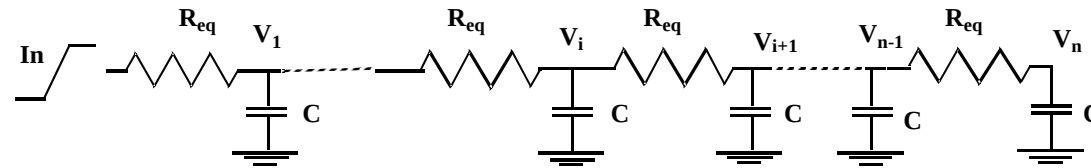
- 内部节点短路降低了等效电阻,但同时增大了内部节点电容

串联传输门电路的延时优化

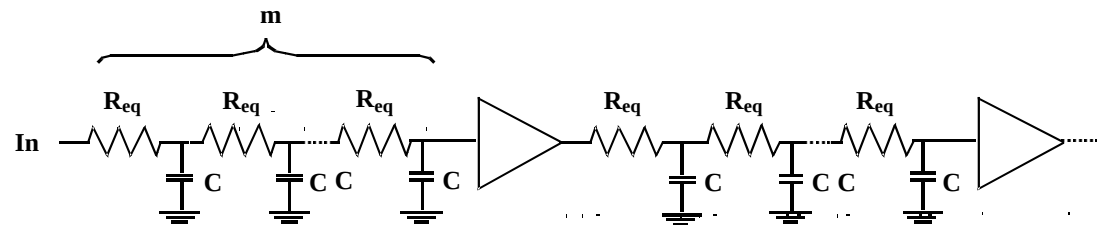
延时随串联传输门的数目迅速增加。



(a)



(b)



(c)

传输管与传输门逻辑小结

- 传输管

- 优点：寄生电容小，速度快，属无比逻辑
- 缺点：阈值损失，噪声容限差，会引起下一级静态功耗；MOS 管的导通电阻随电压变化而变化

- 全传输门

- 优点：无阈值损失，MOS 开关的导通电阻基本为常数
 - 和其它阈值损失补偿方法相比，是无比逻辑、功耗低
- 缺点：必须提供正反控制信号，版图设计效率低，电容大

7.3 传输管逻辑

7.3.3 传输管逻辑的信号完整性问题

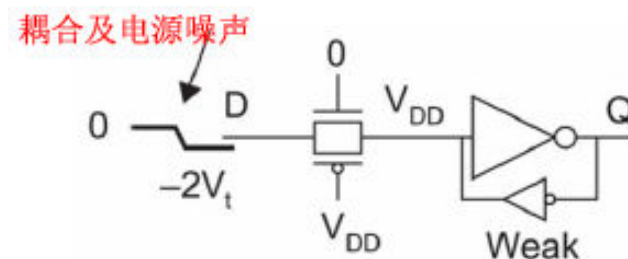
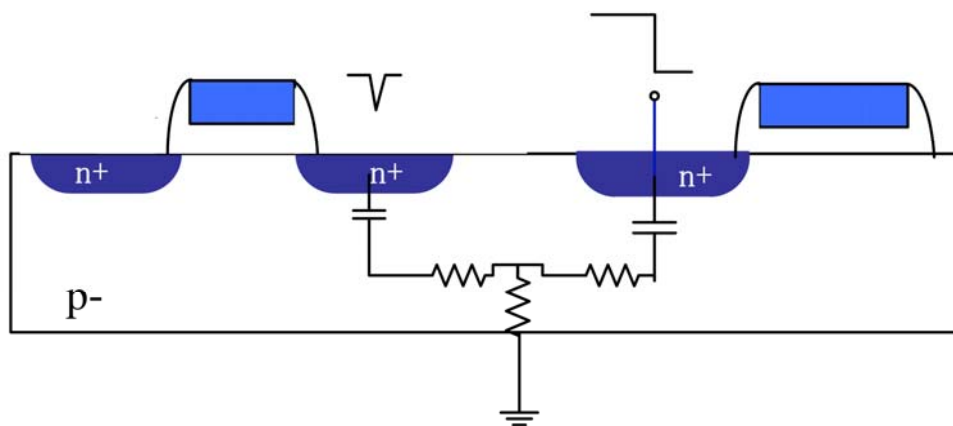
传输管逻辑的信号完整性问题

- 阈值损失问题
- 传输管和传输门都不具有“再生特性”，级联时会累加噪声——输出端需静态反相器
- 扩散区输入
- 非单向性

扩散区输入

a11

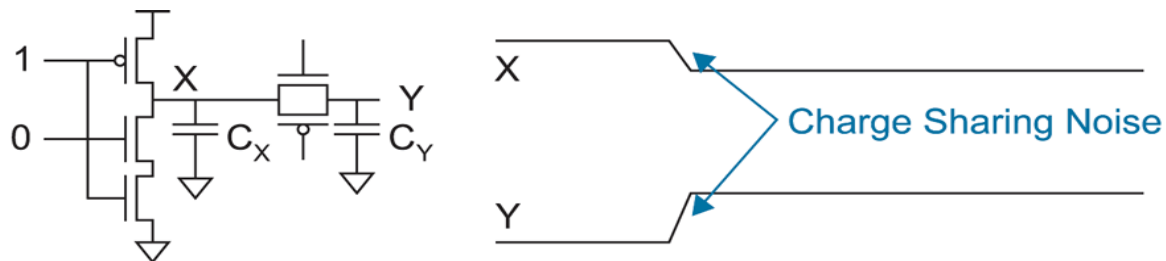
- 当输入信号直接驱动传输管（传输门）的扩散区时，输入端易受到噪声干扰，因为扩散区和衬底接触，存在耦合电容（扩散区pn结电容），比较容易受到衬底耦合噪声的干扰。



a11 扩散区比较容易受到衬底耦合噪声的干扰
adm, 2011/11/7

非单向性

- 传输管不是单向性的，特别是当它由动态节点驱动时可能存在电荷共享问题。



传输管与传输门逻辑小结（1）

- 电路简单、对驱动电路的负载小，但是
 - 扩散区较易受到衬底耦合噪声的干扰，扩散区输入的噪声容限（ V_T ）小；
 - 扩散区输入需要在输入端放置接触孔，增大了输入扩散区的面积；
- 无比逻辑（但增加电平恢复电路后存在有比问题）；
- 传输管和传输门都不具有“再生特性”，级联时会累加噪声，输出端常需静态反相器；
- 静态逻辑：设计传输管、传输门网络时，应使所有情形下遵守“输出端低阻抗连至 V_{DD} 或Gnd”的原则；
- 传输管和传输门逻辑直接级联延时随着级数增加迅速增加，中间需要插入静态反相器缓冲；
- 功耗低：消除阈值损失（或使 $V_{Tn} < V_{Tp}$ ）的情况下无静态功耗。

传输管逻辑小结（2）

- 解决阈值损失问题可以采用

- 电平恢复电路

- 单端逻辑可以采用反馈上拉PMOS
- 差分逻辑可以采用交叉耦合PMOS，交叉耦合反相器（SRPL）等不同形式恢复电平
- 增加了内部节点电容，关断时有信号竞争（注意电平恢复器尺寸的设计），降低了门的速度
- PMOS 的导通加速了上拉，因而减少了输出（反相器）的下降时间

- 低阈值晶体管——存在较大的亚阈值漏电流
- 采用传输门代替NMOS传输管构成电路（CMOS传输门逻辑）——增加了面积（有源区需要放置接触孔）和互补的控制线

传输管逻辑小结（3）

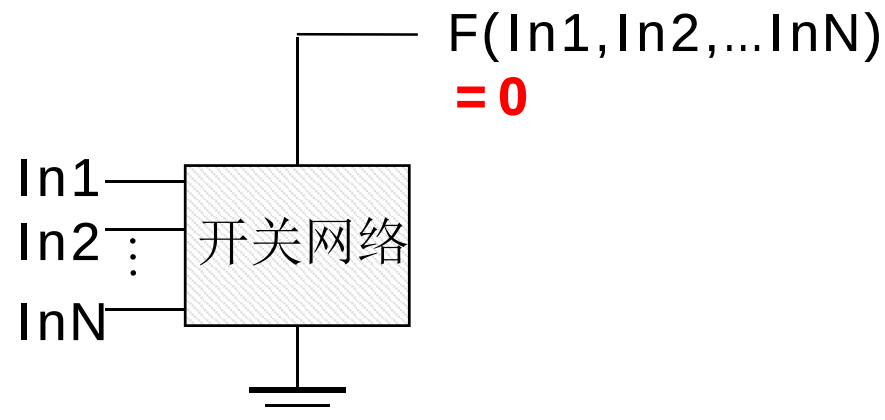
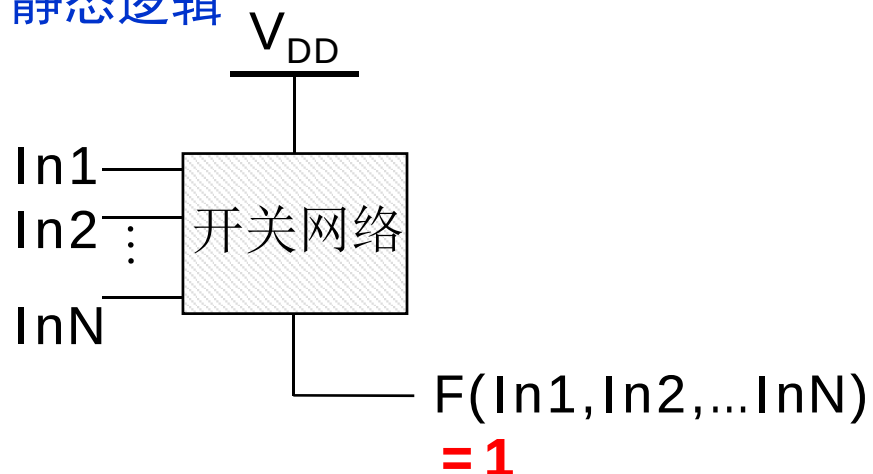
- 为了解决信号再生的问题，传输管逻辑需要插入静态反相器恢复电平
 - 一般采用LO-skewed反相器以弥补NMOS传输管传输高电平的能力不足
 - 传输管（门）延时计算要考虑驱动电路的情况
 - 传输管（门）的静态漏电流要考虑前后级反相器中的电流通路
- 传输门具有较高的可靠性和简单的结构，普遍用于标准单元库中MUX、latch、XOR门、乘法/加法器等逻辑的实现。

第七章 组合逻辑门的设计（下）

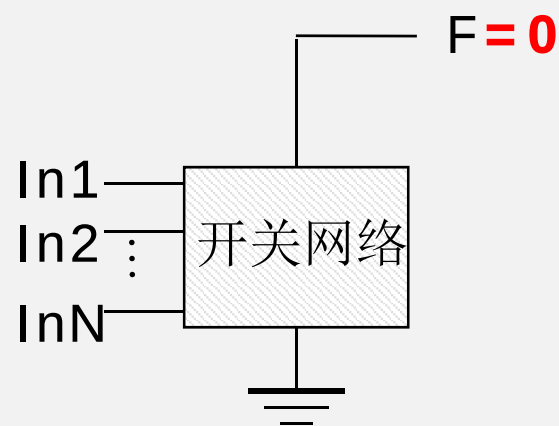
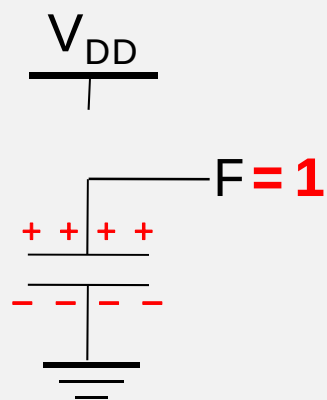
7.4 动态逻辑

静态逻辑与动态逻辑

静态逻辑



动态逻辑



a1

幻灯片 2

a1

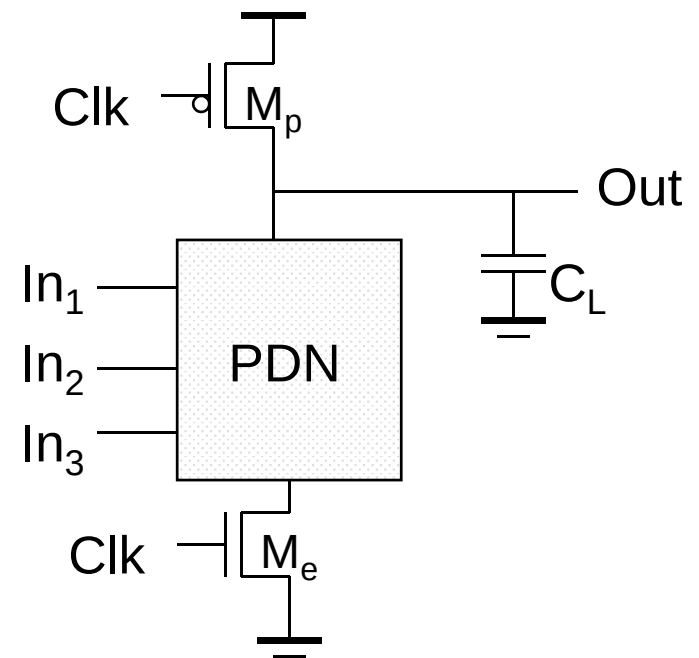
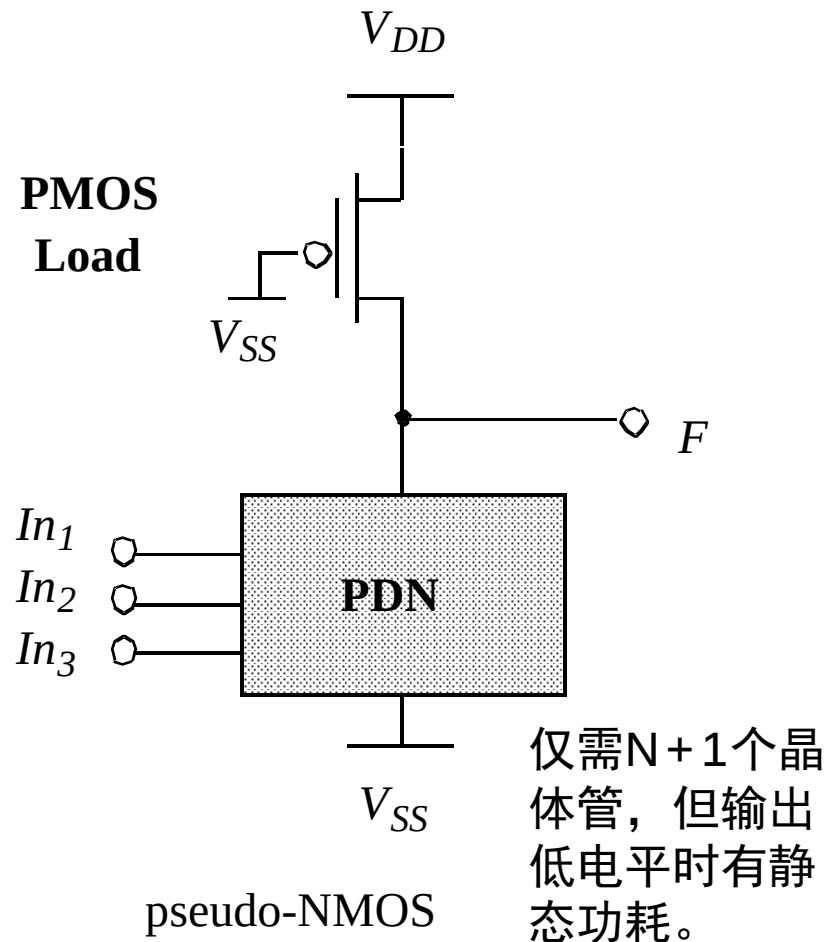
在静态电路中，任何时候（除去翻转时）输出总是通过低阻路径连至GND 或VDD

动态电路依靠把信号值暂时存放在高阻抗节点的电容上来代替低阻抗路径（与伪NMOS的晶体管数相同但没有静态功耗）。

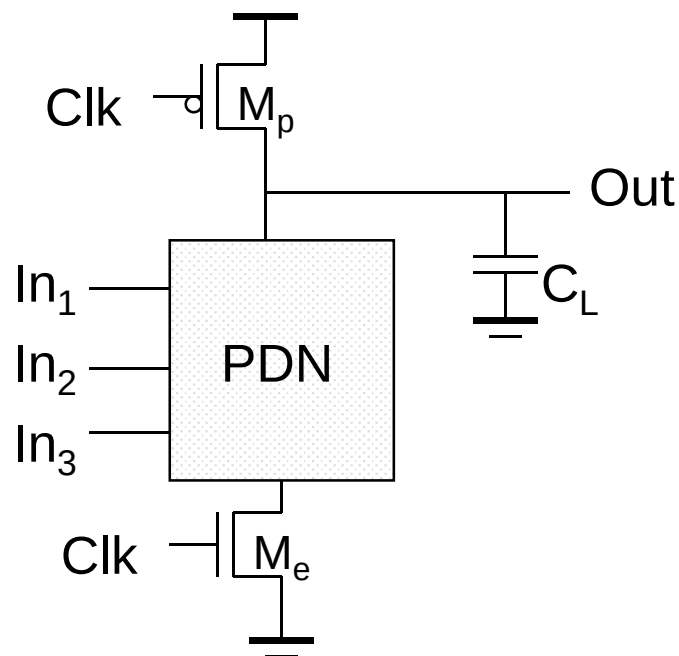
N输入逻辑N+2个晶体管

adm, 2011/11/11

7.4.1 动态逻辑基本原理



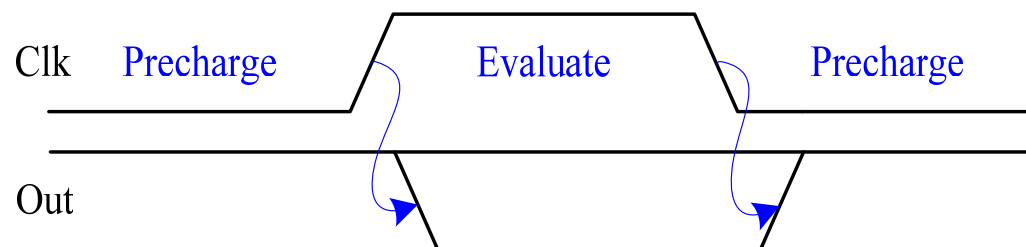
动态门



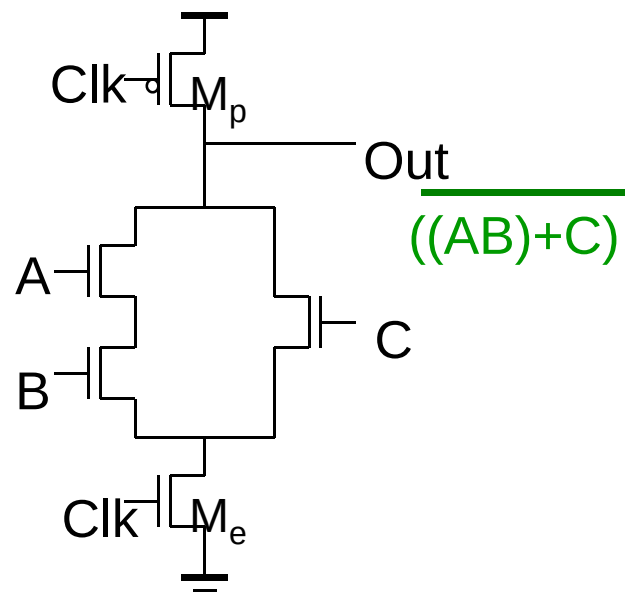
两个相位工作

预充 (CLK = 0)

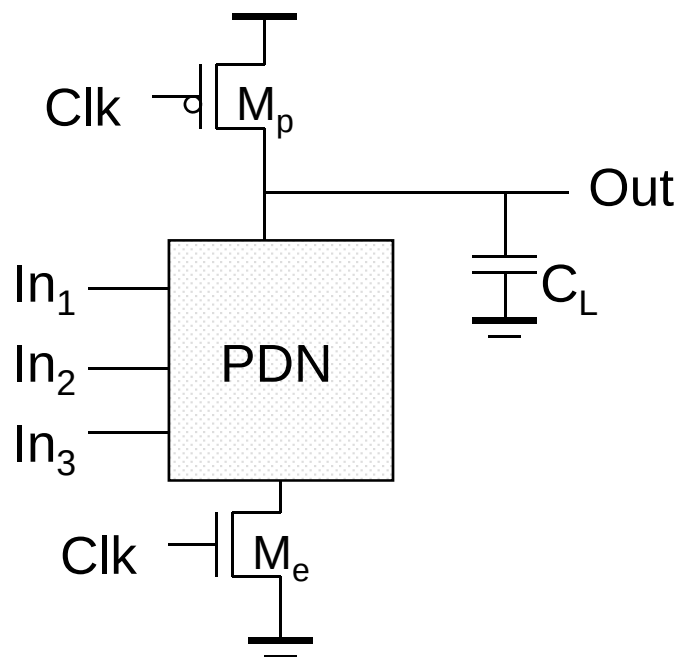
求值 (CLK = 1)



动态门举例

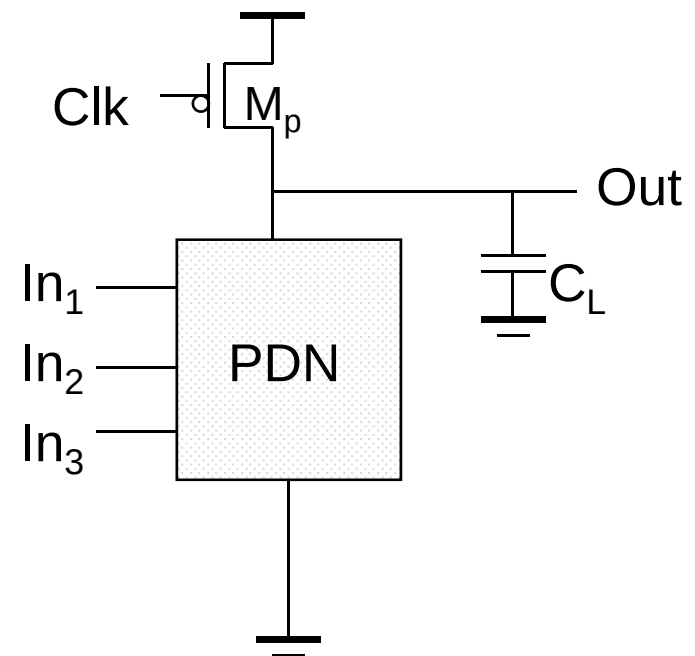


有足动态逻辑和无足动态逻辑



有足门 (footed gate)

预充时可以避免出现竞争



无足门 (unfooted gate)

确保预充阶段PDN不会导通时

工作条件

- ❖ 要求一系列的周期性预充和求值，纯粹的静态分析不再适用
- ❖ 一旦动态门的输出被放电，它直到下一个预充电阶段之前不可能再次被充电。
- ❖ 因此，动态门的输入在求值期间最多只能有一次过渡。
- ❖ 在求值期间或求值之后输出可以处于高阻态（PDN off），求值之后的状态存放在 C_L 上。
- ❖ 输出信号不会有毛刺

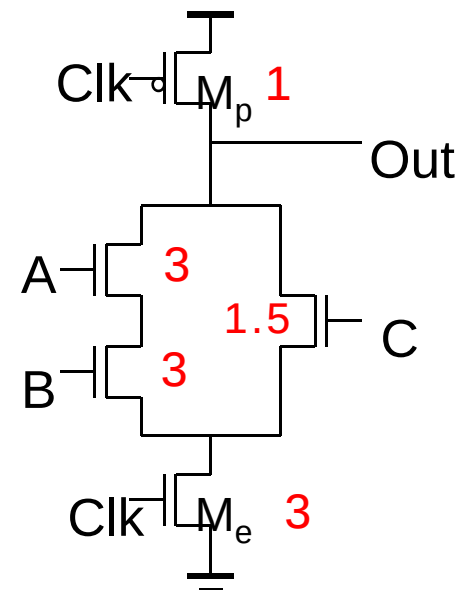
动态逻辑的特性

- ❖ 全摆幅输出
- ❖ 无比逻辑
- ❖ 一旦输入信号超过 V_{Tn} ，PDN 就开始工作，因此 V_M ， V_{IH} 和 V_{IL} 等于 V_{Tn}
- ❖ 噪声容限小（ V_{Tn} ），对噪声敏感，易受噪声耦合的影响

动态逻辑门的设计

对器件尺寸的要求

- 下拉电阻等于单位电阻。
- 可选择较小的预充电管尺寸，使预充电管的电阻为两倍单位电阻（这可以降低时钟负载电容和输出端寄生电容，但会使充电（上升）延时加大）。
- 选择求值管的尺寸比其他NMOS管大。（但这增加了时钟负载。）



LX1

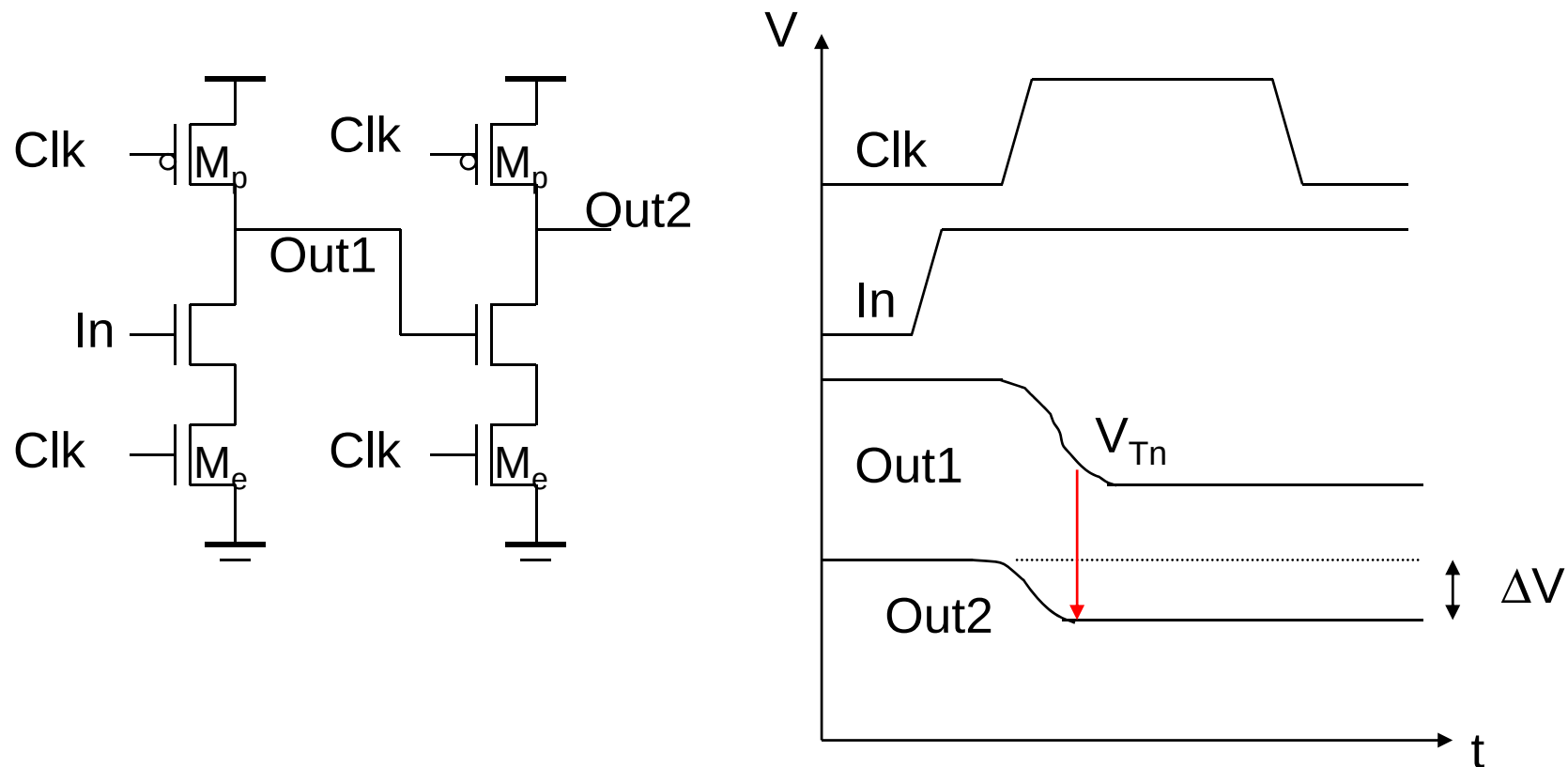
预充可在电路空闲时进行,并且预充电速度常可以慢一些, 因此可选择较小的预充电管尺寸, 使预充电管的电阻为两倍单位电阻 (这可以降低时钟负载电容和输出端寄生电容, 但会使充电 (上升) 延时加大)。

为降低其他输入端的逻辑努力, 选择求值管的尺寸比其他NMOS 管大。(但这增加了时钟负载。)

Li XY, 2015/11/22

7.4.2 串联动态门

动态门串联会发生什么？



a18

a18

Out2 should remain at VDD since Out1 transitions to 0 during evaluation. However, since there is a finite propagation delay for the input to discharge Out1 to GND, the second output also starts to discharge.

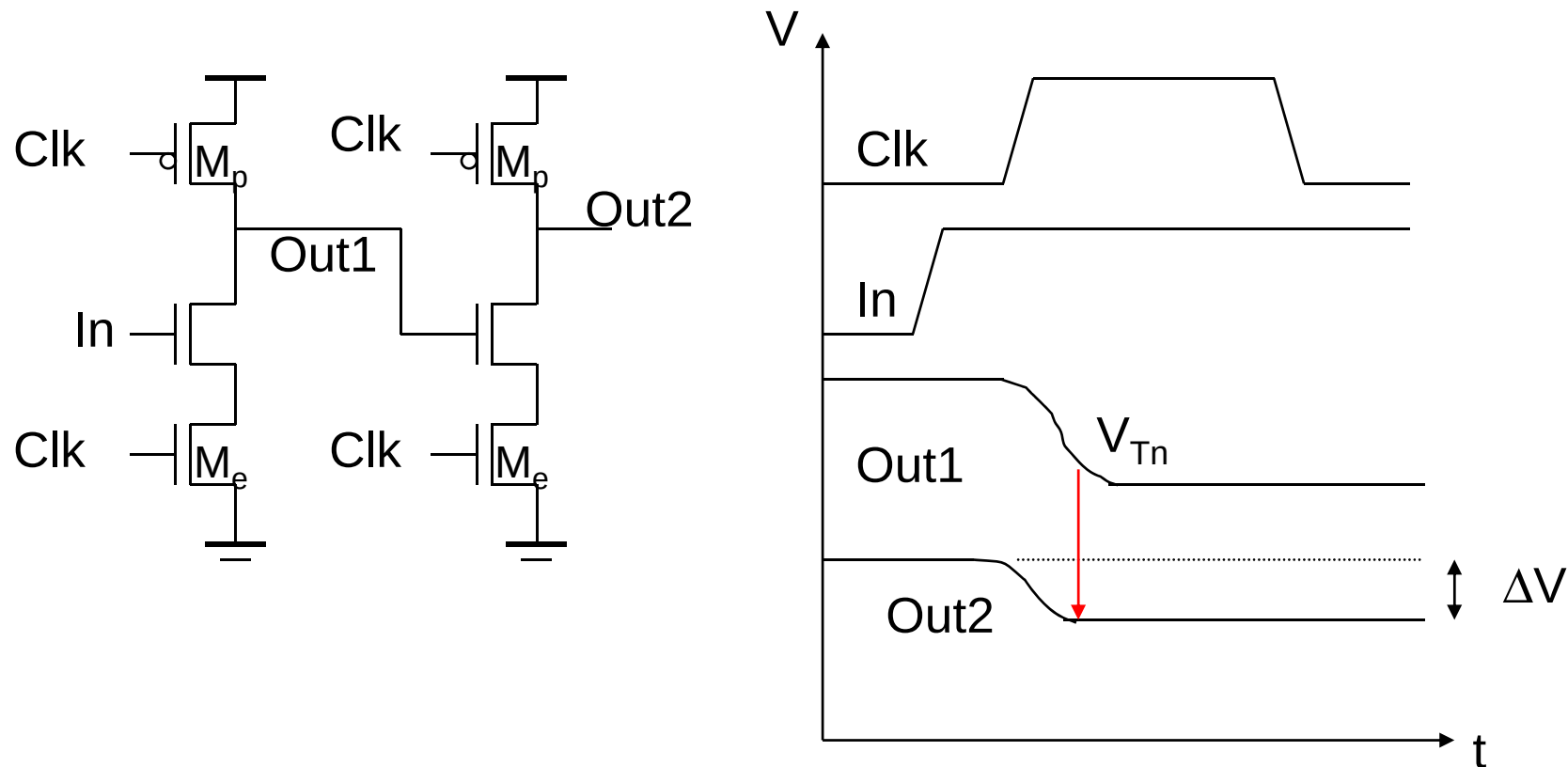
The second dynamic inverter turns off (PDN) when Out1 reaches V_{Tn} .

Setting all inputs of the second gate to 0 during precharge will fix it.

Correct operation is guaranteed (ignoring charge redistribution and leakage) as long as the inputs can only make a single 0 → 1 transition during the evaluation period.

adm, 2011/11/14

动态门串联会发生什么？



前一级门的输出被预充至1，在求值周期开始时可能造成后一级无意的放电。

a18

简单的动态CMOS 逻辑门不能串联！

a18

Out2 should remain at VDD since Out1 transitions to 0 during evaluation. However, since there is a finite propagation delay for the input to discharge Out1 to GND, the second output also starts to discharge.

The second dynamic inverter turns off (PDN) when Out1 reaches V_{Tn} .

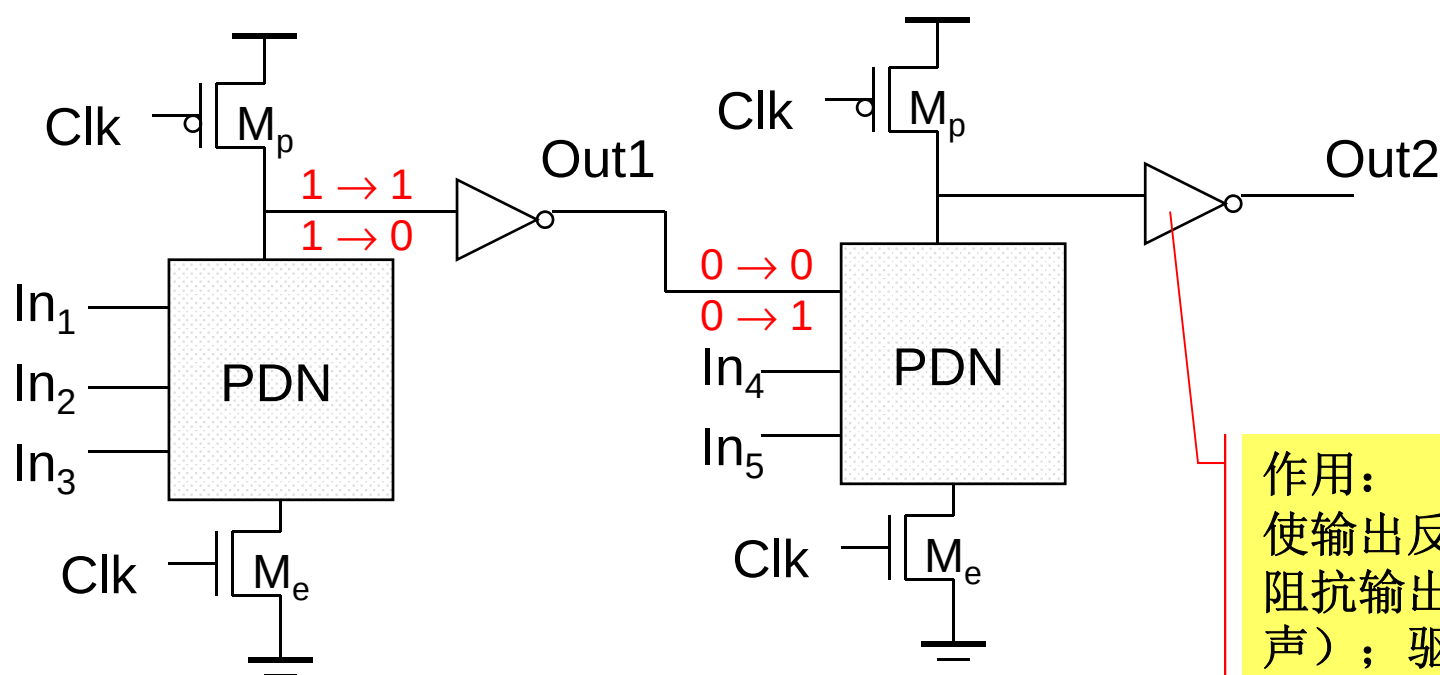
Setting all inputs of the second gate to 0 during precharge will fix it.

Correct operation is guaranteed (ignoring charge redistribution and leakage) as long as the inputs can only make a single 0 → 1 transition during the evaluation period.

adm, 2011/11/14

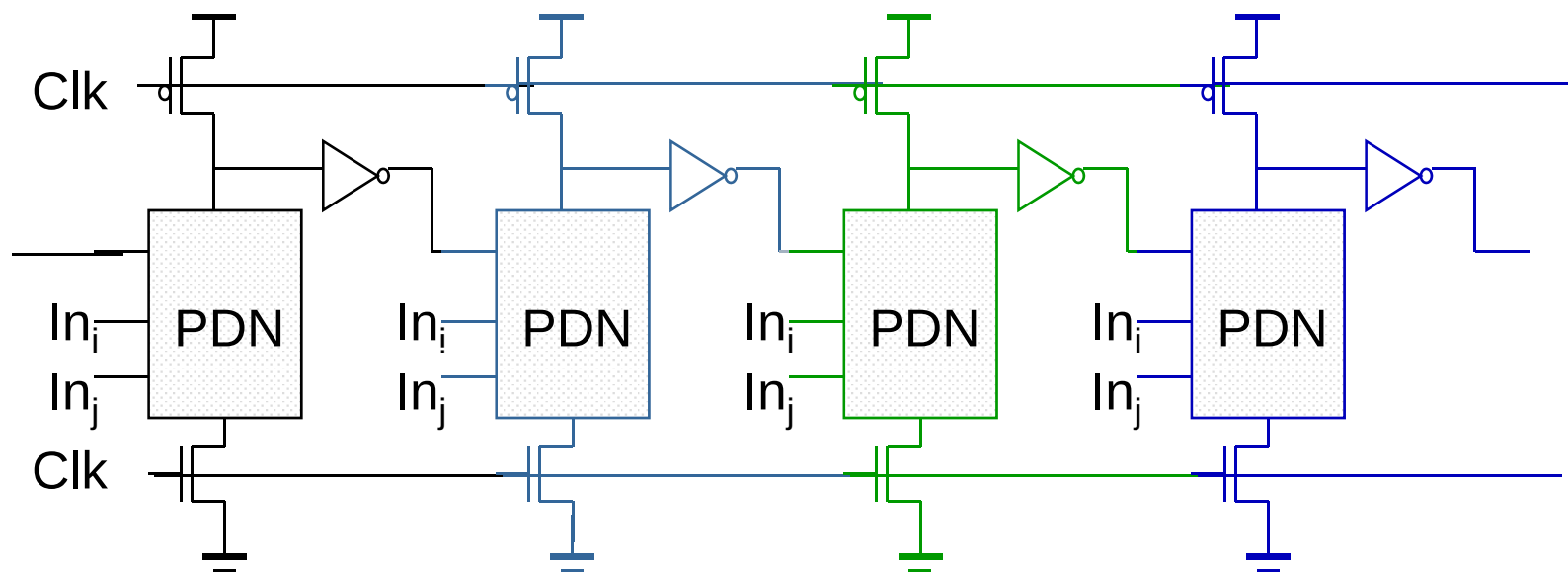
多米诺逻辑

定义：动态逻辑+ 反相静态逻辑



只能实现**非反相的逻辑**（所有的多米诺门均为非反相逻辑门。

标准的多米诺（*Domino*）逻辑

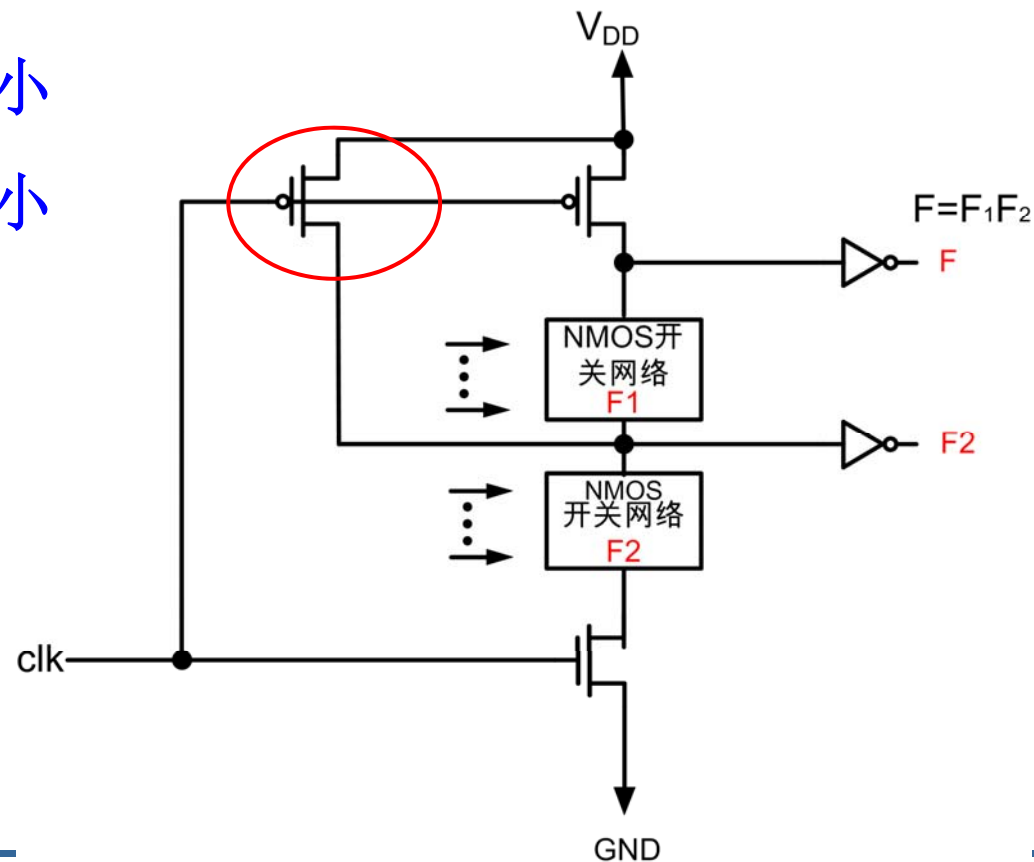


多米诺逻辑可以串联
逻辑求值的传播如同多米诺骨牌的倾倒。

多输出多米诺逻辑

MODL (Multiple-Output Domino)

面积小
扇出小



a20

幻灯片 16

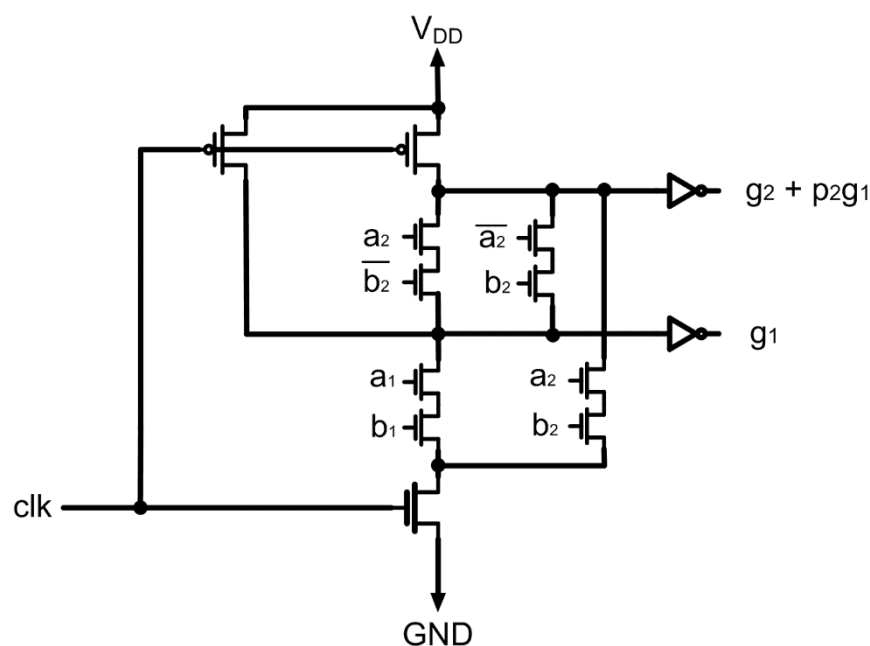
a20

减少了晶体管数目，但是预充管数目没有减少；输入管复用，减小了前级电路的扇出。

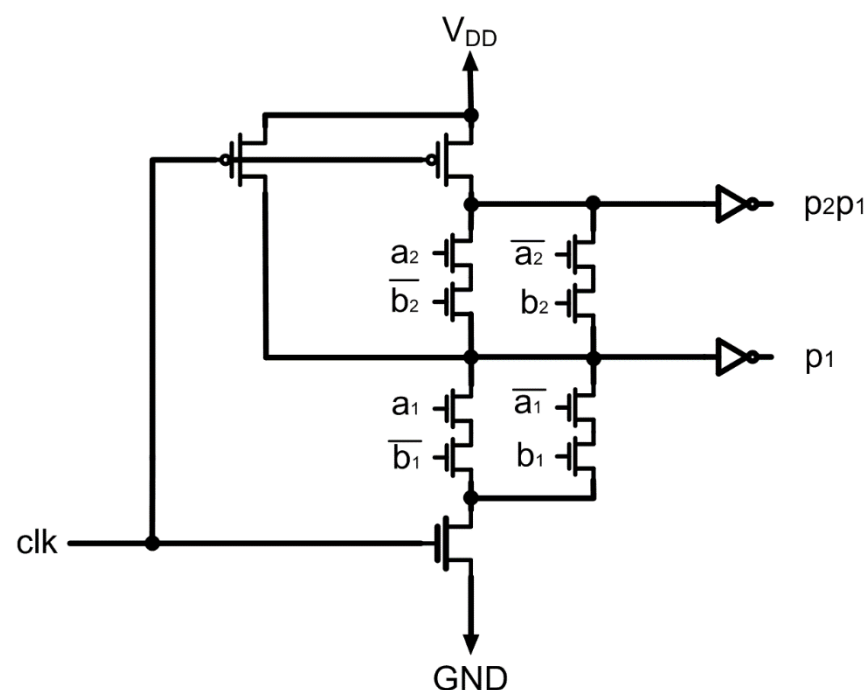
adm, 2011/11/14

多输出多米诺（MODL）的例子

超前进位加法器逻辑: $g_1 = a_1 b_1, g_2 = a_2 b_2, p_1 = a_1 \oplus b_1, p_2 = a_2 \oplus b_2$



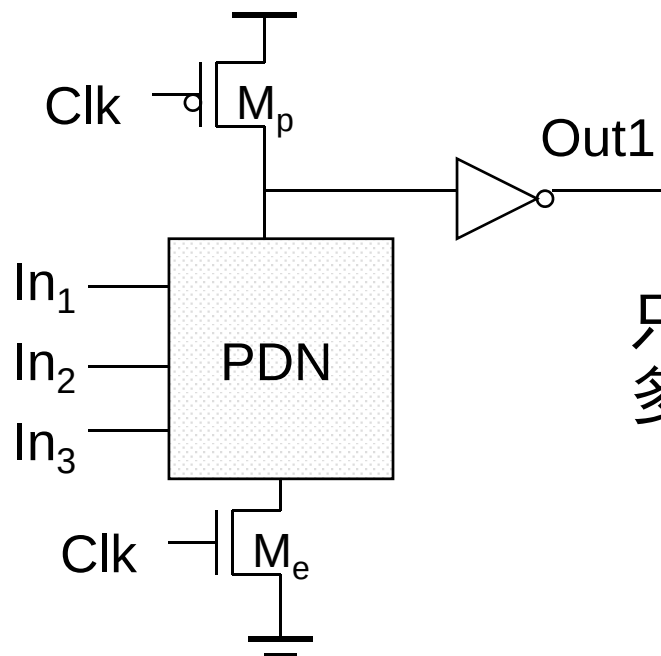
“进位产生”
信号电路



“进位传播”
信号电路

多米诺逻辑

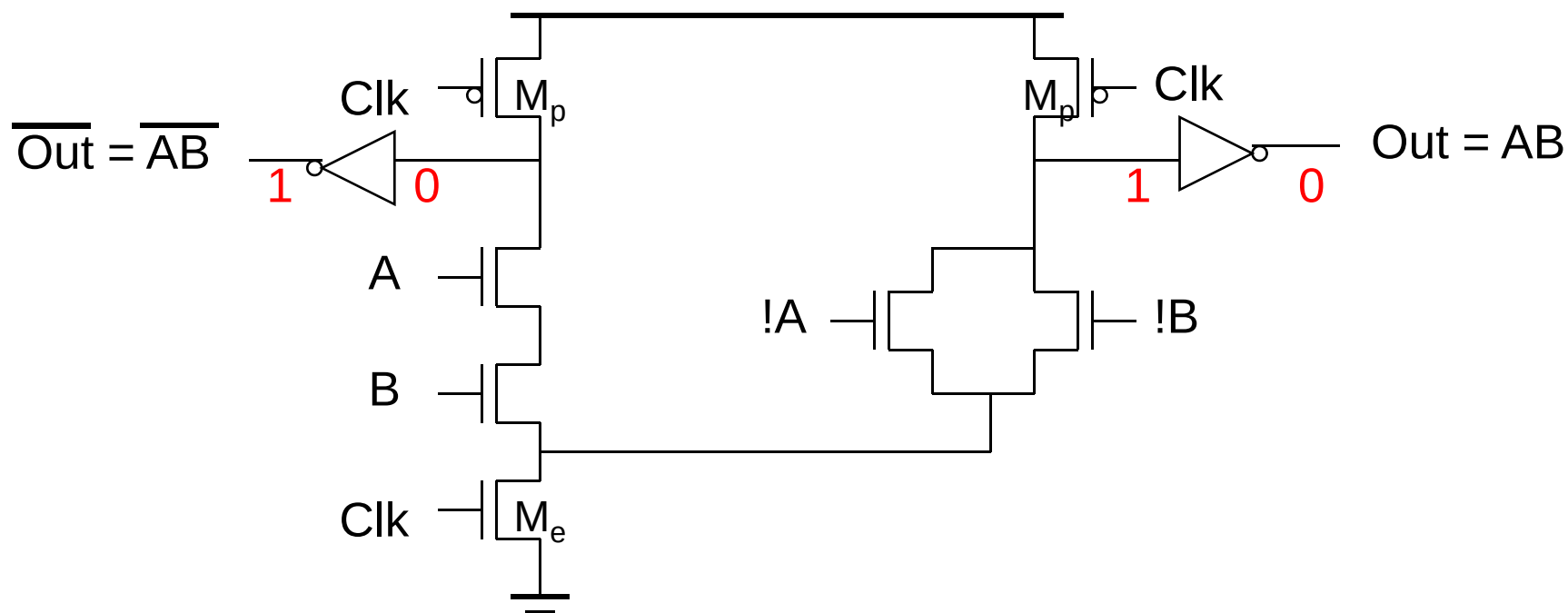
动态逻辑（反相逻辑）+ 反相静态逻辑



只能实现**非反相的逻辑**（所有的多米诺门均为非反相逻辑门。

差分 (双轨, Dual Rail) 多米诺

解决只能实现非反相逻辑的问题

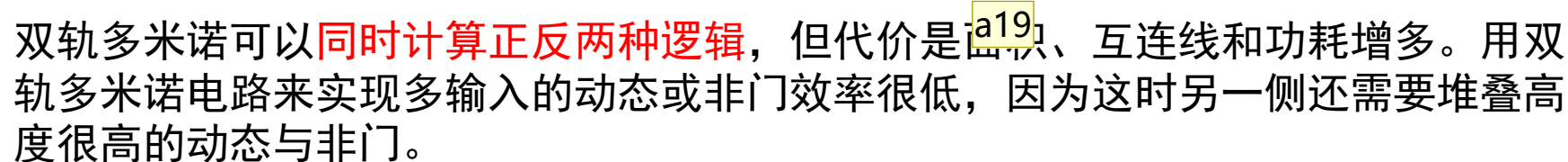


双轨多米诺可以同时计算正反两种逻辑。

双轨编码

A	! A	逻辑值
1	0	1
0	1	0
0	0	空闲 (idle)
1	1	无效 (invalid)

解决只能实现非反相逻辑的问题



a19

AND/NAND differential logic gate. The inputs and their complements come from other differential DR gates and thus all inputs are low during precharge and make a conditional transition from 0 to 1.

Annotations show state during evaluate cycle ($CLK = 1$)

Expensive - but can implement any arbitrary function.

Use significant power since they have a guaranteed transition every single clock cycle (regardless of signal statistics, since either Out or !Out will transition from 0 to 1).

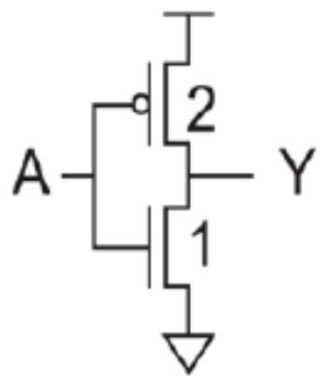
Not ratioed (even though have a cross-coupled PMOS pair)

adm, 2011/11/14

7.4.3 动态逻辑的速度

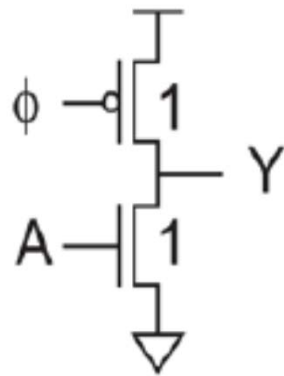
动态逻辑的逻辑努力

计算方法：只有下拉逻辑努力，其值等于与具有相同PDN的等效电阻的反相器的输入电容之比。



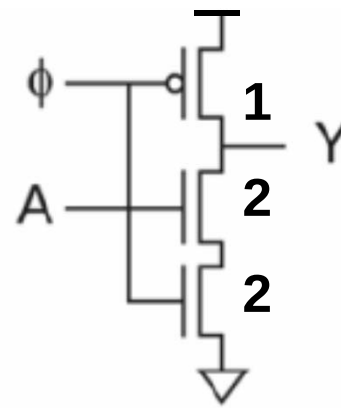
(a) 静态反相器

$$g_d = 1$$



(b) 无足动态反相器

$$g_d = 1/3$$



(c) 有足动态反相器

$$g_d = 2/3$$

动态逻辑的逻辑努力比静态CMOS的逻辑努力小；
有足逻辑的逻辑努力比无足逻辑门的逻辑努力大。

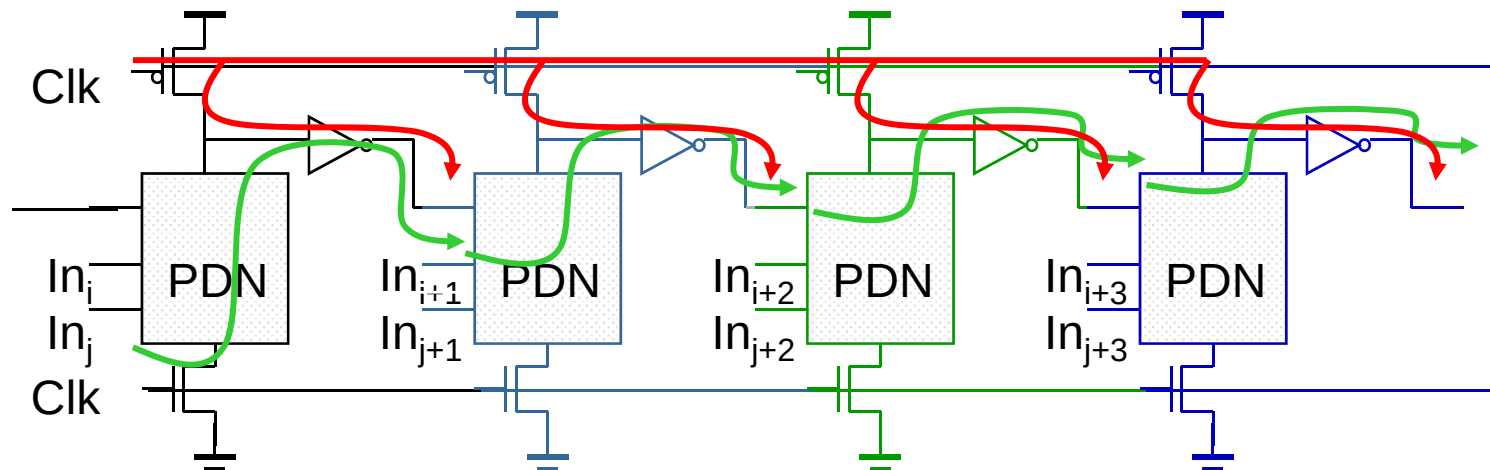
a17

动态逻辑开关速度快，因为（1）输入电容（ C_{in} ）小（作为负载被驱动时，对驱动器的负载电容小）（2）无短路电流 I_{sc} ，因此由PDN提供的电流均用来使CL放电（比伪nMOS快）

adm, 2013/11/11

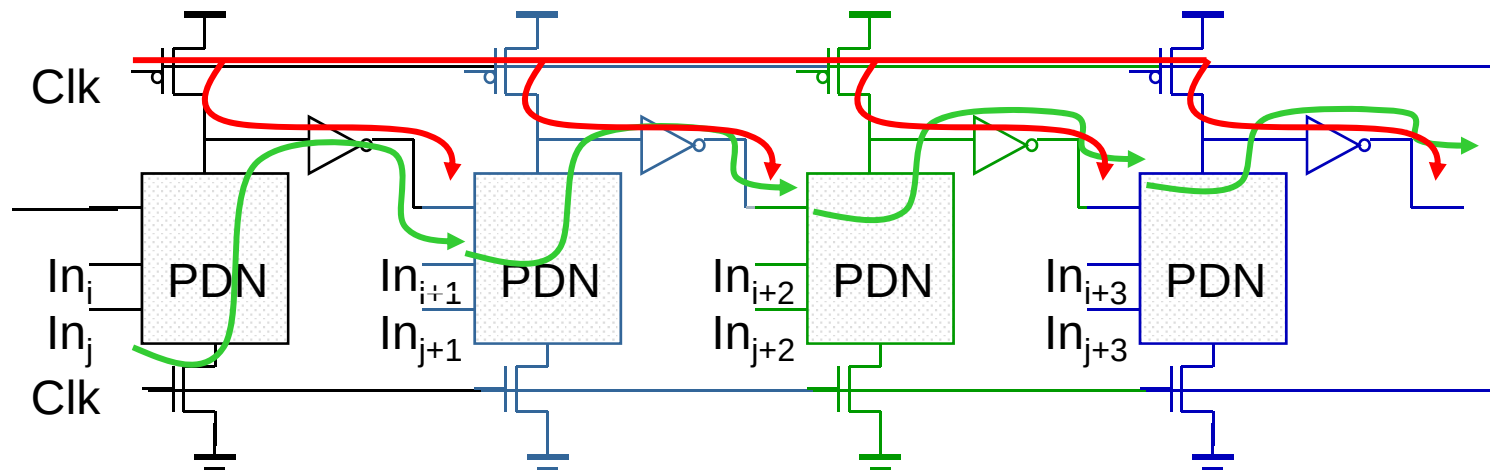
速度

- ❖ 预充电是同时并行完成的（只需要两级门的延时）；求值过程是逐级完成的。
- ❖ 求值速度主要由PDN决定，预充电时间主要由PMOS决定；

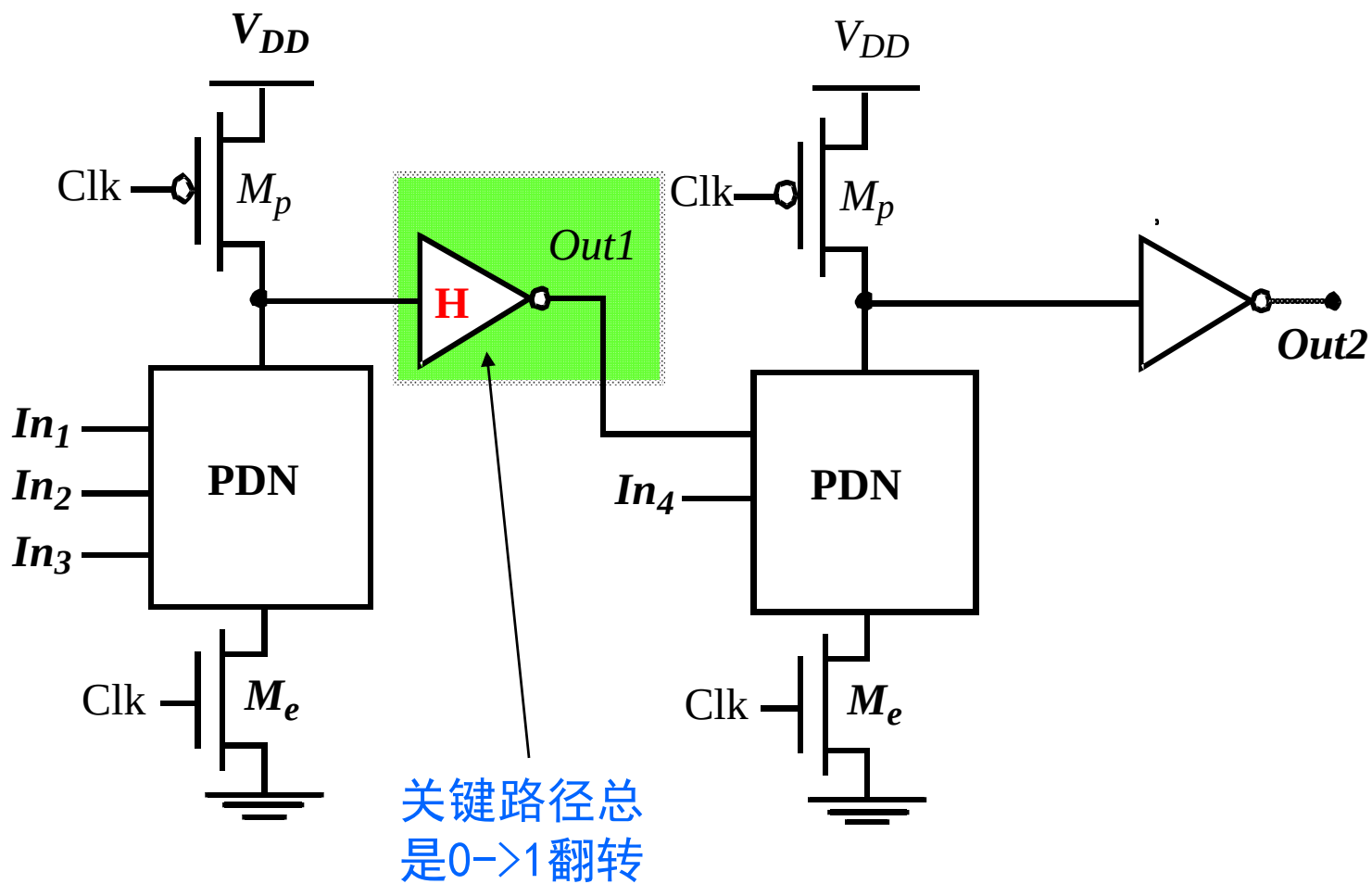


速度

- ❖ 求值时间等于各级门求值时间之和。
- ❖ 预充（求值）脉冲宽度要大于输出上升（下降）的最长时间。——时钟频率上限



多米诺逻辑设计



7.4.4 动态逻辑的功耗

功耗

优点

- 实际电容小——晶体管少，每个扇出的负载只有一个晶体管
- 无毛刺

缺点

- 时钟功耗

动态电路的开关活动性

动态逻辑门当输出为0时，在下一个周期就需要进行充电，因此开关活动性=输出为0的概率(p_0)。

静态电路的开关活动性：

$$\alpha_{0 \rightarrow 1sta} = p_0 p_1$$

动态电路的开关活动性：

$$\alpha_{0 \rightarrow 1dyn} = p_0$$

功耗

优点

- 实际电容小——晶体管少，每个扇出的负载只有一个晶体管
- 无毛刺

缺点

- 时钟功耗
- 开关活动性高
- 当增加抗漏电器件时可能会有短路电流（见后面部分）

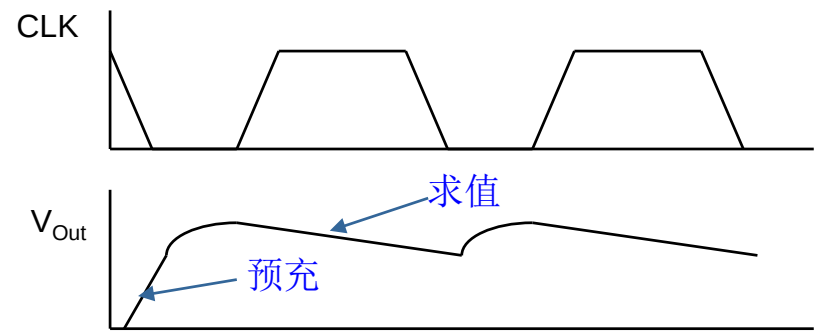
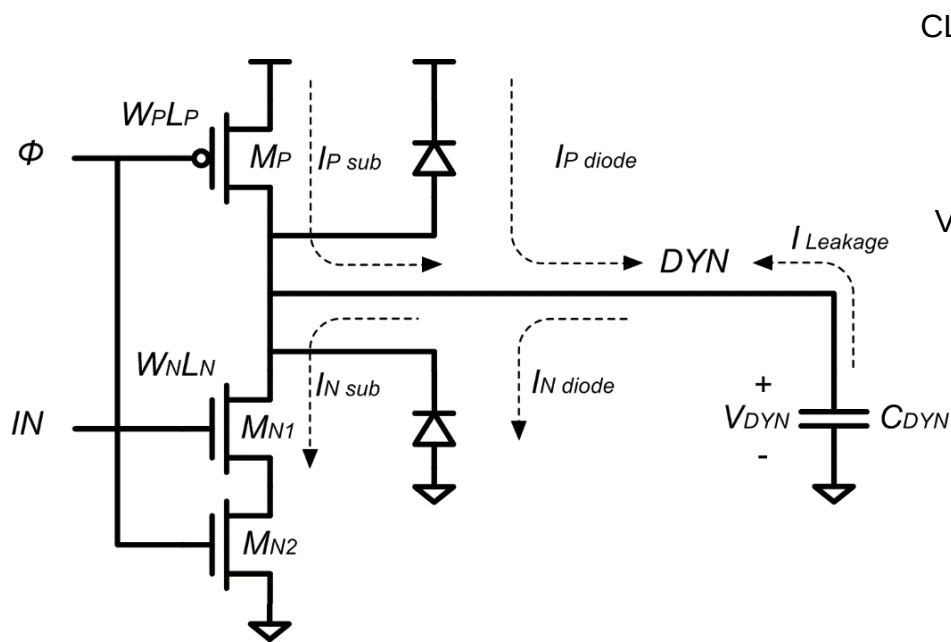
总体上，动态电路的功耗高于静态互补CMOS逻辑。

动态逻辑的动态特点

- ✘ 要求一系列的周期性预充和求值，纯粹的静态分析不再适用
 - 输入只允许在预充电阶段变化，在求值阶段必须保持稳定
 - 输入最多只能有一次低至高的过渡
- ✘ 开关速度快
- ✘ 需要预充电/求值时钟，时钟是一个全局信号，往往有很大的负载。
- ✘ 只有动态功耗，无静态功耗，总功耗通常高于静态**CMOS**

7.4.5.1 电荷泄漏

电荷泄漏

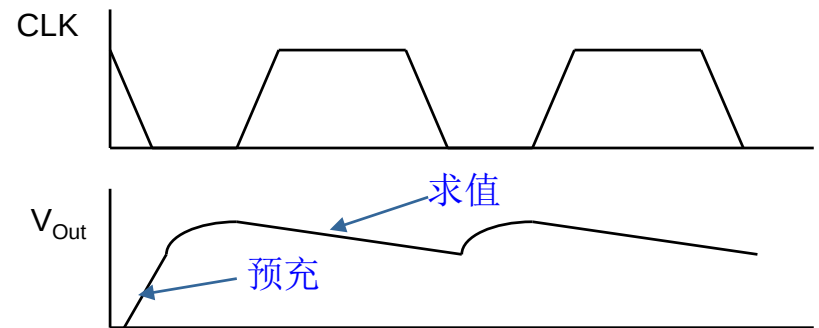
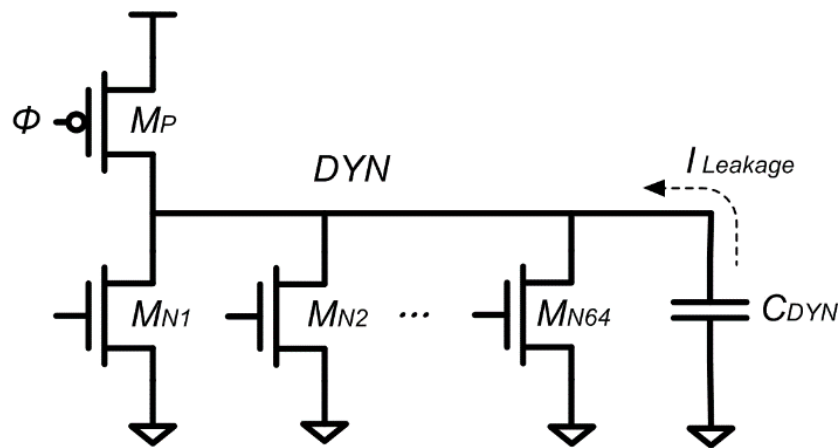


当 $I_{Leak} > 0$ 时就存在电荷泄漏

$$I_{Leak} = (I_{N\ sub} + I_{N\ diode}) - (I_{P\ sub} + I_{P\ diode})$$

I_{sub} ——亚阈值漏电, I_{diode} ——pn结漏电

电荷泄漏



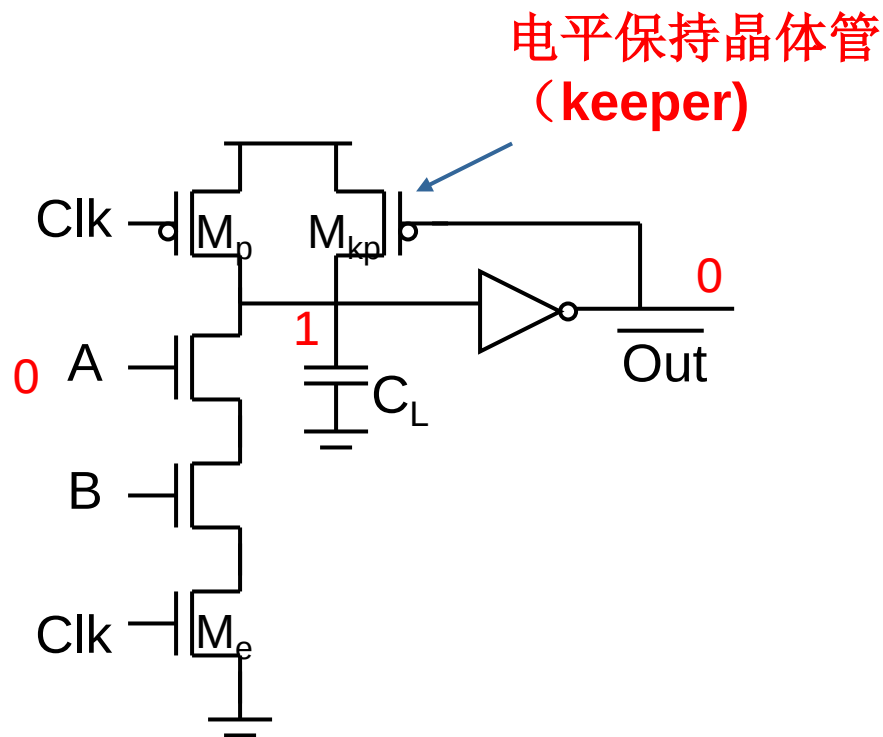
I_{Leak} 与扇入数有关

动态节点电荷泄漏掉的时间:

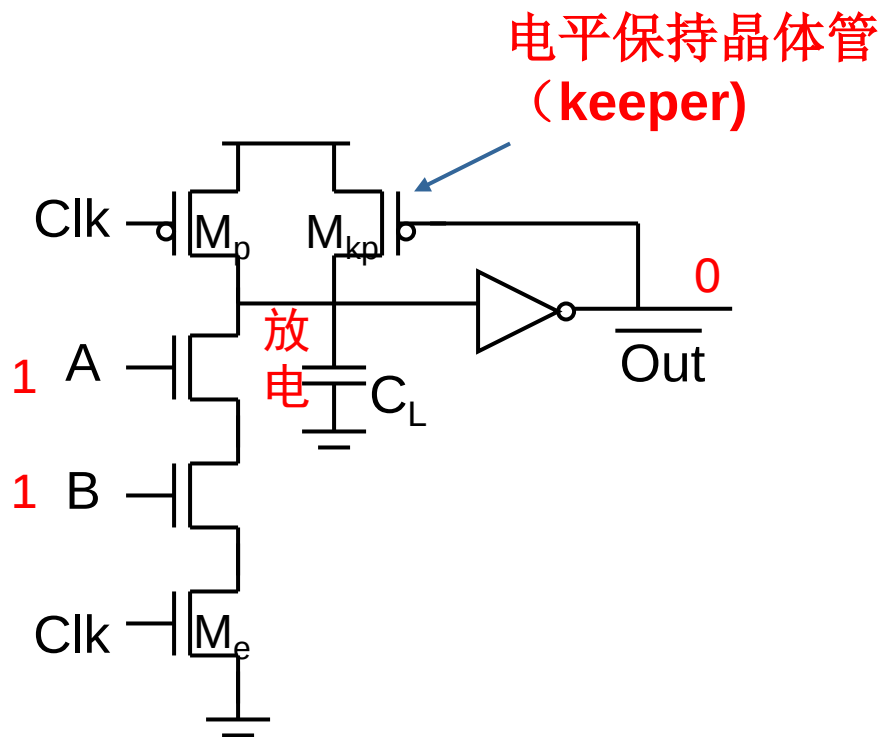
$$t_{sw} = (C_{DYN} \cdot V_{SW}) / I_{Leak}$$

电荷泄露事件决定了动态逻辑的最低时钟频率。

电荷泄漏的解决办法

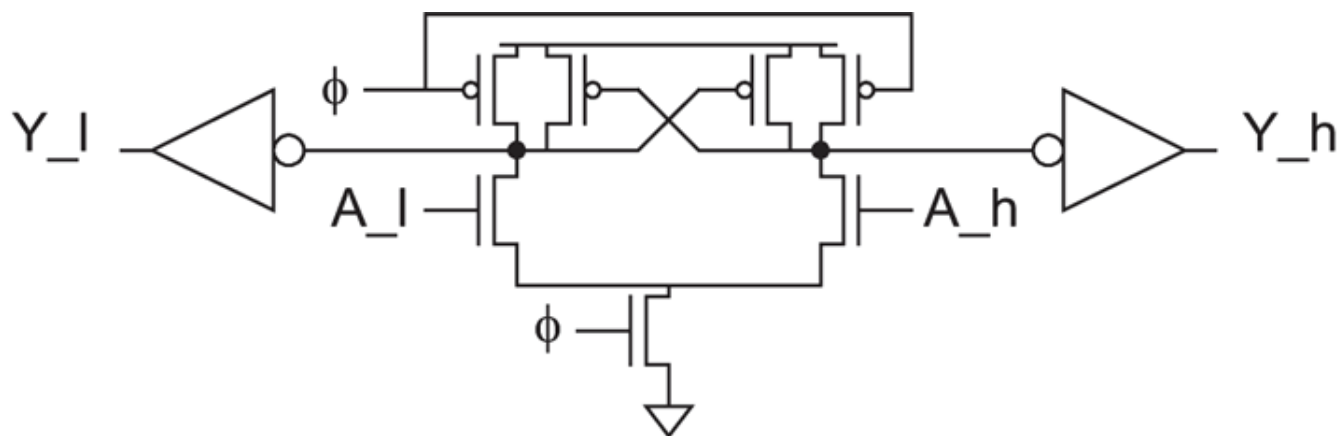


电荷泄漏的解决办法



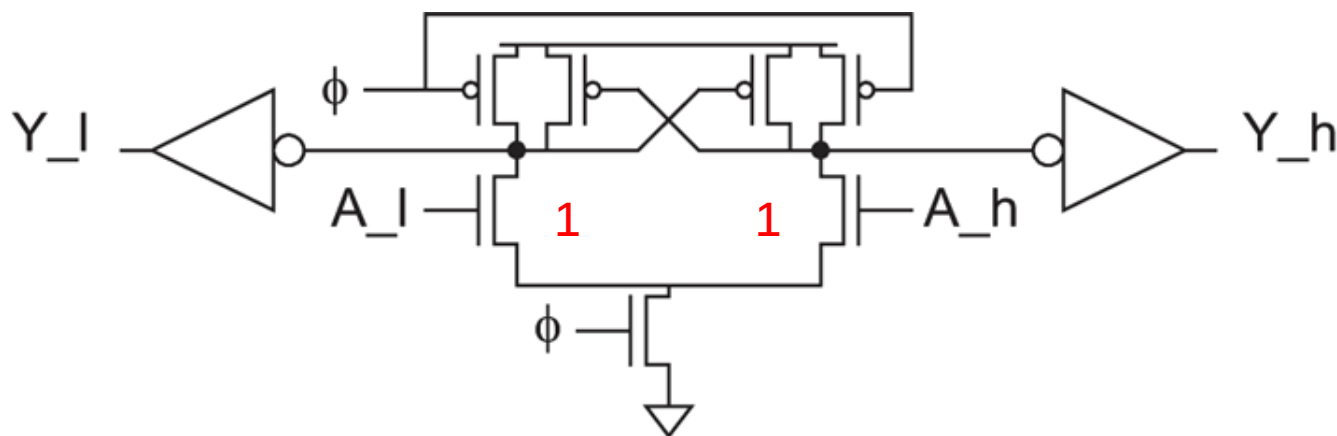
keeper管需要非常弱（PDN的1/10），
，否则会增加值阶段的延时。

差分电平保持器



✚ 用于差分电路（如差分多米诺逻辑）

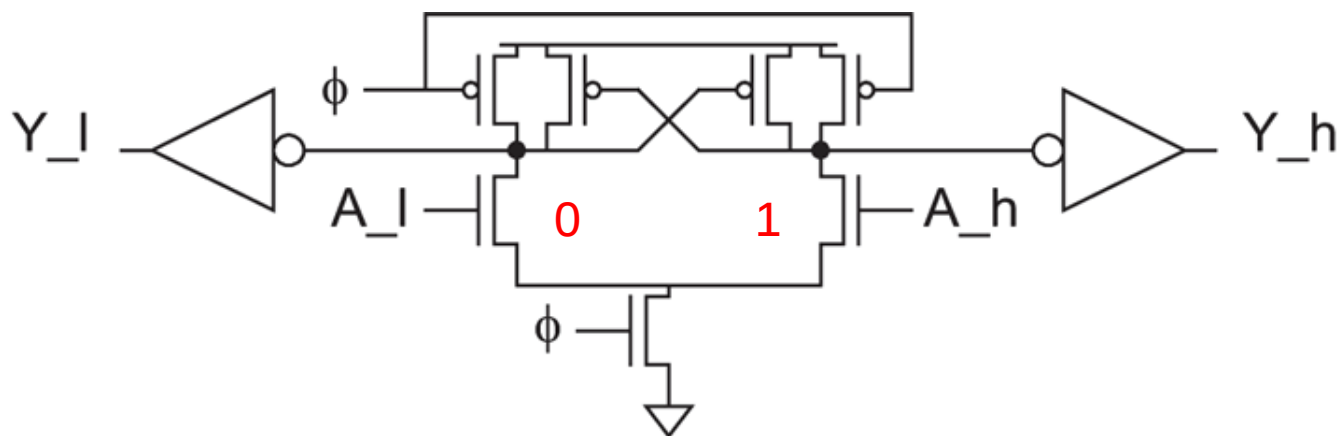
差分电平保持器



预充

❏ 用于差分电路（如差分多米诺逻辑）

差分电平保持器

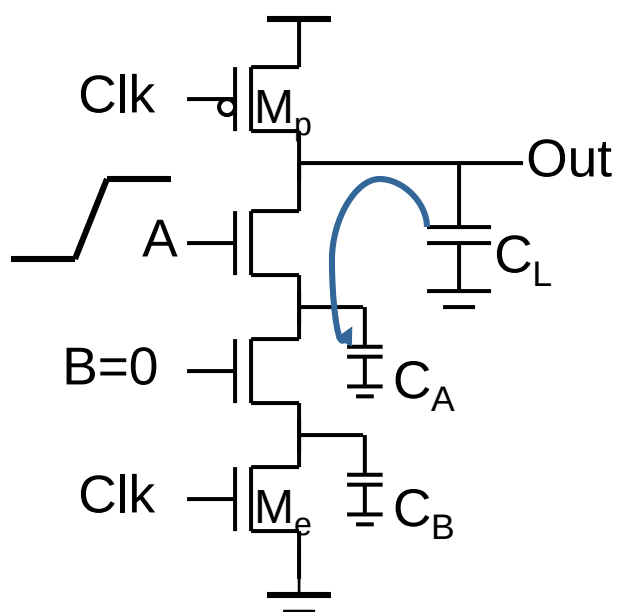


求值

上拉的一侧不会阻碍另一侧的下降，因为预充之后两个电平保持管都是关闭状态，所以采用差分保持器的电路的速度比采用单端保持器的更快。

7.4.5.2 电荷共享

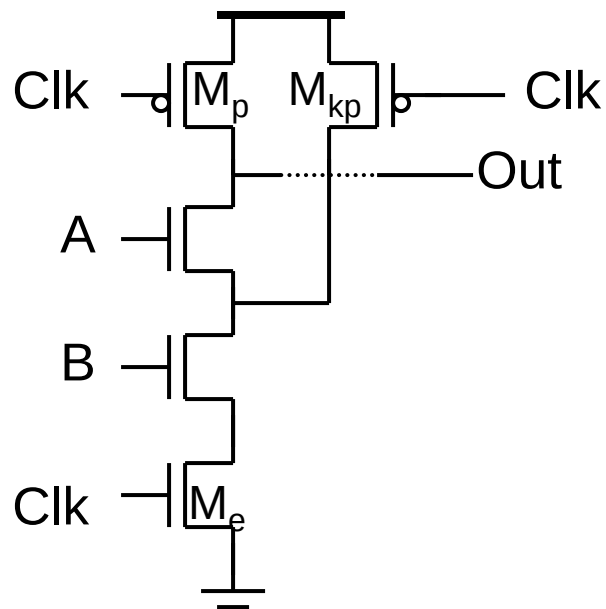
电荷共享



C_A 初始状态无电荷, C_L 充满电荷

原先存放在 C_L 上的电荷由 C_L 和 C_A 重新分布（分享），导致鲁棒性降低。

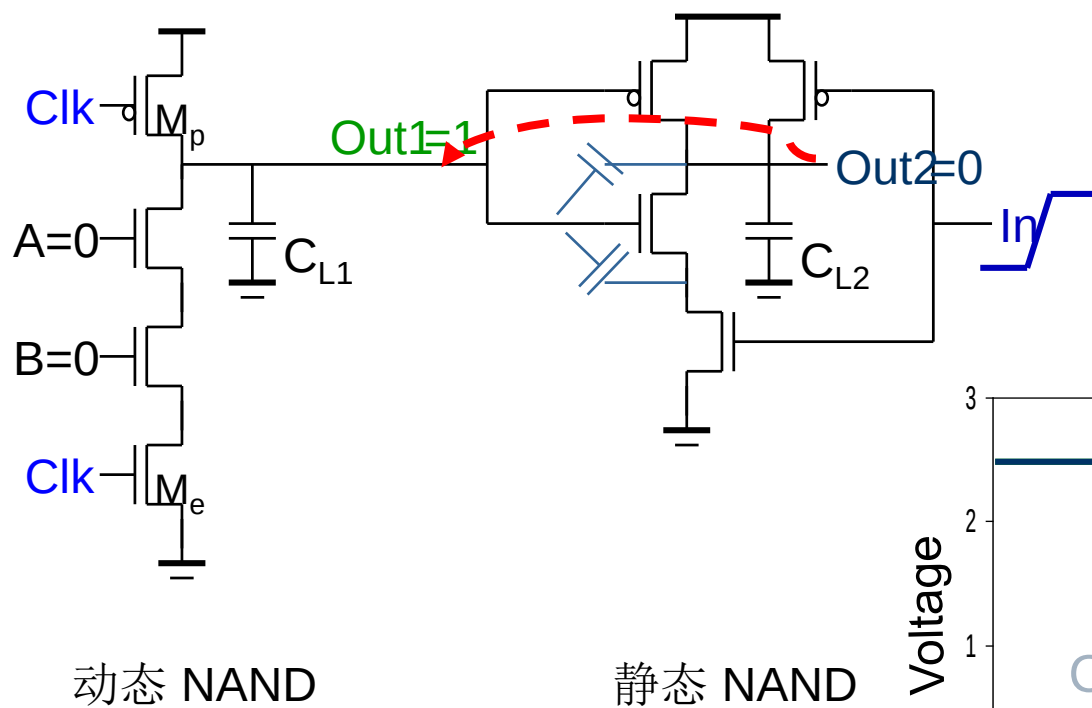
电荷再分配的解决方法



采用时钟驱动的晶体管预充电内部节点
(代价: 增加了面积与功耗)

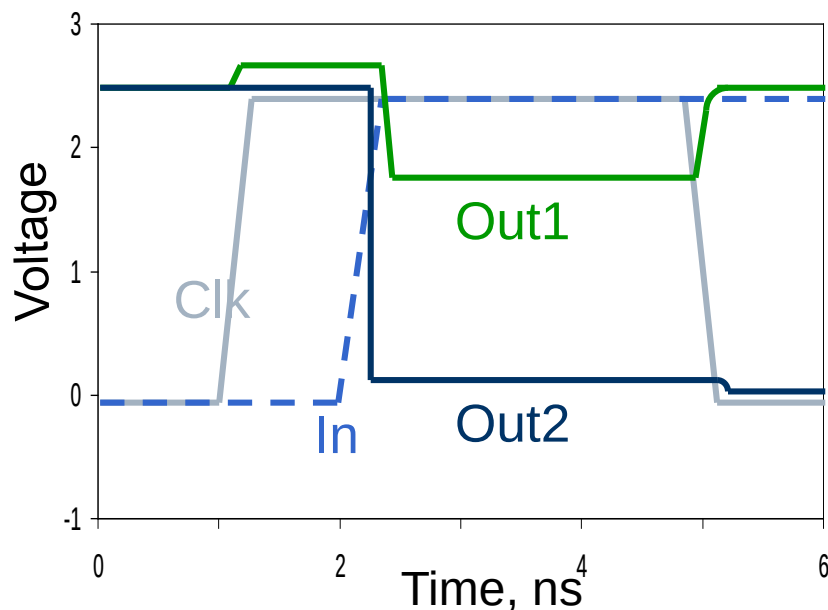
7.4.5.3 电容耦合

回栅电容耦合

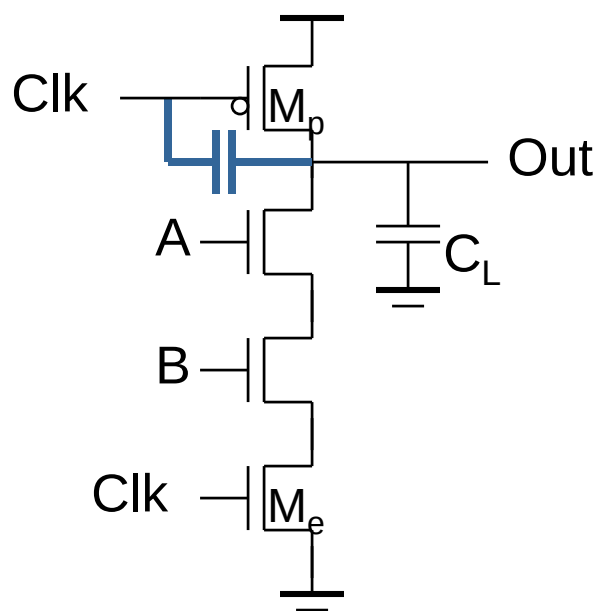


解决方法:

动态门驱动静态门时, 应当驱动静态门中靠近电源/地的管子。

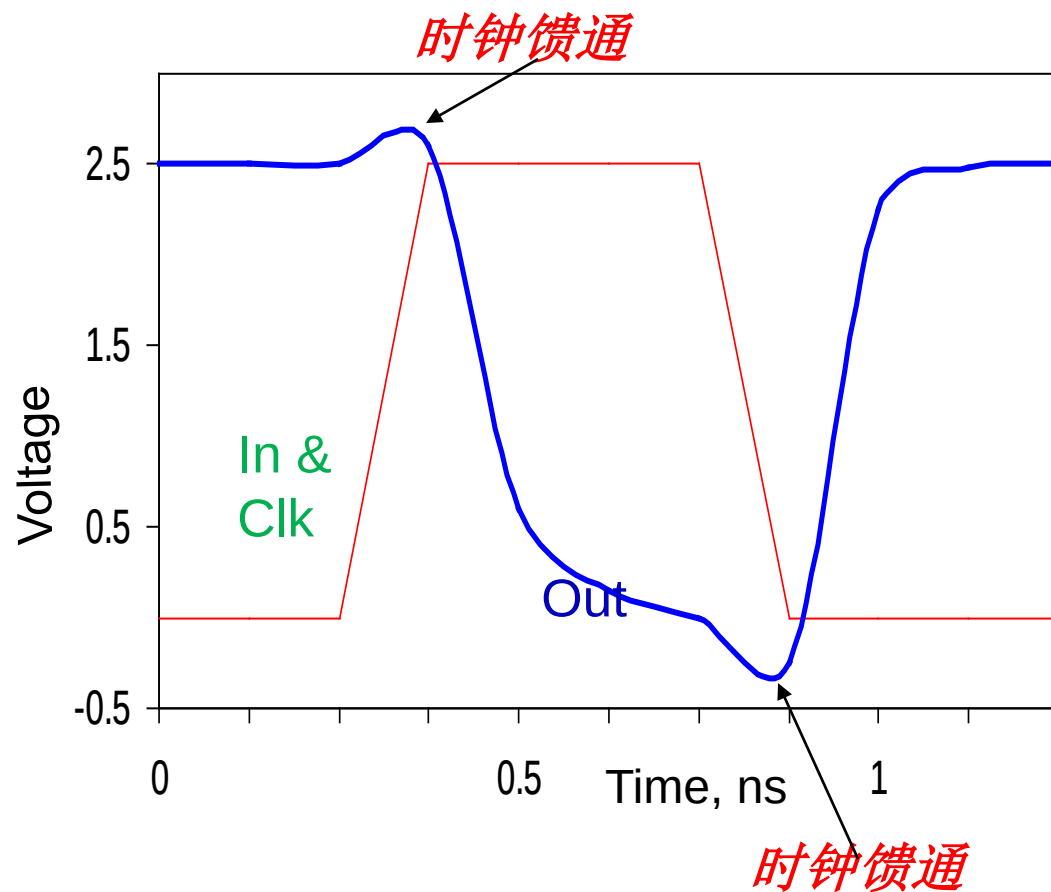
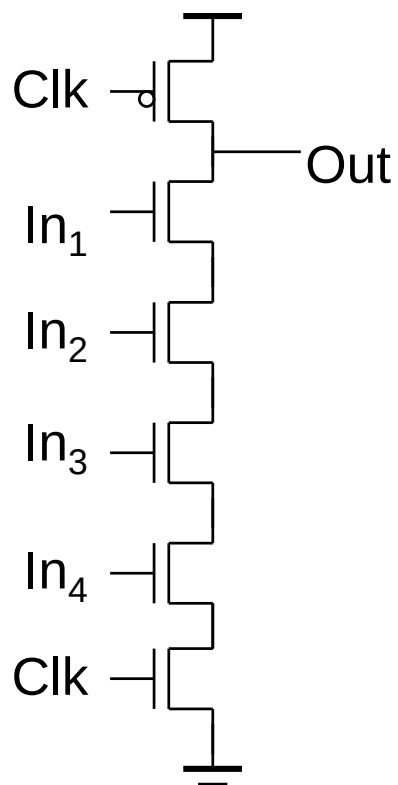


时钟馈通(Clock Feedthrough)



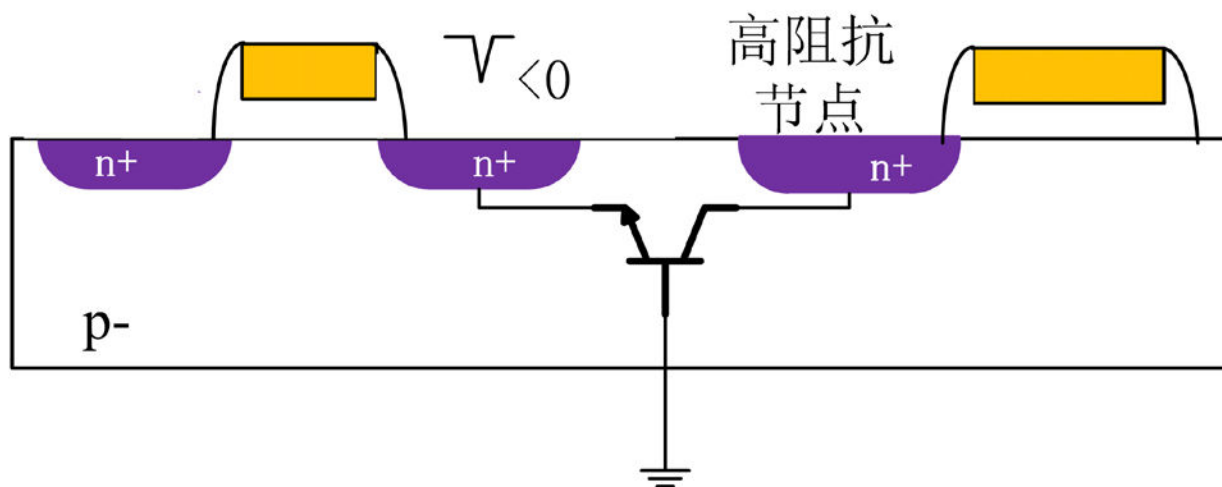
时钟的快速上升（和下降沿）耦合至 **Out**. 输出电压可能上升至 V_{DD} 以上，或下降至 GND 以下。

时钟馈通



时钟馈通

- 时钟馈通可能引起反偏结二极管变为正偏，引起电子注入衬底，



它们可能被附近的高电平高阻抗节点所收集，产生错误。

时钟馈通会对相邻的其它逻辑门的动态节点产生干扰。

时钟馈通的避免措施

1. 版图动态节点远离其它预充/求值管
2. 在动态节点周围放置保护环

其它效应的影响

动态节点易受干扰，无法恢复

- 电容耦合（导线间耦合）
- 衬底耦合
- 其它原因引起的少子电荷注入
 - 如，10单元的翻转
- 电源噪声

上述效应都会改变动态节点的电荷。

解决办法：

- 增大动态节点的电容；增加电平保持器；远离干扰源或增加保护、屏蔽措施。

动态逻辑的特点

- (1) 全摆幅无比逻辑
- (2) 面积小
- (3) 速度高（寄生电容小、无竞争）
- (4) 设计仔细时可以无毛刺
- (5) 只有动态功耗，无静态功耗,总功耗通常高于静态**CMOS**
- (6) 信号完整性差，易受干扰,设计和模拟验证必须全面、周到、仔细

7.5 组合逻辑类型

如何选择合适的逻辑类型

静态CMOS

- 对绝大多数CMOS电路来说是最好的选择。
- 抗噪声能力（noise immunity）强，不需要电平恢复电路。
- 没有静态功耗。
- 速度较快。
- 有高度自动化的综合工具、布线工具和单元库。
- 采用扇入大的逻辑门实现给定逻辑时功耗较低，但扇入大的逻辑门有较大的逻辑努力，因此当速度是主要考虑时，最好把它分成多级的简单门来实现。
- 尺寸缩小潜力大，易于在可靠性和功耗之间综合考虑（trade-off）。

如何选择合适的逻辑类型（续）

伪NMOS

- 消除了PMOS 逻辑，因此扇入大的布尔函数（特别是NOR 门）宜用伪NMOS 实现。例如：ROM，可编程逻辑阵列（**PLA**）。
- 静态功耗需要认真考虑。

如何选择合适的逻辑类型（续）



多米诺逻辑

- 消除了PMOS 逻辑，是实现高性能（高速）微处理器的一种选择。
- 噪声容限小。
- 需要时钟，开关活动性大，导致动态功耗大。
- 设计要求高，当没有从版图反标注、进行全面模拟的工具时，一般不考虑采用。
- 纳米尺度下，由于电源电压降低、节点电容下降使得节点电荷量减少，同时信号完整性问题加重，使得动态逻辑的弱点更加突出。
- 时钟频率受到限制

如何选择合适的逻辑类型（续）

传输管

- 综合起来看，最好的传输管电路是CPL。适于实现XOR、MUX、全加器。

其它电路系列

- 在特定的应用场合具有优势或潜力。
- 必须考虑这些电路的缺陷和局限性。

总结

- 🔥 静态互补CMOS：主流、自动化程度高
 - 对偶的PDN和PUN，任何时候只有其中之一导通
- 🔥 有比逻辑：
 - 节省晶体管，适于大扇入门
 - 静态功耗问题
- 🔥 传输门逻辑：适于特殊功能
 - 电路简单，要避免长串联
 - 阈值损失和电平恢复
- 🔥 动态逻辑：高性能、设计困难
 - 电荷存储在电容节点上来保存状态
 - 速度快，鲁棒性差（信号完整性问题）
 - 时钟问题：驱动、频率范围

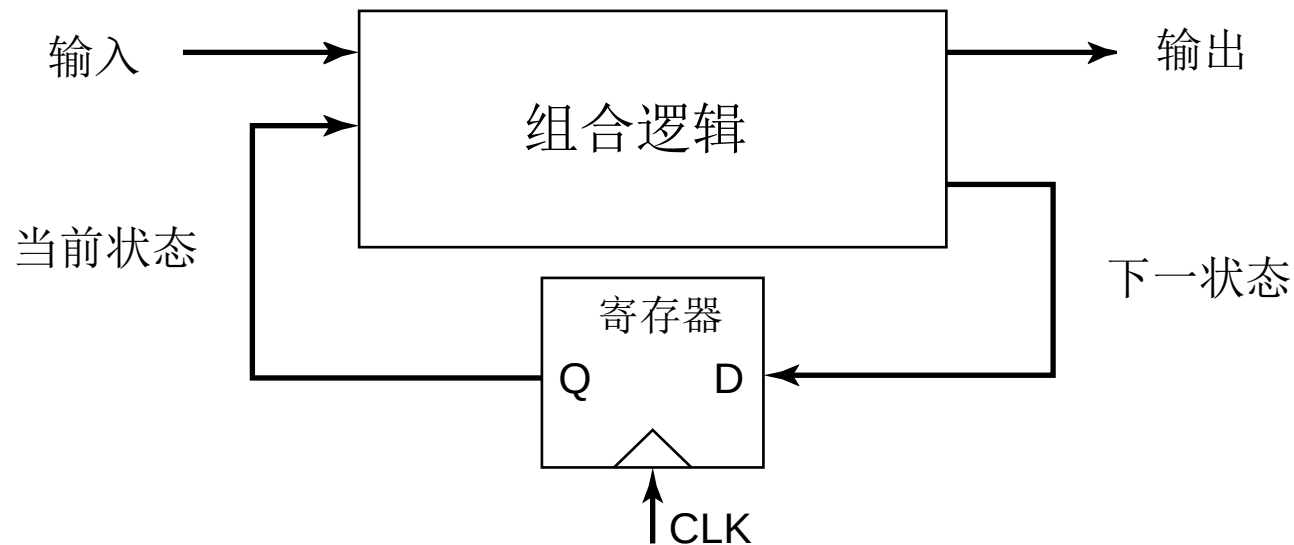
第八章 时序逻辑

8.1 时序逻辑和时序单元

时序逻辑的定义

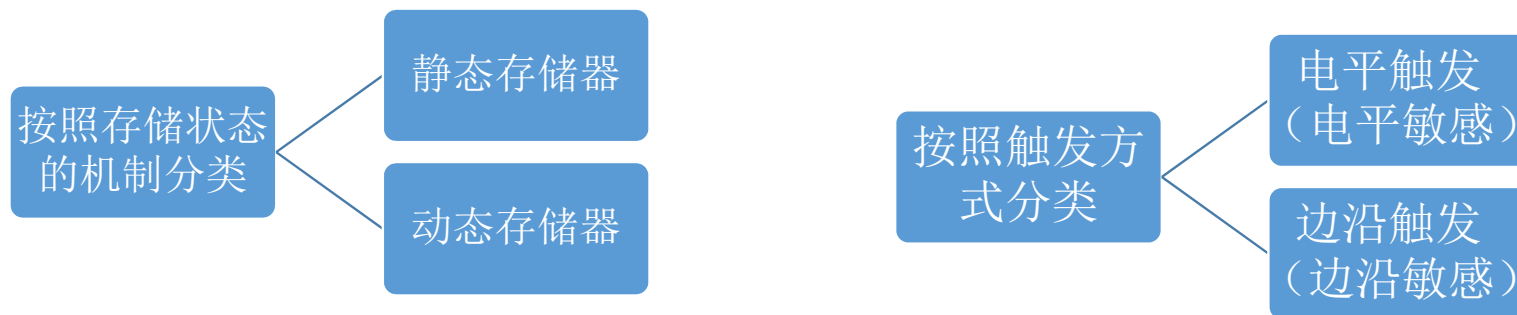
- 时序逻辑电路:

- 具有记忆功能的电路。
- 输出不仅取决于当前的输入还取决于原先的输入值。
- 输出是当前输入和当前状态^{状态}的函数。



时序单元（存储单元）

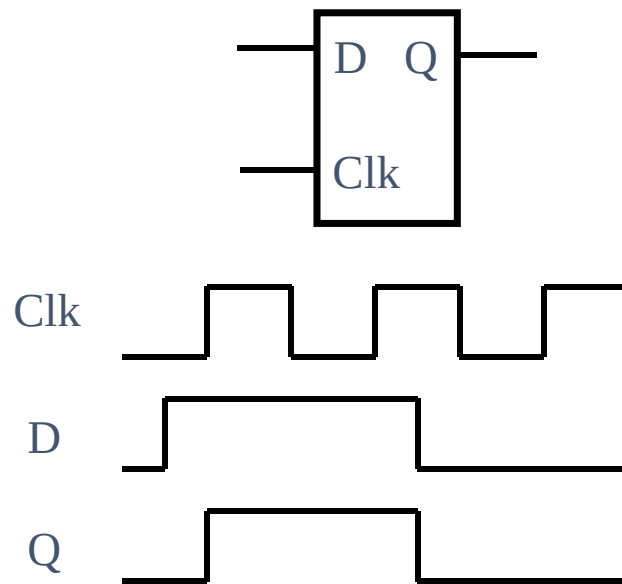
- 时序单元：时序电路中用来存储状态的电路单元。
- 要求：能够保持状态，状态能够被写入（触发）
- 分类



锁存器和寄存器

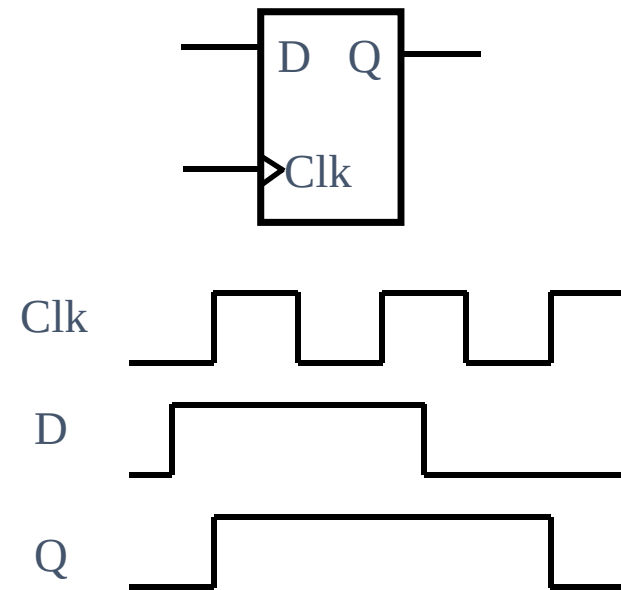
□ 锁存器 (Latch)

时钟为高时写入数据



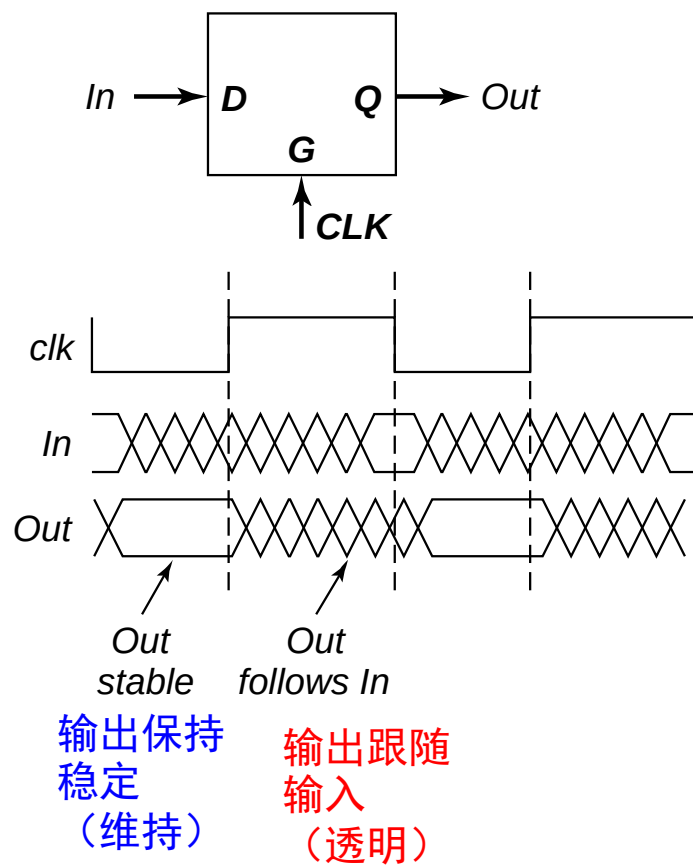
▣ 寄存器

时钟上升时存储数据

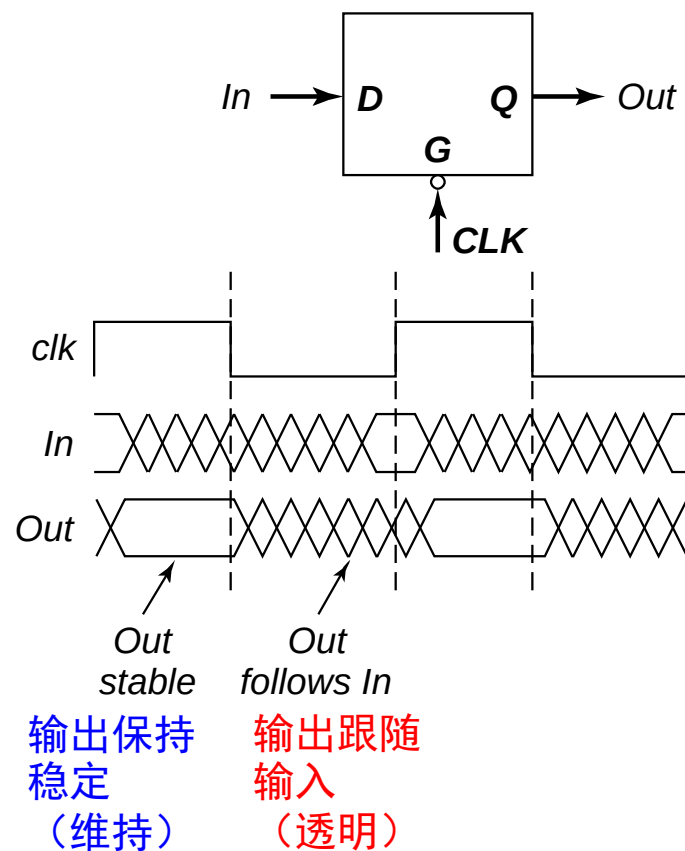


锁存器

正电平锁存器

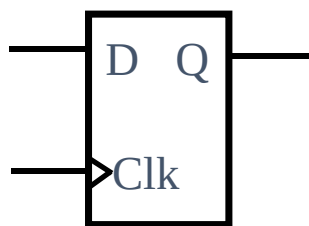


负电平锁存器

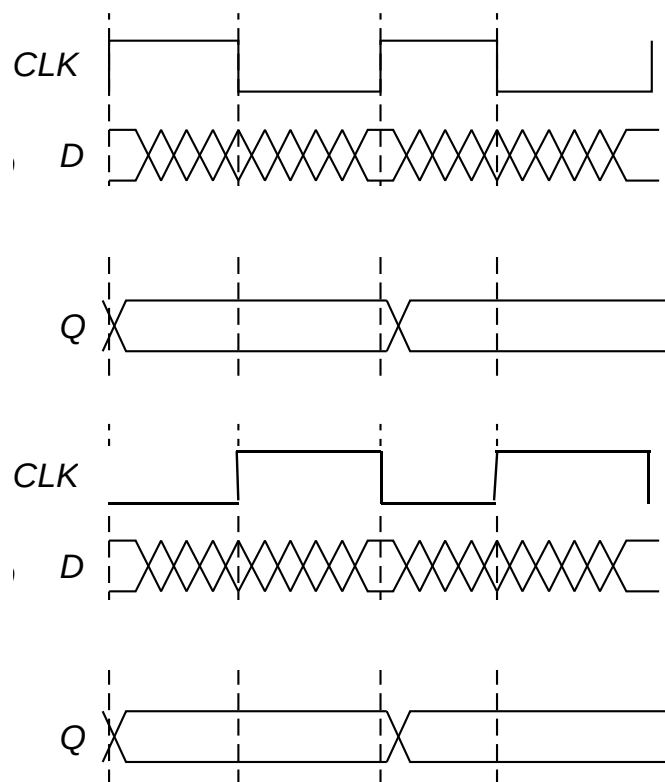
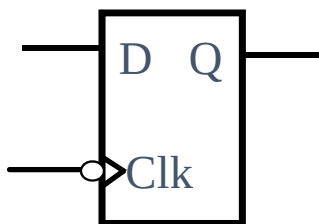


寄存器（边沿触发器）

- 上升沿触发的寄存器



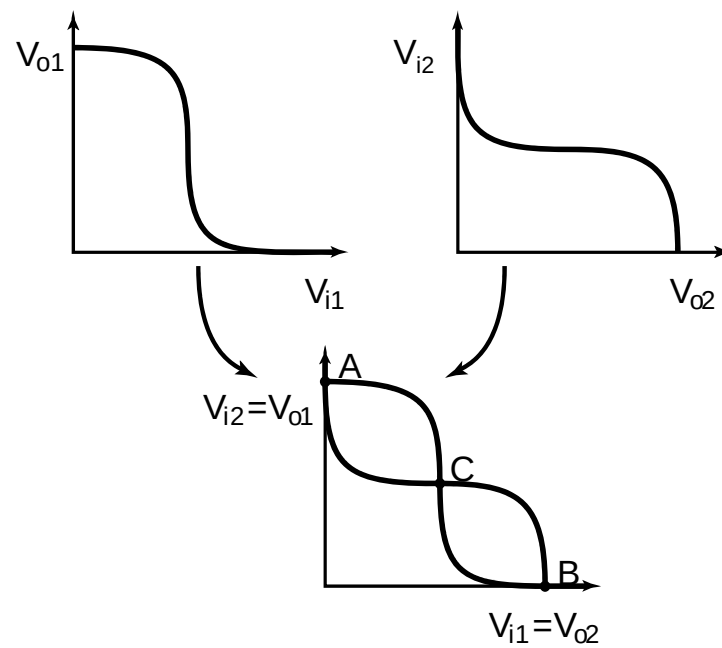
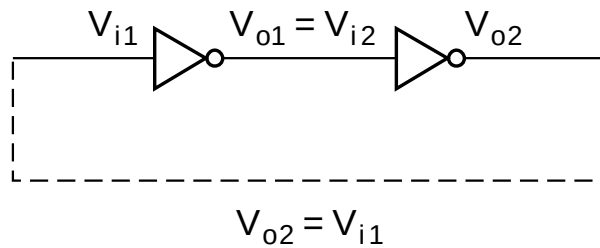
- 下降沿触发的寄存器



8.2 静态时序单元

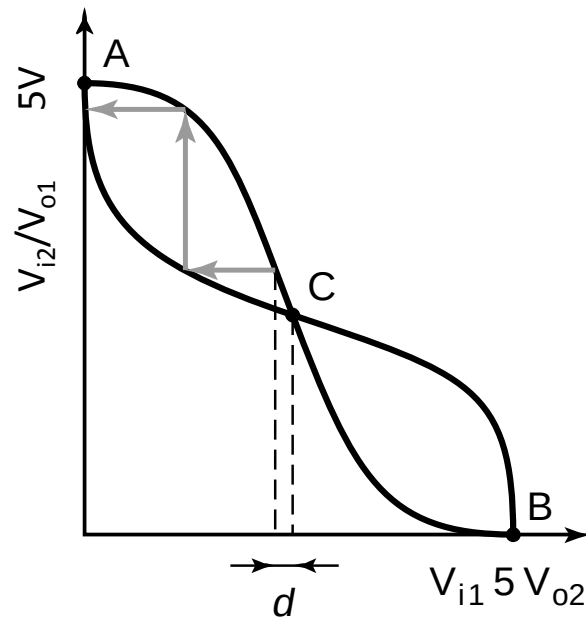
8.2.1 双稳态原理

正反馈：双稳态

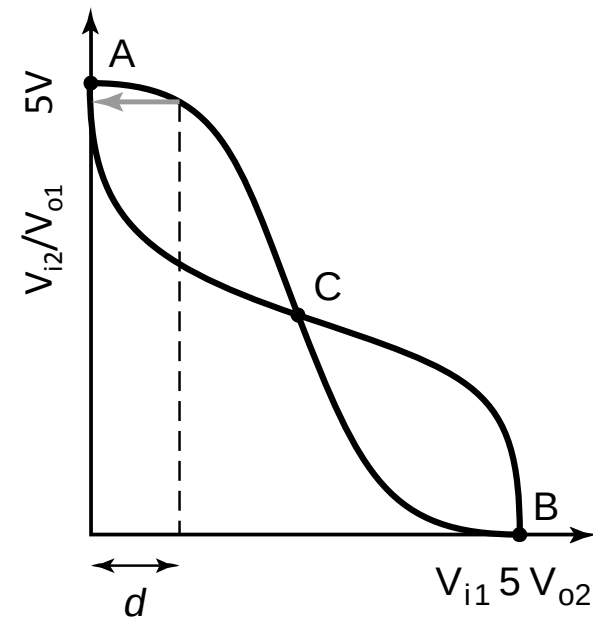


稳态和亚稳态

C点，偏移被放大



A点，偏移被减小直至消失



A、B为稳态工作点，C为亚稳态点（Meta-Stability）

双稳态锁存

特点：

- (1) 静态：信号可以“无限”保持
- (2) 鲁棒性好：对扰动不敏感
- (3) 对触发脉冲宽度的要求：
触发脉冲的宽度须稍大于沿环路总的传播时间，也即这两个反相器平均延时的两倍
- (4) 尺寸大

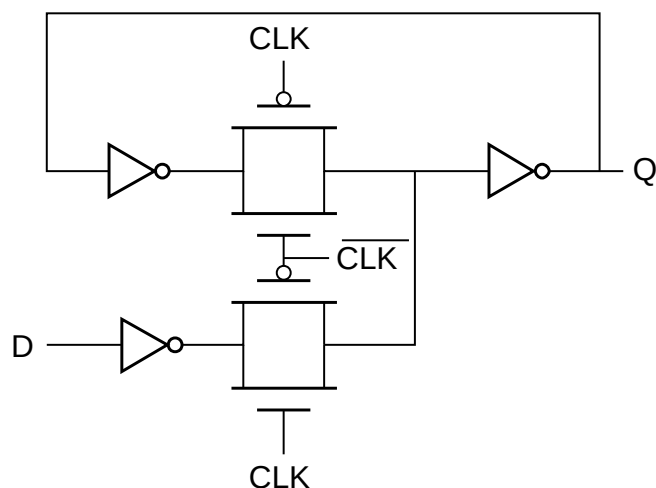
触发翻转的方法:

- (1) 切断反馈环 (2) 强制驱动

8.2 静态时序单元

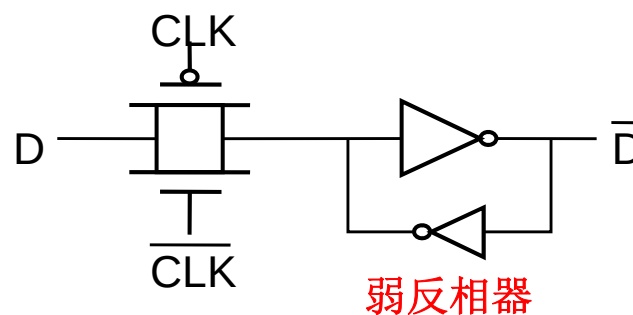
8.2.2 锁存器

静态锁存器的电路



多路开关锁存器

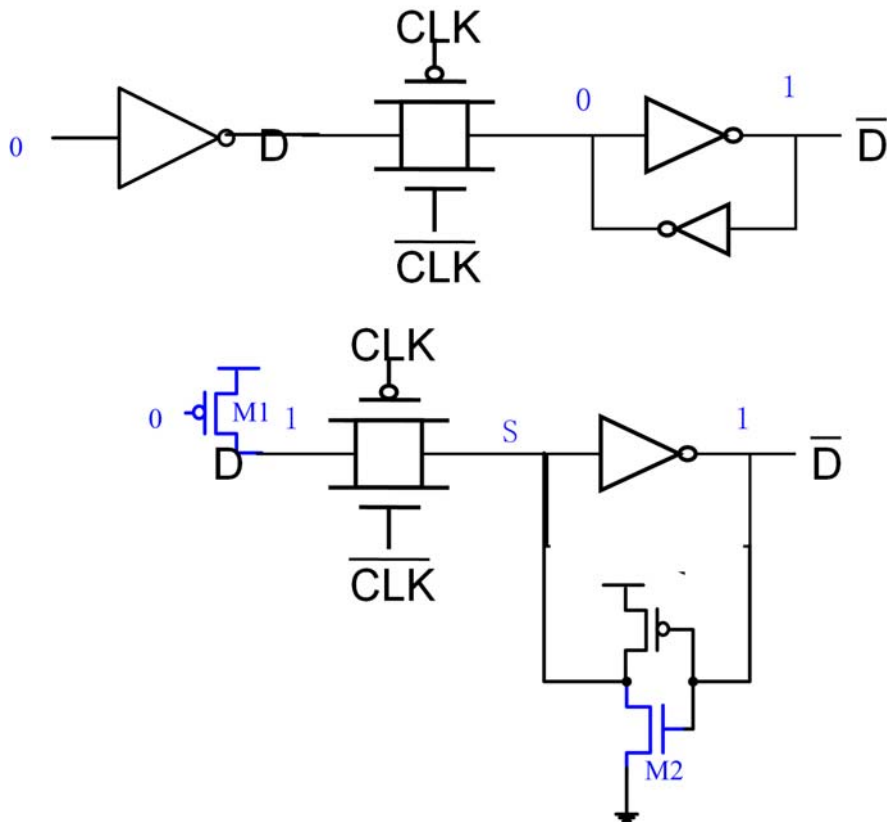
- 晶体管尺寸容易设计
- 晶体管数目多（时钟负载和功耗大）



强制写入锁存器

- 晶体管尺寸需要精心设计
- 晶体管数目少（时钟负载小）

强制写入锁存器的尺寸设计



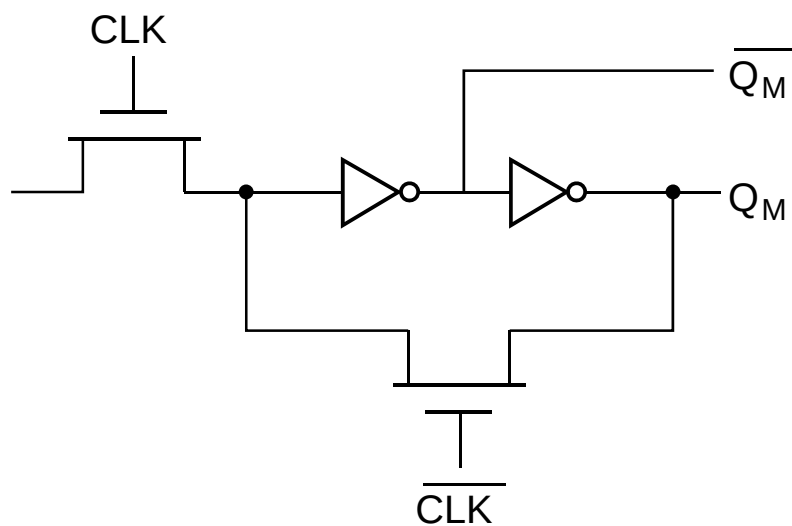
锁存器翻转条件：

S点电位高于反相器的 V_M

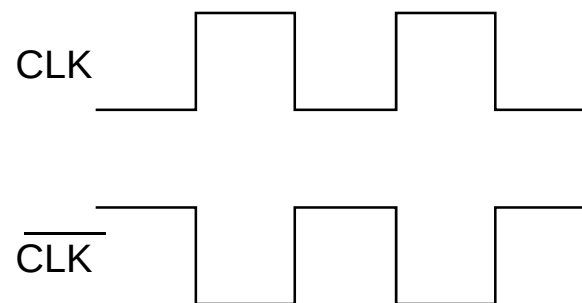
晶体管尺寸必应满足：

$$\frac{\text{M2的导通电阻}}{\text{M1-传输门的串联电阻+M2的导通电阻}} V_{DD} > V_M$$

NMOS多路开关锁存器



NMOS 传输管



不重叠时钟

- 时钟负载小
- 阈值损失，反相器存在静态功耗

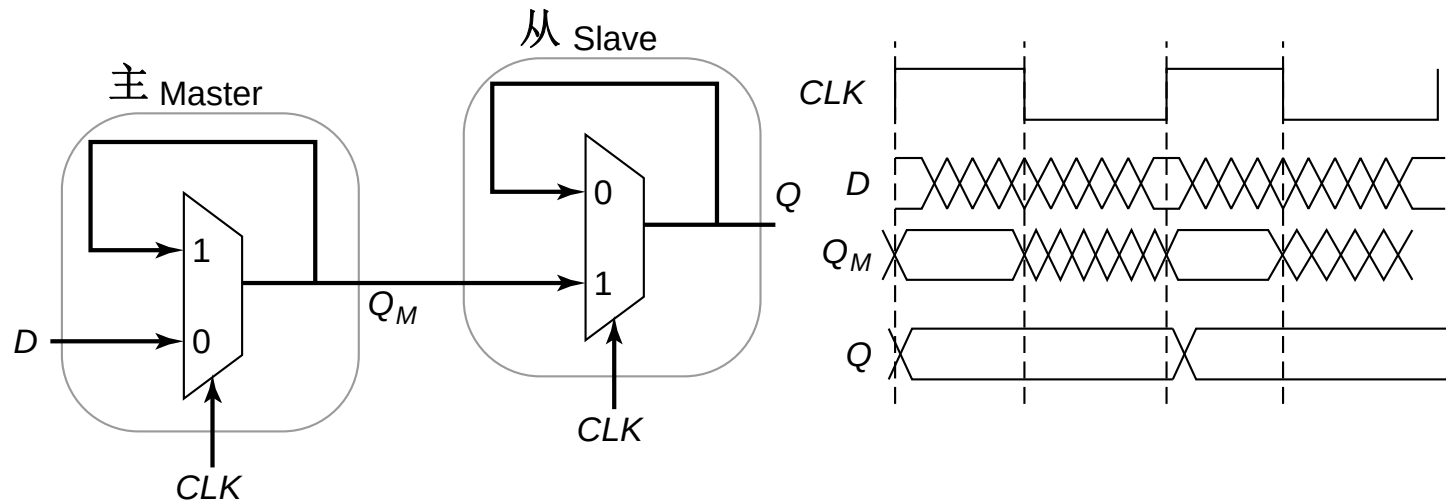
时钟负载

- 数字VLSI中往往存在大量的寄存器，每一个寄存器都带有一定的时钟负载。
例：DEC公司的第二代Alpha处理器的时钟信号的总负载为3.75nF！
- 时钟信号开关活动性=1
- 降低每个寄存器的时钟负载对于减小整个芯片的动态功耗具有重要意义！

8.2 静态时序单元

8.2.3 主从边沿触发寄存器

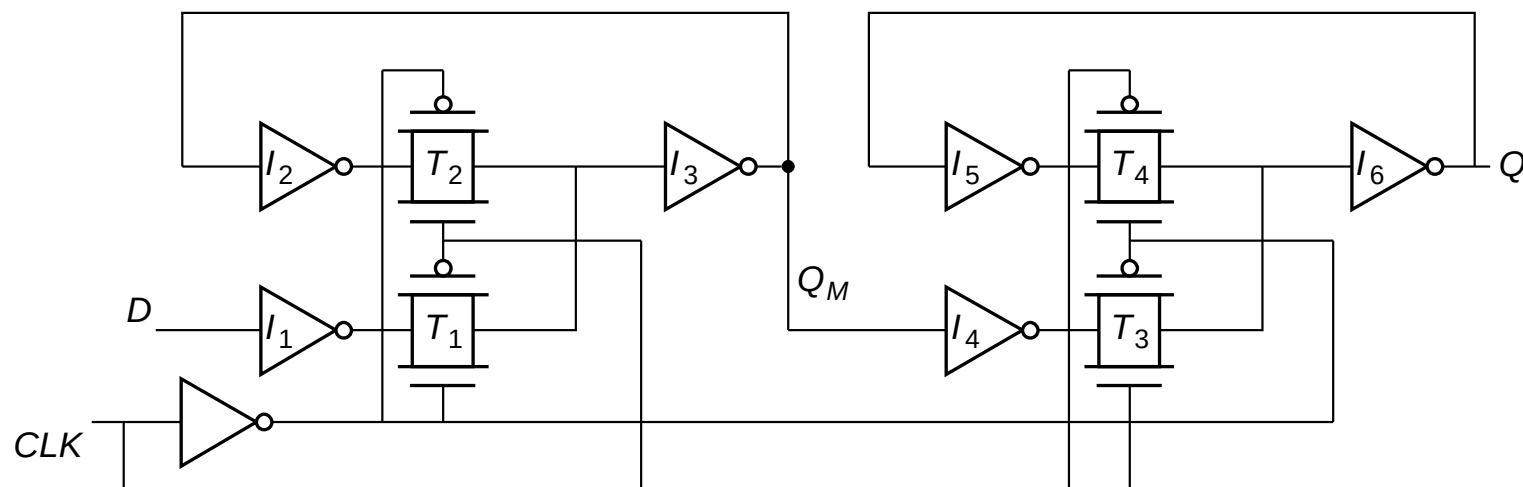
主从（边沿触发）寄存器



- 由一个正电平灵敏Latch和一个负电平灵敏Latch 构成(主从Latch 对) 。
- CLK 高电平和低电平的脉冲宽度应分别大于主（从）latch 的传播延时。

主从寄存器电路

基于多路选择器的锁存器对。



基于强制写入的静态主从寄存器

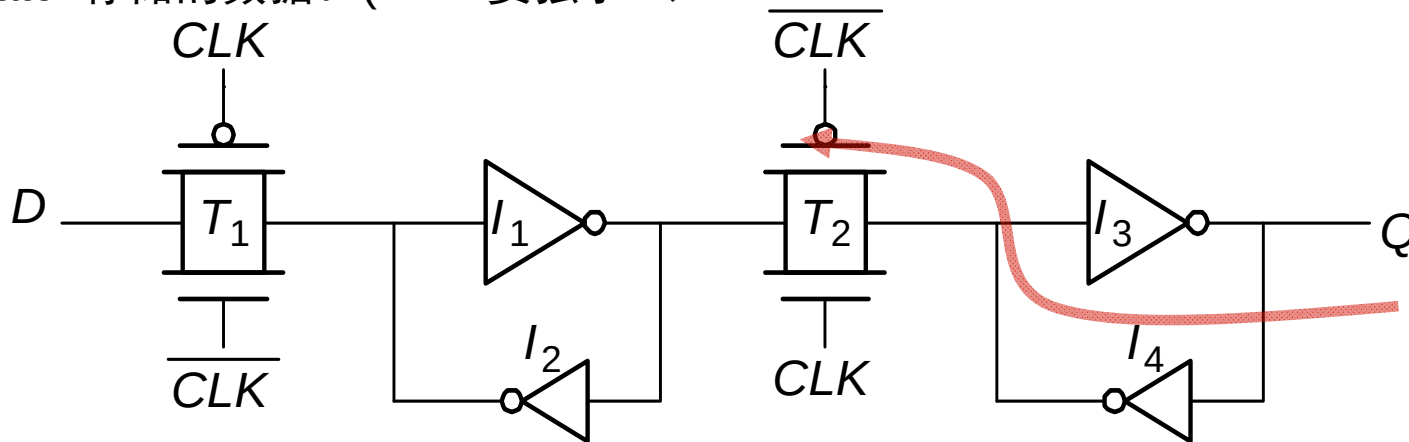
采用弱反相器可减少一个时钟控制的传输门——时钟负载小

但存在缺点：

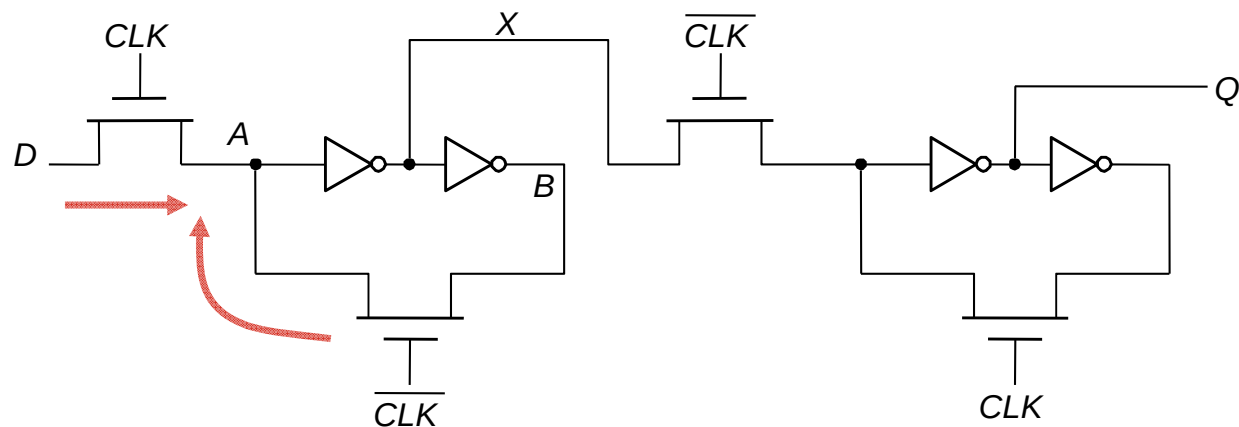
(1) 设计复杂性增加：尺寸设计要保证能强制写入（ T_1 和 D 的驱动电路要强于 I_2 ）

(2) 反向导通：

当 T_2 导通时，第二个latch有可能通过传输门 T_2 的耦合而影响第一个latch存储的数据。（ I_1 - T_2 要强于 I_4 ）

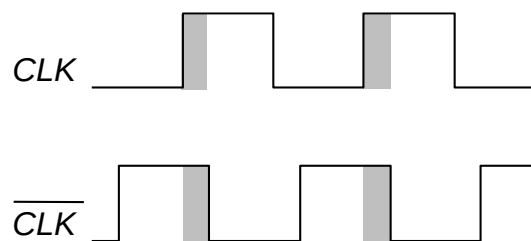


避免时钟重叠



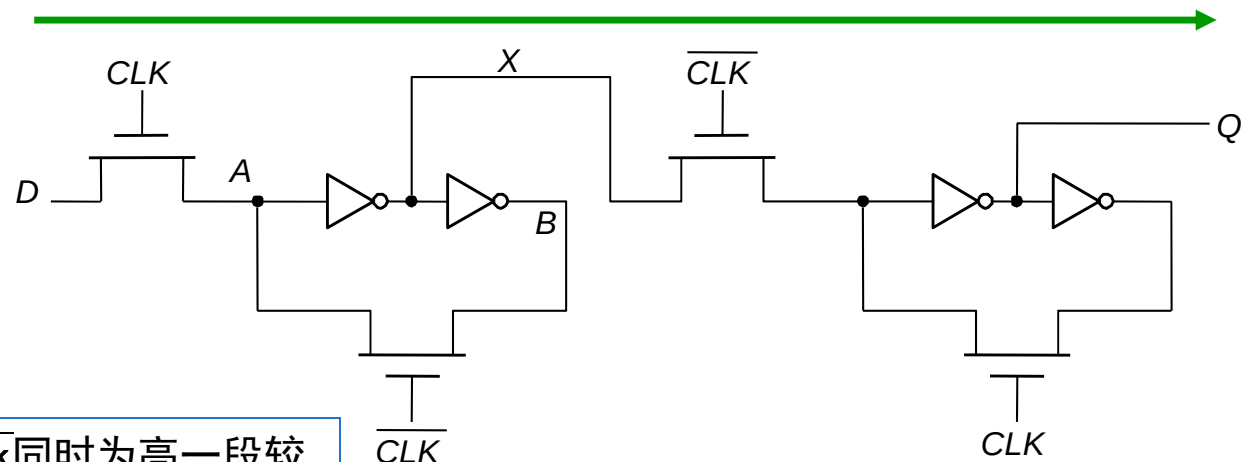
(a) 电路图

(1) 当 CLK 和 $\overline{CLK}$ 同时为高时， A 点同时被 D 和 B 点驱动，造成不定状态



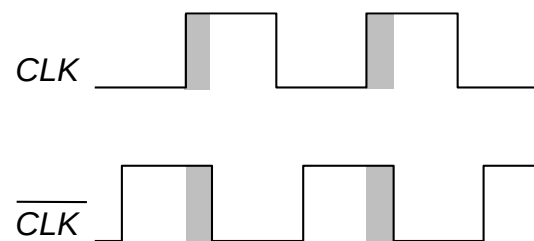
(b) 一对重叠的时钟

避免时钟重叠



(2) 当C1k和C1k同时为高一段较长时间时，D可以**直接穿透**经过主从触发器

(a) 电路图



(b) 一对重叠的时钟

8.3 时序参数

8.3.1 时序参数的定义

时序参数

(1) 建立 (**set-up**) 时间: t_{su}

(2) 保持 (**hold**) 时间: t_{hold}

(3) 时钟至输出 (**clk-q**) 时间: t_{clk-q}

最大值 (**max**): 称为时钟至输出的传播延时 (最长延时)。

最小值 (**min**): 称为时钟至输出的污染延时。

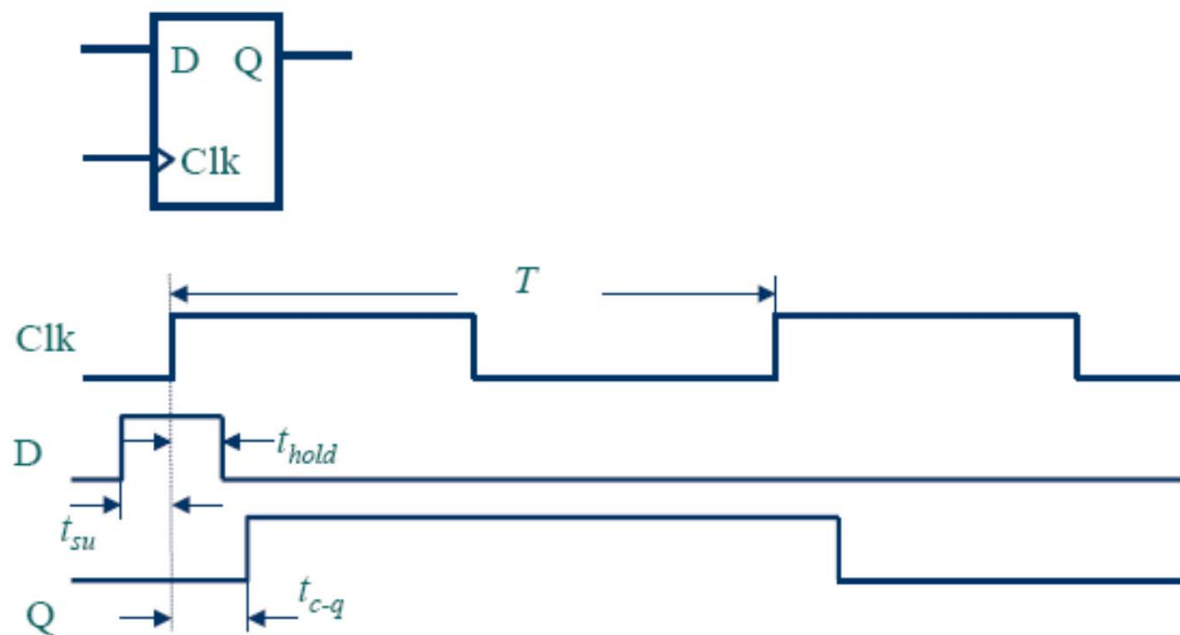
(4) (锁存器) 输入至输出 (**d-q**) 时间: t_{d-q}

最大值 (**max**): 称为输入至输出的传播延时 (最长延时)。

最小值 (**min**): 称为输入至输出的污染延时。

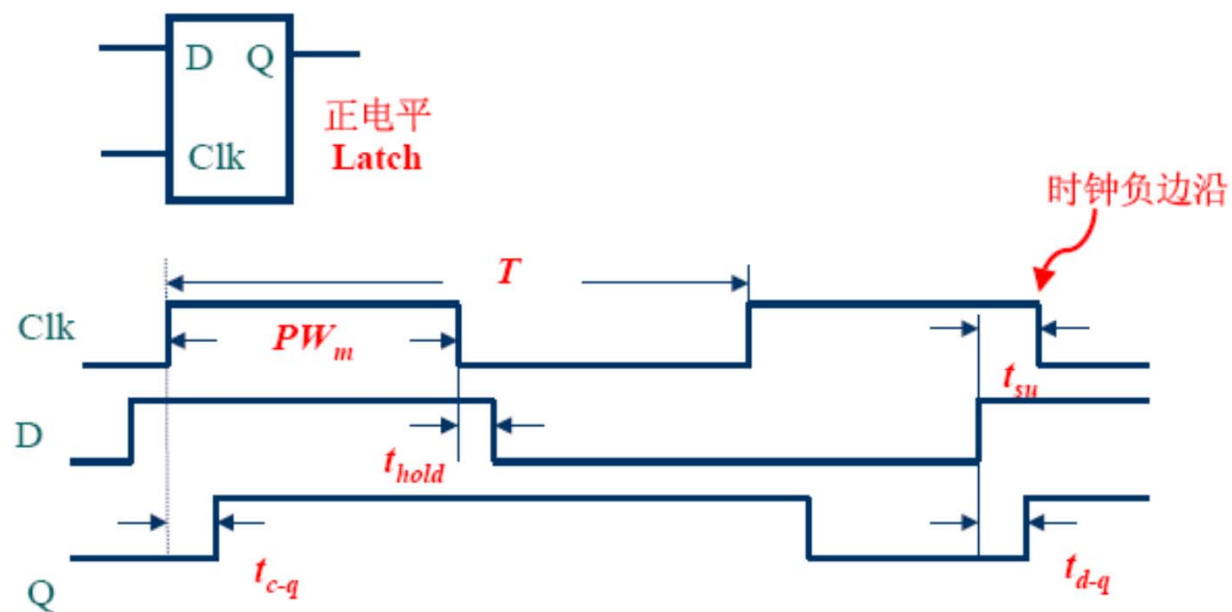
(5) 时钟周期: T

寄存器的时间参数



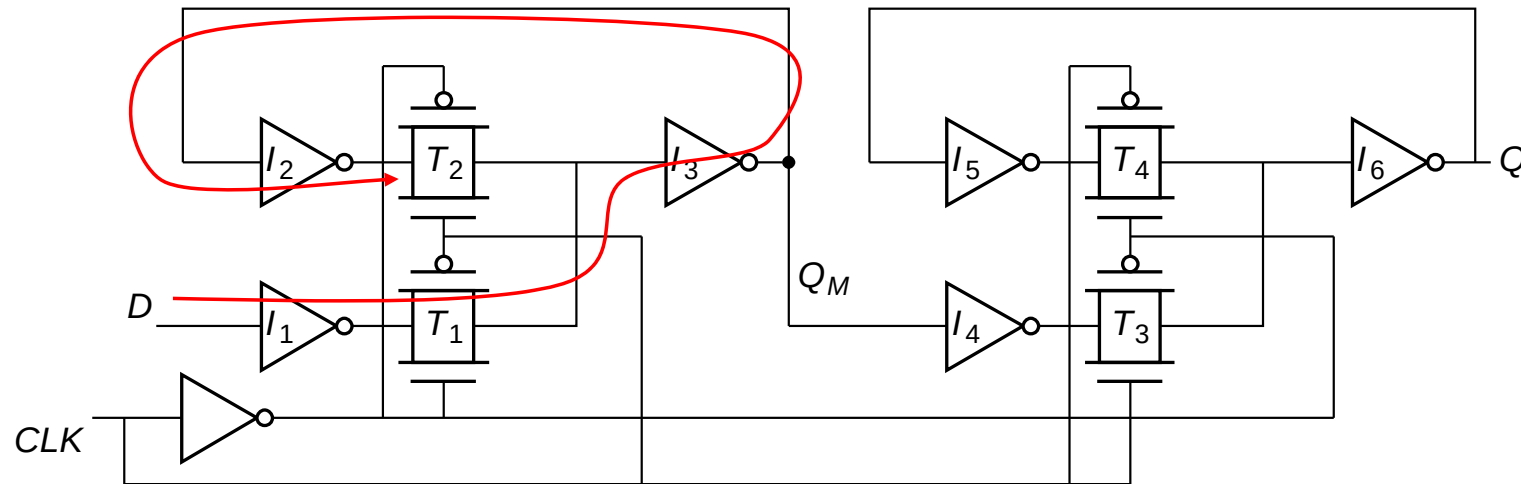
注意：数据的上升和下降时间不同时，延时将不同。

锁存器的时间参数



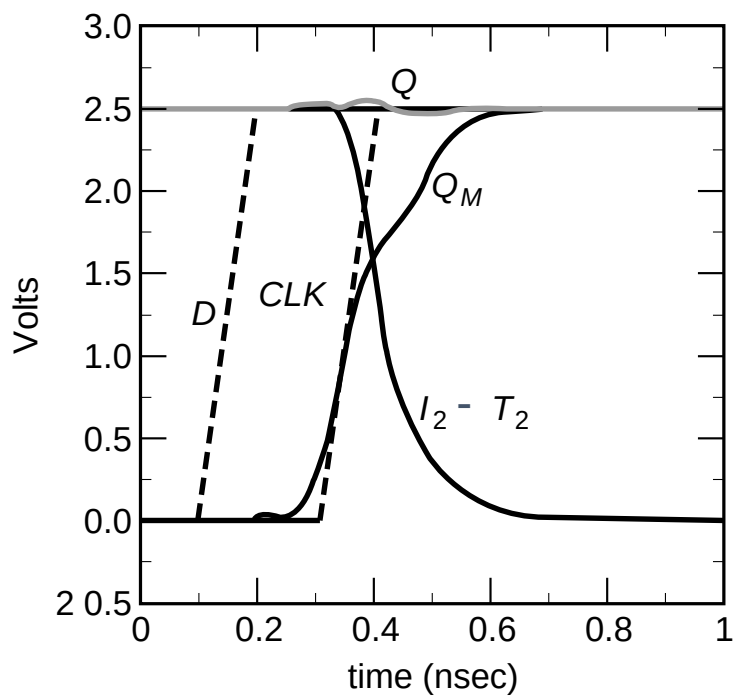
注意：数据的上升和下降时间不同时，延时将不同。

寄存器的延时——建立时间



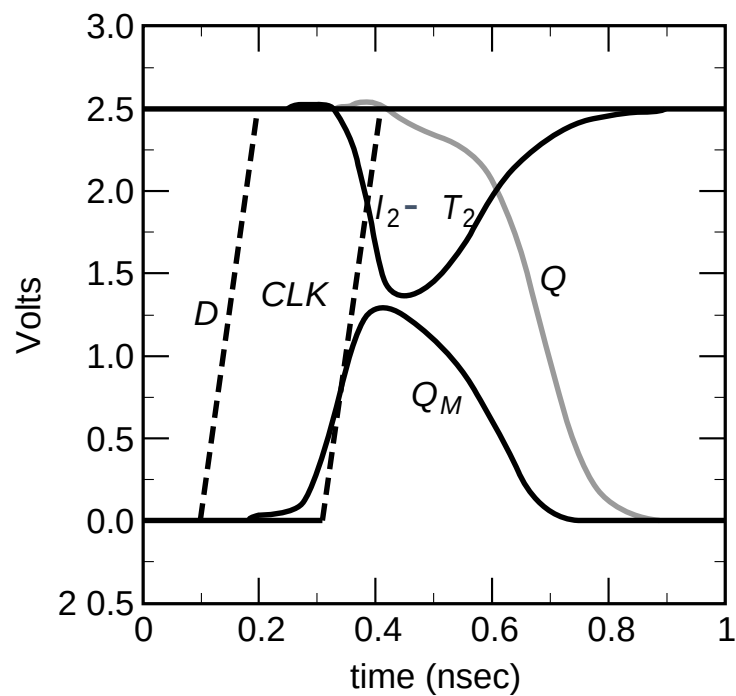
CLK上升 (T_2 导通) 之前D端数据必须传递到 T_2 的两端, 否则会引起竞争。

建立时间



(a) $T_{\text{setup}} = 0.21 \text{ nsec}$

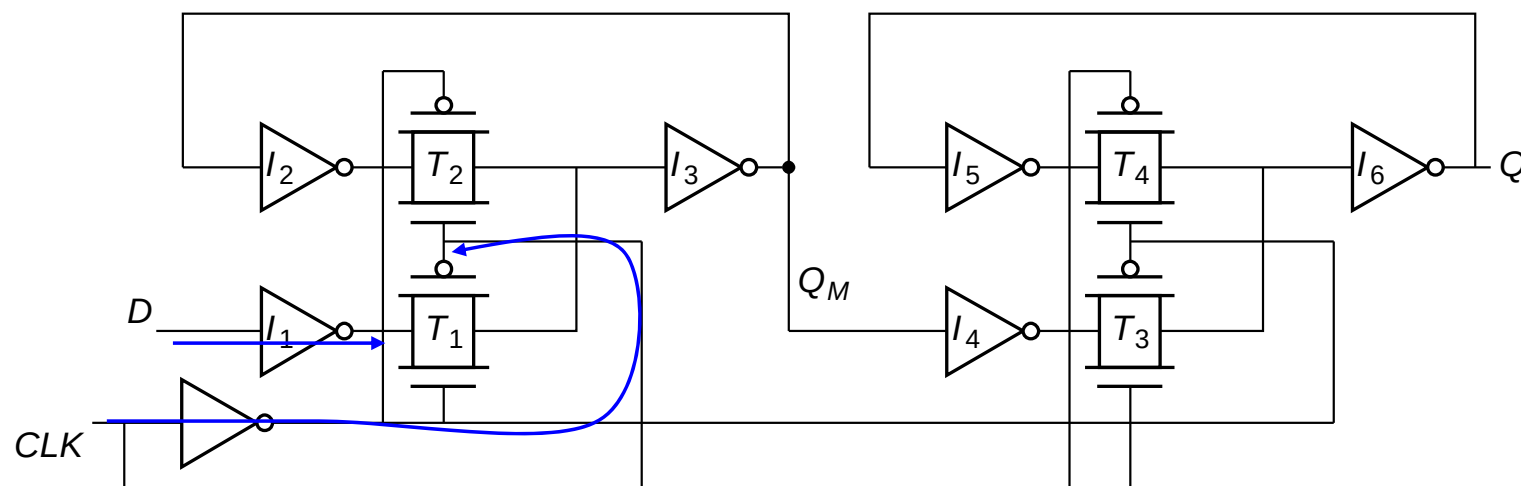
满足建立时间要求



(b) $T_{\text{setup}} = 0.20 \text{ nsec}$

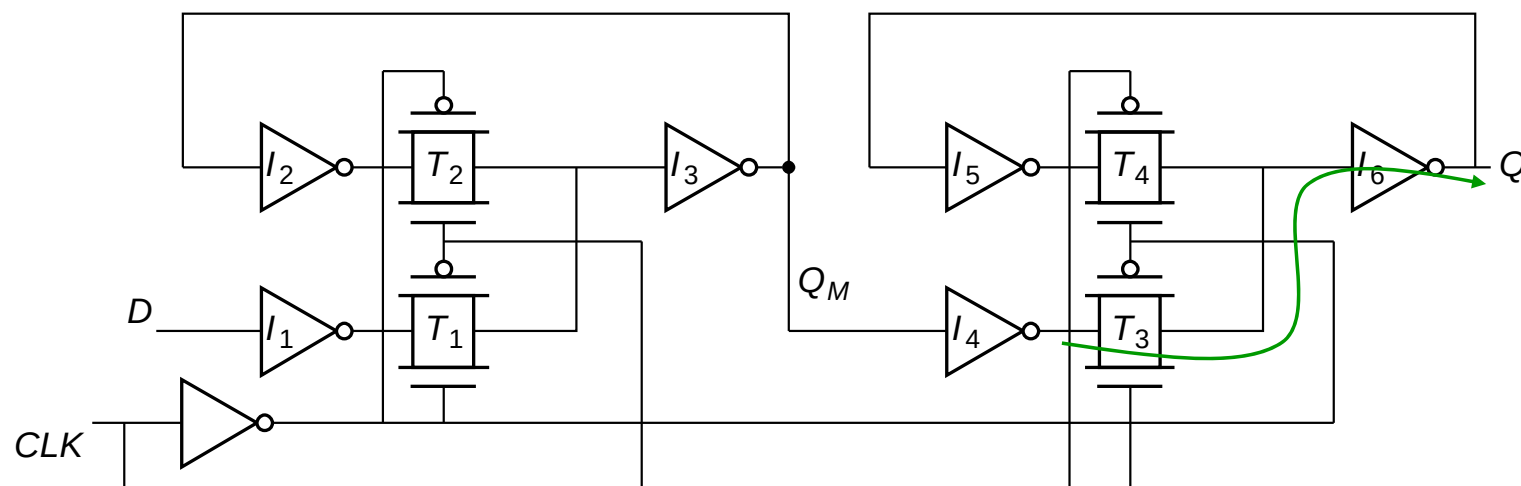
不满足建立时间要求

寄存器的延时——保持时间



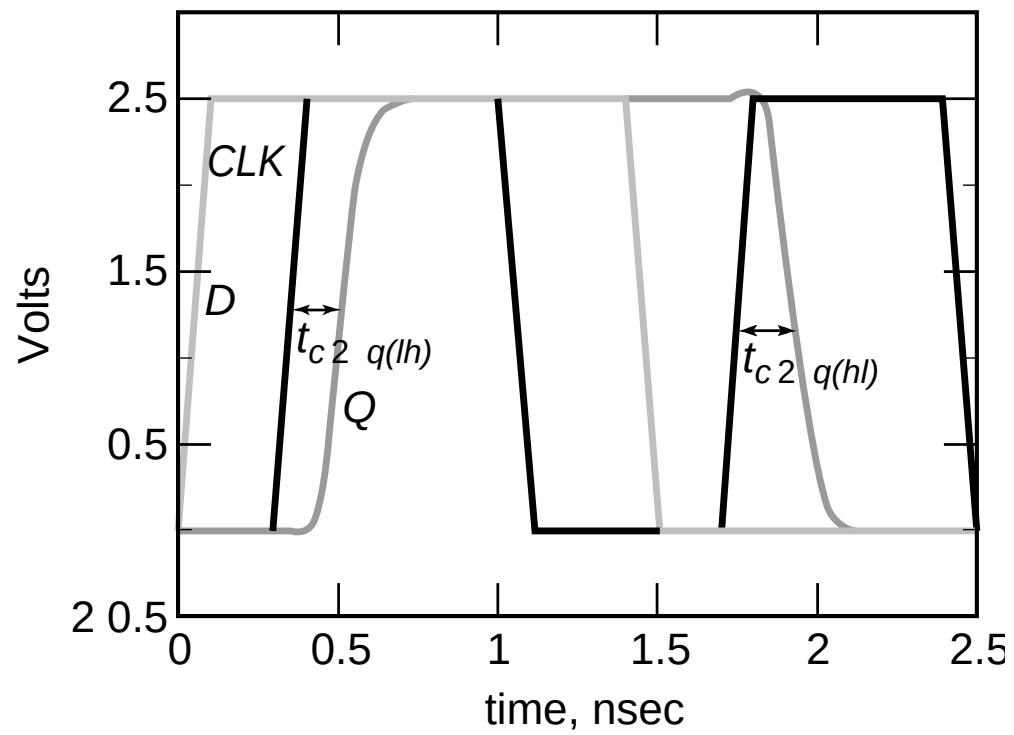
D的数据必须维持到**CLK**把**T₁**完全关闭后，否则会写入错误的值，即最少维持**CLK**输入到**T₁**的传播时间（**D**和**CLK**的传播时间相等，一般为0）。

寄存器的延时——传播延时



时钟上升后锁存的数据要经过 T_3 - I_6 的延时才能到达 Q 。

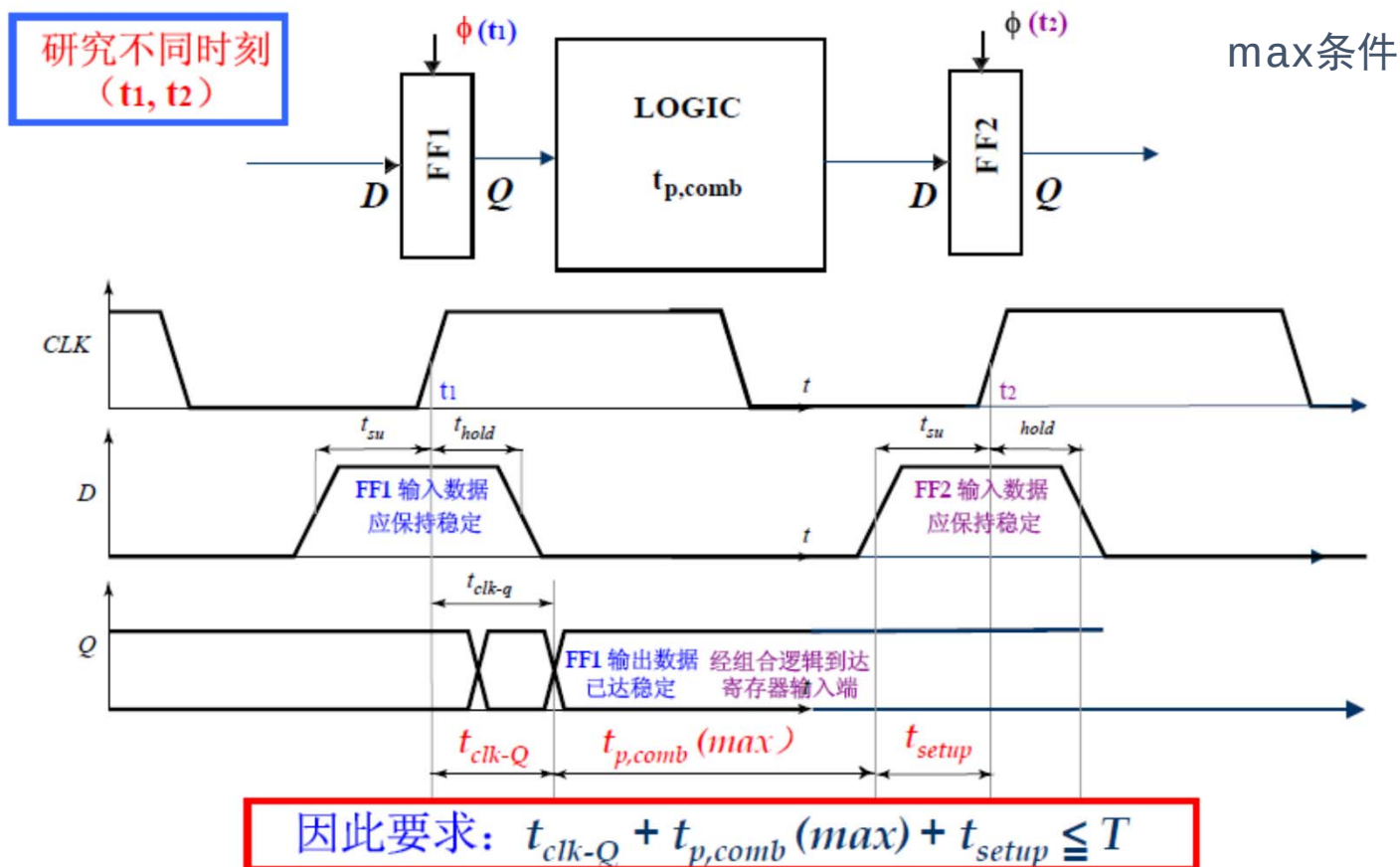
Clk-Q Delay



8.3 时序参数

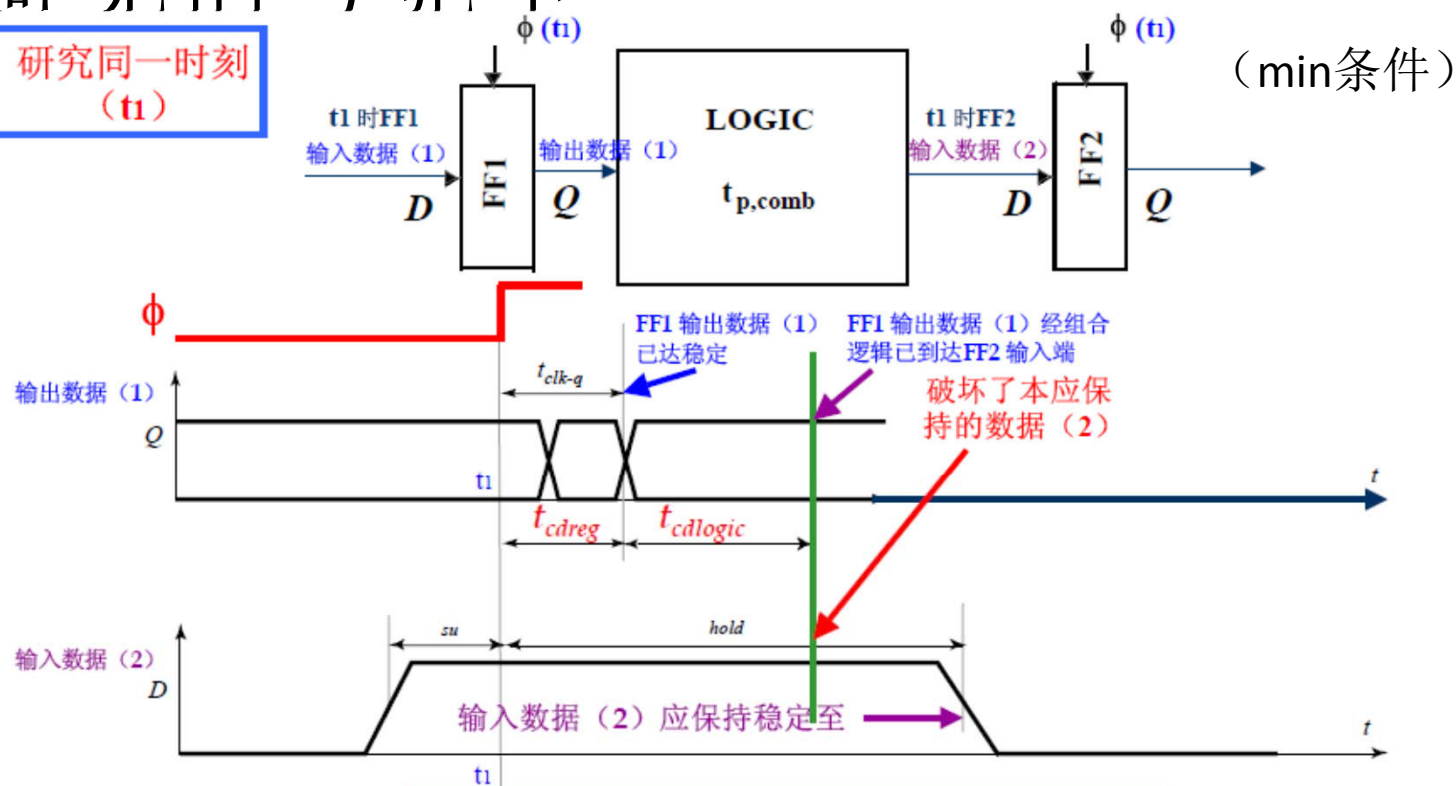
8.3.2 时序参数对同步系统的影响

允许的最高时钟频率（最短时钟周期）



必须避免信号竞争

研究同一时刻
(t_1)



(min条件)

因此要求: $t_{cdreg} + t_{cdlogic} \geq t_{hold}$

t_{cd} : 污染延时 (contamination delay) = 最小延时 (minimum delay)

时序约束

- 一个时序路径的延时必须同时满足max（setup）条件和min（hold）条件：

$$t_{hold2} \leq t_{clk-Q1} + t_{p,comb} \leq T - t_{setup2}$$

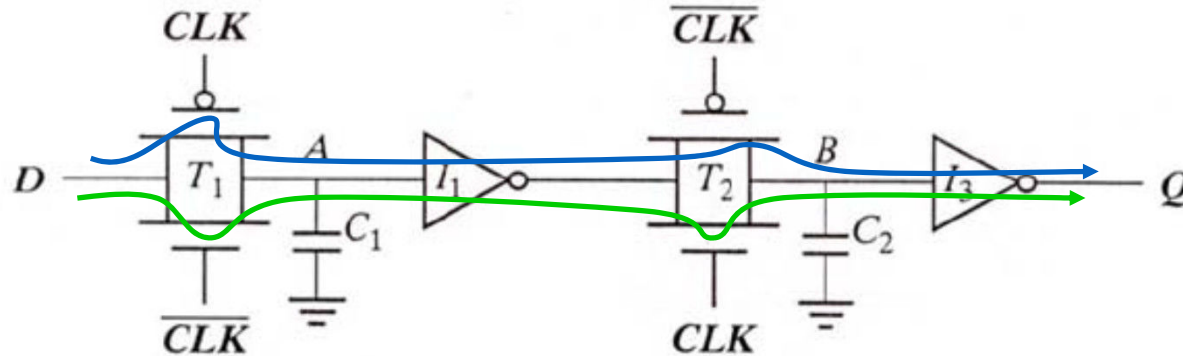
- Max条件决定了最短时钟周期：

$$T_{min} = \max\{t_{clk-Q1} + t_{p,comb}\} + t_{setup2}$$

- 全局时钟的最高时钟频率取决于所有时序路径对应的最短时钟周期的最大值，所对应的时序路径为整个电路的关键路径。

8.4 动态时序单元

动态传输门主从边沿触发寄存器



- 只需8个晶体管，节省功耗和提高性能，甚至可只用NMOS实现。

- 时钟重叠是重要问题：

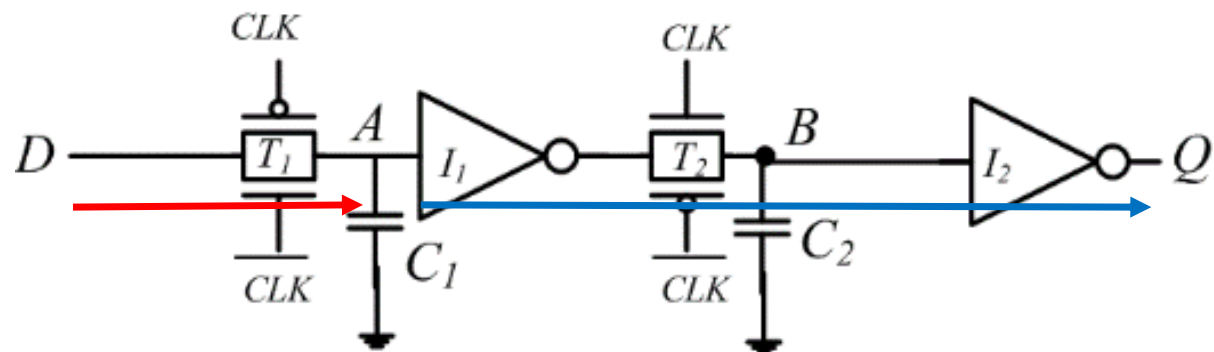
- 0-0重叠（发生在下降沿）： T_1 - T_2 的PMOS同时导通，D-Q馈通，重叠时间超过D-Q传播延时Q可能在下降沿改变

- 1-1重叠（发生在上升沿）： T_1 - T_2 的NMOS同时导通，D-Q馈通

$$t_{overlap0-0} < t_{T1} + t_{I1} + t_{T2}$$

$$t_{hold} > t_{overlap1-1}$$

时序参数



建立时间: T_1

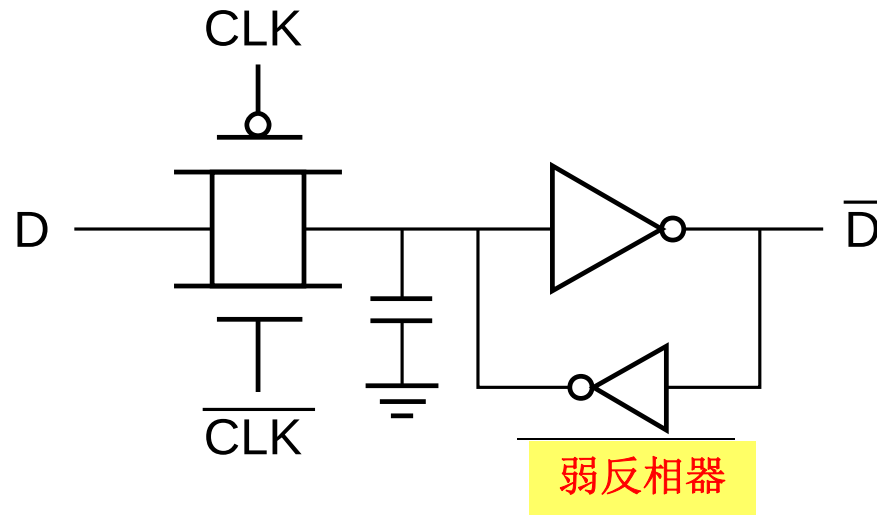
维持时间: 0

传播延时(t_{c-q}): $I_1 + T_2 + I_2$

伪静态传输门锁存器

纯动态锁存器存在一些问题：

1. 高阻抗的内部动态节点易受噪声源的干扰
 2. 为了控制动态节点的电荷泄漏，必须保证一定的时钟频率，CLK长时间停止会造成电路状态丢失，因此无法采用时钟调节等低功耗技术。
 3. 内部动态节点的电压并不跟踪电源电压的变化，从而降低噪声容限。
- 在实际设计中动态锁存器/寄存器一般都应当设计成伪静态形式。



总结

- 构成时序电路的基本原理
 - 双稳态电路（反相器交叉耦合）
 - 利用动态节点电容
- 静态时序电路：
 - 基于多路选择器
 - 切换方法：断开反馈环/强驱动
- 时钟重叠问题
- 时序
 - setup\hold\clk-to-Q\锁存器的D-to-Q时间
 - 时序参数对时序的影响

总结（续）

- 动态时序电路：
 - 采用伪静态电路
- 时序单元设计中，动态功耗方面主要考虑时钟负载问题。

第九章 时钟技术

9.1 同步时序逻辑的时钟

9.1.1 同步时序

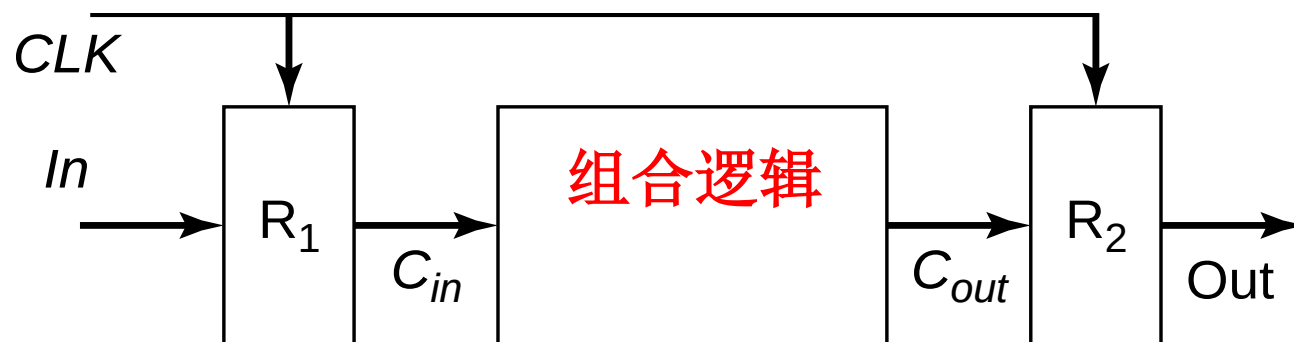
时序与定时

- 时序：事件发生的时间关系（先后顺序、时间间隔）
 - 为了保证功能正确必须保证正确的时序关系
- 定时：为保证时序关系而确定事件（如时序单元采样）的时刻

时钟是一种全局分布的周期性信号，它的作用是为系统操作提供定时。

同步时序逻辑电路

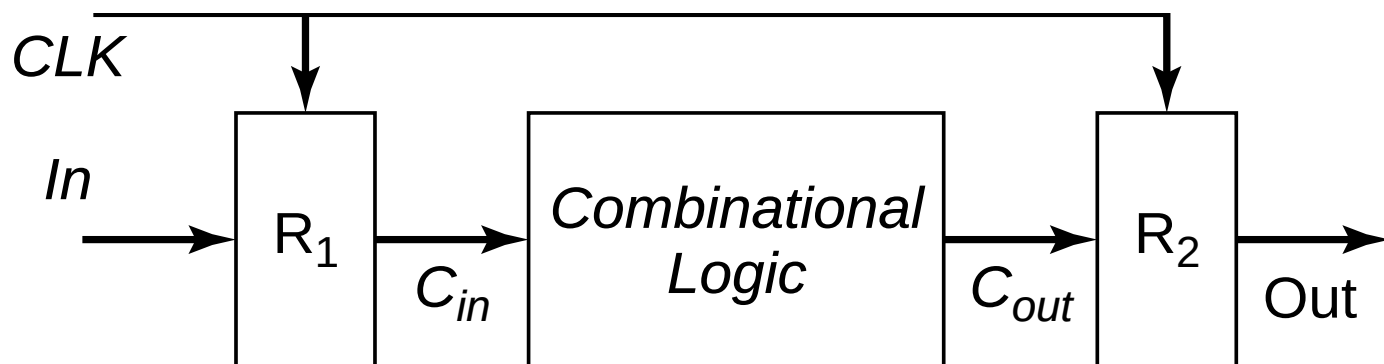
- 依靠全局时钟来定时的时序电路称为同步时序逻辑
- 在同步时序逻辑电路中有一个公共的时钟信号，电路中各记忆元件（存储单元）受它统一控制，只有在该时钟信号到来时，记忆元件的状态才能发生变化，从而使时序电路的输出发生变化，而且每来一个时钟信号，记忆元件的状态和电路输出状态才能改变一次（即与时钟同步）。
- 同步：两个信号具有**完全相同的频率**，且保持**固定的相位差**。



设计情景假设

- 通常的数字电路
 - 具有一个全局的时钟信号，所有时序单元都由这个时钟信号控制
 - 采用寄存器作为记忆元件
 - 所有寄存器采用上升沿触发。
 - 理想时钟信号：芯片中各处（每个寄存器的时钟端）的时钟信号总是同时翻转，即具有相同的时钟周期和相位。

时序路径



时钟周期为T

时序路径：从发射寄存器（ R_1 ）的时钟端到采样寄存器（ R_2 ）的数据输入端的逻辑路径。

电路中任意两个彼此通过组合逻辑连接的寄存器之间都构成一条时序路径。

时序约束：

$$t_{clk-Q} + t_{p,logic} + t_{setup} \leq T$$

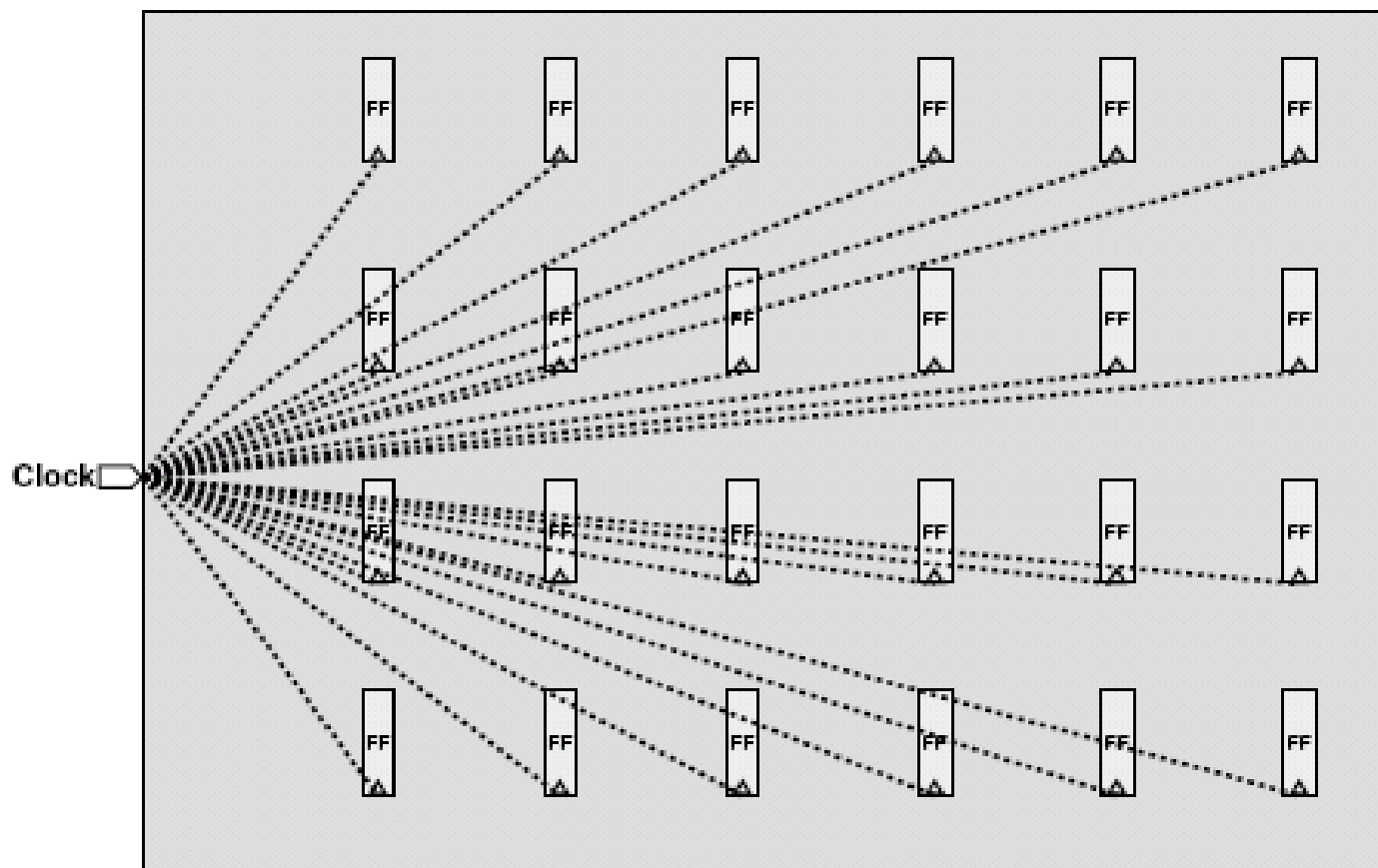
$$t_{cdreg} + t_{cdlogic} \geq t_{hold},$$

在同一个时钟域中，所有时序路径对应的时钟周期T是同一个值。

9.1 同步时序逻辑的时钟

9.1.2 时钟系统

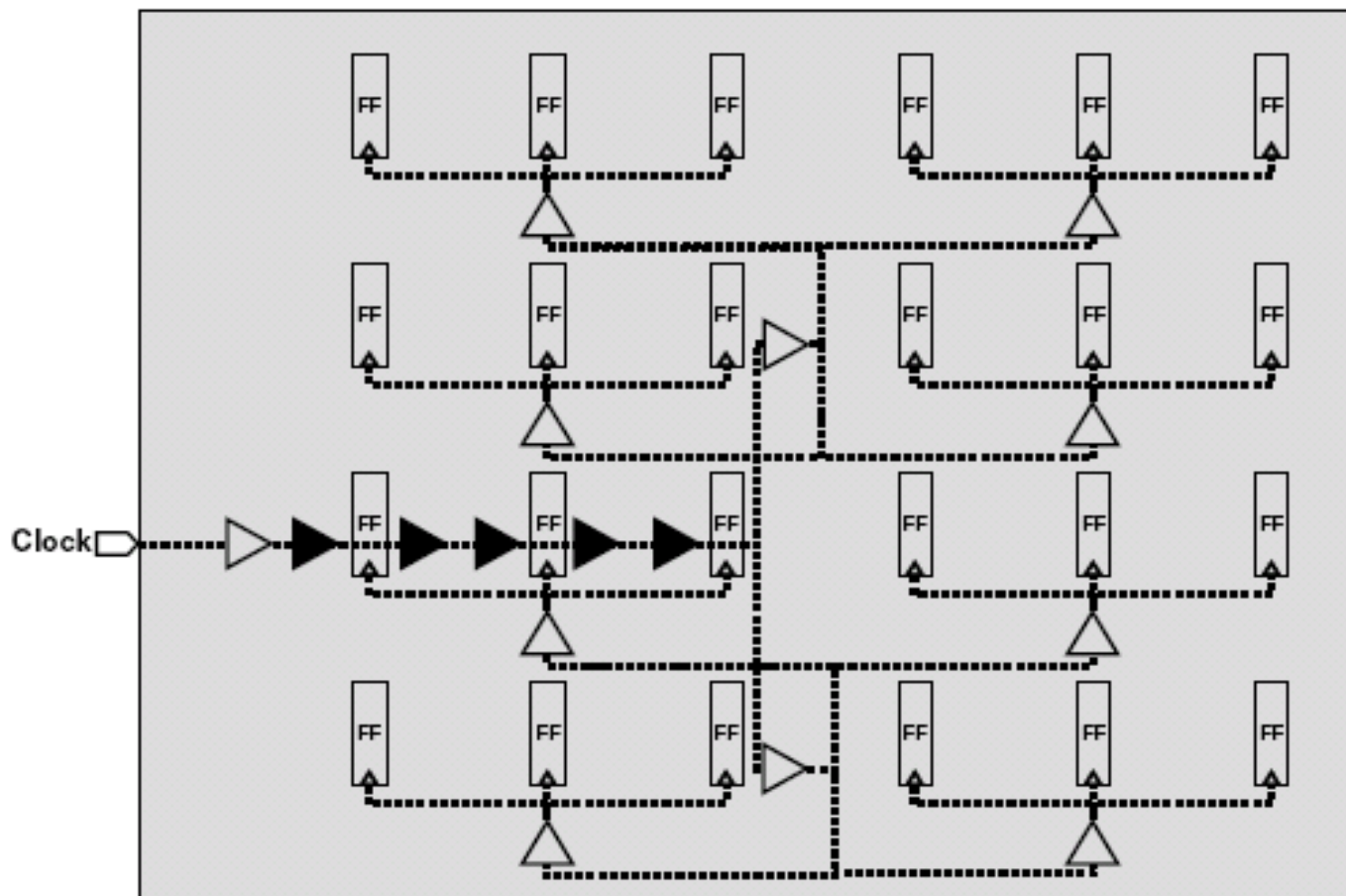
时钟信号的逻辑结构



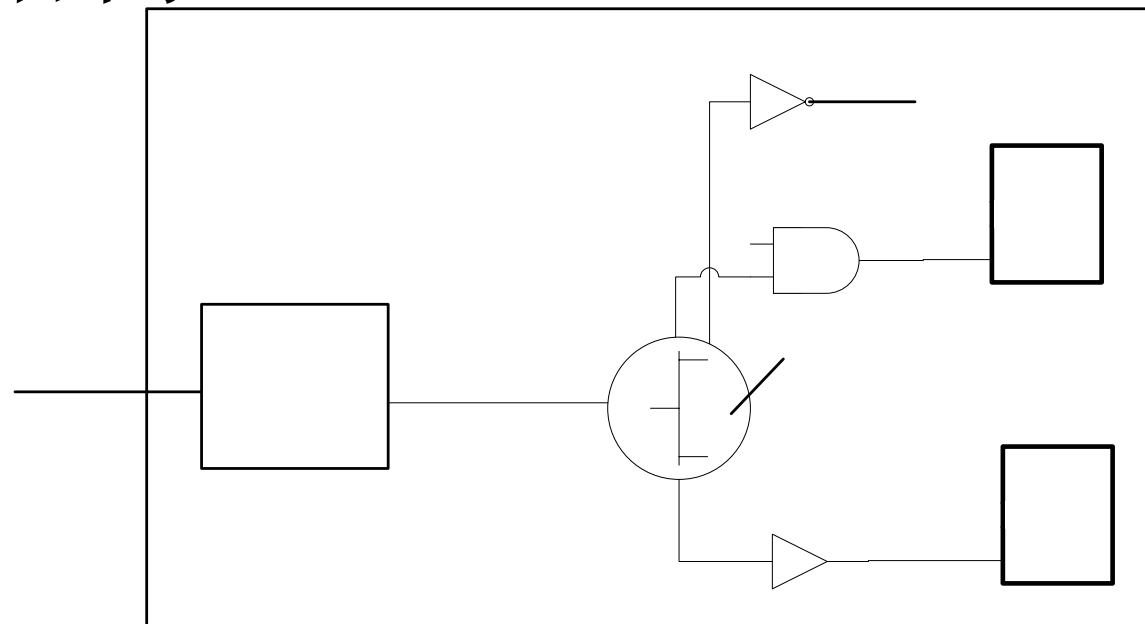
直接连接时序单元的问题:

1. 非常大的负载电容，需要有非常大的驱动电路；
2. 需要大量的布线资源
3. 时钟信号线在各处会受到干扰
4. 难以保证信号传播到芯片各处的时间相同

时钟信号网络的物理结构



时钟系统结构



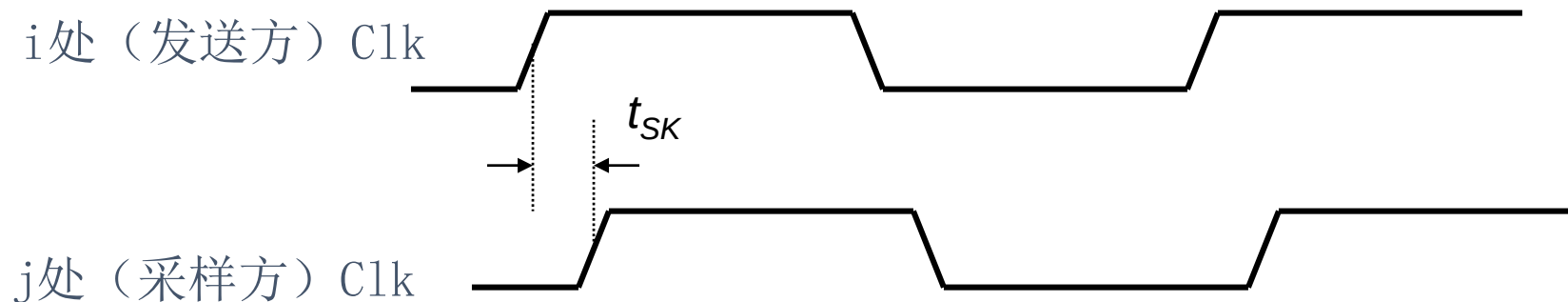
- 外部时钟作为参考时钟
- 分布网络把全局时钟信号分配到各个时序元件附近变成了局部的时钟信号
 - 分布网络由时钟缓冲器(buffer)和互连线组成
- (时钟) 末端驱动器:
 - 本地时钟通过本地的逻辑门连接到时序元件的时钟端上。
 - 缓冲器、时钟门控单元（用于根据条件停止时钟）和反相器（产生反相时钟）等

9.2时钟偏差和时钟抖动

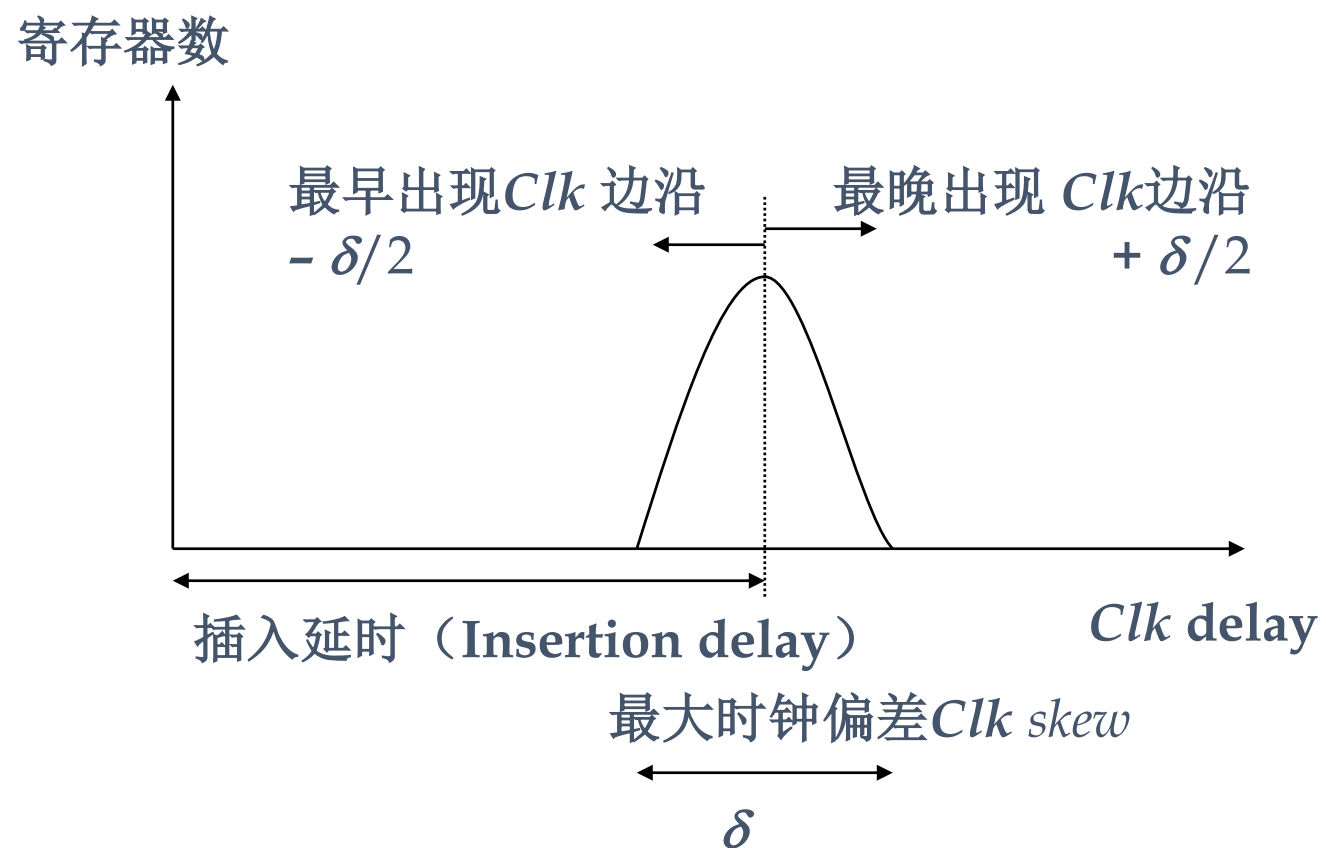
9.2.1 时钟偏差

时钟偏差（Clock Skew）

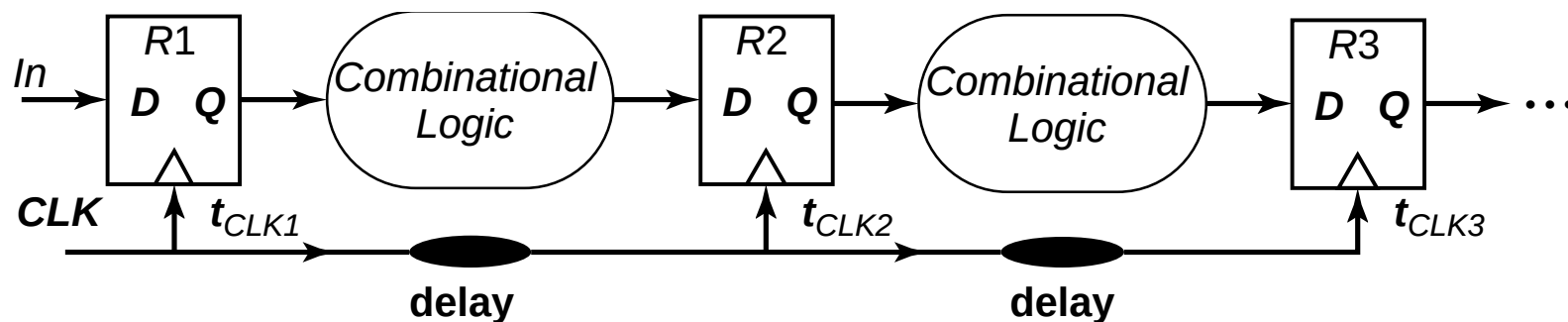
- 定义：空间上两个不同点处时序上等同的两个时钟沿到达时间的偏差
$$\delta(i, j) = t_i - t_j$$
- 由时钟路径上的失配及时钟负载上的差别引起；因数据与时钟布线的相对方向，时钟偏差可正可负。
- 具有确定性（从一个周期至下一个周期是时不变的）
- 时钟偏差不会引起周期的变化，只会引起相移（相位偏移）



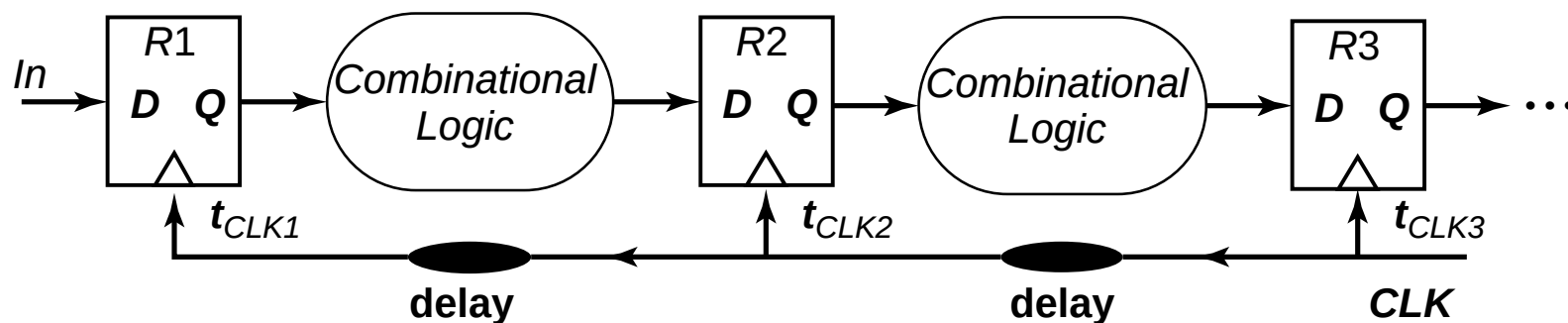
时钟延时的分布



正时钟偏差和负时钟偏差

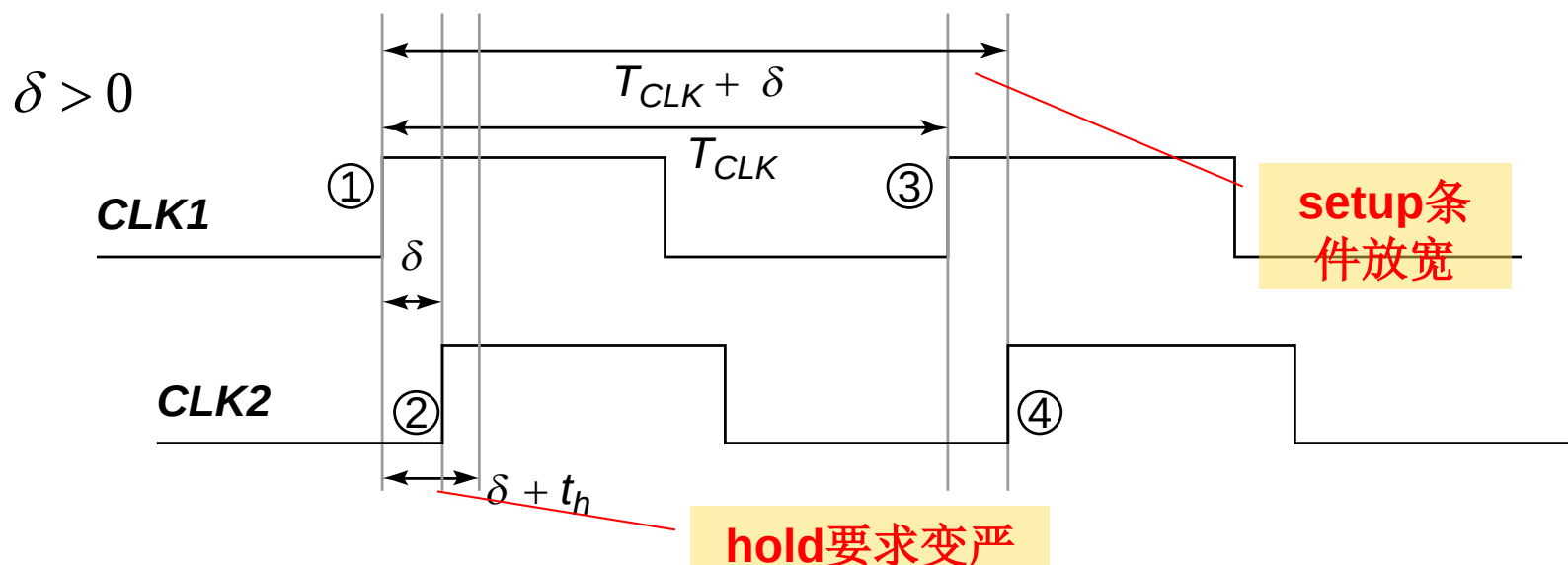
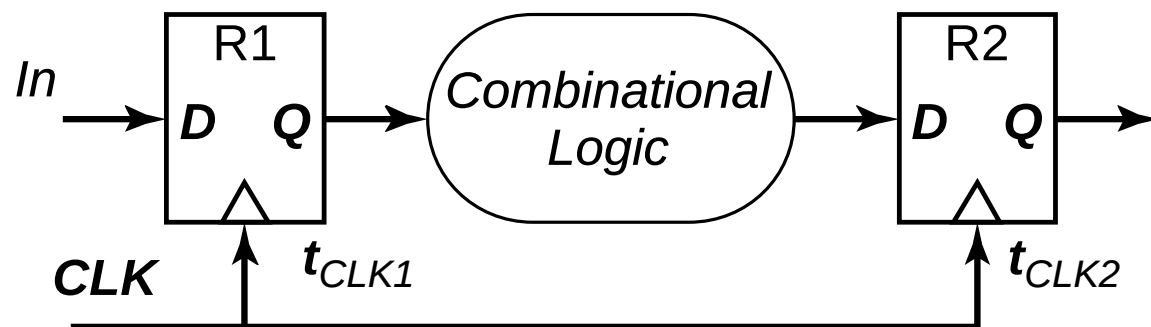


(a) 正时钟偏差（时钟与数据同向传播）

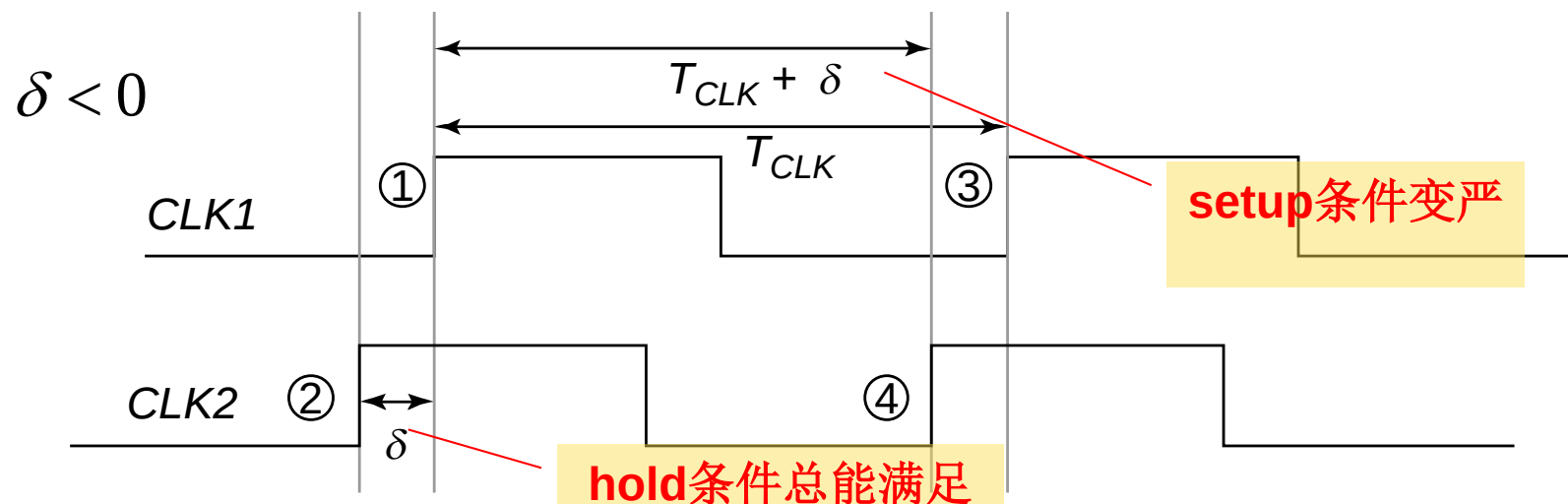
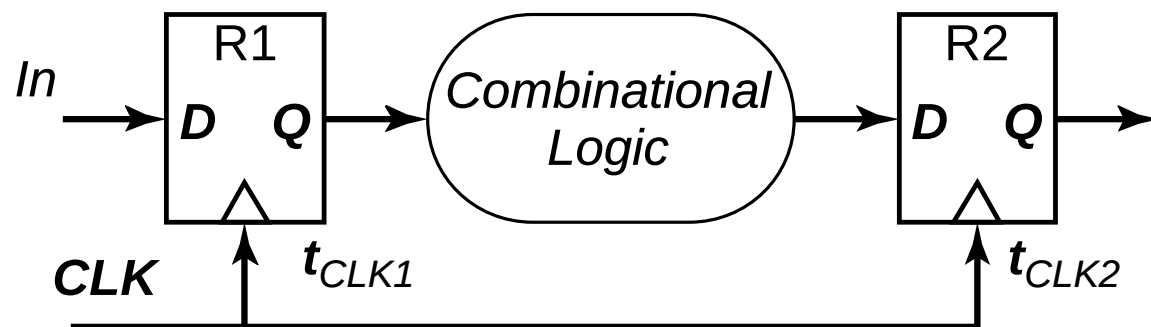


(b) 负时钟偏差（时钟与数据相向传播）

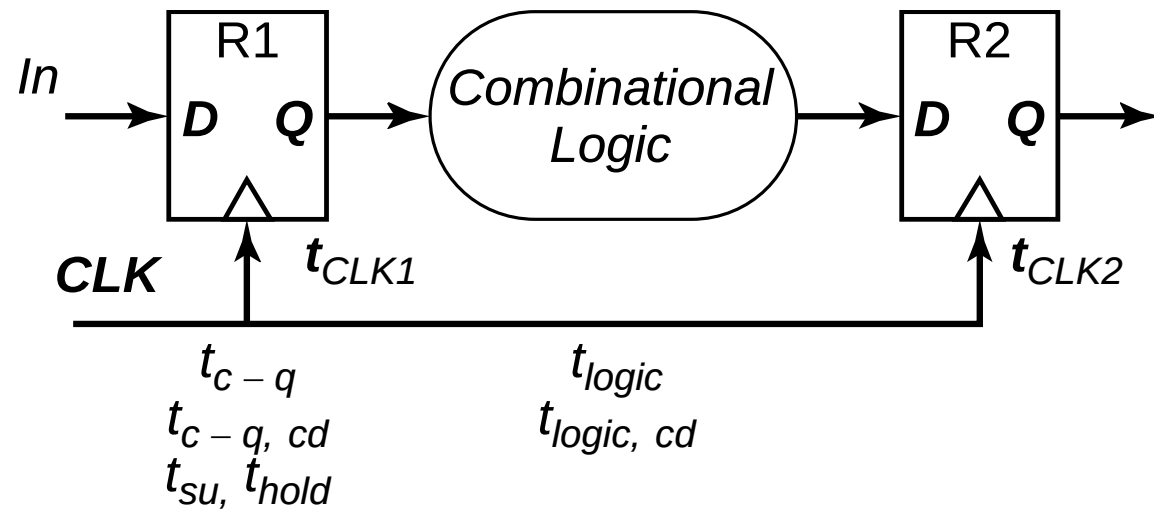
时钟偏差对性能和功能的影响



时钟偏差对性能和功能的影响



时序约束 (setup)

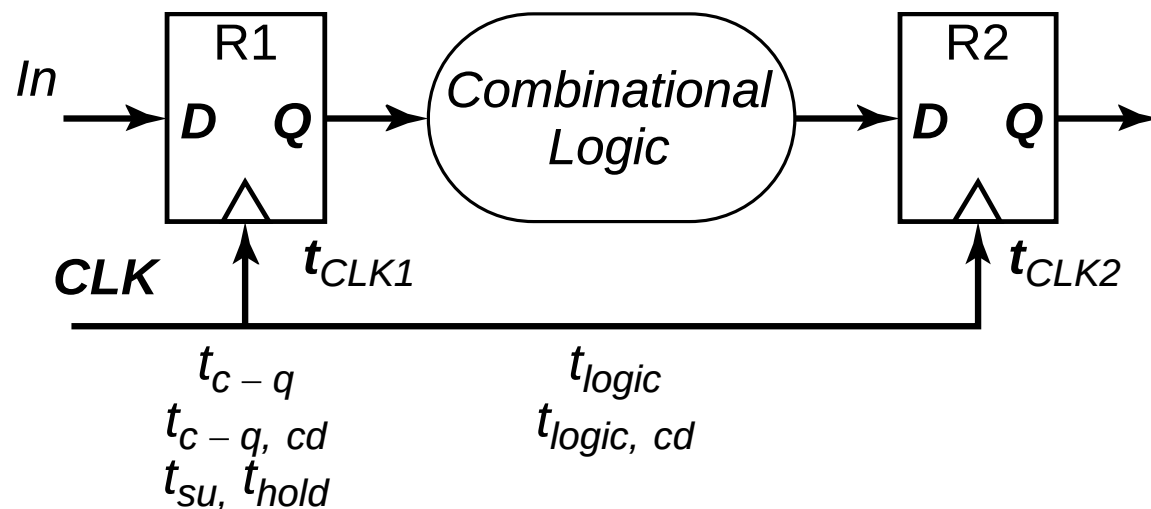


最小周期:

$$T + \delta = t_{c-q1} + t_{su2} + t_{logic}$$

最坏情况是负时钟偏差时

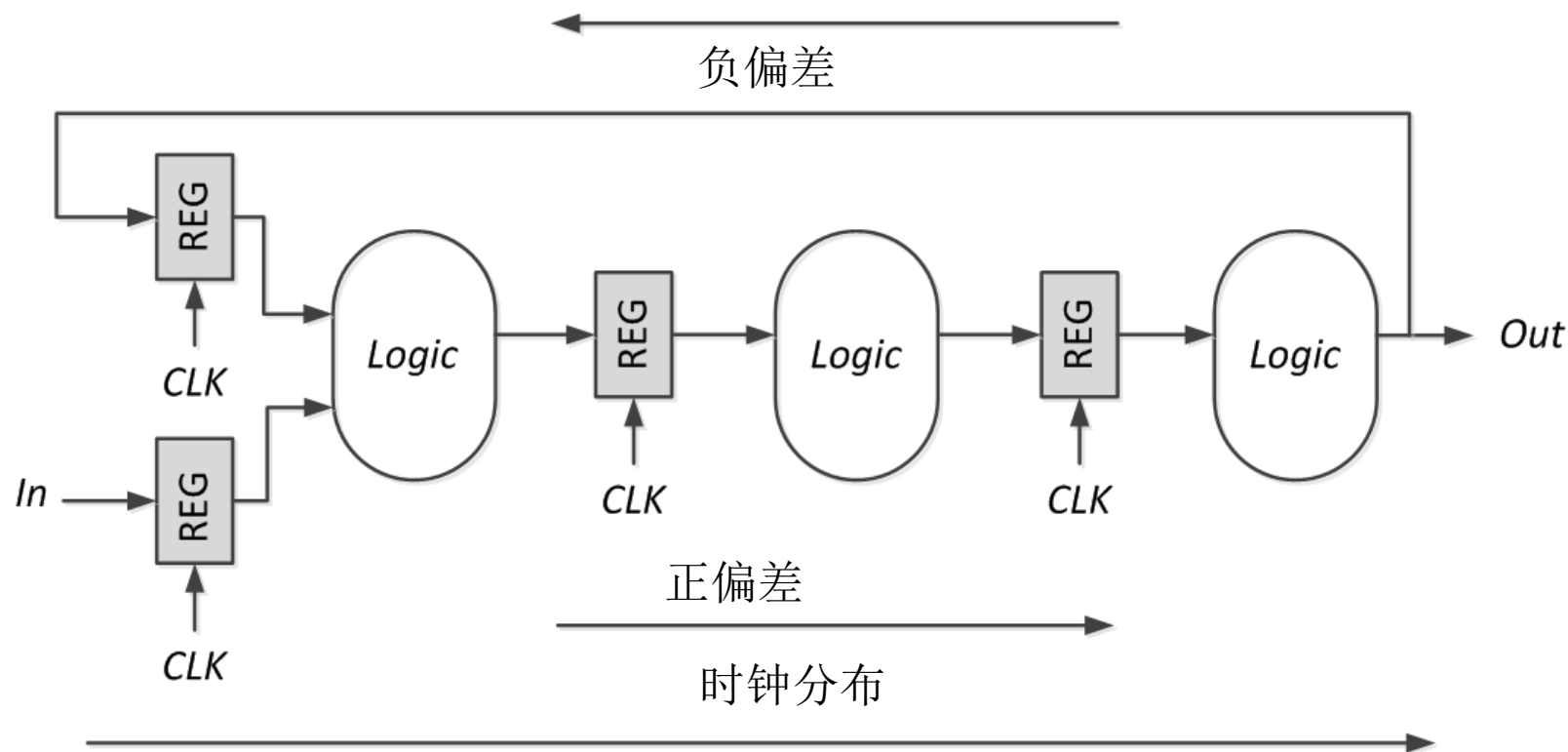
时序约束(hold)



Hold 时间约束:

$$t_{(c-q, cd)1} + t_{(logic, cd)} > t_{hold2} + \delta$$

最坏情况是时钟偏差为正的情况。



一般电路中正时钟偏差和负时钟偏差同时存在，因此，时钟偏差对性能和竞争裕量均有影响，所以要尽量减小时钟偏差。

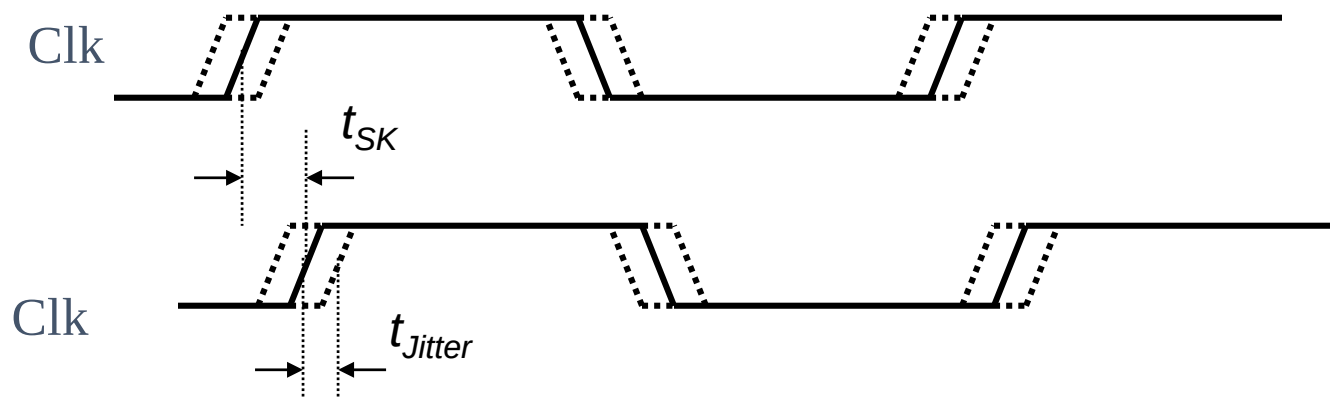
9.2时钟偏差和时钟抖动

9.2.2 时钟抖动

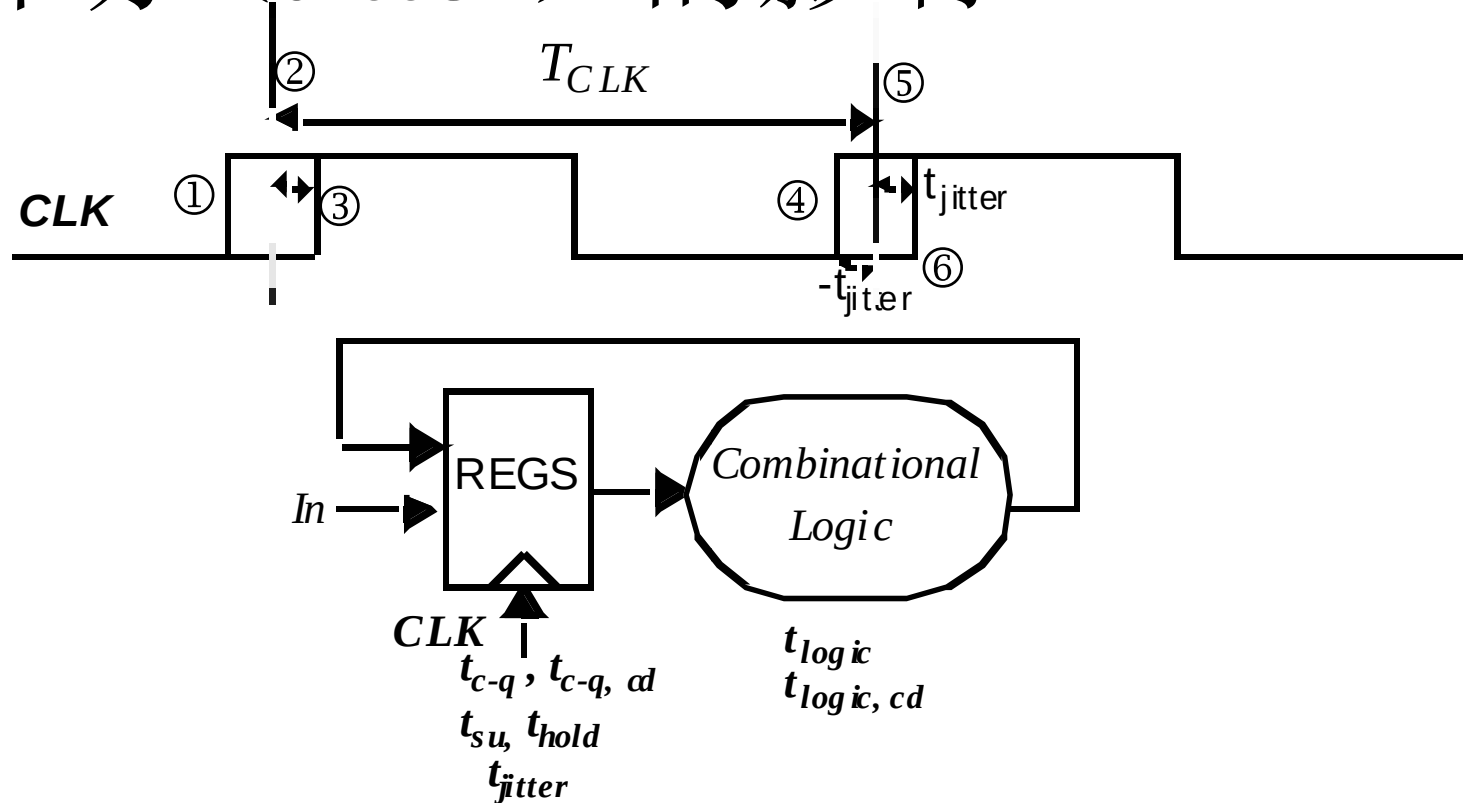
时钟抖动 (clock jitter)

- 定义：空间上**同一个点处时钟周期**随时间的变化。
- 抖动是一个平均值为零的随机变量
- 绝对抖动 (t_{jitter})：某点处一个时钟边沿**相对于理想参照时钟边沿**在最坏情况下偏差的绝对值
- 周期至周期抖动 (T_{jitter})：单个时钟周期相对于理想参照时钟时钟周期的时变偏离。（最坏情况下等于 $2 t_{jitter}$ ）

在位置*i*处 $T_{jitter} = t_{clk,n+1}^i - t_{clk,n}^i - T_{CLK}$



时钟抖动 (Jitter) 的影响

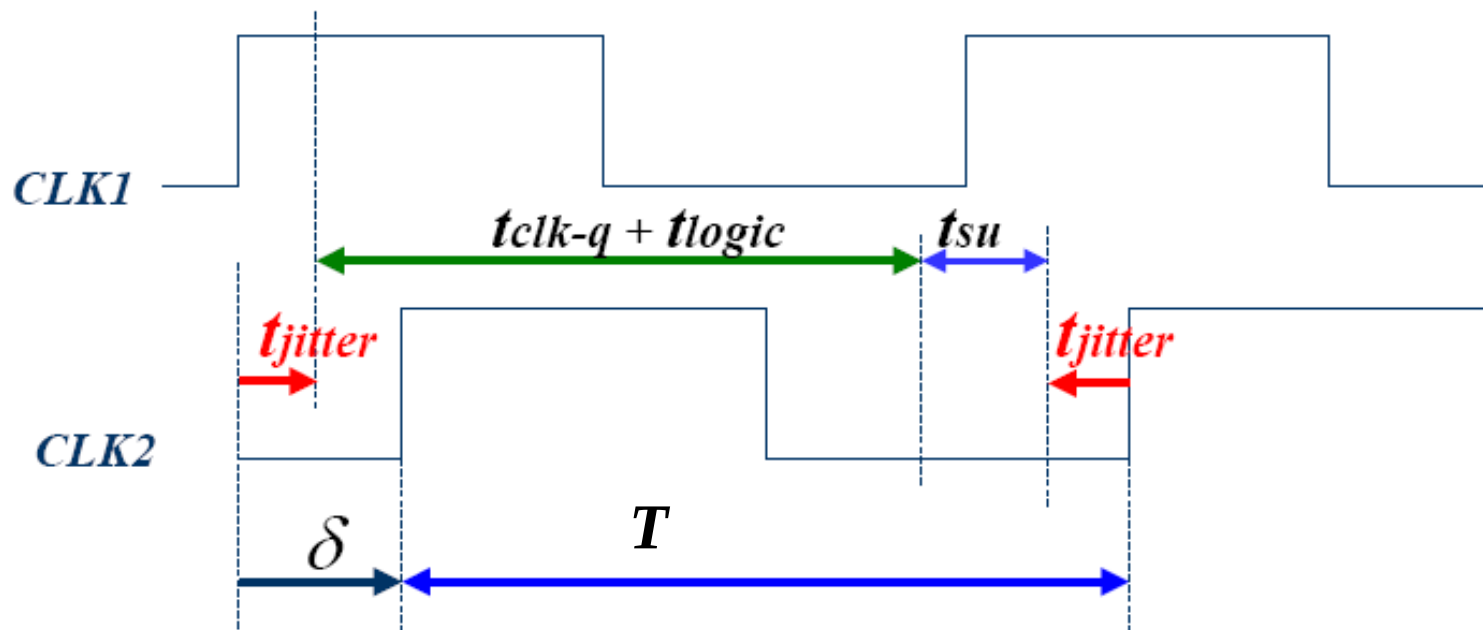


$$T_{CLK} - 2t_{jitter} > t_{c-q} + t_{logic} + t_{su}$$

$$T_{CLK} > t_{c-q} + t_{logic} + t_{su} + 2t_{jitter}$$

因此，抖动会降低性能

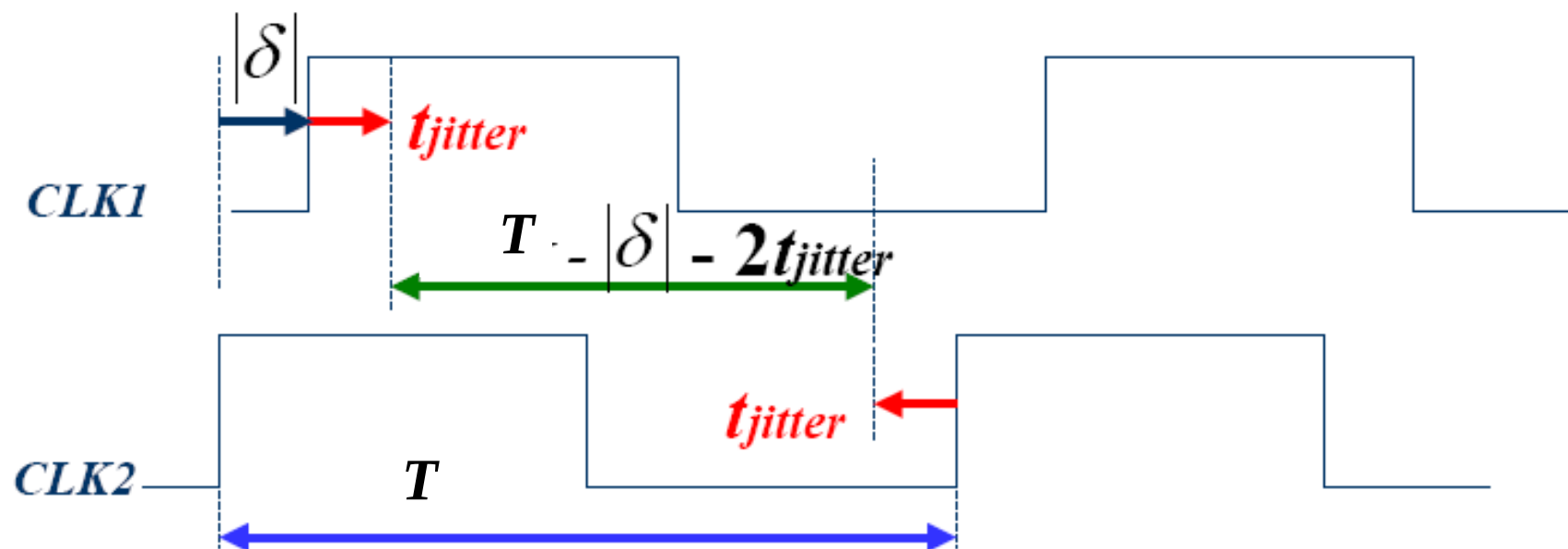
Skew 和 *Jitter* 共同作用的影响



$$T + \delta - 2t_{jitter} \geq t_{clk-q} + t_{logic} + t_{su}$$

$$T \geq t_{clk-q} + t_{logic} + t_{su} - \delta + 2t_{jitter}$$

最长时钟周期（最低时钟频率）发生在：

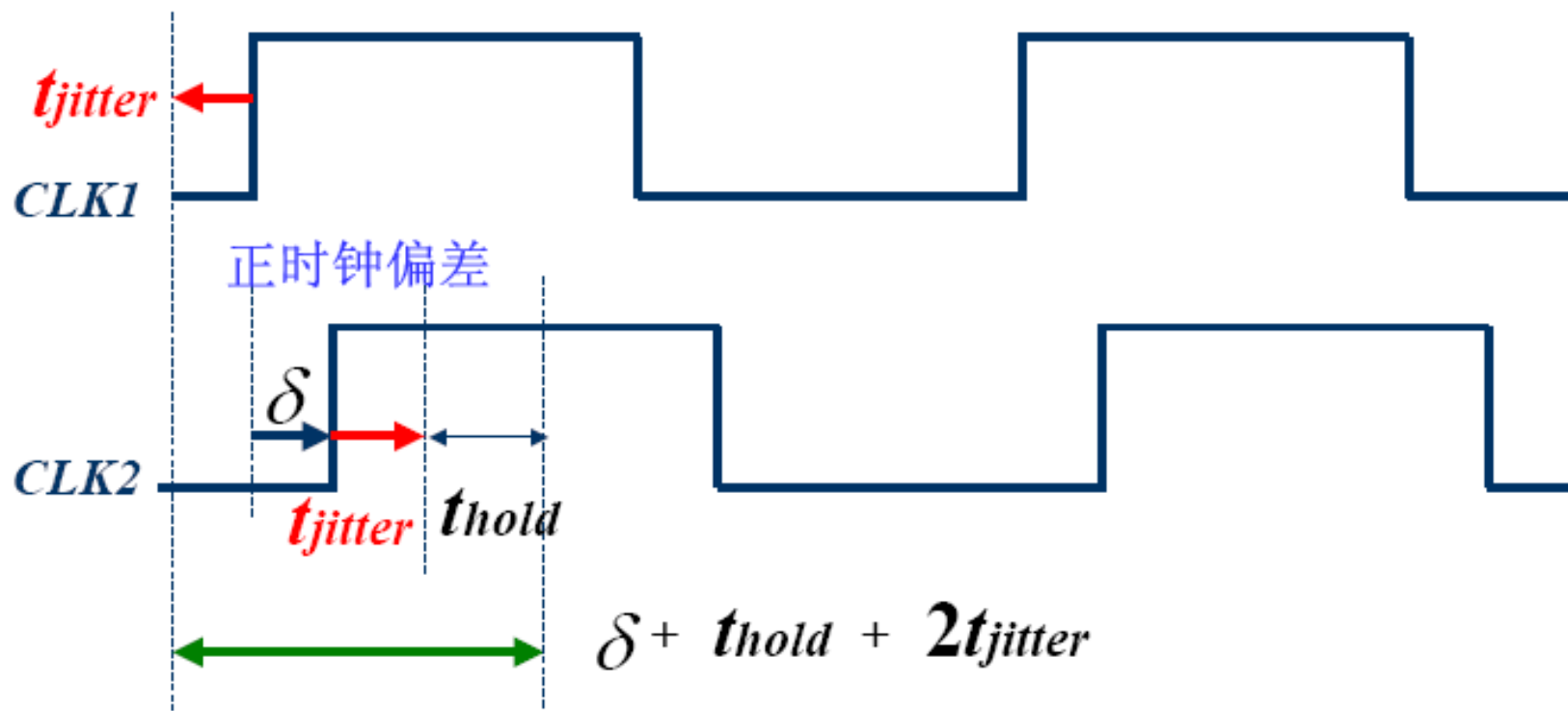


$$T_{CLK} - |\delta| - 2t_{jitter} \geq t_{clk-q} + t_{logic} + t_{su}$$

$$T_{CLK} > t_{clk-q} + t_{logic} + t_{su} + |\delta| + 2t_{jitter}$$

负时钟偏差将降低性能，抖动将降低性能

信号竞争最容易发生在：



$$\delta + t_{hold} + 2t_{jitter} < t_{clk-q,cd} + t_{logic,cd}$$

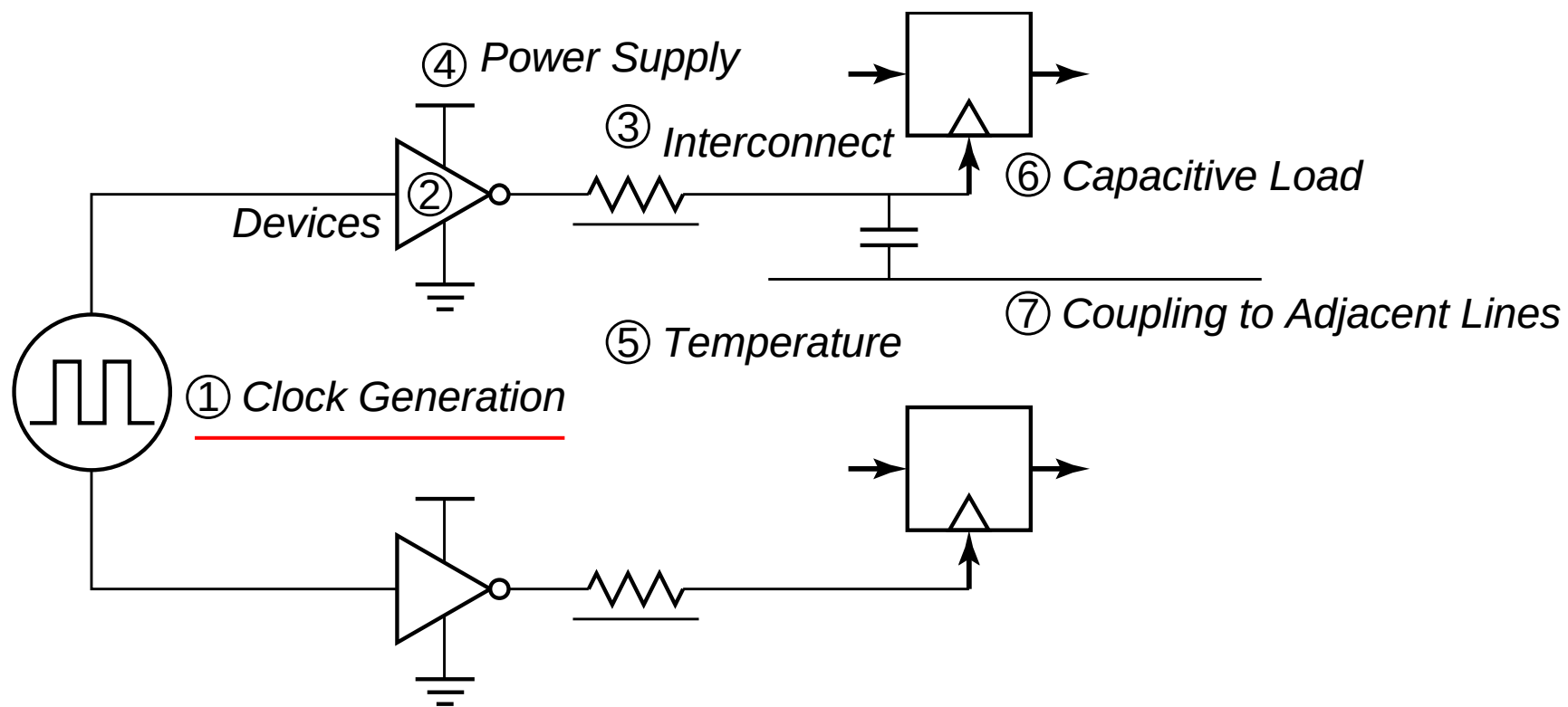
小结

- 负时钟偏差会带来性能的降低，正偏差会使竞争更容易发生
- 时钟抖动总是导致性能降低并增加竞争发生的风险
- 在设计中通常要尽力减小时钟偏差，但有时也利用它来解决竞争问题。
- 在设计中总是尽力减小时钟抖动。

9.2时钟偏差和时钟抖动

9.2.3 时钟偏差和抖动的来源

时钟的不确定性来源

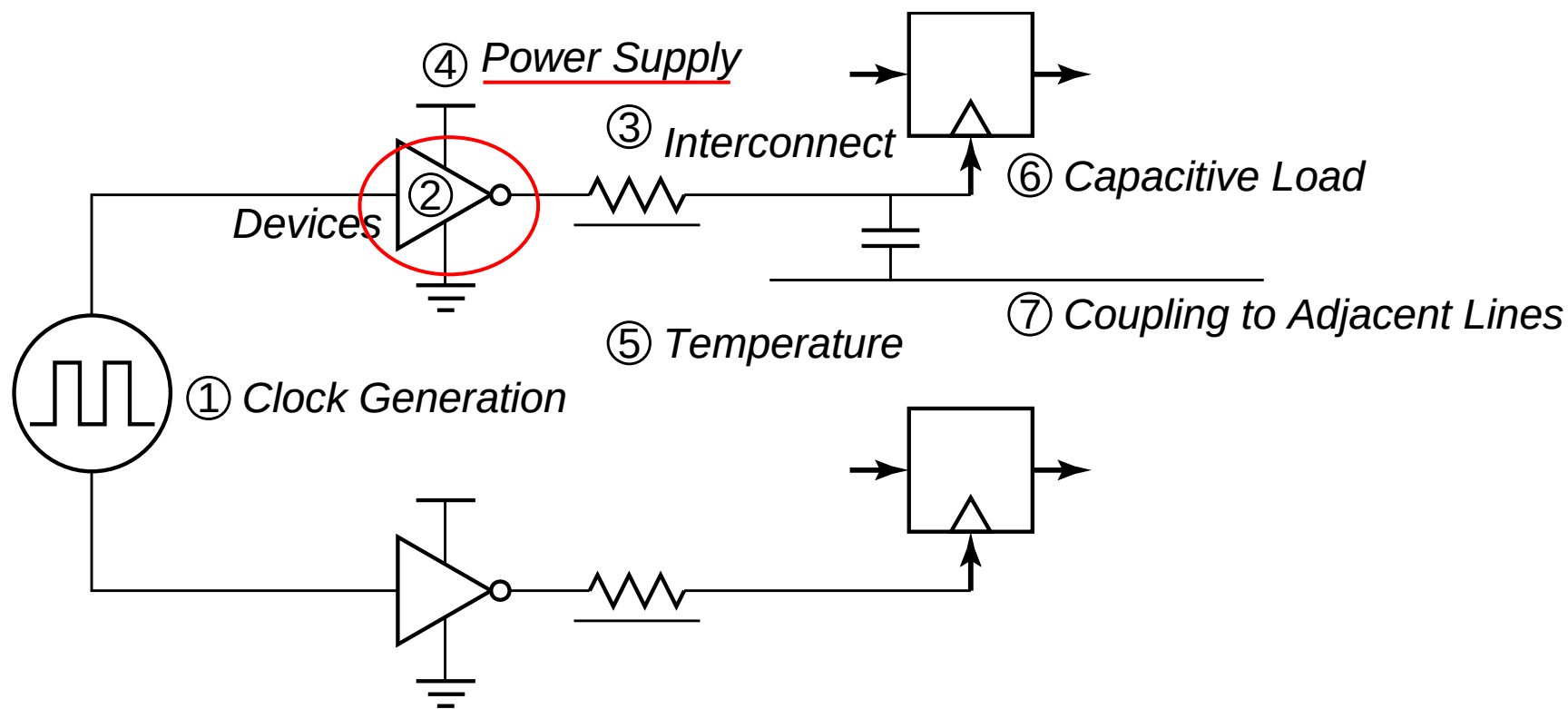


时钟不确定性的来源

时钟源的非理想因素

- 时钟源的频率不稳定会引起时钟频率的动态变化——引起时钟抖动。

时钟的不确定性来源

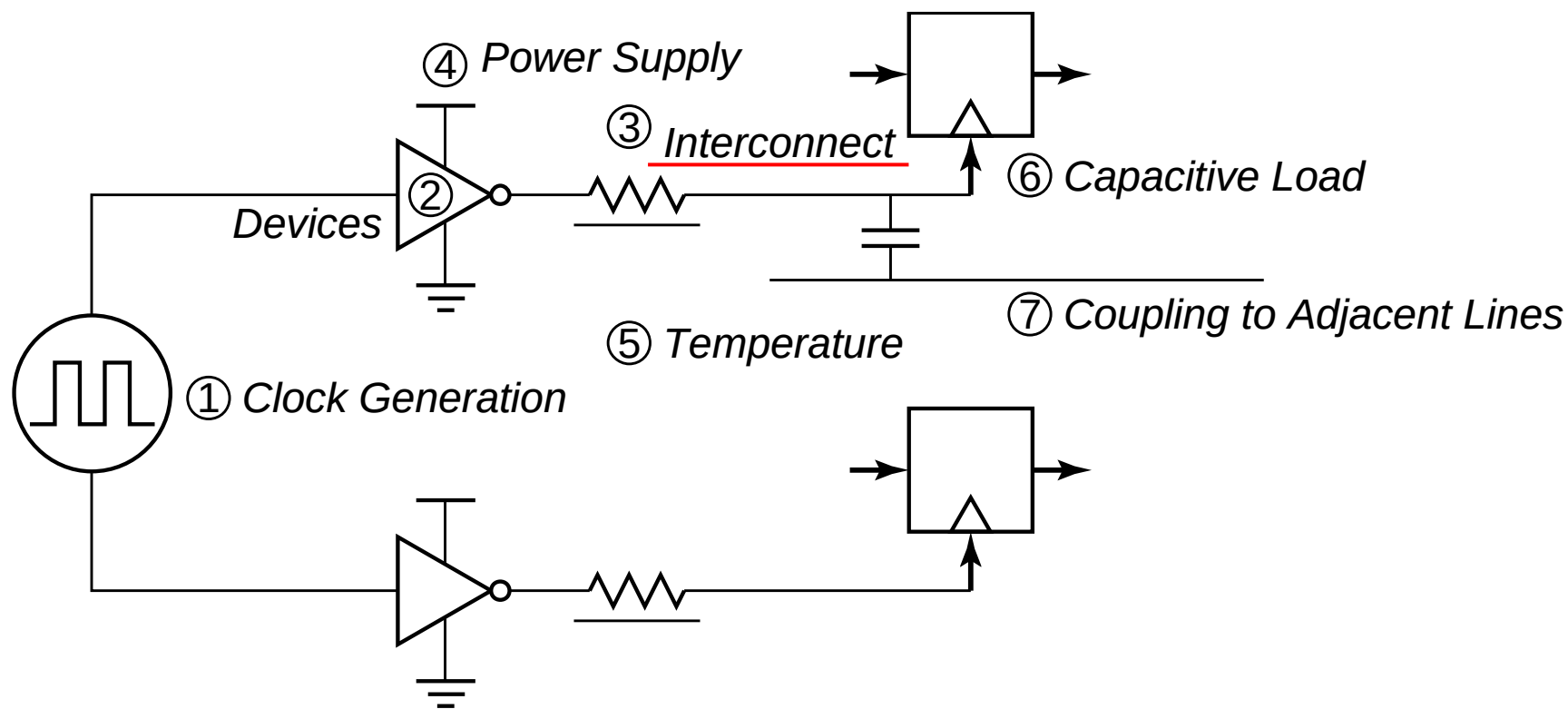


时钟不确定性的来源

时钟驱动器的影响

- 时钟驱动器之间的器件参数失配——引起时钟偏差
- 时钟延时因电源电压的波动而引起的波动——引起时钟抖动
- 时钟延时因电源电压的差异（IR drop）而引起的失配——引起时钟偏差

时钟的不确定性来源

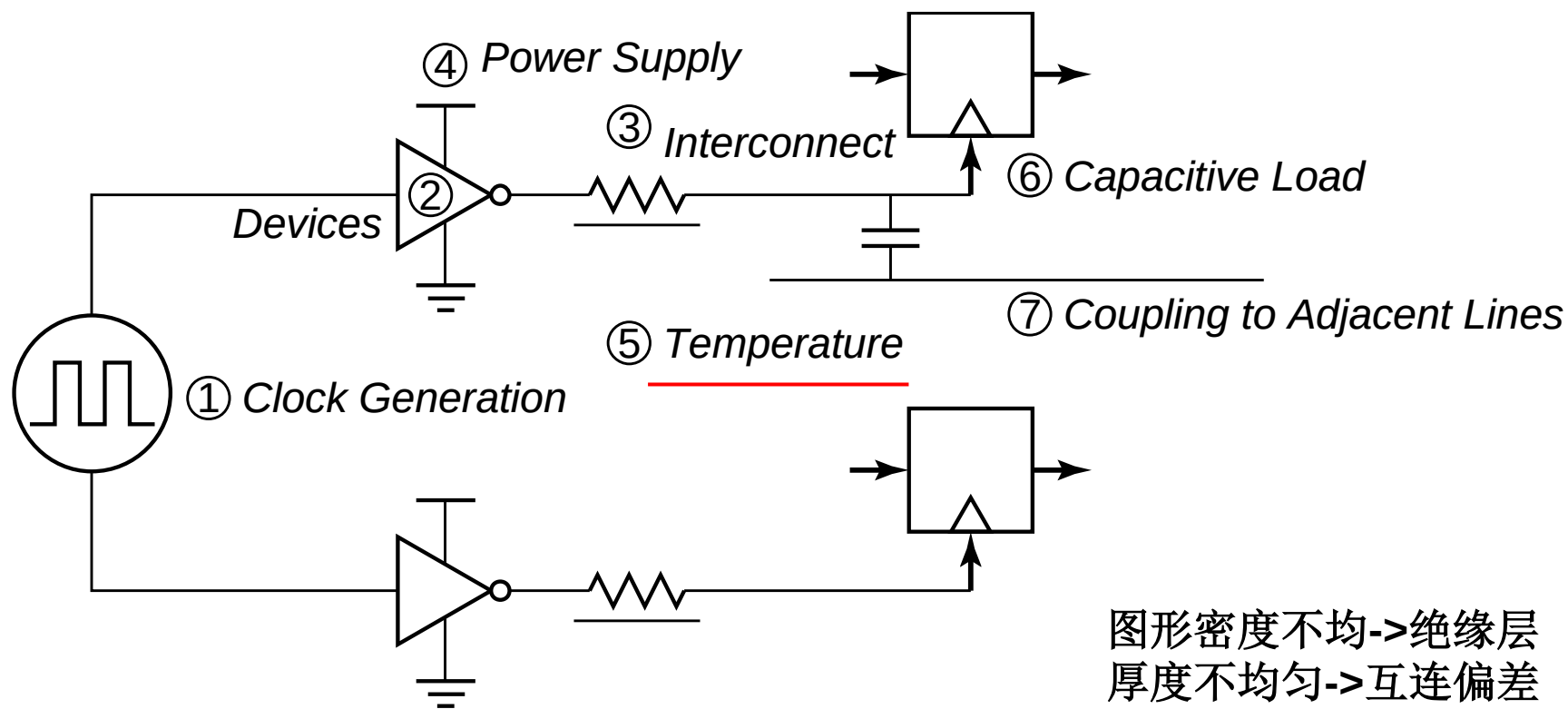


时钟不确定性的来源

时钟信号线的影响

- 互连线寄生参数的失配——引起时钟偏差
- 造成互连线寄生参数失配的主要原因是绝缘介质厚度不均匀。
——各处绝缘介质的厚度与金属图形的密度有关。

时钟的不确定性来源

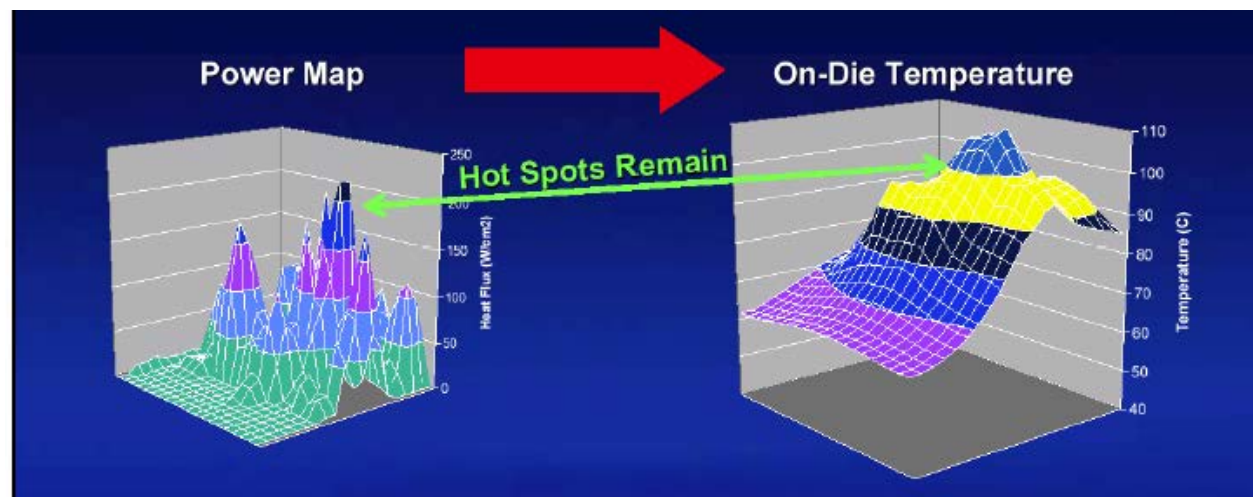


图形密度不均->绝缘层
厚度不均匀->互连偏差

时钟不确定性的来源

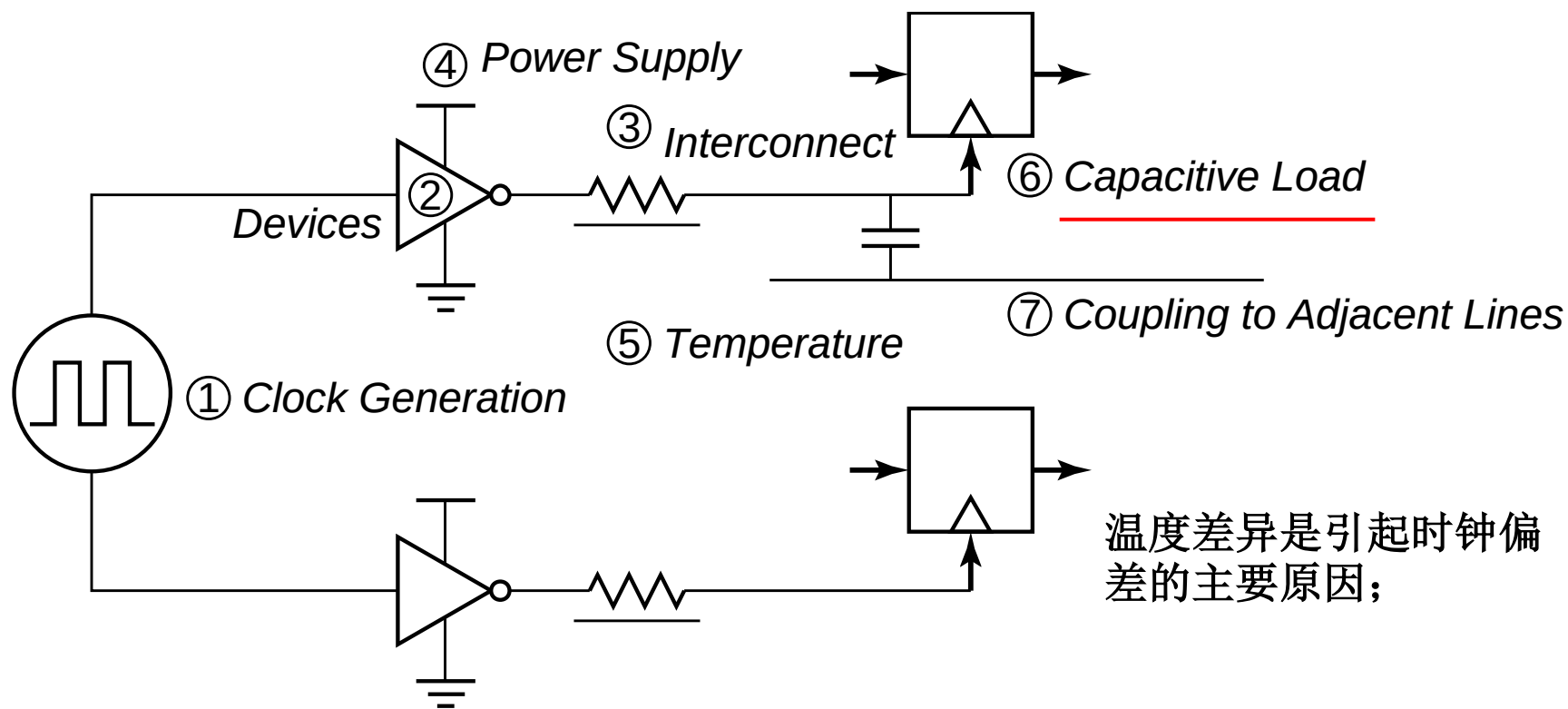
温度梯度引起的失配

- 芯片表面温度分布不均匀导致驱动器的延时和互连线的寄生电阻值失配——引起时钟偏差的主要原因。



芯片表面的功耗分密度分布和温度分布
(来源: Intel Corp.)

时钟的不确定性来源



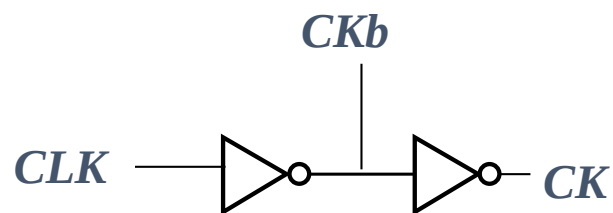
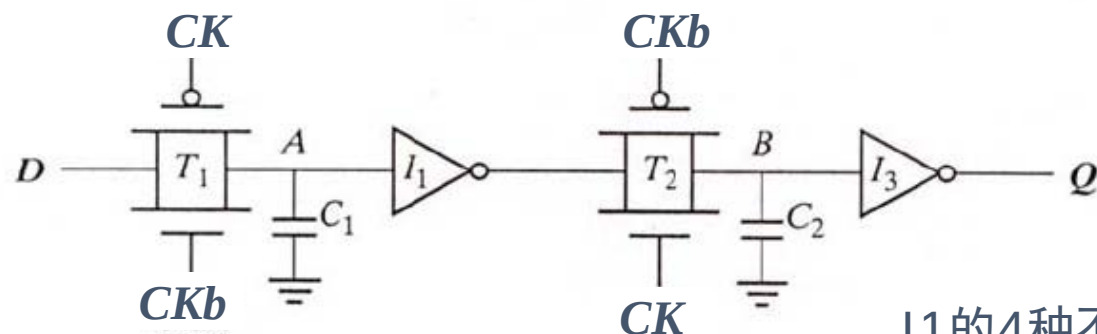
温度差异是引起时钟偏差的主要原因；

时钟不确定性的来源

时钟负载的失配

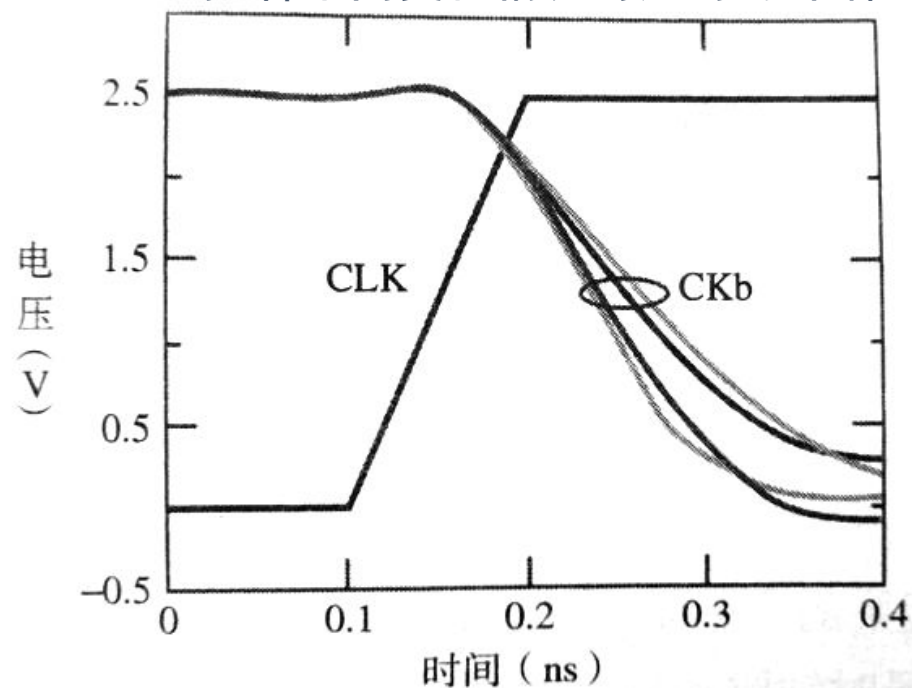
- 时钟分布网络末梢处的负载差异引起的末端驱动器延时差异——引起时钟偏差
- 差异来源：
 - 时序单元的时钟负载不同
 - 本地时钟驱动器类型的差异：如，门控时钟单元

与数据有关的抖动

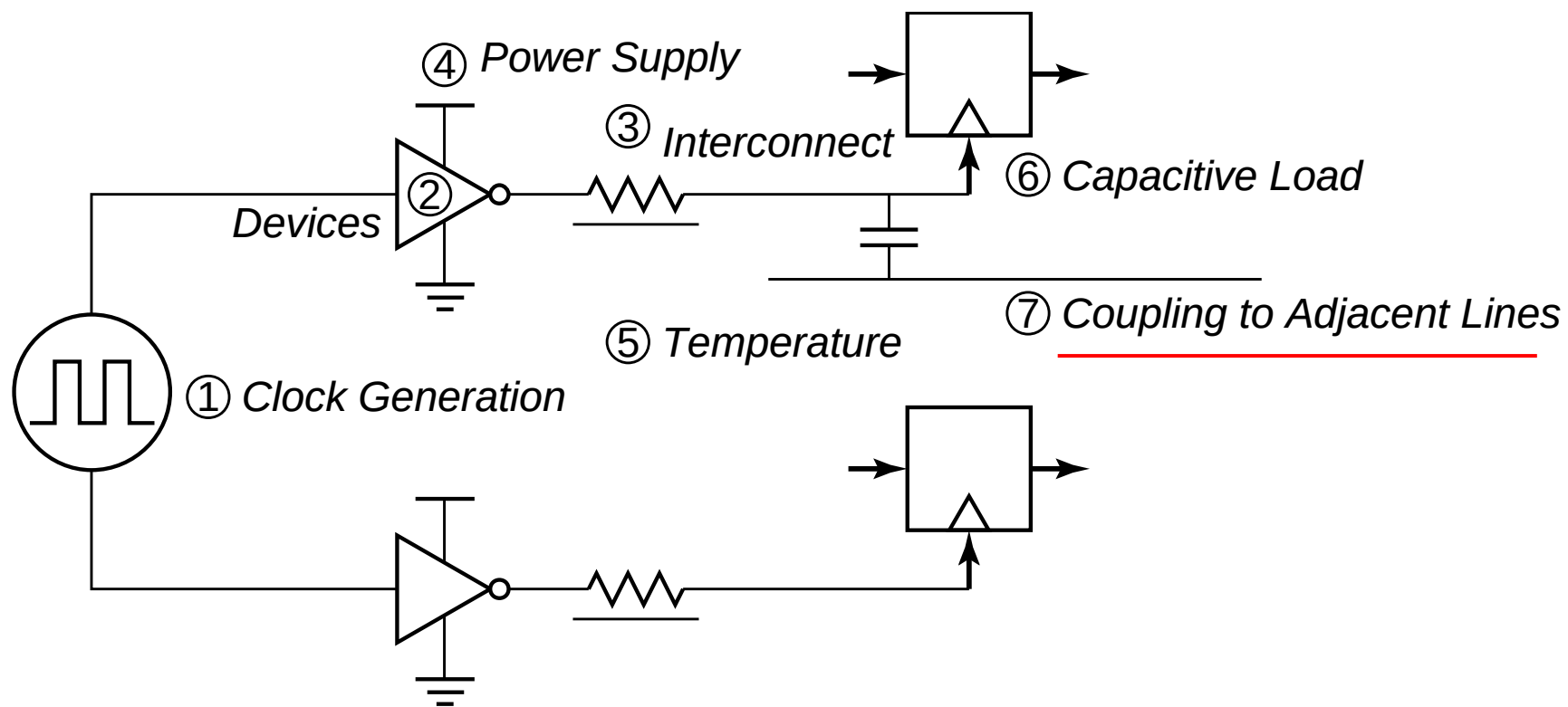


——引起时钟抖动

I1的4种不同变化情况对应的时钟沿



时钟的不确定性来源

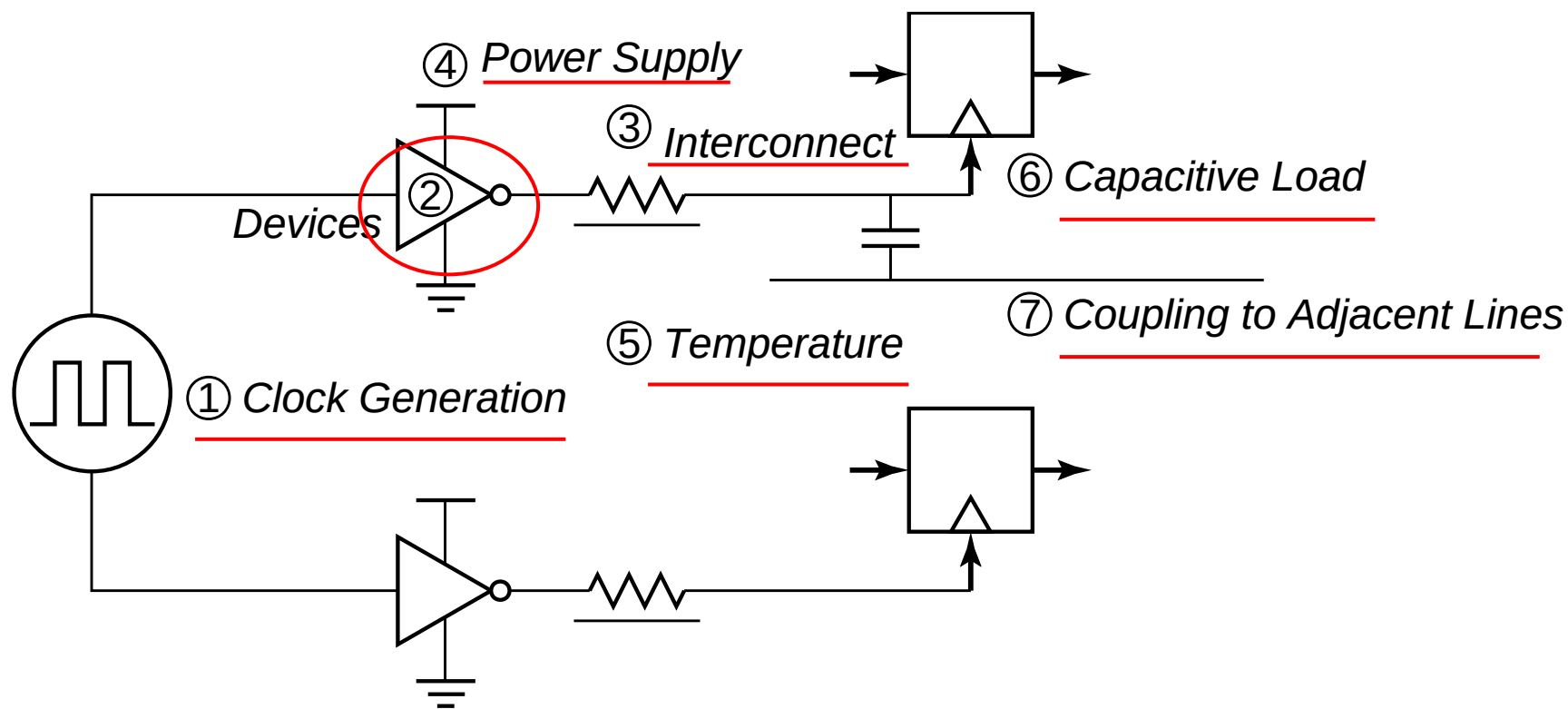


时钟不确定性的来源

噪声干扰

- 时钟线邻近信号线对时钟的串扰
- 串扰与信号翻转方向相关，且干扰信号可看做是随机变化的信号，因此串扰会引起时钟信号延时的随机波动——引起时钟抖动。

时钟的不确定性来源



时钟不确定性的来源

影响时钟质量的因素小结

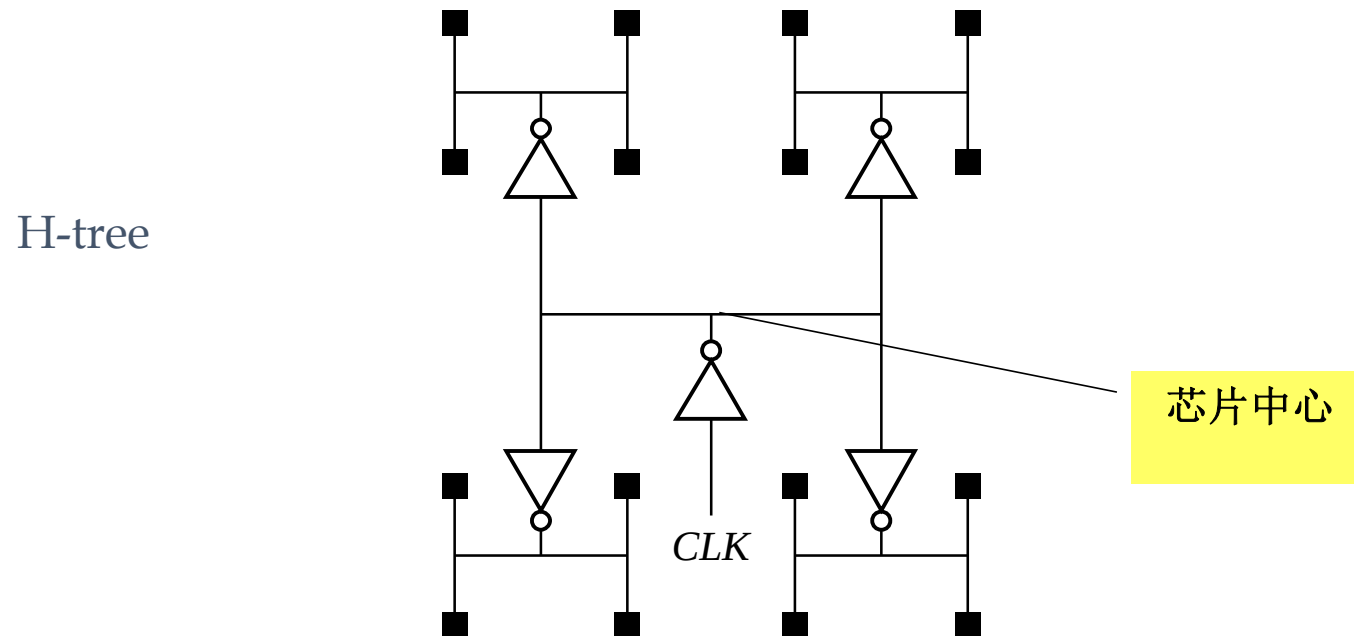
因素	时钟偏差	时钟抖动
时钟源		时钟源噪声、抖动
时钟分布	时钟驱动器和互连线的工艺偏差、 温度分布差异	IR drop ，串扰（数据线对时钟的干扰）
时钟树末端	负载差异	数据相关的时钟抖动

改善时钟偏差和抖动的方法

- 在时钟驱动器附近插入去耦电容。
- 在版图中插入虚假金属图形，使得版图中的金属图形密度均匀。
- 控制芯片表面的温度梯度（功耗密度）。
- 避免使用门控时钟。
- 减小IR drop。
- 为了避免相邻的信号线对时钟信号的串扰引入数据相关的边沿抖动，应该在时钟线旁边（两侧甚至上下）放置电源线或地线进行屏蔽。

9.3 时钟分布网络

H-树时钟网络



用一个树状网络把参照时钟（全局时钟）分布到芯片的各个部分。

——采用均衡路径。

H-树（续）

- 由于所有的子功能块与时钟源是等距离的，因此对于每一个子功能块，时钟信号被延迟一个相同的数量。
- H 树形态是一个理想的模型，只适合于规整的阵列网络。在这样的网络中，所有的元件都是相同的，分布均匀。但这一概念可以延伸为分配时钟时平面布置的原则，即使得时钟信号至末端的互连线具有相同的长度（或相同的延迟时间）。

RC匹配分布网路

芯片被划分成负载相同的**10**个瓦片（**tile**），
全局时钟被分布到**10**个瓦片时钟驱动器上，
瓦片时钟驱动器通过
较低层次的**RC**匹配树
驱动其内部的驱动器。

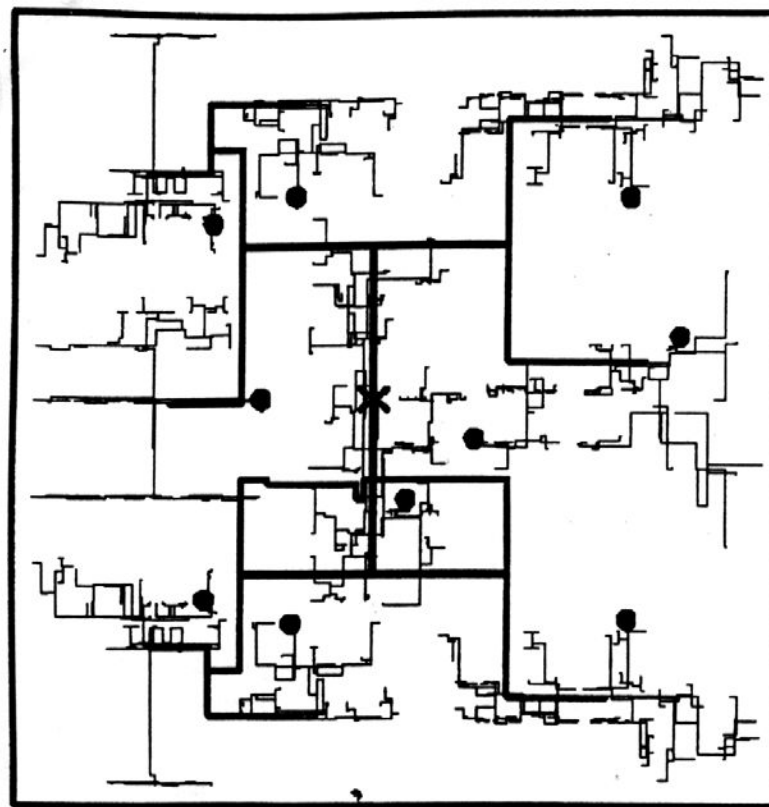


Fig. 1. CMOS S/390 microprocessor first- and second-level wiring tree, with dots showing the measurement points for waveforms in Fig. 3. The wire widths are not to scale. The wide wires are the first-level wiring tree driving nine sector buffers, and the narrow lines are the second-level trees.

[Restle98]

RC匹配时钟网络布线的一般方法:

- 对时钟信号**仔细布线**。
- **层次化的时钟缓冲技术**使本地时钟的延时相同，从而控制时钟**Skew**。中间究竟应当有几个时钟的缓冲层取决于互连线的材料尺寸，也取决于时钟网络的扇出。
- **缓冲器的作用**:
 - 把本地的时钟节点与时钟源隔离，以减轻时钟源的驱动负载。
 - 改善因时钟线的**RC**网络造成的时钟波形的变差，以便产生陡直的时钟沿
 - 减少了绝对的延时值。

这一方法并不能达到零的Skew 。实际上也没有必要达到零的Skew ，而只需要把Skew 限定在一定的范围内。

9.4 时钟技术小结

时钟技术小结

- **Skew:**

- 利用**H**树及一般的匹配树结构可以均衡从中心至各个钟控单元的**Skew**。
- 布线时钟树时，应使每条路径时钟的等效负载（包括器件与连线）相同。
- 通过虚材料填充可以减小由于层间介质厚度不同引起互连线电容的变化（因而减小所产生的**Skew**）。
- 使时钟沿与数据流相反的方向布线可以避免信号竞争，但将损失性能。

时钟技术小结（续）

- **Jitter:**

- 为节省功耗采用**门控时钟**时会使负载状况与数据相关，从而引起严重的**Jitter**
- 可采用差分寄存器解决因时钟负载与数据相关而引起的严重的**Jitter**。
- 电源电压是**Jitter**的重要来源，必须充分降低IR drop
- 增加去耦电容可减小高频的电源变化，但需很多芯片面积，为此要有高效的封装
 - 在时钟驱动器附近插入去耦电容

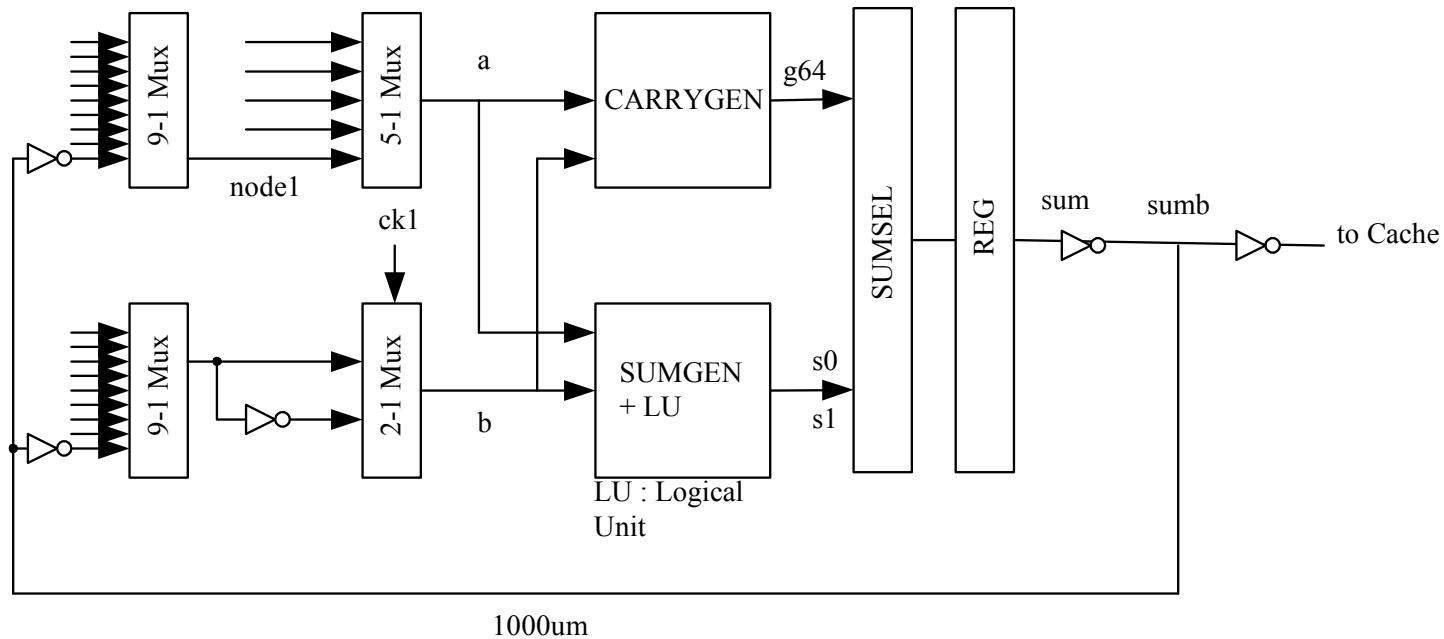
时钟技术小结（续）

- 屏蔽
 - 使时钟线与相邻的信号线相屏蔽可避免与数据相关的噪声。
 - 如果不采用屏蔽，应将时钟线与相邻信号线的间距拉大
- 温度影响
 - 温度变化将引起时钟缓冲器延时的变化，可采用基于延时锁定环（**delayed -locked loop**）的方法来进行补偿。

第十章 数据通路

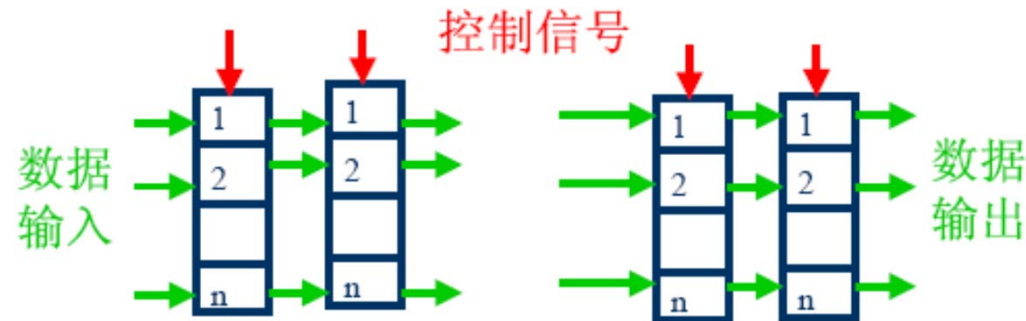
10.1 引言——数据通路的特点

Intel Itanium 处理器的数据通路



数据通路的特点：

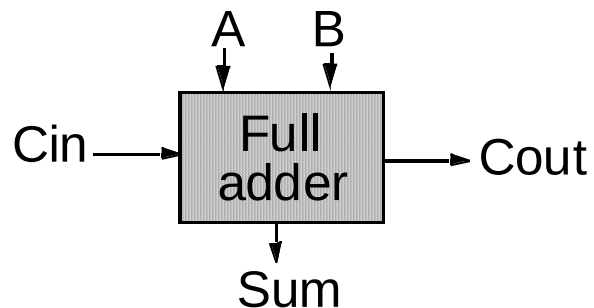
- 规整性：位片式实现
- 局域化：运算顺序执行，级联结构
- 正交性：数据流、控制流
- 层次化（位片式结构）：高位->低位，多位->少位，不同层次不同实现
- 模块化：各种IP 模块，如：加法器、乘法器



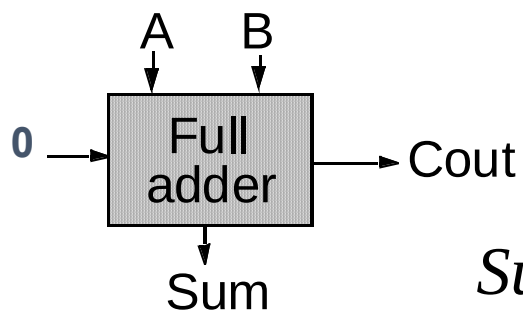
10.2 加法器

10.2.1 数字电路中的加法运算

二进制加法



全加器



半加器

A	B	C_i	S	C_o	Carry status
0	0	0	0	0	delete
0	0	1	1	0	delete
0	1	0	1	0	propagate
0	1	1	0	1	propagate
1	0	0	1	0	propagate
1	0	1	0	1	propagate
1	1	0	0	1	generate
1	1	1	1	1	generate

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A + B) \quad (\text{major运算})$$

进位产生、进位取消、进位传播信号

- 进位产生信号 (G) $G = AB$
- 进位消除信号 (K或D) $K = \overline{A}\overline{B}$
- 进位传播信号 (P) $P = A \oplus B$

$$S(G, P) = P \oplus C_i$$

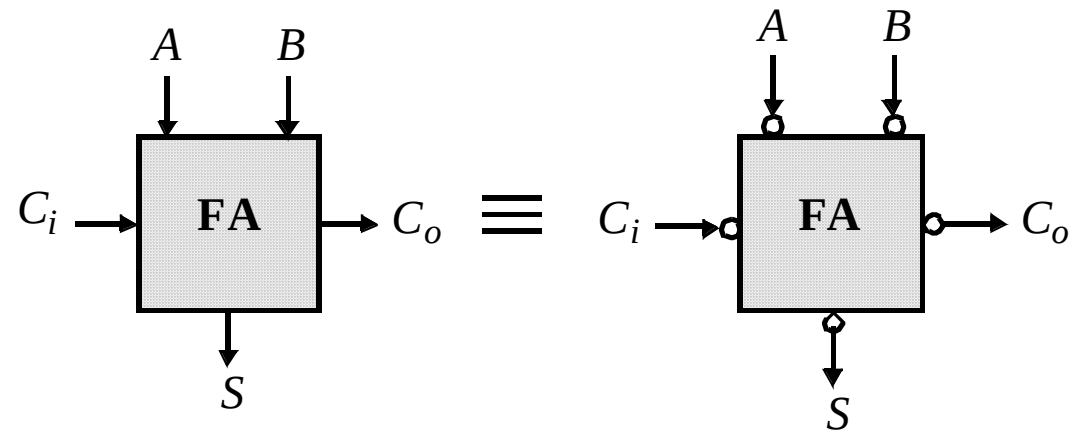
$$C_o(G, P) = G + PC_i$$

也可以取 $A+B$

$$C_o = AB + BC_i + AC_i$$

$$S = ABC_i + \overline{C_o}(A + B + C_i)$$

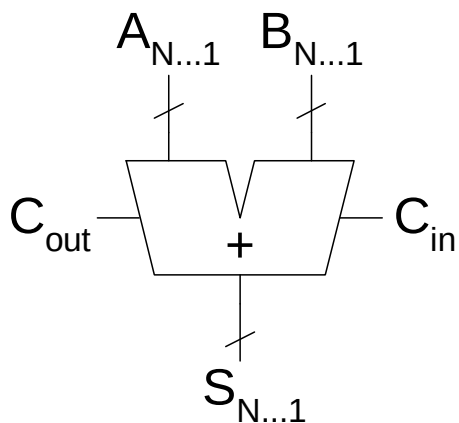
反相特性



$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i)$$
$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i)$$

进位传播加法器

- N位加法器
 - 每位和与前面所有位的进位相关



$$\begin{array}{r}
 \textcircled{0}\textcircled{0}\textcircled{0}\textcircled{0}\textcircled{0} \\
 1111 \\
 +0000 \\
 \hline
 1111
 \end{array}$$

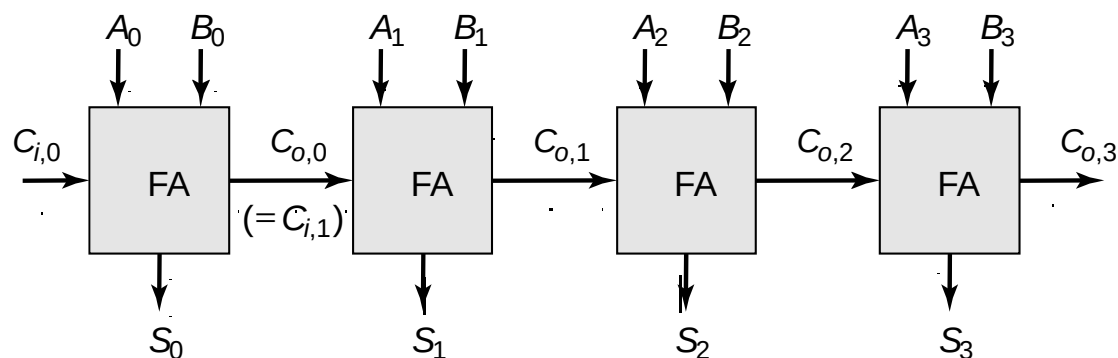
Diagram illustrating a 5-bit ripple-carry adder. The inputs are 00000 (with the first 0 circled and labeled C_{in}) and 1111. The output is 1111. The carry-out C_{out} is indicated by an arrow pointing to the first 0.

$$\begin{array}{r}
 \textcircled{1}\textcircled{1}\textcircled{1}\textcircled{1}\textcircled{1} \\
 1111 \\
 +0000 \\
 \hline
 0000
 \end{array}$$

Diagram illustrating a 5-bit ripple-carry adder. The inputs are 11111 (with the first 1 circled and labeled C_{in}) and 1111. The output is 0000. The carry-out C_{out} is indicated by an arrow pointing to the first 1. The label "carries" is next to the first 1.

N位加法器

行波进位（Ripple-Carry）加法器



最坏情况的延时与加数位数N成线性关系

$$t_d = O(N)$$

$$t_{adder} = (N-1)t_{carry} + t_{sum}$$

关键： 加快进位信号传播的速度

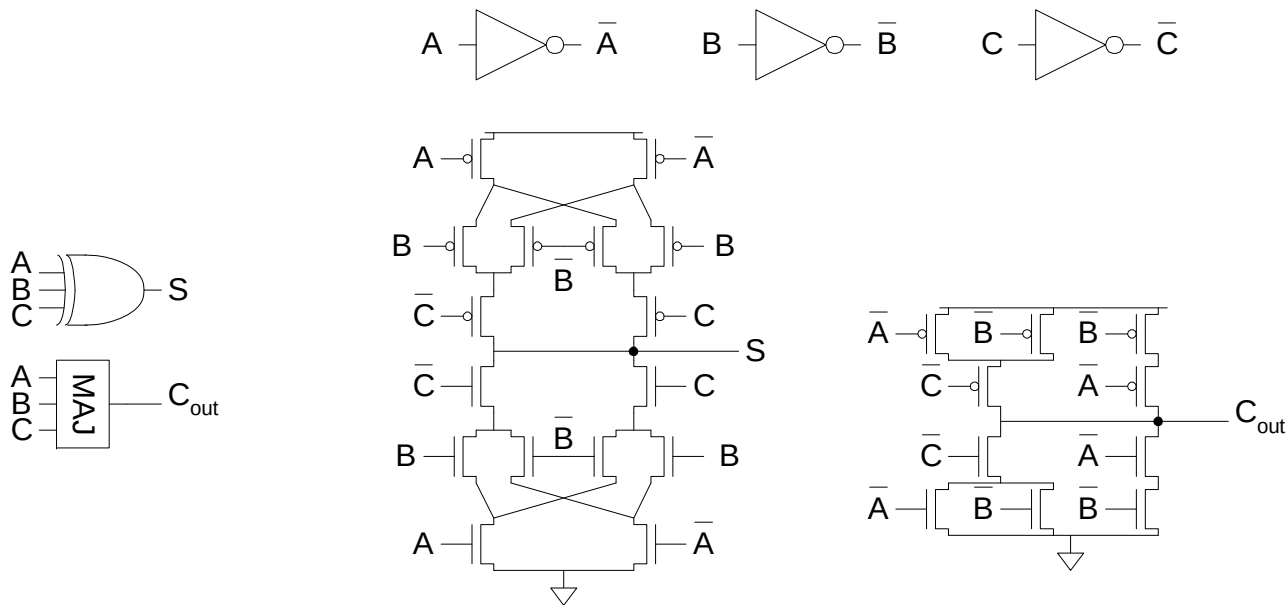
10.2 加法器

10.2.2 静态互补CMOS全加器

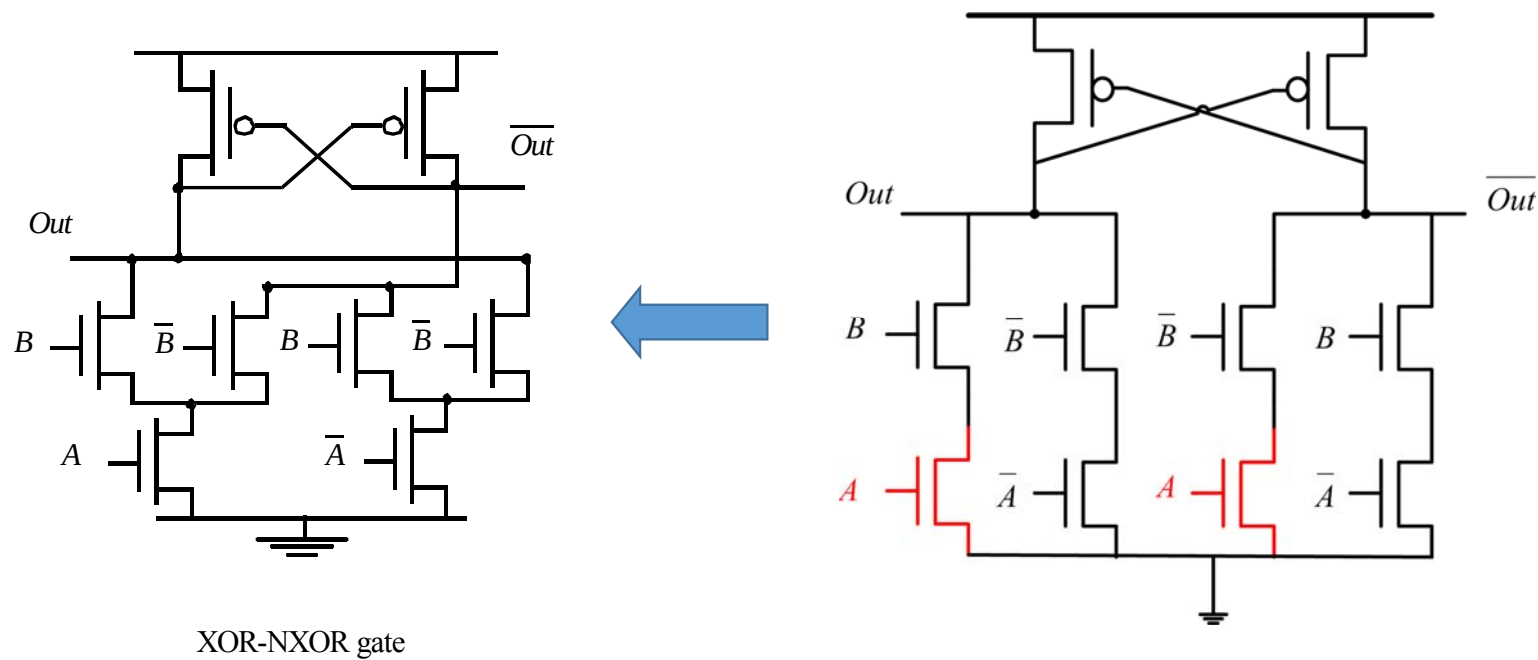
全加器电路（1）——复合CMOS门

$$S = A \oplus B \oplus C_i = ABC_i + \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i$$

$$C_o = AB + C_i(A + B)$$



第七章回顾： 逻辑共享



全加器电路（1）——复合CMOS门

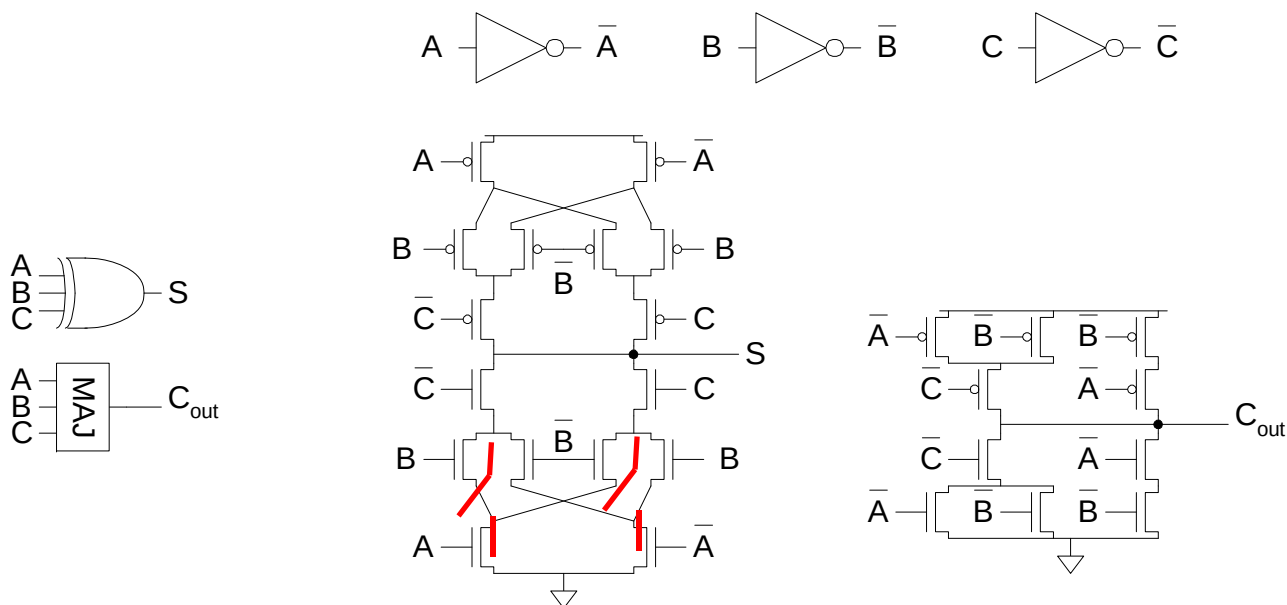
$$S = A \oplus B \oplus C_i = ABC_i + \overline{A}\overline{B}C_i + \overline{A}B\overline{C}_i + A\overline{B}\overline{C}_i \quad \text{16 个管子}$$

$$C_o = AB + C_i(A + B) \quad \text{10 个管子}$$

3个反相器

6 个管子

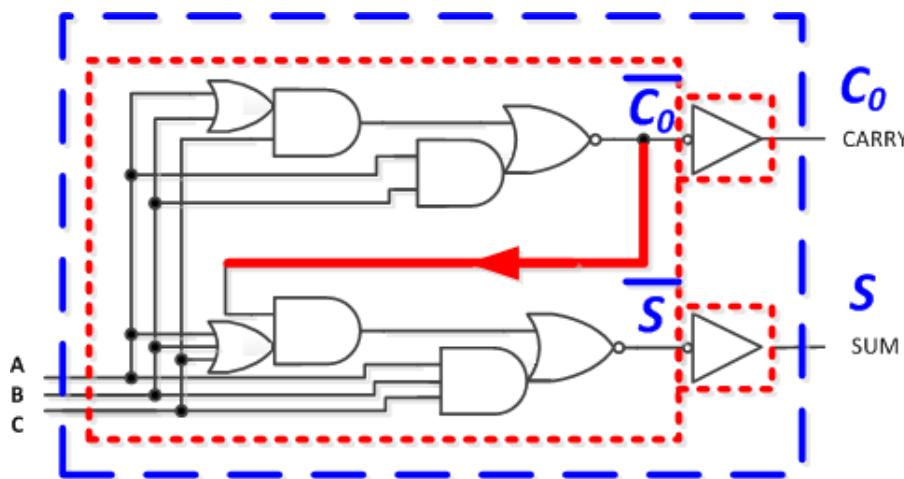
共32个管子（多个管子串联）



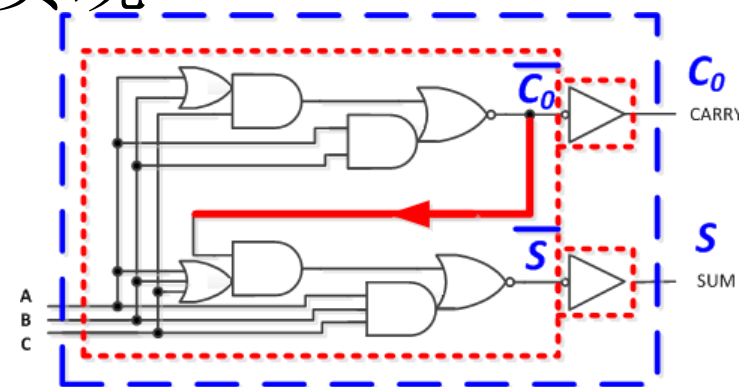
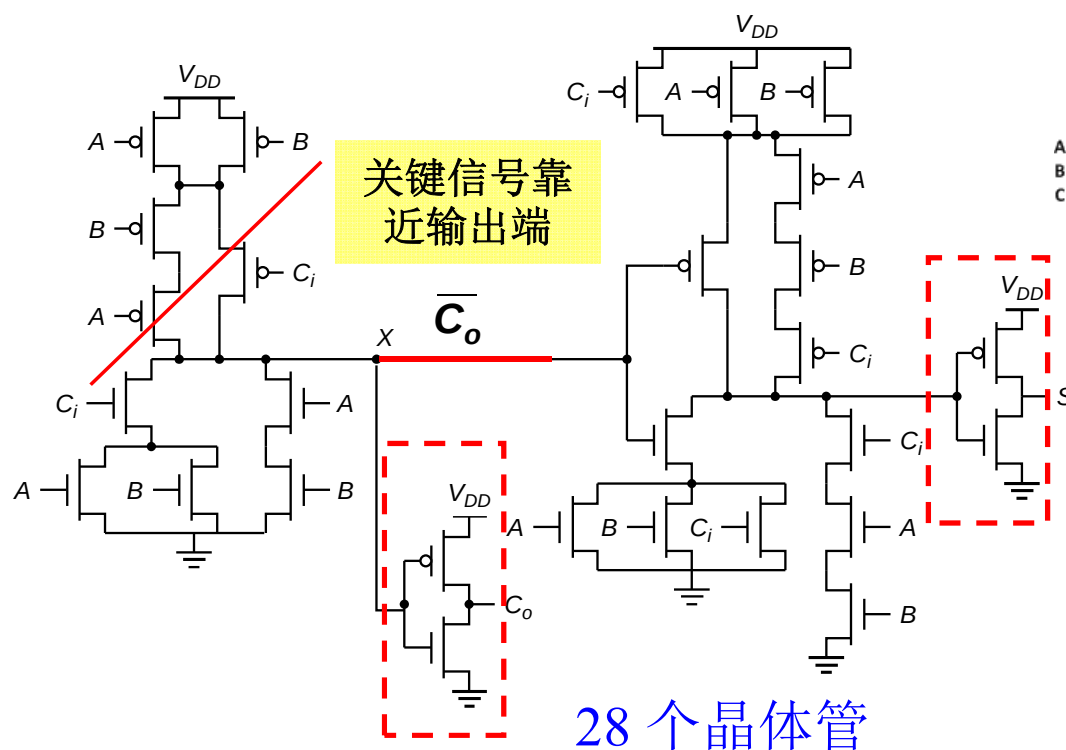
全加器电路（2）——无XOR复合门实现

利用输出进位产生“和”位（“和”位与“进位”子电路之间共享逻辑）

$$C_o = AB + BC_i + AC_i$$
$$S = ABC_i + \overline{C_o}(A + B + C_i)$$



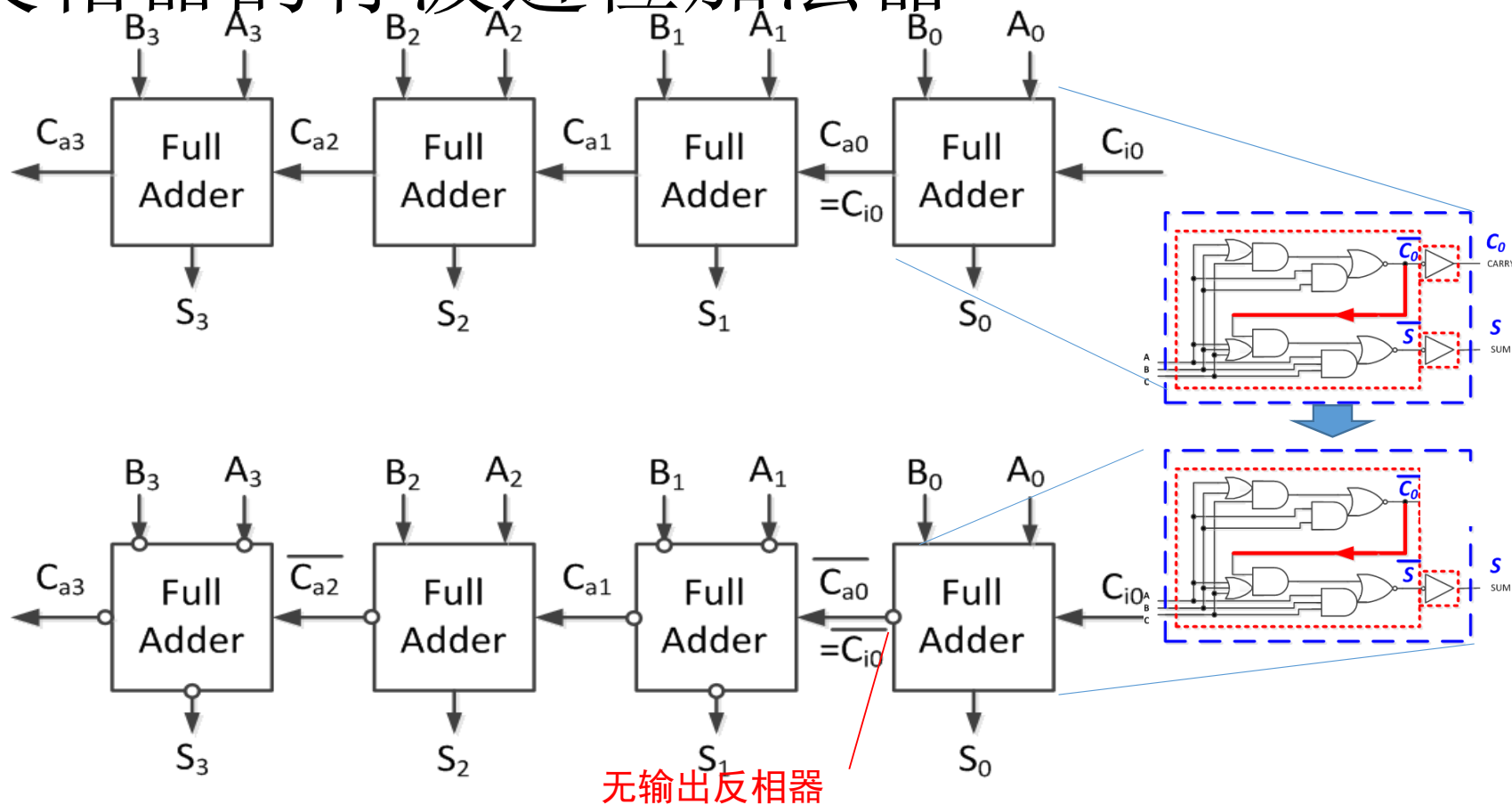
全加器电路（2）——无XOR复合门实现



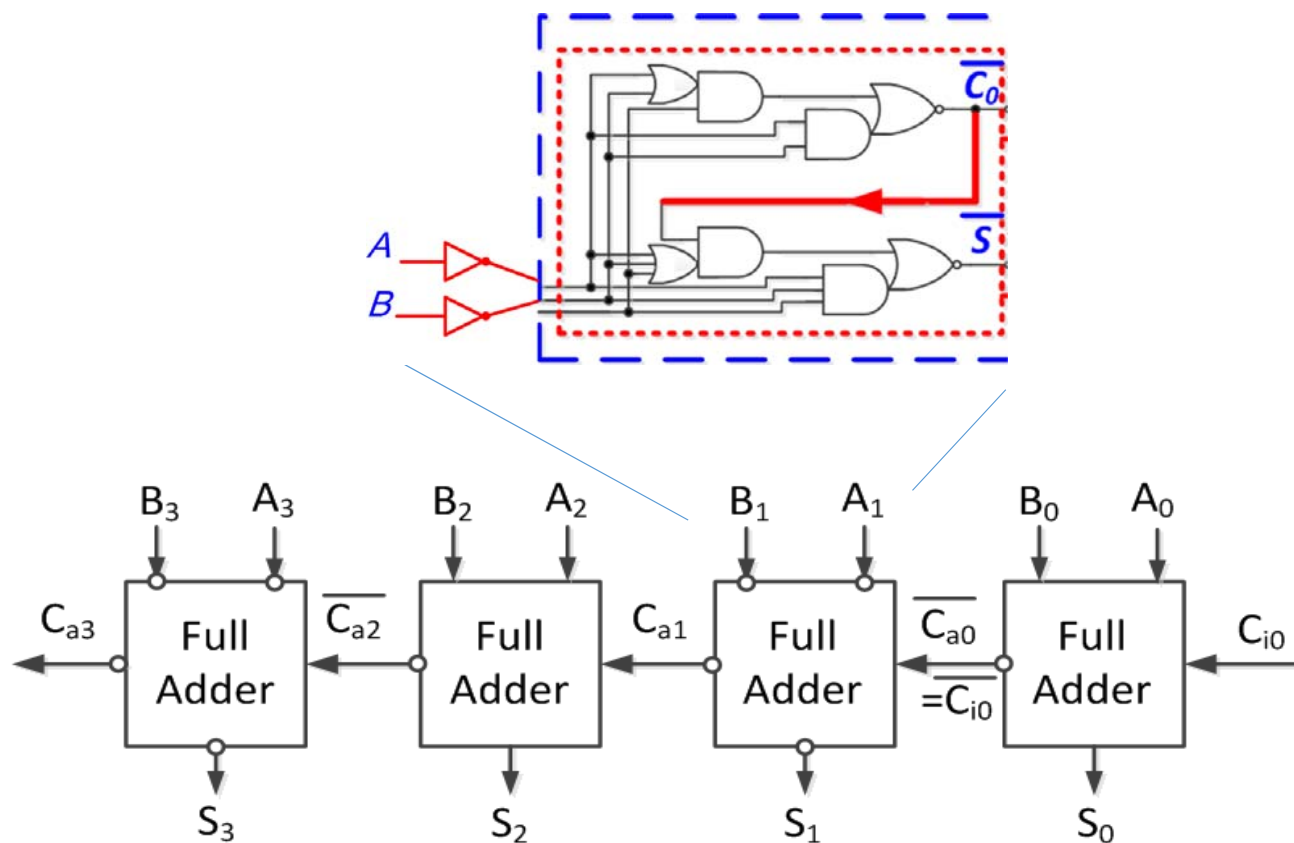
$$C_o = AB + BC_i + AC_i$$

$$S = ABC_i + \overline{C_o}(A + B + C_i)$$

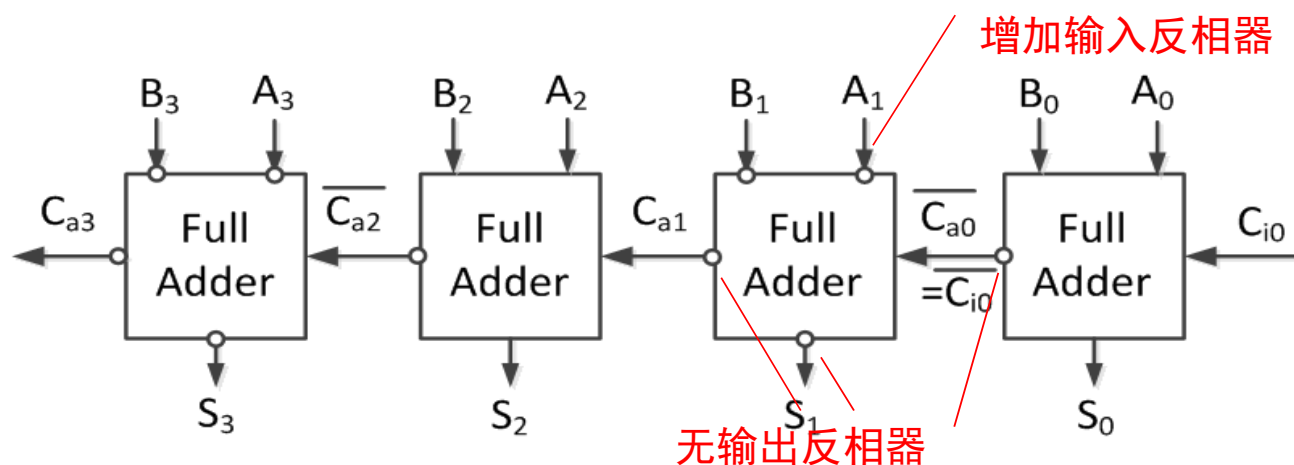
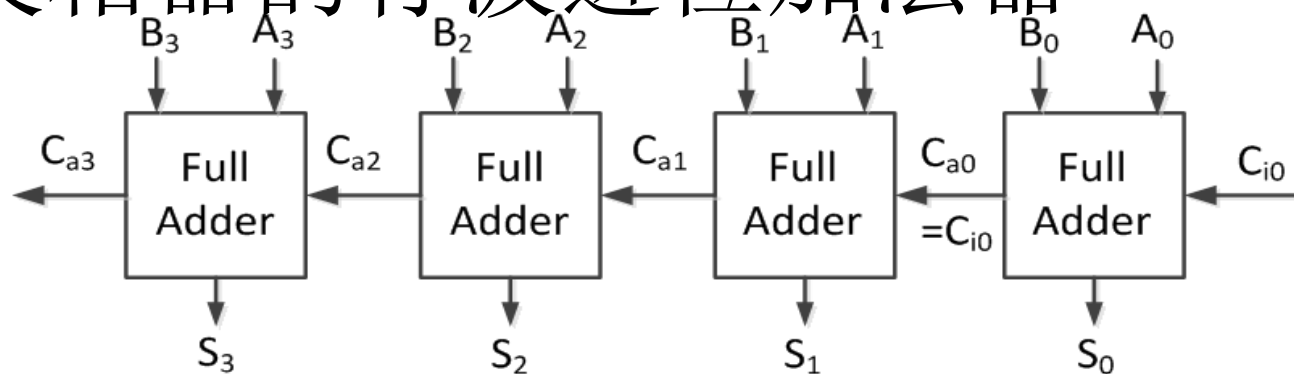
消除反相器的行波进位加法器



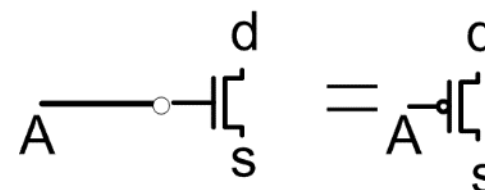
消除反相器的行波进位加法器



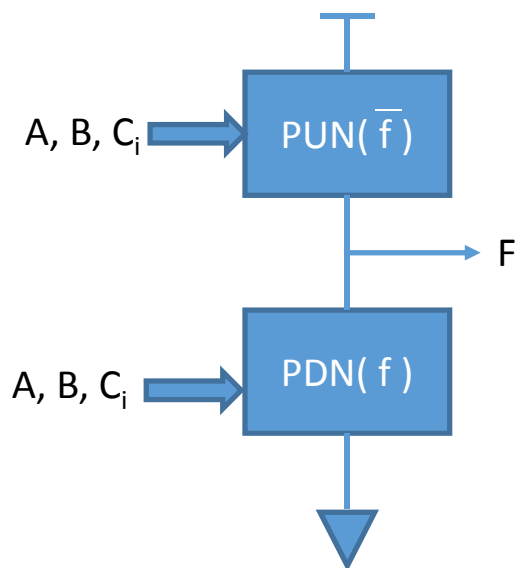
消除反相器的行波进位加法器



全加器镜像结构原理

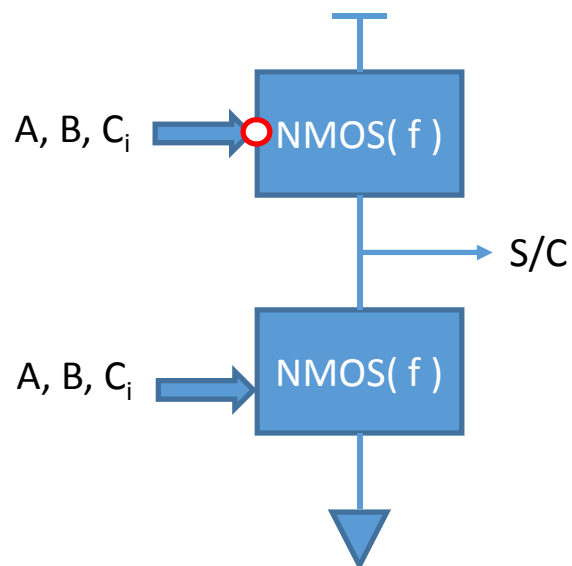


一般逻辑



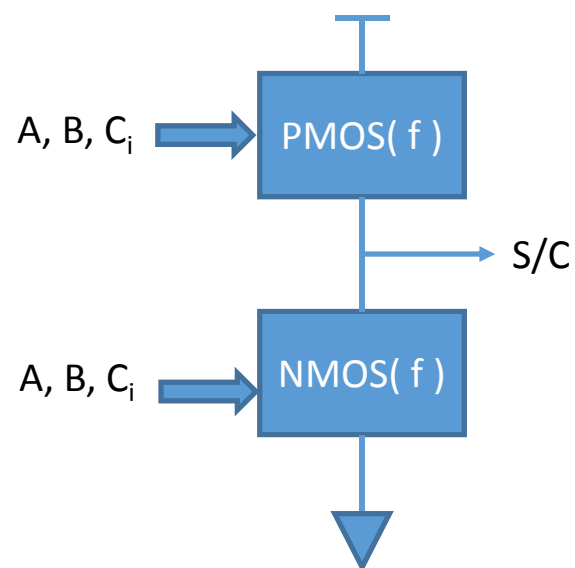
$$\bar{f}(A, B, C_i) = \overline{f(A, B, C_i)} = F$$

加法逻辑



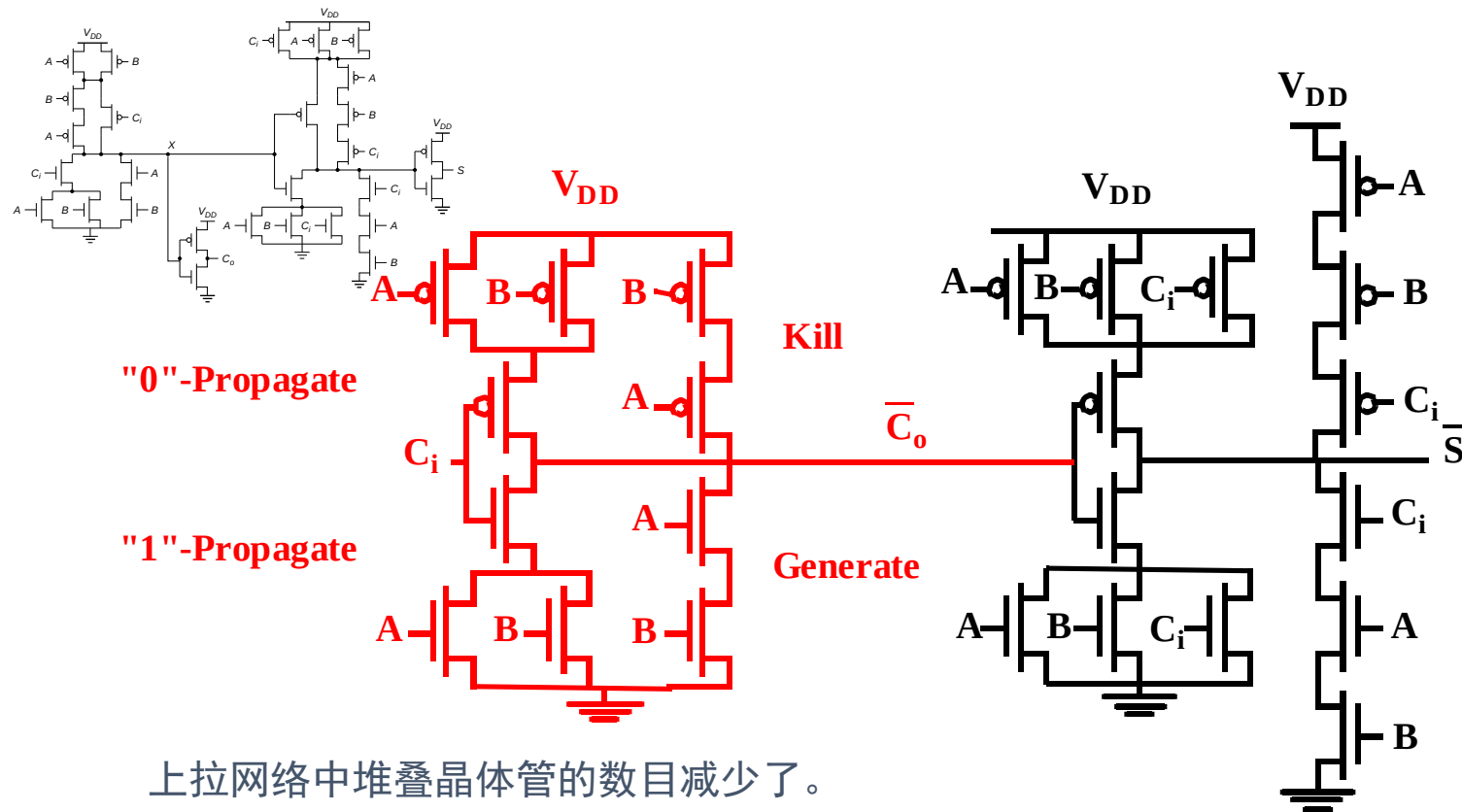
$$f(\bar{A}, \bar{B}, \bar{C}_i) = \overline{f(A, B, C_i)}$$

镜像加法器



上拉的P管网络和下拉的N管网络具有相同的拓扑结构

全加器电路（3）——镜像加法器



上拉网络中堆叠晶体管的数目减少了。

24 transistors（已消除反相器）

10.2 加法器

10.2.3 传输管逻辑全加器

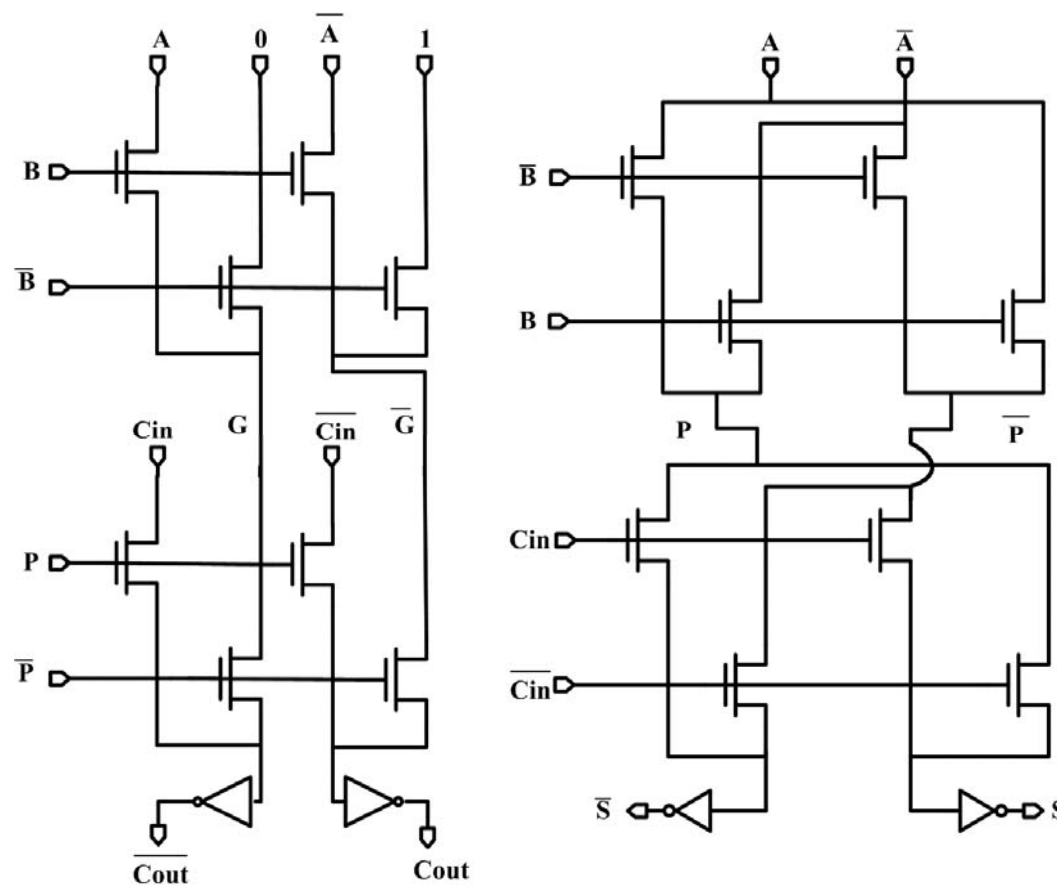
全加器电路（4）——CPL全加器

$$G = AB$$

$$P = A \oplus B$$

$$S = P \oplus C_{in}$$

$$C_{out} = G + PC_{in}$$



24个晶体管

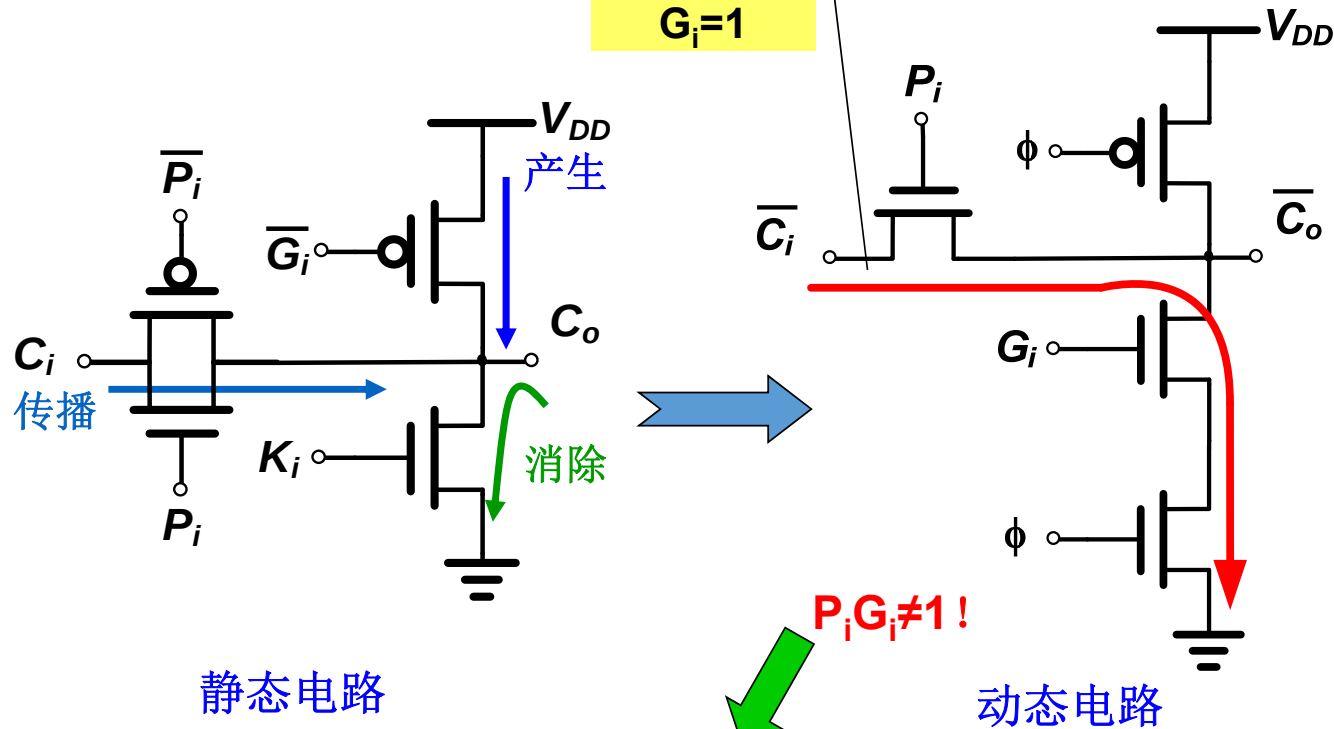
10.2 加法器

10.2.4 动态逻辑全加器

全加器电路（5）——曼彻斯特进位门

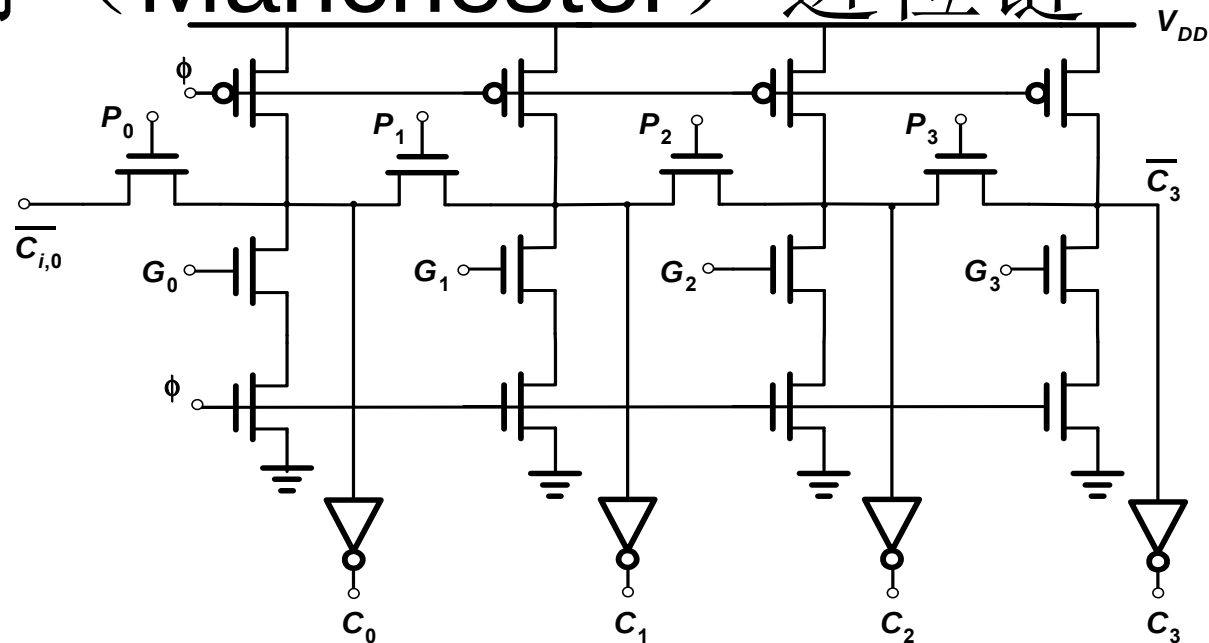
曼彻斯特（Manchester）进位门

若 $P_i=1$ 且
 $G_i=1$



$$P_i = a_i \oplus b_i, G_i = a_i b_i$$

曼彻斯特（Manchester）进位链

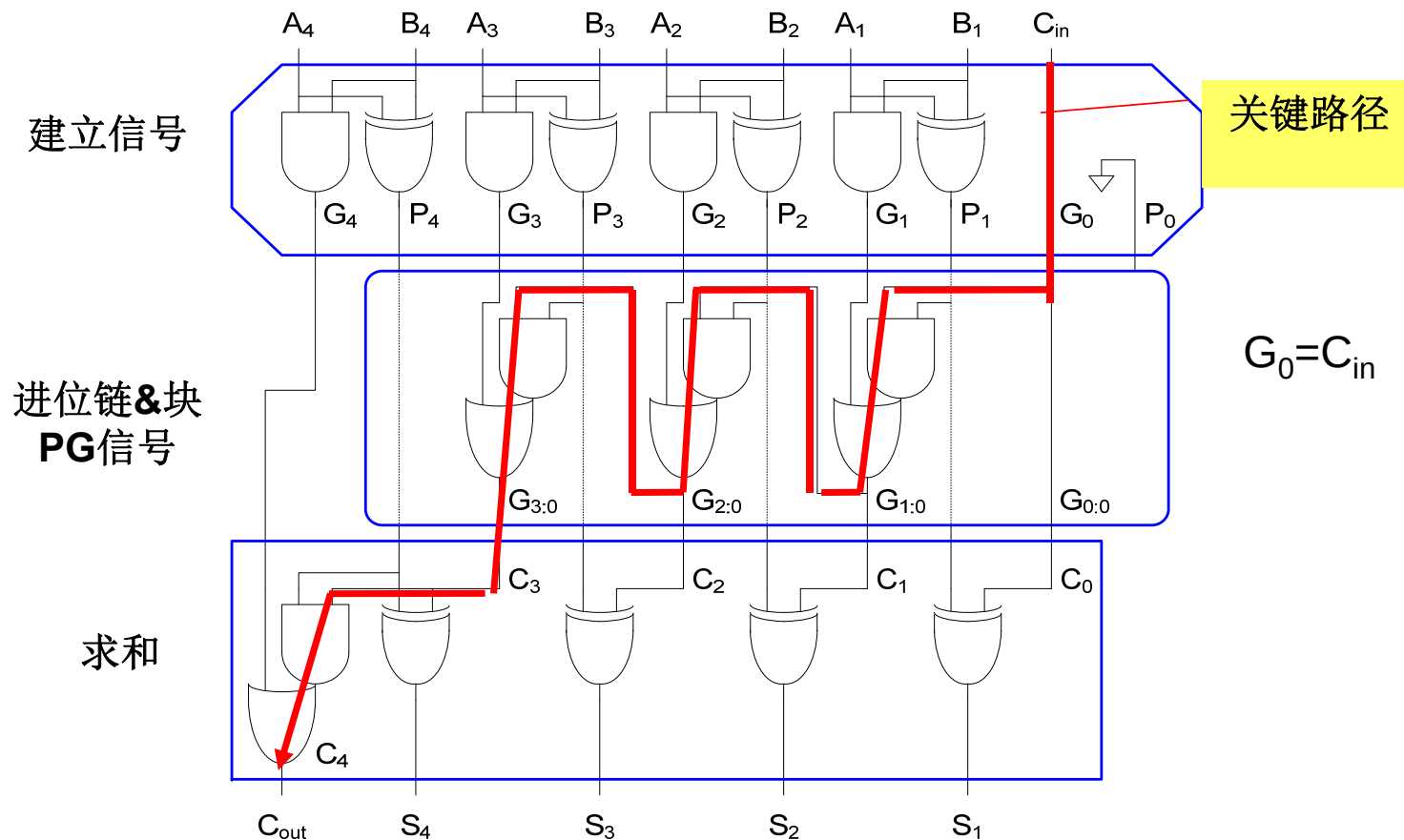


- 采用动态逻辑降低复杂性和加快速度。
- 进位链传输管只用 N 管，节点电容很小，为四个扩散电容。

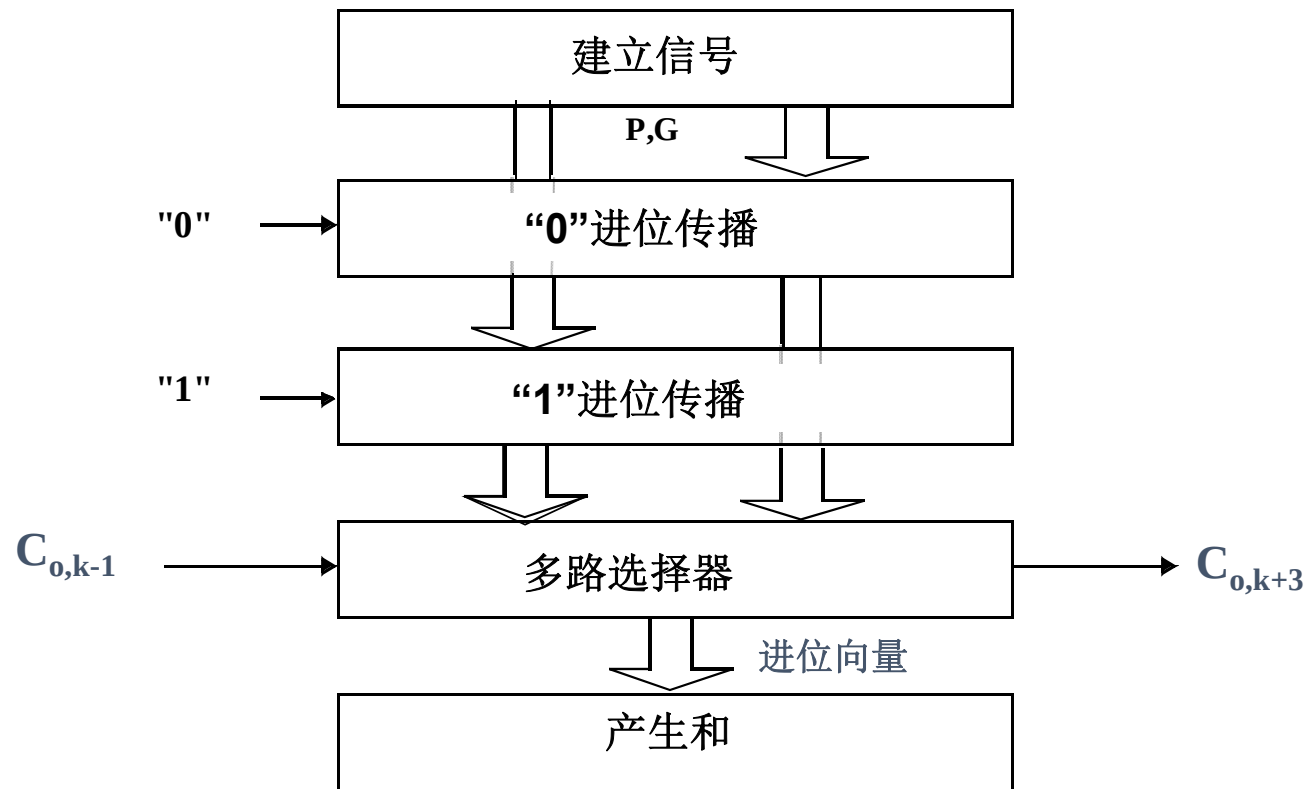
10.2 加法器

10.2.5 进位选择加法器

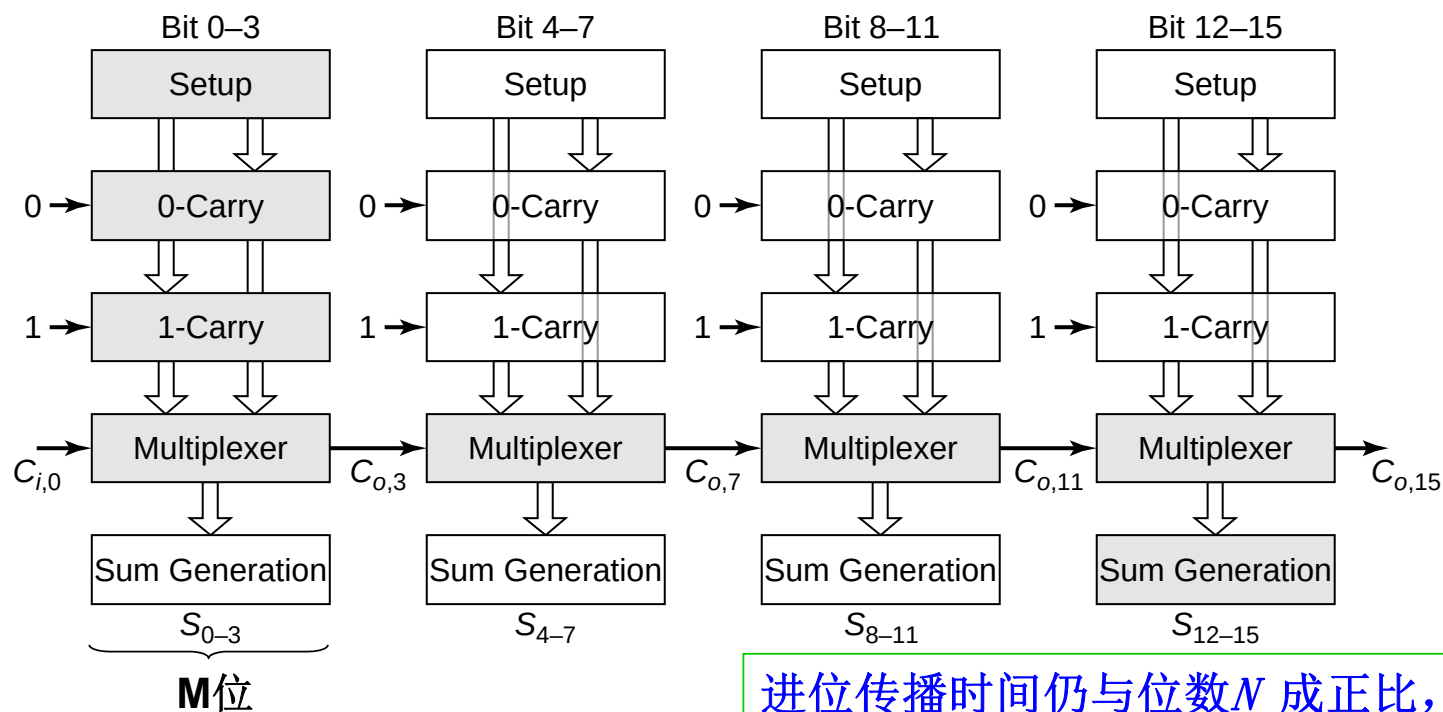
串行进位加法器的逻辑门实现



进位选择加法器



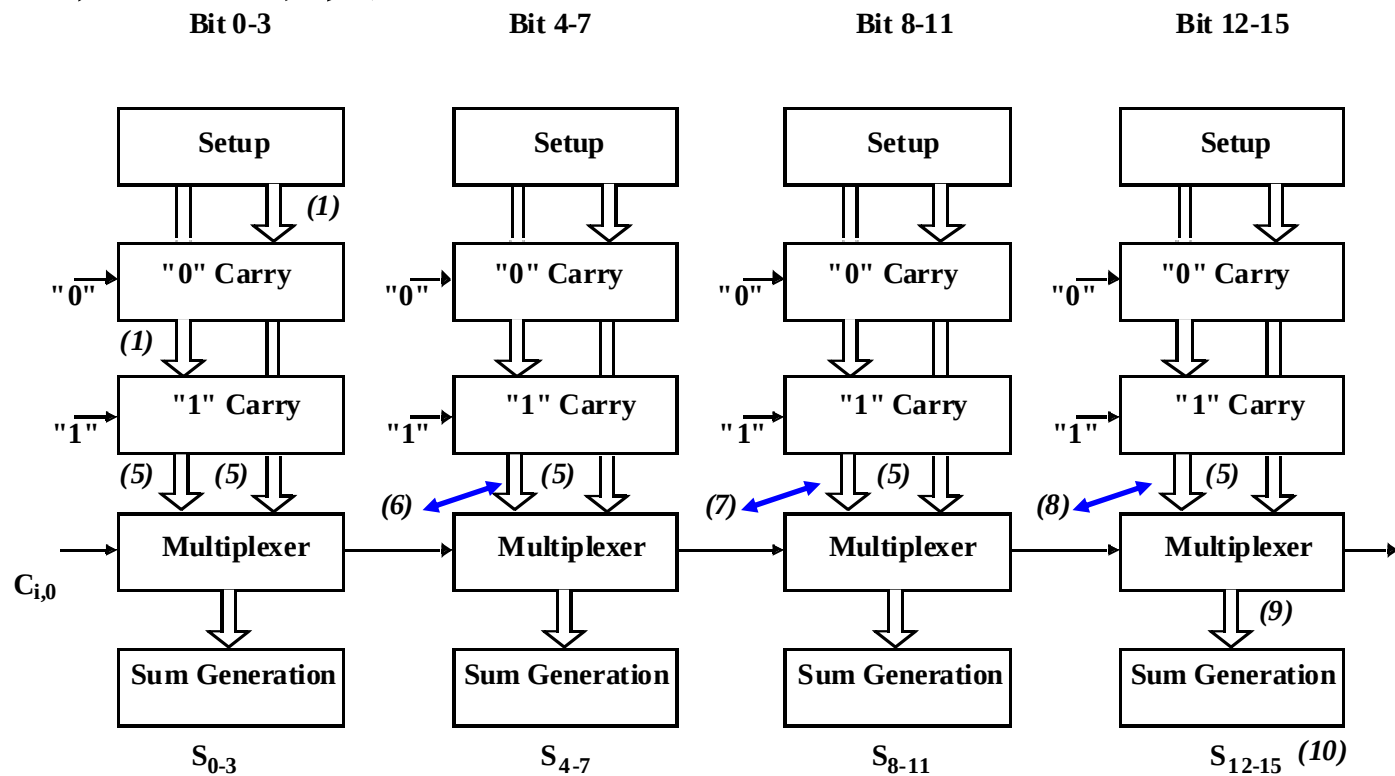
进位选择加法器: 关键路径



$$t_{add} = t_{setup} + Mt_{carry} + \left(\frac{N}{M} \right) t_{mux} + t_{sum}$$

进位传播时间仍与位数 N 成正比，但比逐位进位加法器快。
硬件开销为一个额外的进位路径和一个MUX，大约比一个逐位进位加法器多30%

线性进位选择加法器

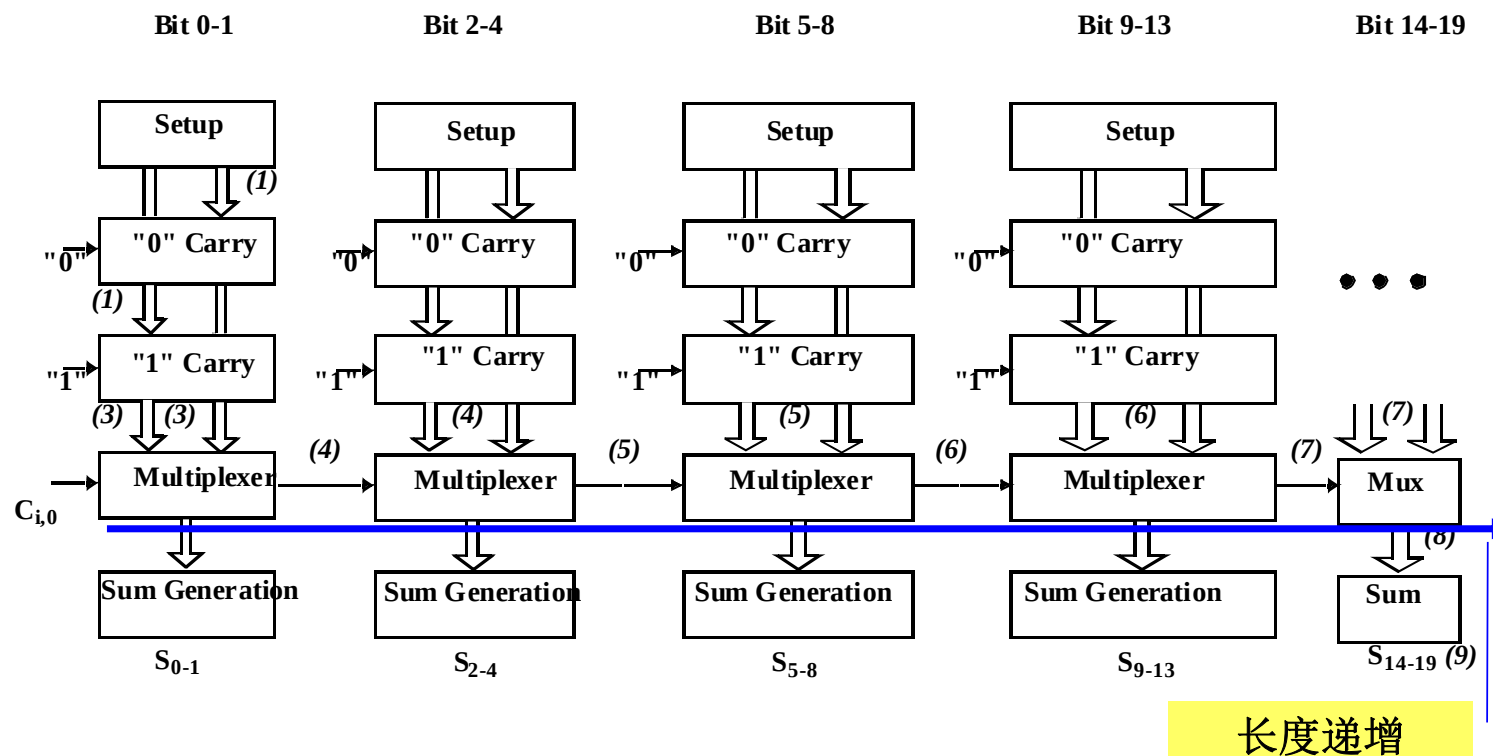


$$t_{add} = t_{setup} + Mt_{carry} + \left(\frac{N}{M}\right)t_{mux} + t_{sum}$$

存在时间浪费

平方根进位选择加法器

考虑到前级的进位输出要经过一个**MUX**才到达本级的进位输入，因此在两条信号路径之间相差一个延时时间，故本级的位数可以比前一级多一级。



平方根进位选择加法器的延时:

假设 N 位的加法器含有 P 个级，且第一级是 M 位，后续级逐级增加一位，于是：

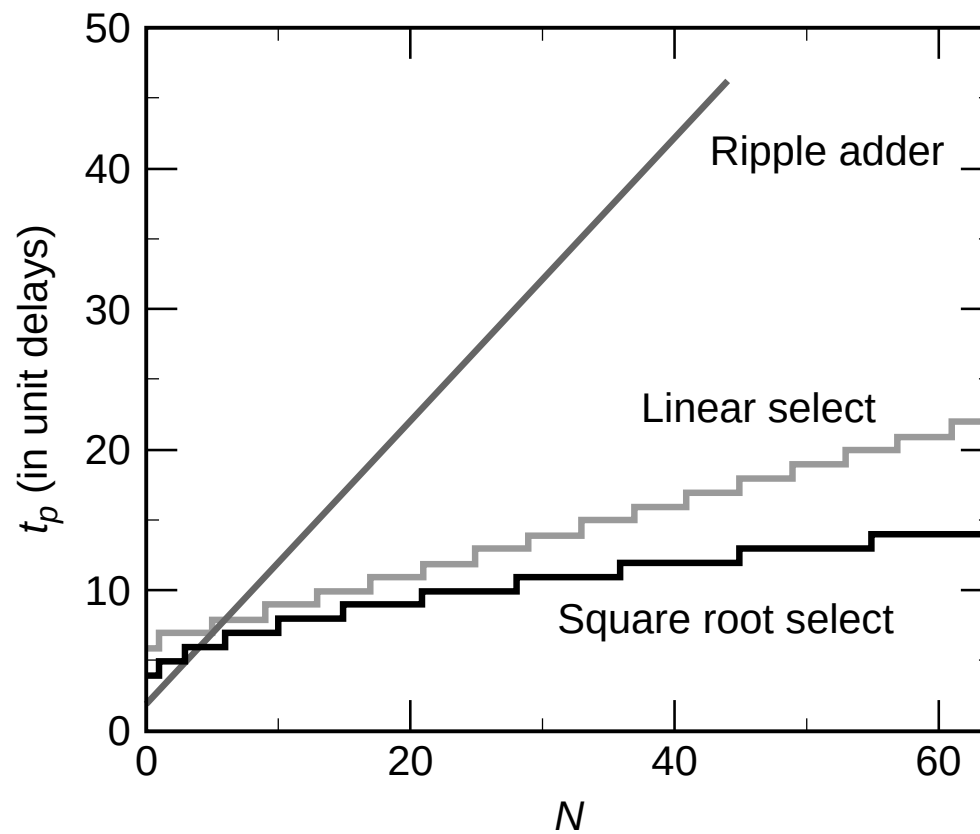
$$\begin{aligned} N &= M + (M + 1) + (M + 2) + (M + 3) + \dots + (M + P - 1) \\ &= MP + \frac{P(P-1)}{2} = \frac{P^2}{2} + P(M - \frac{1}{2}) \end{aligned}$$

$$\text{如果 } M \ll N, \text{ 则 } N \approx \frac{P^2}{2} \text{ 或 } P \approx \sqrt{2N}$$

$$t_{add} = t_{setup} + Mt_{carry} + \sqrt{2N}t_{mux} + t_{sum}$$

此时延时正比于 $\sqrt{2N}$ （亚线性关系），而不是 N （线性关系）
当 N 很大时，延时几乎变为常数。

加法器延时比较



10.2 加法器

10.2.6 超前进位加法器

超前进位（Carry Lookahead）加法器

- 基本原理

$$C_{o,k} = G_k + P_k C_{o,k-1} \quad \text{展开:}$$

$$\begin{aligned} C_{o,k} &= G_k + P_k (G_{k-1} + P_{k-1} (\dots + P_1 (G_0 + P_0 C_{i,0})) \dots) \\ &= \underline{G_k + P_k (G_{k-1} + P_{k-1} (\dots (G_1 + P_1 G_0) \dots))} + \underline{P_k \dots P_1 P_0 C_{i,0}} \\ &\triangleq G_{k:0} + P_{k:0} C_{i,0} \end{aligned}$$

$$S_k = P_k \oplus C_{o,k-1}$$

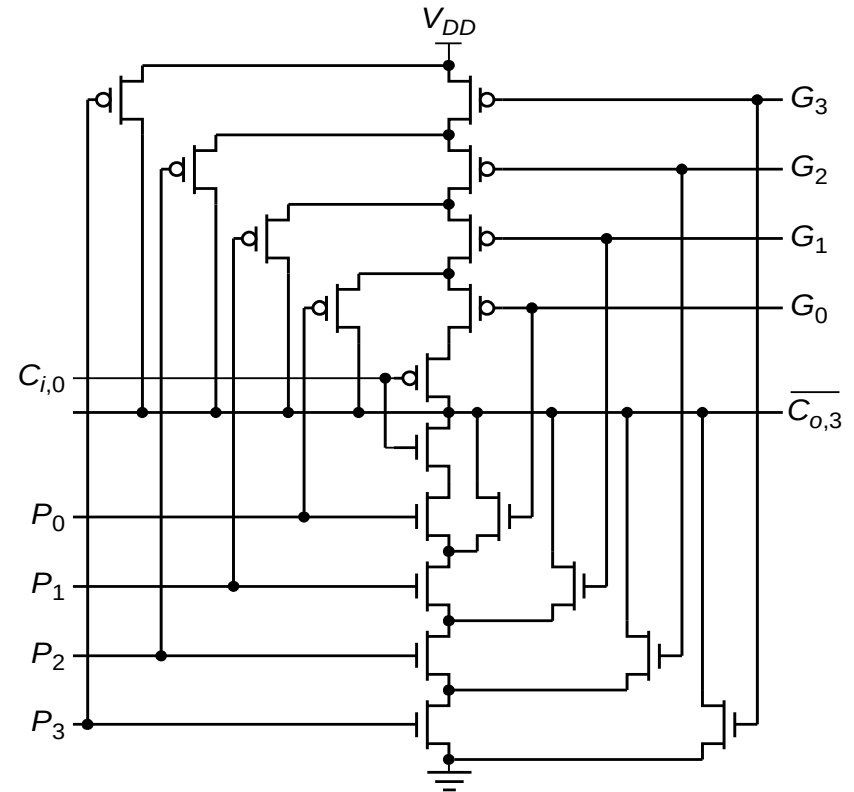
- 由此可知，每一位的进位输出只与最初的进位输入有关，而与比它低位的其他进位输入无关，因此**加法时间与位数 N 无关**。

超前进位加法器的拓扑结构

超前进位加法器(CLA):

直接由建立信号和原始进位输入计算
“和”与进位输出。

$$C_{o,k} = G_k + P_k(G_{k-1} + P_{k-1}(\dots(G_1 + P_1 G_0)\dots)) + P_k \dots P_1 P_0 C_{i,0}$$



超前进位加法器

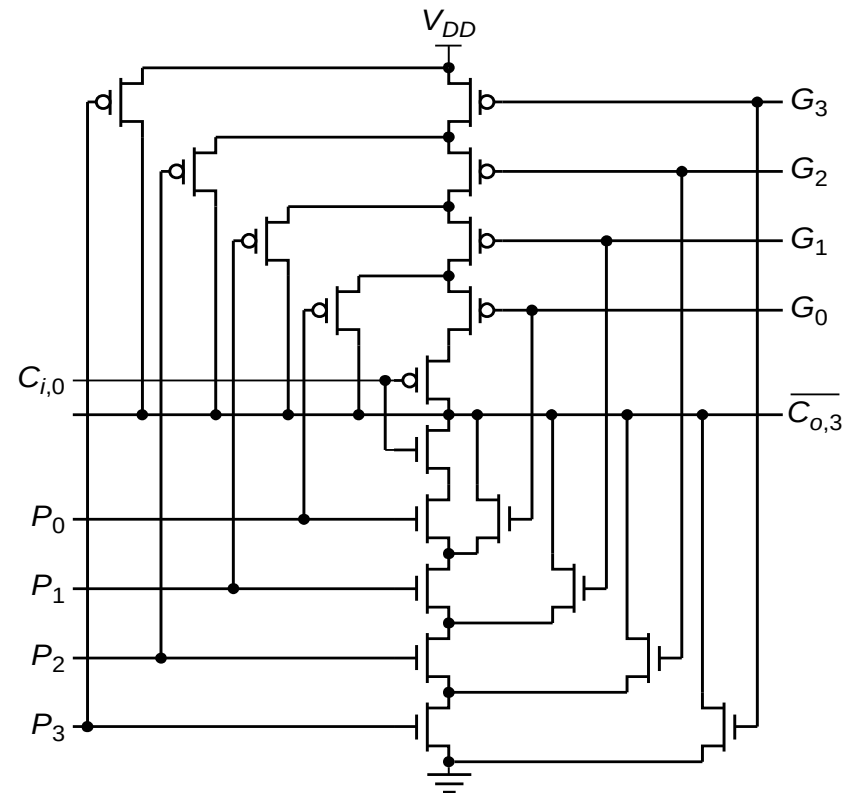
超前进位加法器(CLA):

当位数 N 较大时,

- 某些信号的扇出会很大 (P_0, G_0), 使加法器的延时加大。

- 实现面积也会随位数 N 的增加而增加。

因此这一超前进位只对小的 N 值 ($N \leq 4$) 有效。



CLA小结

- 延时与位数无关的超前进位加法器只是一种理想，实际当中是无法实现的。
- 真正的延时至少随 N 呈线性增长。
- 实际当中，很少使用超过4位的CLA，但是它所蕴含的关系却可以被用来实现高速的加法器。

10.2 加法器

10.2.7 树形加法器

块建立信号

$$C_{o,0} = G_0 + P_0 C_{i,0}$$

$$C_{o,1} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0} = \underline{(G_1 + P_1 G_0)} + \underline{P_1 P_0} C_{i,0} = \underline{G_{1:0}} + \underline{P_{1:0}} C_{i,0}$$

$$C_{o,2} = G_2 + P_2 C_{o,1} = (G_2 + P_2 G_{1:0}) + P_2 P_{1:0} C_{i,0} = G_{2:0} + P_{2:0} C_{i,0}$$

$$C_{o,3} = G_3 + P_3 C_{o,2} = G_3 + P_3 (G_2 + P_2 C_{o,1}) = \underline{G_3} + \underline{P_3 G_2} + \underline{P_3 P_2} C_{o,1} = \underline{G_{3:2}} + \underline{P_{3:2}} C_{o,1}$$

...

$$\begin{aligned} C_{o,i} &= G_i + P_i C_{o,i-1} = G_i + P_i (G_{i-1:j} + P_{i-1:j} C_{o,j}) = (G_i + P_i G_{i-1:j}) + P_i P_{i-1:j} C_{o,j} \\ &= G_{i:j} + P_{i:j} C_{o,j} \end{aligned}$$

块建立信号

$$G_{i:j} = G_i + P_i(G_{i-1} + \dots(G_{j+2} + P_{j+2}(G_{j+1} + P_{j+1}G_j))\dots)$$

$$P_{i:j} = \prod_{k=j}^i P_k$$

$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j}, P_{i:j} = P_{i:k} \cdot P_{k-1:j}, i < k < j$$

$$C_i = G_{i:j} + P_{i:j}C_j$$

块建立信号是位建立信号的一般化。

基本进位算子

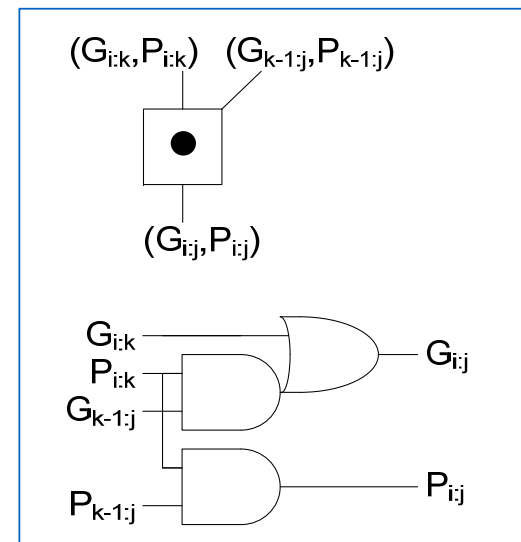
定义：基本进位算子

$$(G, P) \bullet (G', P') = (G + PG', PP')$$

基本进位算子满足结合律

$$(G_{i:j}, P_{i:j}) = (G_{i:k}, P_{i:k}) \bullet (G_{k-1:j}, P_{k-1:j}), i > k > j$$

$$\begin{aligned} (G_{i:j}, P_{i:j}) &= (G_i, P_i) \bullet (G_{i-1:j}, P_{i-1:j}) \\ &= (G_i, P_i) \bullet (G_{i-1}, P_{i-1}) \dots \bullet (G_j, P_j) \\ &= ((G_i, P_i) \bullet (G_{i-1}, P_{i-1})) \dots ((G_k, P_k) \dots \bullet (G_j, P_j)) \end{aligned}$$



N位加法器的问题抽象

重新定义 $G_{0:0} = C_{in}$, 则

$$G_{i:i} \equiv G_i = A_i B_i, P_{i:i} \equiv P_i = A_i \oplus B_i$$

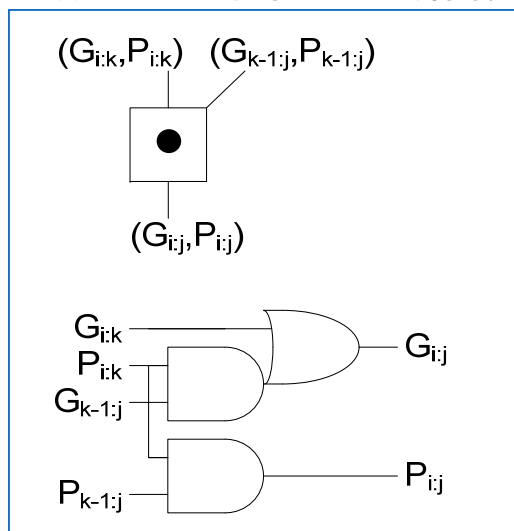
$$S_i = P_i \oplus G_{i-1:0}$$

$$C_{out} = G_{N:0} = G_N + P_N G_{N-1:0}$$

加数A, B的各位从1开始编号。

进位计算问题:

$$\begin{aligned} (G_{i:0}, P_{i:0}) &= (G_i, P_i) \bullet (G_{i-1:0}, P_{i-1:0}) \\ &= (G_i, P_i) \bullet (G_{i-1}, P_{i-1}) \dots \bullet (G_0, P_0) \end{aligned}$$



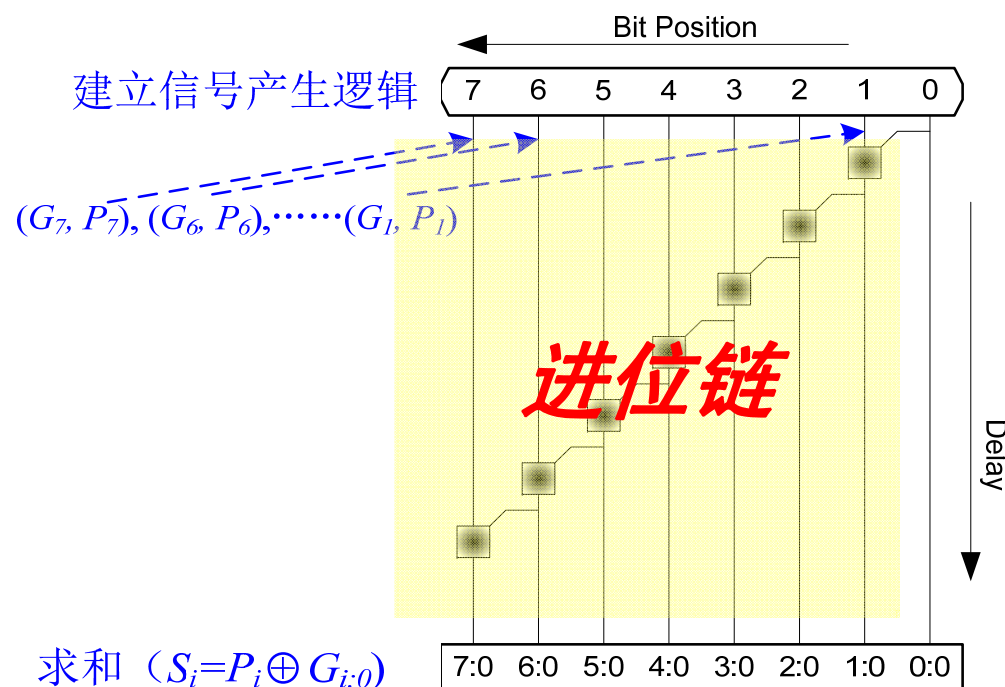
N位加法器的问题抽象

$$G_{0:0} = C_{in}$$

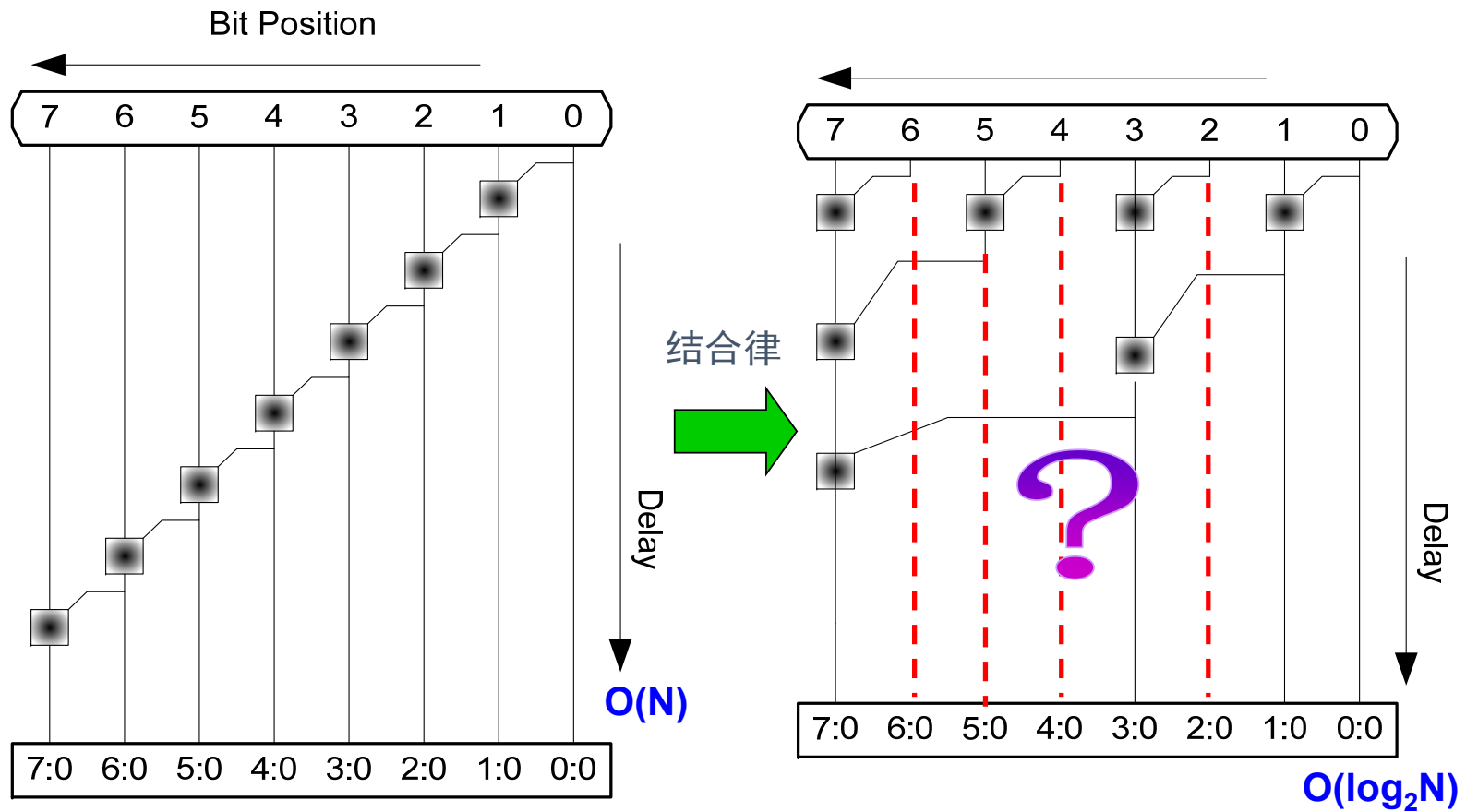
$$G_{i:i} \equiv G_i = A_i B_i, P_{i:i} \equiv P_i = A_i \oplus B_i$$

$$S_i = P_i \oplus G_{i-1:0}$$

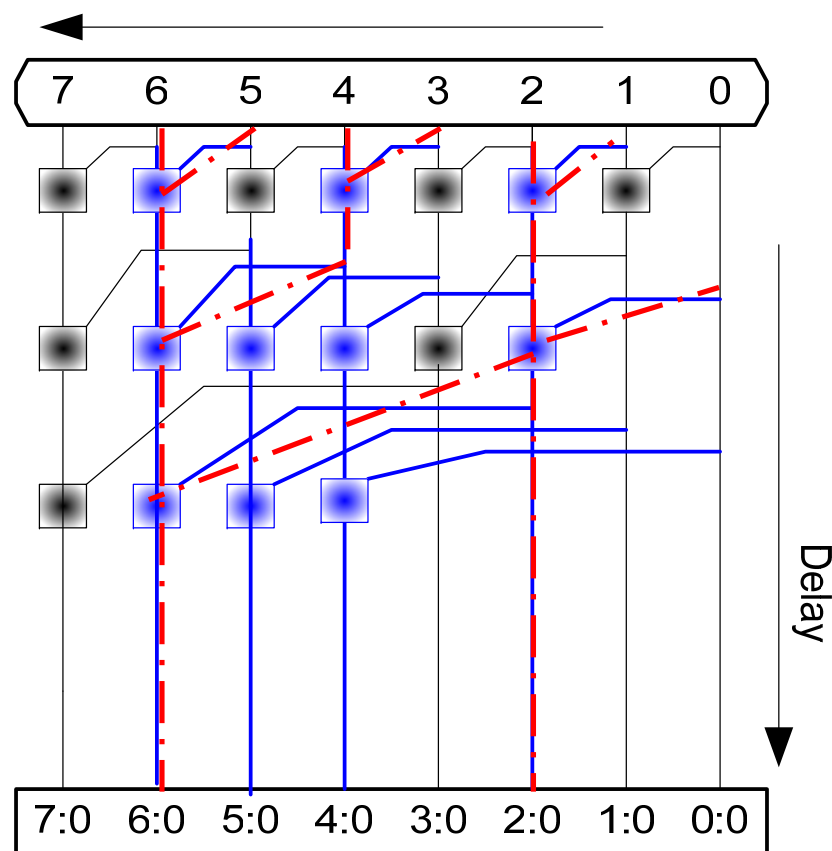
$$C_{out} = G_{N:0} = G_N + P_N G_{N-1:0}$$



并行进位计算结构



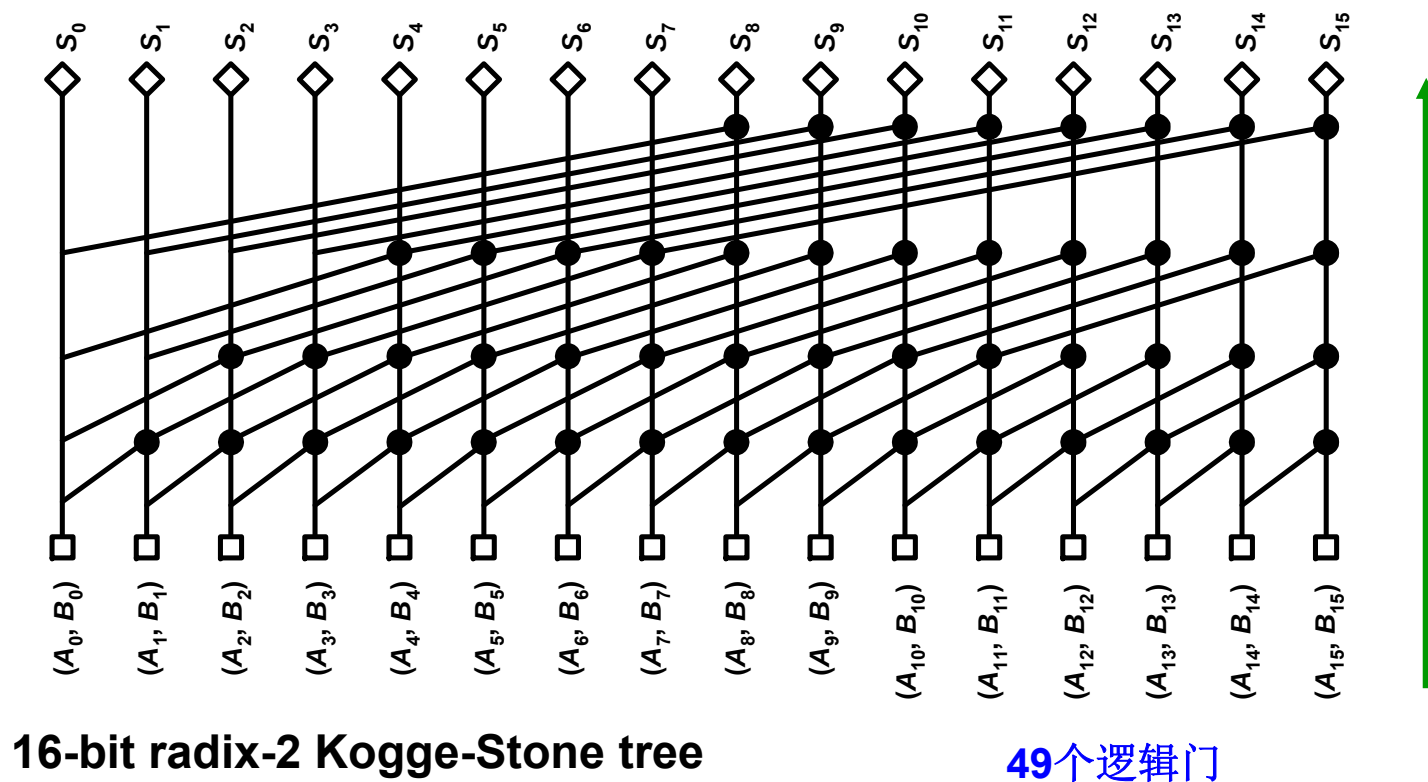
Kogge-Stone树结构



在每一位上都重复二进制树结构

$$D(N) = \log_2 N$$

Kogge-Stone树形加法器



Kogge-stone加法器的特点

- 逻辑层数正比于 $\log_2 N$ 因此传播延时也正比于 $\log_2 N$
- 结构规则，关键路径扇出较小而且基本是一个常数，确定尺寸容易
- 门数较多
- 各级之间有很多长连线

广泛应用于高性能的32位加法器和64位加法器。

加法器小结

- 结构:
 - **Ripple（逐位进位）加法器：**
 1. 面积小，简单，速度相对可以接受，应首先考虑使用。
 2. 易改成流水线加法器。
 - **Carry-Select Adder:**
 1. 较快的n 位加法器。
 2. 容易用Ripple Adder 和MUX 组装。
 3. 面积较大。
 - **Carry Look Ahead (CLA) 加法器：**
 1. 非常快的非流水线加法器。（连线不规则）
 2. 功耗相对较大。
 3. $N < 4$

加法器小结(续)

- 结构（续）：
 - 树形加法器
 - 速度快（对数加法器）、规模大，适用于大的加法器
 - Kogge-Stone加法器由于结构规则、扇出小，可以获得很高的性能。
- 电路实现：
 - 传输门、传输管加法器在实际当中使用最为广泛
 - Manchester 加法器：
 - 1. 规则，较小，较快。
 - 2. 适合于全定制设计的数据通路，字宽从16~ 32位。

10.3 乘法器

10.3.1 数字电路中的乘法运算

二进制乘法

M位XN位

$$X = \sum_{i=0}^{M-1} X_i 2^i$$

$$Y = \sum_{j=0}^{N-1} Y_j 2^j$$

$$Z = X \times Y = \sum_{k=0}^{M+N-1} Z_k 2^k$$

$$= \left(\sum_{i=0}^{M-1} X_i 2^i \right) \left(\sum_{j=0}^{N-1} Y_j 2^j \right)$$

$$= \sum_{i=0}^{M-1} \left(\sum_{j=0}^{N-1} \underline{X_i Y_j} 2^{i+j} \right) \text{ AND}$$

	1	0	1	0	1	0		被乘数(Multiplicand)
x					1	0	1	乘数 (Multiplier)
		1	0	1	0	1	0	
		1	0	1	0	1	0	
		0	0	0	0	0	0	
+	1	0	1	0	1	0		
	1	1	1	0	0	1	1	1
	1	1	1	0	0	1	1	1

}

部分积
(Partial products)

结果

有符号数与补码

- N位2进制数表示无符号数：0~2^N-1
- 负数可以由正数的相反数定义：a为正数，b为它的相反数，a和b都用n比特来表示
- 解决办法：取余
把所有要表示的数值都用一个有限元素的数域中的元素来表示：
- $a + b = 0 \rightarrow (a + b) \bmod 2^N \equiv 0$
- $b \equiv (-a) \bmod 2^N \equiv (2^N - a) \bmod 2^N$, 令 $b = 2^N - a = (100 \cdots 0)_2 - a$

$$\begin{array}{r} a_{n-1} \ a_{n-2} \cdots a_0 \\ \hline \overline{a_{n-1}} \ \overline{a_{n-2}} \cdots \overline{a_0} \end{array}$$

$$+ \qquad \qquad \qquad 1$$

$$1 \ 0 \ 0 \ \cdots \ 0$$

b=a按位取反+1 （a的2's补码）

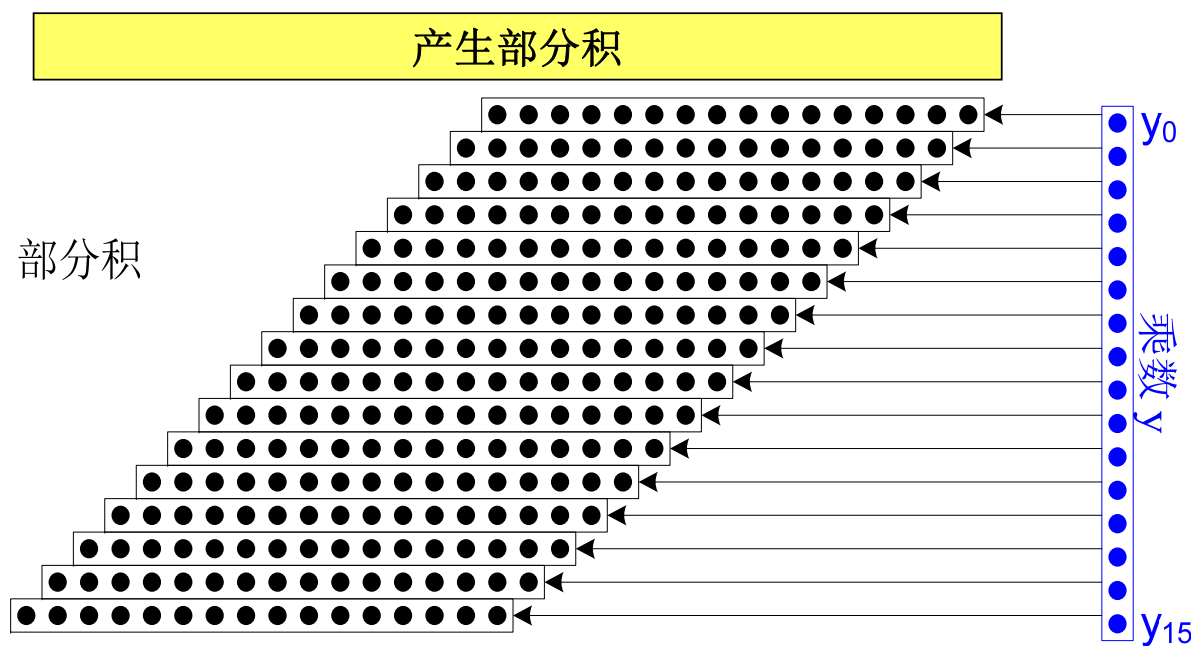
- 有符号数：最高位为符号位（正数-0，负数-1），在计算补码时（相反数）符号位和有效数字位一起取反加1。

补码举例

- 5位正数: $(01010)_2 = (10)_{10}$
- 它的相反数: $(-10)_{10} = (10101)_2 + 1 = (10110)_2$
- 有符号数参与的乘法要考虑符号问题!

乘法器结构

每个点代表部分积的一个比特



阵列乘法器实现:

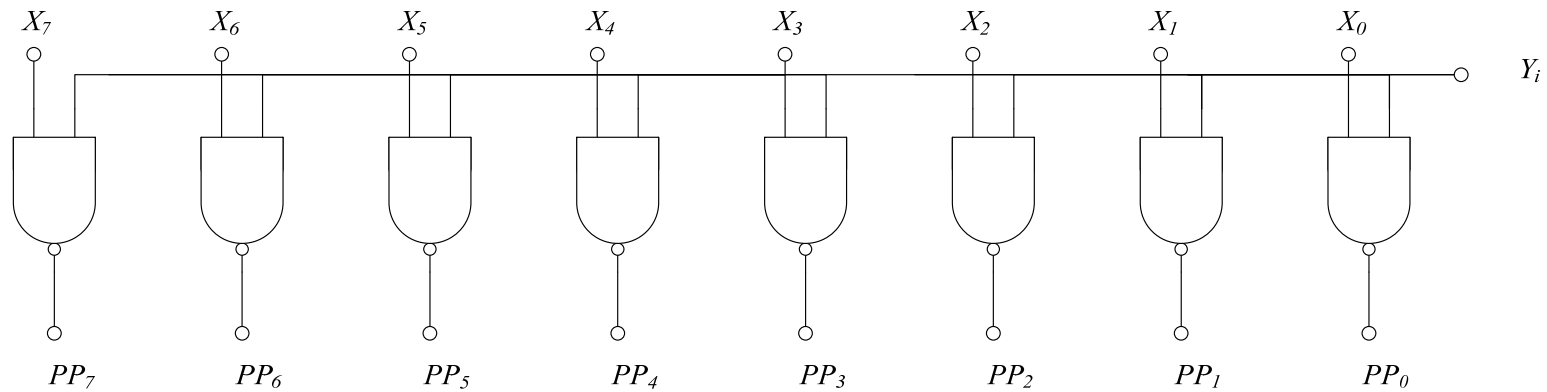
部分积产生-部分积累加-最终相加

10.3 乘法器

10.3.2 部分积产生

部分积产生

- 部分积是被乘数 X 和一个乘数位 Y_i 相乘的结果，一位二进制乘法由AND实现。
- 部分积产生逻辑
 - 共 N 个部分积， $M \times N$ 个AND（NAND）



部分积压缩——波兹编码

- 乘数中等于0的位所对应的部分积对结果没有影响，因此可以只计算非零位的部分积，减少部分积的数目
- 通过编码的方法可以减少非零行的数目
 - 改进的波兹编码（ Booth ）：每次乘数中取 k 位（例如 $k=2$ ）与被乘数相乘产生部分积,并在乘数编码中引入-1来减少非零位。

改进的波兹（Booth）编码

通过同时观察3位乘数进一步简化

a20

即：在乘数的低位上加一“考察位”：考察 $y_{i+1}y_iy_{i-1}$ ，其中 y_{i-1} 为“考察位”。下次考察时考察 $y_{i+3}y_{i+2}y_{i+1}$ ，其中 y_{i+1} 为“考察位”。相邻两次考察时重叠一位乘数（ y_{i+1} ）。

对于有符号数(2's补码表示)的乘数Y

$$Y = -y_{n-1}2^{n-1} + \sum_{j=0}^{n-2} y_j 2^j$$
$$= \sum_{i=0}^{i=n/2-1} \underbrace{(y_{2i-1} + y_{2i} - 2y_{2i+1})}_{\{-2, -1, 0, 1, 2\}} 2^{2i}, y_{-1} = 0$$

编码公式

改进波兹编码的证明

$$X \bullet Y = (y_{n-1}2^{n-1} + y_{n-2}2^{n-2} + \cdots + y_12^1 + y_02^0) \bullet X$$

$$= \overbrace{(y_{-1} + y_0 - 2y_1)}^{\equiv 0} \bullet X \bullet 2^0 + \\ + (y_1 + y_2 - 2y_3) \bullet X \bullet 2^2 + \\ + (y_3 + y_4 - 2y_5) \bullet X \bullet 2^4 + \\ + \cdots +$$

$$+ (y_{n-1} + \overbrace{y_n}^{\equiv 0} - 2\overbrace{y_{n+1}}^{\equiv 0}) \bullet X \bullet 2^n$$

$$= (y_02^0 + y_12^1 + y_22^2 + y_32^3 + \cdots + y_{n-1}2^{n-1}) \bullet X \\ = X \bullet Y$$

$$\begin{aligned} &= \overbrace{(y_{-1} + y_0 - 2y_1)}^{\equiv 0} \bullet X \bullet 2^0 + \quad (n \text{ 为奇数}) \\ &+ (y_1 + y_2 - 2y_3) \bullet X \bullet 2^2 + \\ &+ \cdots + \\ &+ \overbrace{(y_{n-2} + y_{n-1} - 2y_n)}^{\equiv 0} \bullet X \bullet 2^{n-1} = X \bullet Y \end{aligned}$$

(若 n 为偶数)

改进波兹编码乘法

$y_{i+1}y_iy_{i-1}$	部分积
000	0
001	+X
010	+X
011	+2X
100	-2X
101	-X
110	-X
111	0

$$y_{2i-1} + y_{2i} - 2y_{2i+1}$$

举例：X=1101, Y=1001，4位乘4位得8位数
将X, Y看成无符号数（正数），则X=13, Y=9, XY=117

乘数 Y =

0	0	1	0	0	1	0			
	+X		-2X		+X				
0	0	0	0	1	1	0	1		+X
1	0	0	1	1	0				-2X
1	1	0	1						+X
0	1	1	1	0	1	0	1		= 117

注意：-2X 为
左移，求反，
加1（或求反，
加1，左移）。
左移时右边添0。

X: 001101
2x: 011010
-2x: 100101+1=100110

改进波兹编码乘法

$y_{i+1}y_iy_{i-1}$	部分积
000	0
001	+X
010	+X
011	+2X
100	-2X
101	-X
110	-X
111	0

$$y_{2i-1} + y_{2i} - 2y_{2i+1}$$

X=1101, Y=1001,
将X, Y看成2's补码（负数），则X= -3, Y= -7, XY=21

乘数 Y =

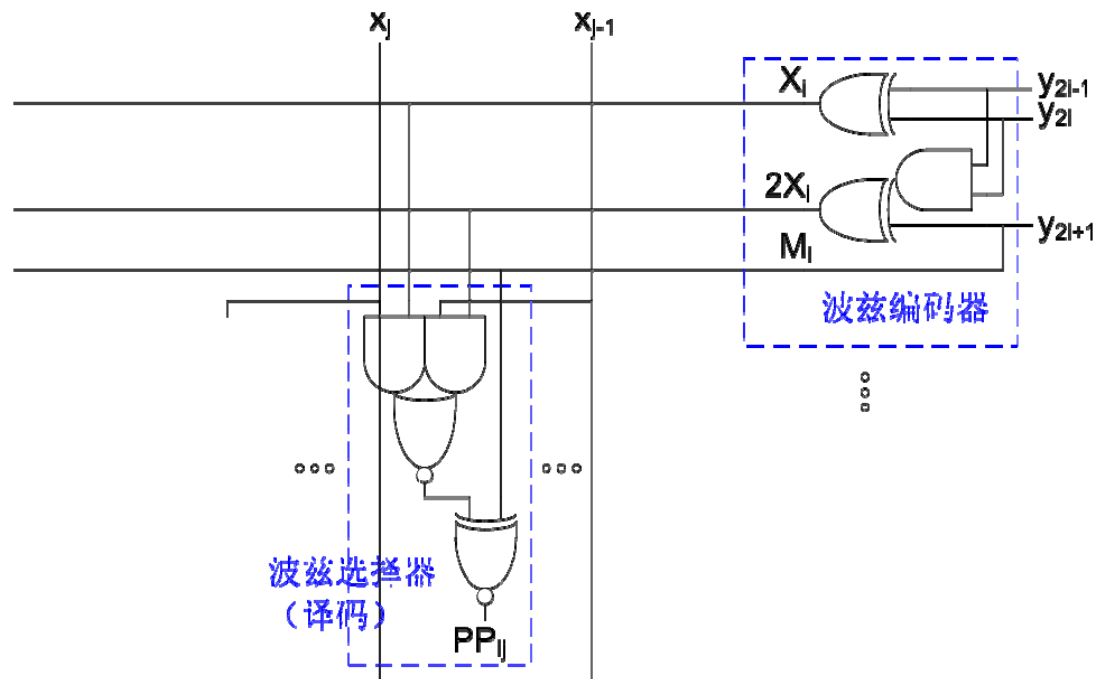
1	1	1	0	0	1	0
	0		-2x		+x	
1	1	1	1	1	0	1
0	0	0	1	1	0	
0	0	0	1	0	1	0
= 21						

计算与无符号
数相同

X: 111101
-x: 000010+1=000011
-2x: 000110

Booth编码器及部分积选择器

- 波兹编码的部分积计算需要编码、译码、移位和（求反+1）



X	2X	M	PP
0	0	/	0
1	0	0	X
0	1	0	2X
1	0	1	-X
0	1	1	-2X

改进Booth编码乘法的特点

- 优缺点：
 - 部分积数目减为原来的一半。可用于无符号数及2's 补码数。
 - 代价是：部分积除0，X 外还有 $+2X$ ， $-2X$ ， $+X$ ， $-X$ ，因此需要译码。对负项还要“求反加1”。
- 在部分积累加时每产生一个部分积要左移2位

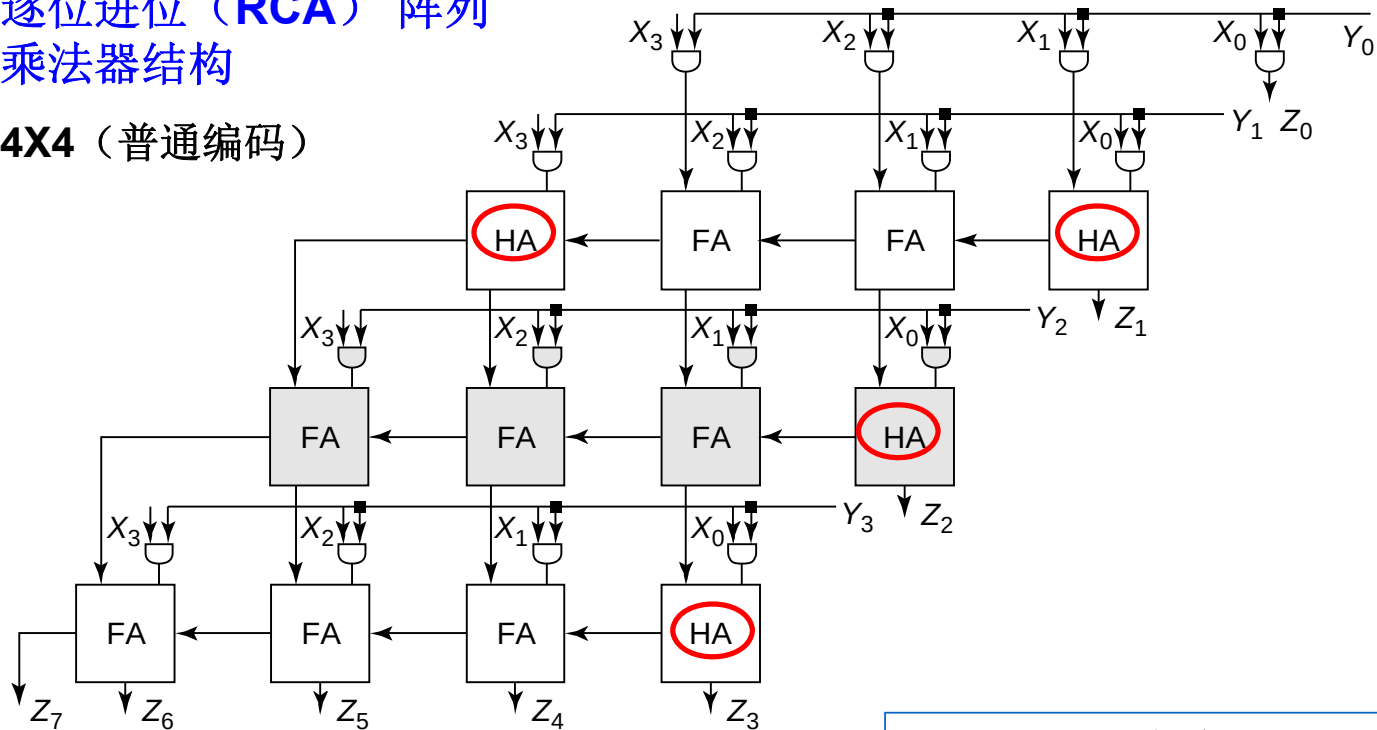
10.3 乘法器

10.3.3 部分积累加

阵列乘法器

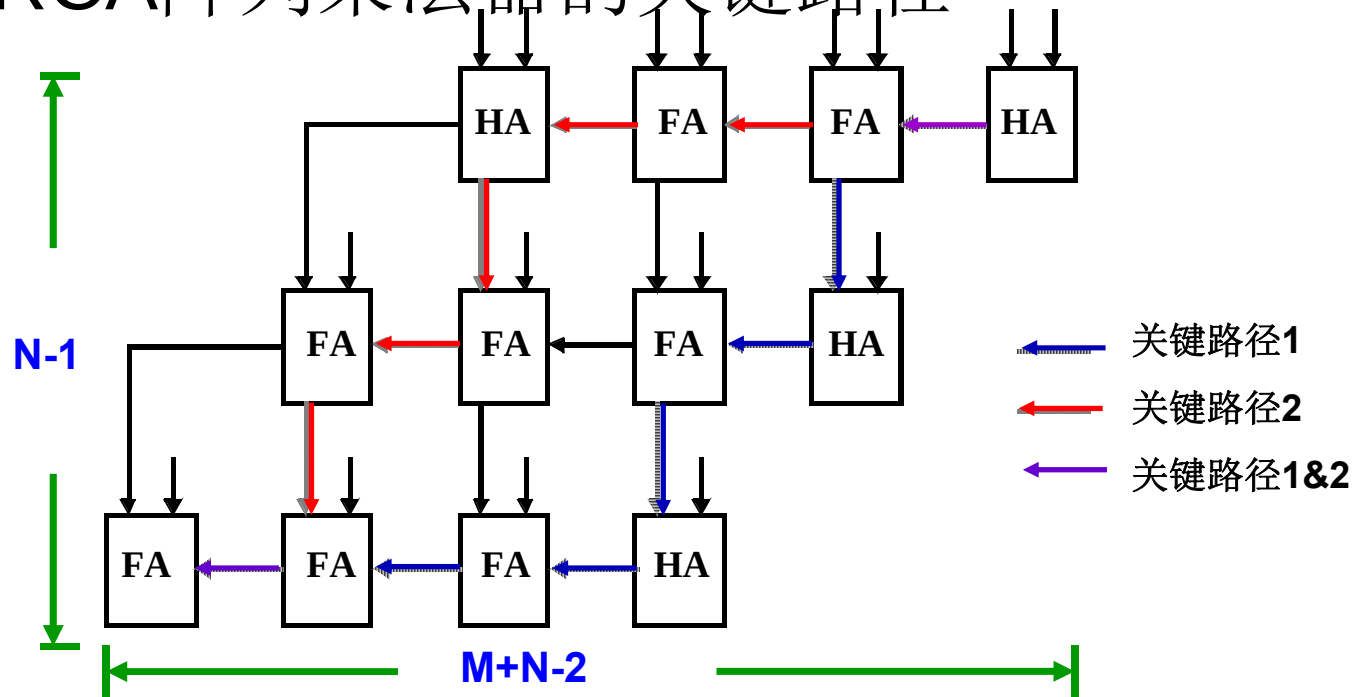
逐位进位（**RCA**）阵列
乘法器结构

4X4（普通编码）



$m \times n$ 位乘法器，共需：
 $n-1$ 个 M 位的加法器

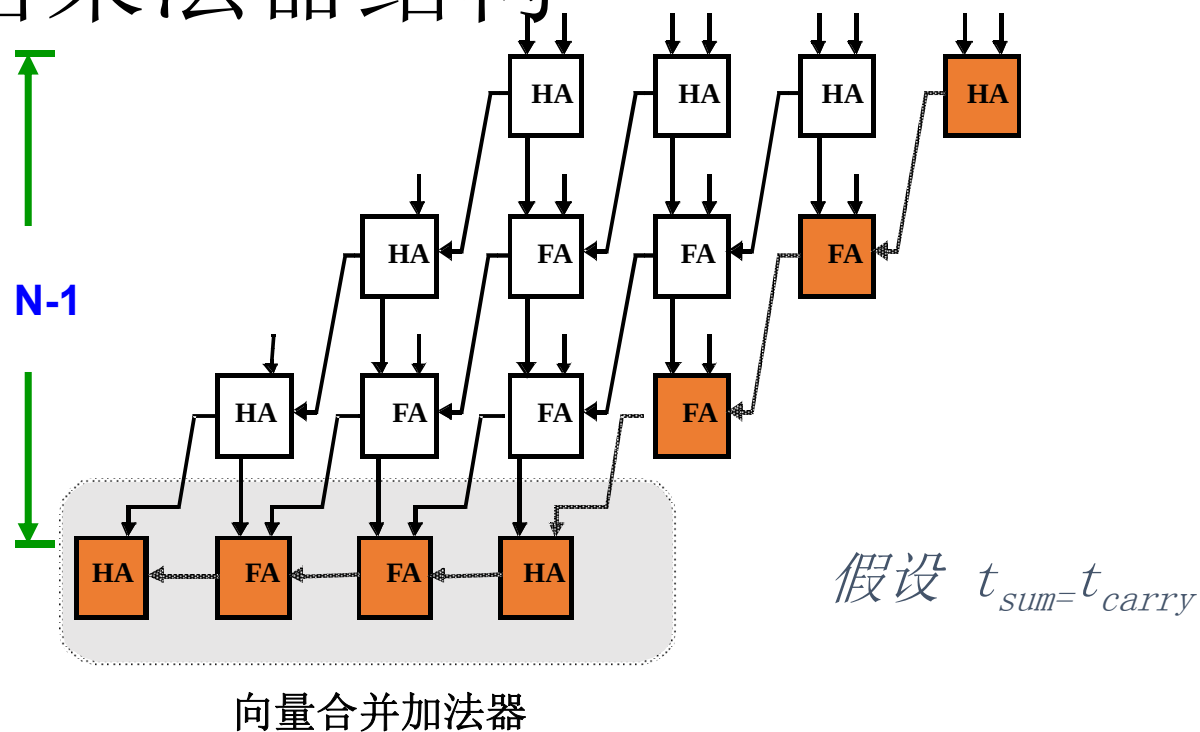
MxN RCA阵列乘法器的关键路径



$$t_{mult} = [(M - 1) + (N - 2)]t_{carry} + (N - 1)t_{sum} + t_{and}$$

多条长度相同的关键路径； t_{sum} 和 t_{carry} 的延时同等重要

进位保留乘法器结构



$$t_{mult} = t_{and} + (N-1)t_{carry} + t_{merge}$$

进位保留加法器

- 一个n 位的进位保留加法器（Carry-Save Adder, CSA）：由n个不相关的一位全加器构成，在这些全加器之间没有进位关系，即在一个n 位CSA加法器的内部没有Carry 传播。
- 它的输入是要被相加的三个n 位数, 它的输出是两个字：
 - 一个字是n 位的“和”： $S(S_{n-1}, \dots, S_1, S_0)$
 - 另一个字是n 位的“进位”： $C(C_{n-1}, \dots, C_1, C_0)$
- S 和C 可以被送入另一个n 位的CSA 与第四个n 位的数W 相加；一般地，多个n 位的数可以通过CSA 树结构相加而产生。
- 为得到最后的和，S 和C 必须最终用一个带进位传播的通常的加法器（CPA）相加（最终相加）。
- 在CSA中为了加快连续加法的速度，减少门延时，可以采用正码和反码两种CSA，在阵列中交叉使用。
- 进位保留乘法器在最坏情形下的关键路径是唯一确定的。

最终相加

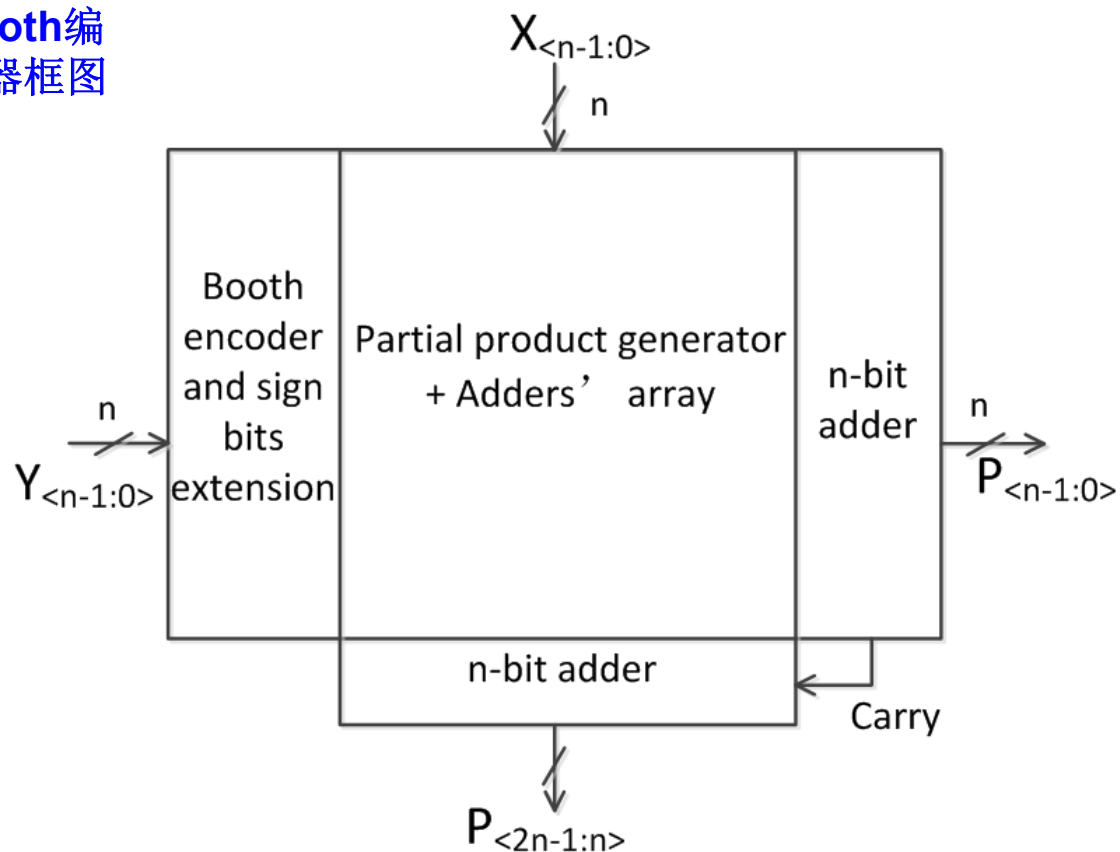
- 实现CSA最后的向量合并
- 由进位传播加法器（CPA）实现
 - 如串行进位加法器
 - 进位选择加法器

10.3 乘法器

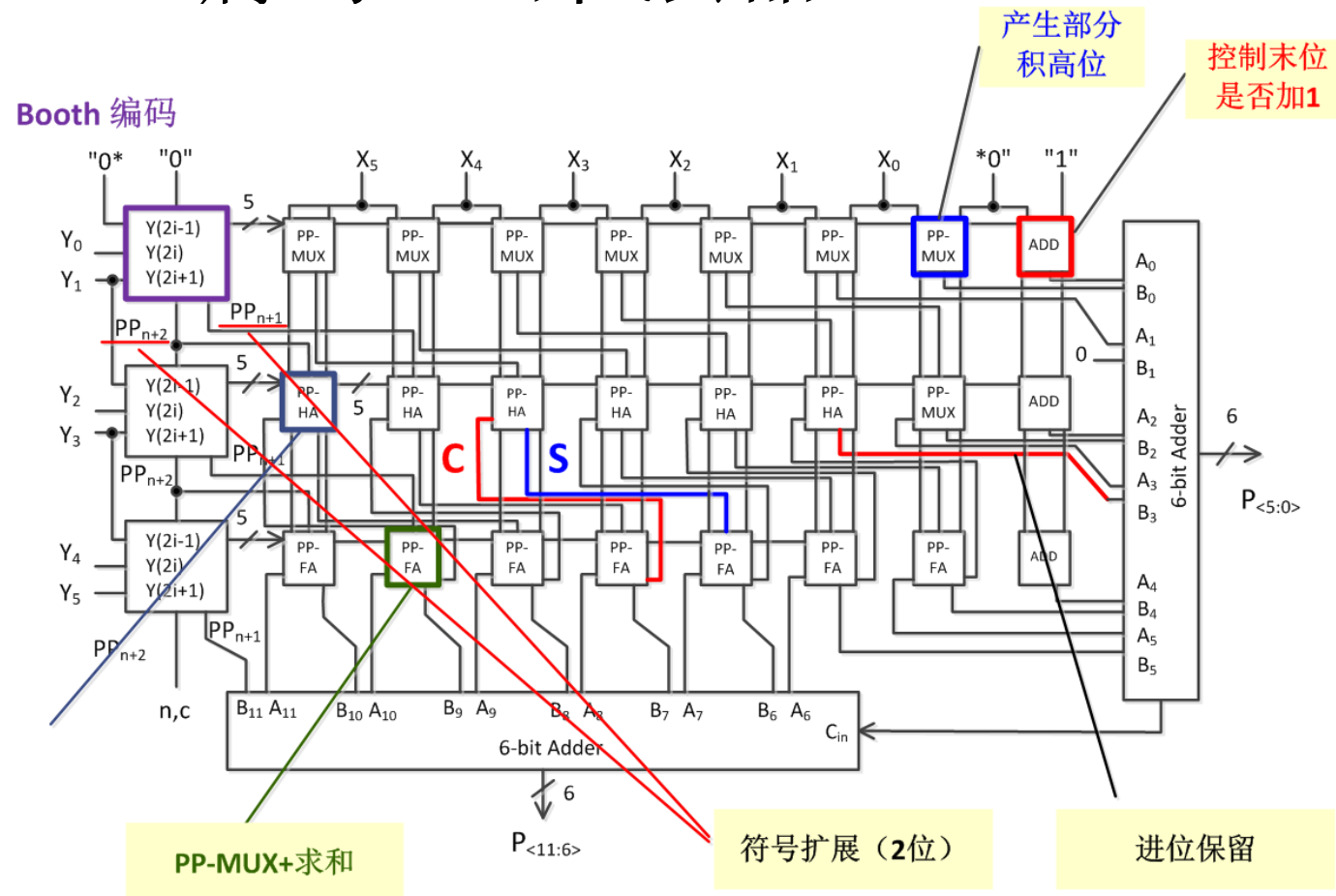
10.3.4 乘法器小结

Booth编码CSA乘法器

改进Booth编
码乘法器框图



6X6 Booth编码CSA乘法器



乘法器小结

- 第一次系统级的优化
- 优化目标和二进制加法器不同
- 又一次出现**关键路径**的概念：识别关键路径至关重要

10.4 本章小结

总结

- 数据通路的特点：
 - 位片式
 - 主要优化指标是性能和功耗，结构的规则性也非常重要
- 加法器：关键路径是进位传播路径
- 乘法器：
 - 总延时=部分积时间+部分积累加时间+最终相加时间
 - 可以通过波兹编码减少部分积数量

总结（续）

- 数据通路的优化方法
 - 线性结构的旁路：超前进位加法器、进位保留
 - 先计算后选择：进位选择
 - 分组并行处理：树形加法器
 - 流水线
 - 把关键路径上的操作移到非关键路径上：利用加法器的反相特性，平方根进位选择.....
 - 预计算：在设计阶段完成常数运算，简化电路（波兹编码）